ShortsAI 코드 상세 안내서

왼쪽: 실제 코드 (줄 번호 포함) · 오른쪽: 그 부분 설명 · 초보자용 · v0.9.2 기준 · 2026-10-13

이 문서는: ShortsAI를 이루는 코드를 **파일마다 처음부터 끝까지** 보여 주고, 블록마다 무슨 일을 하는지 쉬운 말로 설명한다. 왼쪽 코드는 실제 파일에서 프로그램으로 그대로 가져온 것이고, 모든 줄이 설명 칸에 포함됐는지 자동으로 확인했다.

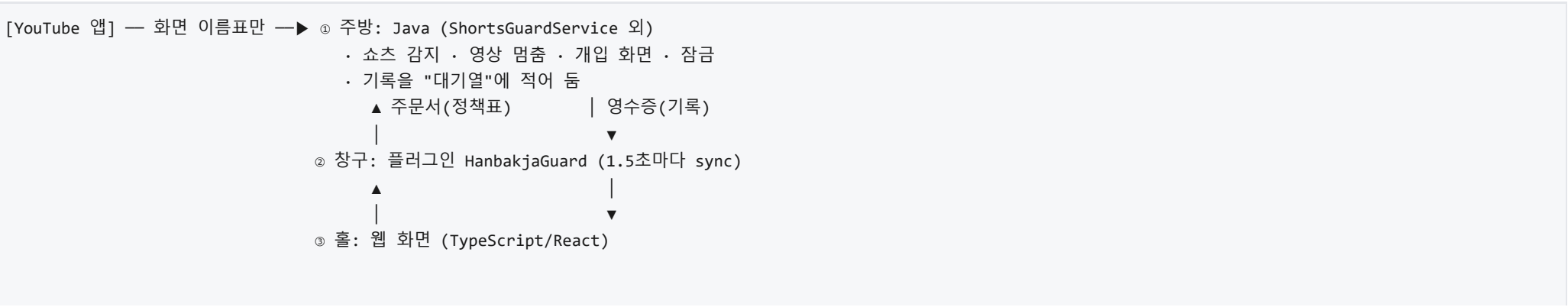
읽는 순서 추천: 1부(용어, 큰 그림, 흐름)를 먼저 읽고 → 관심 있는 파일로 이동. 처음이라면 `GuardScreens.java`(쇼츠 판별), `GuardFlow.java`(개입 화면 순서), `lib/policy.ts`(정책표), `components/guard-bridge.tsx`(연결) 순서를 추천한다.

빠진 것: 자동으로 만들어지는 파일(`package-lock.json`, `gradlew`, 빌드 결과물)과 그림 파일은 넣지 않았다.

1부. 먼저 읽을 것: 용어, 큰 그림, 흐름

용어	쉬운 설명
빌드 / APK	코드(설계도)를 폰에서 돌아가는 프로그램으로 바꾸는 작업 / 그 결과인 안드로이드 설치 파일
네이티브 (Java)	안드로이드가 원래 쓰는 방식의 코드. 다른 앱 화면 감지, 소리 제어 같은 깊은 기능을 쓸 수 있다
웹 (TypeScript, React)	웹사이트 기술로 만든 앱 화면(홈, 통계, 설정). 만들기 빠르지만 폰의 깊은 기능은 못 쓴다
접근성 서비스	원래 장애인을 돕기 위해 다른 앱 화면 구조를 읽게 해 주는 안드로이드 기능. 우리는 쇼츠 화면인지 알아내는 데 쓴다
뷰 ID	화면 부품마다 붙은 이름표 (예: `reel_recycler`). 글자 내용이 아니라 이름표만 본다
오버레이 / 바텀 시트	다른 앱 위에 겹쳐 띄우는 창 / 화면 아래에서 올라오는 창. 개입 화면
함수, 클래스	"이 일을 해라"를 묶은 코드 덩어리 / 관련 값과 함수를 묶은 단위
컴포넌트	React에서 화면 한 조각(버튼, 패널)을 만드는 함수
플러그인	웹 코드가 Java 기능을 부를 수 있게 하는 연결 장치 (우리 것은 HanbakjaGuard)
JSON	데이터를 글자로 적는 형식. `{"goal": 20}`. 웹과 Java가 이 형식으로 주고받는다
SQLite / SharedPreferences	폰 안의 작은 데이터베이스(진짜 기록 보관) / 안드로이드의 작은 메모장(Java가 잠깐 들고 있는 값)
null, undefined	"값이 없음"을 뜻하는 특별한 값
테스트	"이 입력이면 이 결과"를 적어 두고 자동으로 확인하는 코드

큰 그림 (식당 비유)



```
· 기록을 SQLite에 저장 → "저장했어(ack)"
· 기록으로 정책표(목표, 단계, AI 결과)를 만들어 넘김
```

쇼츠 하나 열었을 때 (파일 → 함수)

#	일어나는 일	코드 위치
1	화면이 바뀌었다는 신호	`ShortsGuardService.onAccessibilityEvent` → `evaluate`
2	쇼츠 화면인지 판단	`ShortsGuardService.classifyActiveWindow` → `GuardScreens.isYoutubeShorts`
3	"들어옴/나감" 한 번으로 정리	`ShortsSessionTracker.onSample` → `onEntered`
4	무엇을 할지 결정 (잠금? 약속 이어가기? 개입?)	`ShortsGuardService.decide`
5	영상 멈춤	`ShortsGuardService.pausePlayback`
6	개입 화면 순서 (방식 → 이유 → 약속)	`GuardFlow.entry / method / reason / commitment`, 그리기는 `GuardOverlay`, 문구는 `GuardCopy`
7	AI가 방식 고르기	`GuardBandit.pick` (표는 `lib/ai.ts banditTables`가 만듦)
8	기록을 대기열에	`GuardFlow.write` → `GuardState.addIntervention / endSession`
9	웹이 가져가 저장, 정책표 넘김	`components/guard-bridge.tsx` → `GuardPlugin.sync` → `lib/logic.ts convertNativeBatch` → `components/store.tsx mergeNative` → `lib/db.ts applyOps`, 정책표는 `lib/policy.ts buildPolicy`
10	화면에 표시	`components/home-view.tsx`, `stats-view.tsx`, `report-view.tsx`

Java 문법 미리 보기: `package`(파일이 속한 묶음) · `import`(다른 기능 가져오기) · `class`(묶음 단위) · `static`(객체 없이 바로 쓰는 것) · `final`(한 번 정하면 못 바꿈) · `private`(이 파일 안에서만) · `void`(돌려주는 값 없음) · `if/else`(조건) · `for/while`(반복) · `try/catch`(오류가 나도 앱이 멈추지 않게 잡기) · `->`(짧게 쓰는 함수, 람다)

TypeScript 문법 미리 보기: `const`(바꾸지 않는 값) · `let`(바꿀 수 있는 값) · `function` 또는 `(a) => ...`(함수) · `export`(다른 파일에서 쓸 수 있게 공개) · `type`(데이터 모양 정의) · `a?.b`(a가 있을 때만 b) · `a ?? b`(a가 없으면 b) · `async/await`(오래 걸리는 일을 기다림) · `[...목록]`(목록 펼쳐 복사) · `{ ...객체, x: 1 }`(객체 복사하며 x만 바꾸기)

2부. 휴대폰 쪽 코드 (Java)

위치 `android/app/src/main/java/app/focuson/shorts/`. 작은 파일부터 큰 파일 순서로 본다. 모든 Java 파일 첫 줄의 `package app.focuson.shorts;`는 "이 파일은 app.focuson.shorts 묶음에 속한다"는 뜻이고, `import` 줄들은 안드로이드나 Java가 미리 만들어 둔 기능을 가져다 쓰겠다는 선언이다. 이 두 가지는 파일마다 반복되므로 처음 몇 번만 자세히 설명한다.

MainActivity — 앱의 출입문 `android/app/src/main/java/app/focuson/shorts/MainActivity.java (12줄)`

홈 화면에서 ShortsAI 아이콘을 누르면 열리는 화면. 직접 그리지 않고 Capacitor가 웹 화면(홈, 통계, 설정)을 띄운다.

```java
package app.focuson.shorts;

import android.os.Bundle;
import com.getcapacitor.BridgeActivity;
```

1줄 이 파일이 속한 묶음 이름.

3~4줄 가져다 쓸 기능: `Bundle`(안드로이드가 화면을 다시 만들 때 넘겨주는 보따리), `BridgeActivity`(Capacitor가 만든 "웹 화면을 띄우는 화면").

```java
public class MainActivity extends BridgeActivity {
    @Override
    public void onCreate(Bundle savedInstanceState) {
        registerPlugin(GuardPlugin.class);
        super.onCreate(savedInstanceState);
    }
}
```

6줄 `extends BridgeActivity` = BridgeActivity의 기능을 그대로 물려받는다(상속). 그래서 이 짧은 코드만으로 앱 안의 브라우저(WebView)가 웹 화면을 연다.

7~8줄 `onCreate`는 화면이 처음 만들어질 때 안드로이드가 부르는 함수. `@Override`는 "물려받은 함수를 내 방식으로 고쳐 쓴다"는 표시.

9줄 우리 플러그인 `GuardPlugin`을 등록한다. 이 줄이 있어야 웹 코드에서 `HanbakjaGuard.sync()` 같은 Java 함수를 부를 수 있다.

10줄 나머지는 원래 BridgeActivity의 동작에 맡긴다(`super` = 물려준 쪽). 플러그인 등록이 이보다 먼저여야 한다.

GuardTime — 날짜 계산 도우미 `android/app/src/main/java/app/focuson/shorts/GuardTime.java (55줄)`

날짜를 글자로 만들기, 자정을 넘긴 시청 시간을 날짜별로 나누기, 예전 단계 계산. 안드로이드 기능을 쓰지 않아서 자동 테스트(GuardTimeTest)로 검증한다.

```java
package app.focuson.shorts;

import java.util.Calendar;
import java.util.LinkedHashMap;
import java.util.Locale;
import java.util.Map;
import java.util.TimeZone;

/** Day keys, midnight splits and level bands. No Android types so it can be unit tested. */
public final class GuardTime {
    private GuardTime() {}
```

3~7줄 `Calendar`(달력 계산 도구), `LinkedHashMap`(넣은 순서를 기억하는 표), `Locale`(나라별 표기 규칙), `Map`(이름→값 표), `TimeZone`(시간대)을 가져온다.

10줄 `public final class` = 어디서나 쓸 수 있고(public) 더 물려줄 수 없는(final) 클래스.

11줄 생성자를 `private`로 막아 "물건을 만들지 말고 함수만 바로 써라"는 뜻. 도구 모음 클래스에서 흔히 쓰는 방법.

```java
    /** Local date as yyyy-MM-dd, same format as todayKey() on the web side. */
    public static String dayKey(long millis, TimeZone zone) {
        Calendar cal = Calendar.getInstance(zone);
        cal.setTimeInMillis(millis);
        return String.format(Locale.ROOT, "%04d-%02d-%02d",
            cal.get(Calendar.YEAR), cal.get(Calendar.MONTH) + 1, cal.get(Calendar.DAY_OF_MONT
```

dayKey: 시각(1970년 1월 1일부터 지난 밀리초 숫자)을 `"2026-10-13"` 같은 날짜 글자로 바꾼다. 웹의 `todayKey()`와 같은 모양이라 두 쪽 날짜가 맞는다.

1. **15줄** 그 시간대의 달력을 만든다.
2. **16줄** 시각을 넣는다.

```
H));
    }
```

3. **17~18줄** 연, 월, 일을 꺼내 `%04d-%02d-%02d`(4자리-2자리-2자리) 형식으로 만든다. Java 달력은 1월을 0으로 세서 `+ 1`을 한다.

```
    /** Start of the next local day after {@code millis}. */
    static long nextMidnight(long millis, TimeZone zone) {
        Calendar cal = Calendar.getInstance(zone);
        cal.setTimeInMillis(millis);
        cal.set(Calendar.HOUR_OF_DAY, 0);
        cal.set(Calendar.MINUTE, 0);
        cal.set(Calendar.SECOND, 0);
        cal.set(Calendar.MILLISECOND, 0);
        cal.add(Calendar.DAY_OF_MONTH, 1);
        return cal.getTimeInMillis();
    }
```

nextMidnight: 주어진 시각의 **다음 날 0시**를 돌려준다.

25~28줄 시, 분, 초, 밀리초를 0으로 맞춰 "그날 0시"를 만들고, **29줄**에서 하루를 더한다.

```
    /** Seconds of [start, end) that fall on each local date, in date order. */
    public static Map<String, Double> splitByDay(long start, long end, TimeZone zone) {
        Map<String, Double> out = new LinkedHashMap<>();
        if (end <= start) return out;
        long cursor = start;
        while (cursor < end) {
            long boundary = Math.min(end, nextMidnight(cursor, zone));
            String key = dayKey(cursor, zone);
            Double prev = out.get(key);
            out.put(key, (prev == null ? 0 : prev) + (boundary - cursor) / 1000.0);
            cursor = boundary;
        }
        return out;
    }
```

splitByDay: 시청이 밤 11시 50분~다음 날 0시 10분처럼 자정을 넘기면 "어제 600초, 오늘 600초"로 나눈다.

1. **36줄** 끝이 시작보다 앞이면 빈 표를 돌려준다.
2. **37줄** `cursor` = 지금 보고 있는 지점(처음엔 시작).
3. **38~44줄** 끝에 닿을 때까지 반복: 이번 날짜가 끝나는 지점(`boundary`)을 구하고, 그 날짜에 (boundary − cursor)초를 더한 뒤 다음 날짜로 넘어간다.

결과 예: `{"2026-10-12": 600.0, "2026-10-13": 600.0}`

```
    /** Same bands as levelForUsage() in lib/logic.ts. */
    public static int levelFor(double seconds, int level1Minutes, int level3Minutes) {
        double minutes = seconds / 60.0;
        if (minutes < level1Minutes) return 1;
        if (minutes < level3Minutes) return 2;
        return 3;
    }
}
```

levelFor: 예전 Level 1·2·3 계산(웹 `levelForUsage`와 같은 규칙). 초를 분으로 바꾸고, 첫 기준보다 작으면 1, 둘째 기준보다 작으면 2, 아니면 3. 지금은 Instagram(예전 방식) 호환용으로 남아 있다.

GuardNotifier — 잠금 중 알림 `android/app/src/main/java/app/focuson/shorts/GuardNotifier.java (68줄)`

쇼츠가 잠긴 동안 알림창에 "쇼츠 잠금 중"과 남은 시간 카운트다운을 띄운다.

```
package app.focuson.shorts;

import android.app.NotificationChannel;
import android.app.NotificationManager;
import android.app.PendingIntent;
```

3~10줄 알림 관련 도구들. `NotificationCompat`은 안드로이드 버전이 달라도 똑같이 쓸 수 있게 해 주는 호환 도구.

```
import android.content.Context;
import android.content.Intent;
import android.os.Build;
import androidx.core.app.NotificationCompat;
import androidx.core.app.NotificationManagerCompat;

/** Status bar countdown while Shorts is locked. Skipped when notifications are not allowed. */
final class GuardNotifier {
    private static final String CHANNEL = "shorts_lock";
    private static final int LOCK_ID = 7001;

    private GuardNotifier() {}
```

14줄 `CHANNEL`: 알림 종류 이름. 안드로이드 8 이상은 알림을 채널별로 묶고, 사용자가 채널별로 끌 수 있다.

15줄 `LOCK_ID = 7001`: 알림 번호. 같은 번호로 다시 띄우면 새로 쌓이지 않고 내용이 바뀐다.

```
    static void showLock(Context context, long blockedUntil) {
        long remaining = blockedUntil - System.currentTimeMillis();
        if (remaining <= 0) {
            cancelLock(context);
            return;
        }
        NotificationManagerCompat manager = NotificationManagerCompat.from(context);
        if (!manager.areNotificationsEnabled()) return;
        ensureChannel(context);
```

showLock 시작 부분.

20~24줄 남은 시간 = 잠금 끝 시각 − 지금. 0 이하면 알림을 지우고 끝낸다(`return`).

25~26줄 사용자가 이 앱 알림을 꺼 두었으면 아무것도 하지 않는다.

27줄 알림 채널이 없으면 만든다.

```
        Intent open = new Intent(context, MainActivity.class)
            .addFlags(Intent.FLAG_ACTIVITY_NEW_TASK | Intent.FLAG_ACTIVITY_SINGLE_TOP);
        PendingIntent tap = PendingIntent.getActivity(
            context, LOCK_ID, open, PendingIntent.FLAG_UPDATE_CURRENT | PendingIntent.FLAG_IMMUTABLE);
```

알림을 눌렀을 때 우리 앱(MainActivity)이 열리도록 "나중에 실행할 열기 요청"(`PendingIntent`)을 만든다. `FLAG_IMMUTABLE`은 다른 앱이 이 요청을 바꾸지 못하게 하는 보안 설정.

```
        NotificationCompat.Builder builder = new NotificationCompat.Builder(context, CHANNEL)
            .setSmallIcon(R.drawable.ic_stat_shortsai)
            .setContentTitle("쇼츠 잠금 중")
            .setContentText("잠금이 끝나면 이 알림이 사라져요.")
            .setWhen(blockedUntil)
            .setShowWhen(true)
            .setUsesChronometer(true)
            .setChronometerCountDown(true)
            .setOngoing(true)
            .setOnlyAlertOnce(true)
            .setTimeoutAfter(remaining)
            .setContentIntent(tap)
            .setPriority(NotificationCompat.PRIORITY_LOW);
```

알림 내용을 조립한다.

- **35~37줄** 작은 아이콘, 제목 "쇼츠 잠금 중", 내용.
- **38~41줄** 기준 시각을 잠금 끝으로 두고 **카운트다운 시계**를 켠다. 안드로이드가 스스로 초를 줄여 보여 주므로 앱이 매초 고칠 필요가 없다.
- **42줄** `setOngoing`: 밀어서 지울 수 없음. **43줄** 소리는 처음 한 번만. **44줄** 남은 시간이 지나면 저절로 사라짐.
- **45~46줄** 누르면 앱 열기, 중요도 낮음(소리·진동 없이 조용히).

```
        try {
            manager.notify(LOCK_ID, builder.build());
        } catch (SecurityException ignored) {
            // POST_NOTIFICATIONS was revoked between the check and the call.
        }
    }
```

알림을 띄운다. 확인한 뒤 띄우는 사이에 사용자가 권한을 끌 수도 있어서 `try/catch`로 오류를 잡아 앱이 멈추지 않게 한다.

```
    static void cancelLock(Context context) {
        NotificationManagerCompat.from(context).cancel(LOCK_ID);
```

cancelLock: 7001번 알림을 지운다 (기록 초기화, 잠금 해제 때).

```
    }

    private static void ensureChannel(Context context) {
        if (Build.VERSION.SDK_INT < Build.VERSION_CODES.O) return;
        NotificationManager manager = context.getSystemService(NotificationManager.class);
        if (manager == null || manager.getNotificationChannel(CHANNEL) != null) return;
        NotificationChannel channel = new NotificationChannel(
            CHANNEL, "쇼츠 잠금", NotificationManager.IMPORTANCE_LOW);
        channel.setDescription("시간 차단 중 남은 시간을 보여줍니다.");
        channel.setShowBadge(false);
        manager.createNotificationChannel(channel);
    }
}
```

ensureChannel: 알림 채널 만들기.

59줄 안드로이드 8(O) 미만은 채널이 없어서 그냥 돌아간다. **61줄** 이미 있으면 그대로 둔다. **62~66줄** 이름 "쇼츠 잠금", 중요도 낮음, 설명, 앱 아이콘에 숫자 배지 안 붙임.

GuardScreens — 쇼츠 판별 규칙 `android/app/src/main/java/app/focuson/shorts/GuardScreens.java (143줄)`

"지금 화면이 YouTube 쇼츠 플레이어인지, Instagram 릴스인지"를 **화면 부품 이름표만으로** 판단하는 규칙 모음. 안드로이드 기능 없이 계산만 하므로 테스트 7개로 검증한다.

```
package app.focuson.shorts;

import java.util.Collections;
import java.util.List;
import java.util.Locale;

/**
 * Decides whether a window is the YouTube Shorts player or the Instagram Reels
 * player. Resource ids and view class names are structural. A content
 * description is consulted only when it is a short selected tab label
 * (Shorts / 쇼츠 / Reels / 릴스) and is not kept.
 */
public final class GuardScreens {
    public static final String YOUTUBE = "com.google.android.youtube";
    public static final String INSTAGRAM = "com.instagram.android";
    public static final String YOUTUBE_ID = YOUTUBE + ":id/";
```

7~12줄 설명 주석: 이름표와 부품 종류 이름만 보고, 글자 설명은 "Shorts/쇼츠/Reels/릴스" 같은 짧은 탭 이름일 때만 보며 저장하지 않는다.

14~15줄 두 앱의 패키지 이름(앱의 고유 주소).

16줄 YouTube 이름표 앞에 붙는 접두어 `com.google.android.youtube:id/`.

```
    /**
     * Only on the Shorts player. Verified on SM-S936N (ShortsDetection) for the
     * Shorts tab, Shorts under a video, home feed, search results and channel tab.
     */
    public static final String[] YOUTUBE_SHORTS_IDS = {
        YOUTUBE_ID + "reel_recycler",
        YOUTUBE_ID + "reel_player_page_container"
    };
```

쇼츠 플레이어에만 있는 이름표 2개. 실제 갤럭시(SM-S936N)에서 쇼츠 탭, 영상 아래 쇼츠, 홈, 검색, 채널 탭 모두 확인한 값. `String[]`은 글자 목록.

```
    /**
     * Only on the long-form player. Any of these visible means not Shorts. During the
     * long-form to Shorts animation reel_recycler and watch_player show together.
     * Do not use reel_time_bar (also on long-form) or reel_feedback_play/pause
     * (only while tapping).
```

일반 영상에만 있는 이름표 5개. 이 중 하나라도 보이면 쇼츠가 아니다. 일반 영상에서 쇼츠로 넘어가는 애니메이션 중에는 쇼츠 이름표와 일반 영상 이름표가 같이 보이기 때문에 필요하다.

```
     */
    public static final String[] YOUTUBE_LONGFORM_IDS = {
        YOUTUBE_ID + "watch_player",
        YOUTUBE_ID + "watch_panel",
        YOUTUBE_ID + "watch_list",
        YOUTUBE_ID + "player_overlays",
        YOUTUBE_ID + "metapanel_overlay_recycler_view"
    };
```

30~31줄 주석은 실험으로 알아낸 함정: `reel_time_bar`는 일반 영상에도 있고, `reel_feedback_play/pause`는 화면을 누를 때만 나타나서 쓰면 안 된다.

```
    /** Full-screen Reels viewer. Feed previews and stories do not use this. */
    private static final String[] INSTAGRAM_REELS = {
        "clips_viewer"
    };

    private GuardScreens() {}
```

42~44줄 Instagram 릴스 전체 화면의 이름표 조각 `clips_viewer`. 피드 미리보기와 스토리는 해당하지 않는다.

46줄 생성자를 막아 함수만 쓰게 한다.

```
    public static final class Hit {
        public final List<String> ids;
        public final boolean shortsTab;
        public final boolean reelsTab;
        public final boolean clipsClass;

        public Hit(List<String> ids, boolean shortsTab, boolean reelsTab, boolean clipsClass) {
            this.ids = ids == null ? Collections.emptyList() : ids;
            this.shortsTab = shortsTab;
            this.reelsTab = reelsTab;
            this.clipsClass = clipsClass;
        }
    }
```

Hit: 한 화면에서 모은 정보를 담는 상자. 보이는 이름표 목록(`ids`), 쇼츠 탭 선택 여부, 릴스 탭 선택 여부, 릴스 화면 종류 여부.

55줄 목록이 null이면 빈 목록으로 바꿔서 뒤에서 오류가 나지 않게 한다 (`조건 ? A : B` = 조건이 맞으면 A, 아니면 B).

```
    public static boolean isWatchedPackage(String packageName) {
        return YOUTUBE.equals(packageName) || INSTAGRAM.equals(packageName);
    }

    /** View class names such as ClipsViewer. Stories use ReelViewer and do not match. */
    public static boolean viewClassIsReels(String className) {
        if (className == null) return false;
        return className.contains("Clips");
    }
```

isWatchedPackage: 이 앱이 YouTube나 Instagram인지. 다른 앱이면 판별 자체를 하지 않는다.

viewClassIsReels: 화면 부품 종류 이름에 "Clips"가 들어 있는지 (Instagram 릴스용).

```
    /**
     * Selected navigation tab only. Long captions, names, and typed text do not match.
     * @return "youtube", "instagram", or null
     */
    public static String selectedTabKind(String description, boolean selected) {
        if (!selected || description == null) return null;
        String trimmed = description.trim();
        if (trimmed.isEmpty() || trimmed.length() > 24) return null;
        String lower = trimmed.toLowerCase(Locale.ROOT);
        if (lower.equals("shorts") || lower.equals("쇼츠")
            || lower.startsWith("shorts ") || trimmed.startsWith("쇼츠 ")) {
            return "youtube";
```

selectedTabKind: 아래 메뉴에서 **선택된 탭**의 짧은 이름을 보고 "youtube"/"instagram"/null.

- **77줄** 선택되지 않았거나 설명이 없으면 null.
- **78~79줄** 앞뒤 공백을 지우고, 비었거나 **24자보다 길면 무시**. 긴 글, 사람 이름, 사용자가 입력한 글을 읽지 않기 위한 안전장치.
- **80~88줄** 소문자로 바꿔 "shorts"/"쇼츠"로 시작하면 youtube, "reels"/"릴스"면 instagram.

```
        }
        if (lower.equals("reels") || lower.equals("릴스")
            || lower.startsWith("reels ") || trimmed.startsWith("릴스 ")) {
            return "instagram";
        }
        return null;
    }
```

```
    public static String classify(String packageName, List<String> resourceIds) {
        return classify(packageName, new Hit(resourceIds, false, false, false));
    }

    public static String classify(String packageName, Hit hit) {
        if (packageName == null || hit == null) return null;
        if (YOUTUBE.equals(packageName)) {
            return isYoutubeShorts(hit.ids) ? "youtube" : null;
        }
        if (INSTAGRAM.equals(packageName)) {
            return containsMarker(hit.ids, INSTAGRAM_REELS) ? "instagram" : null;
        }
        return null;
    }
```

classify: 최종 판정 함수. 같은 이름의 함수가 두 개인데(오버로딩), 첫 번째는 이름표 목록만 받아 Hit 상자로 감싼 뒤 두 번째를 부른다.

두 번째 함수: YouTube면 `isYoutubeShorts`로, Instagram이면 이름표에 `clips_viewer`가 있는지로 판단해 `"youtube"`, `"instagram"`, null(아님)을 돌려준다.

```
    /**
     * @param visibleIds full YouTube view ids that are visible to the user
     * @return true when a Shorts id is visible and no long-form id is
     */
    public static boolean isYoutubeShorts(List<String> visibleIds) {
        return firstExact(visibleIds, YOUTUBE_SHORTS_IDS) != null
            && firstExact(visibleIds, YOUTUBE_LONGFORM_IDS) == null;
    }
```

isYoutubeShorts — 핵심 규칙. 쇼츠 이름표가 하나라도 보이고(`!= null`) **그리고**(`&&`) 일반 영상 이름표는 하나도 없을 때(`== null`)만 true.

```
    /** First id in {@code wanted} that appears exactly in {@code ids}, or null. */
    public static String firstExact(List<String> ids, String[] wanted) {
        if (ids == null) return null;
        for (String id : wanted) {
            if (ids.contains(id)) return id;
        }
        return null;
    }
```

firstExact: 찾는 이름표 목록 중 실제로 보이는 첫 번째를 돌려주고, 없으면 null. **글자가 완전히 같아야** 한다(`contains` = 목록에 똑같은 것이 있나). 부분 일치를 쓰면 엉뚱한 부품이 걸릴 수 있어서다.

```
    private static String leafOf(String id) {
        if (id == null || id.isEmpty()) return null;
        int slash = id.lastIndexOf('/');
        if (slash >= 0 && slash + 1 < id.length()) return id.substring(slash + 1);
        return id;
    }
```

leafOf: `com.instagram.android:id/clips_viewer_x`에서 마지막 `/` 뒤의 `clips_viewer_x`만 잘라 낸다. `lastIndexOf` = 마지막으로 나온 위치, `substring` = 그 뒤 글자.

```
    private static boolean containsMarker(List<String> resourceIds, String[] markers) {
        if (resourceIds == null) return false;
        for (String id : resourceIds) {
            String leaf = leafOf(id);
            if (leaf == null) continue;
            for (String marker : markers) {
                if (leaf.contains(marker)) return true;
            }
        }
        return false;
    }
}
```

containsMarker: 이름표 끝부분에 표시 글자(`clips_viewer`)가 **들어 있는지** 본다(Instagram은 부분 일치). 이중 반복문: 이름표마다, 표시 글자마다 확인하고 하나라도 맞으면 true.

ShortsSessionTracker — "머문 한 번" 정리기 `android/app/src/main/java/app/focuson/shorts/ShortsSessionTracker.java (103줄)`

화면 신호는 스와이프할 때마다 수없이 온다. 이걸 "쇼츠에 들어옴 → 나감" **한 번**으로 묶는다. 사용자의 별도 프로젝트 ShortsDetection에서 실제 폰으로 검증한 방식을 옮겨 왔다.

```
package app.focuson.shorts;

import android.os.Handler;

/**
 * Turns per-screen samples into Shorts / Reels stays. Ported from ShortsDetection's
 * ShortsSessionTracker:
 * - entering fires once per stay (swipes inside the player do not re-fire)
 * - "not the player" for {@link #EXIT_GRACE_MS} is re-checked with the probe, then ends
 *   the stay at the time it was first seen missing
 * - another app in front ends it at once
 * Main thread only.
 */
final class ShortsSessionTracker {
    static final long EXIT_GRACE_MS = 1000L;
```

3줄 `Handler`: "이 작업을 몇 초 뒤에 실행해 줘"를 맡기는 안드로이드 도구.

5~13줄 규칙 설명: ① 들어옴은 머무는 동안 한 번만 ② 플레이어가 1초 넘게 안 보이면 다시 확인한 뒤, **처음 안 보인 시각**에 끝냄 ③ 다른 앱이 앞에 오면 바로 끝냄.

15줄 기다리는 시간 1000밀리초 = 1초.

```
    interface Listener {
        void onEntered(String kind);

        void onExited(String kind, long endedAt, String reason);
    }

    /** Current player kind ("youtube" / "instagram"), or null when not on a player. */
    interface Probe {
        String probe();
    }
```

Listener: "들어왔을 때 / 나갔을 때" 알려 줄 상대가 지켜야 할 약속(`interface`). ShortsGuardService가 이 역할을 맡는다.

Probe: "지금 화면이 무슨 플레이어인지 다시 봐 줘"를 부탁할 상대. 1초 뒤 재확인에 쓴다.

```
    private final Handler handler;
    private final Probe probe;
    private final Listener listener;

```

이 정리기가 기억하는 값.

- `handler, probe, listener`: 만들 때 받은 도구들.

```
32      private String kind;
33      /** Wall time when "not the player" was first seen. 0 = not waiting to exit. */
34      private long missingSince;
```

- `kind`: 지금 머무는 플레이어 ("youtube"/"instagram"). 안 머무르면 null.
- `missingSince`: 플레이어가 처음 안 보인 시각. 0이면 "기다리는 중 아님".

```
36      private final Runnable graceCheck = new Runnable() {
37          @Override
38          public void run() {
39              if (kind == null || missingSince == 0L) return;
40              if (kind.equals(probe.probe())) {
41                  missingSince = 0L;
42                  return;
43              }
44              exit(missingSince, "플레이어 아님 " + EXIT_GRACE_MS + "ms 지속");
45          }
46      };
```

graceCheck: 1초 뒤 실행될 확인 작업(`Runnable` = 실행할 수 있는 작업).

- **39줄** 이미 끝났거나 기다리는 중이 아니면 할 일 없음.
- **40~43줄** 다시 보니 같은 플레이어면 기다림 취소 (잠깐 가려졌던 것).
- **44줄** 여전히 아니면 **처음 안 보인 시각(missingSince)**으로 끝낸다. 그래야 기다린 1초가 시청 시간에 더해지지 않는다.

```
48      ShortsSessionTracker(Handler handler, Probe probe, Listener listener) {
49          this.handler = handler;
50          this.probe = probe;
51          this.listener = listener;
52      }
53
54      String currentKind() {
55          return kind;
56      }
```

48~52줄 생성자: 만들 때 받은 도구 세 개를 기억한다(`this.handler = handler` = 받은 것을 내 칸에 저장).

54~56줄 `currentKind`: 지금 머무는 플레이어를 알려 준다.

```
58      /** @param sampled the player kind on screen, or null for another screen of a watched app */
59      void onSample(String sampled, long now) {
60          if (sampled != null) {
61              if (sampled.equals(kind)) {
62                  cancelPendingExit();
63                  return;
64              }
65              if (kind != null) exit(now, "다른 플레이어로 바뀜");
66              kind = sampled;
67              missingSince = 0L;
68              listener.onEntered(sampled);
69              return;
70          }
71          if (kind != null && missingSince == 0L) {
72              missingSince = now;
73              handler.postDelayed(graceCheck, EXIT_GRACE_MS);
74          }
75      }
```

onSample — 화면 신호가 올 때마다 호출. `sampled`는 지금 화면의 플레이어 종류, 플레이어가 아니면 null.

- **61~64줄** 이미 같은 플레이어에 머무는 중이면 기다리던 "나감"을 취소하고 끝 (스와이프는 새 들어옴이 아님).
- **65줄** 다른 플레이어에 머물던 중이면 그건 끝낸다.
- **66~69줄** 새로 들어옴: 종류를 기억하고 listener에게 **"들어옴"을 한 번** 알린다.
- **71~74줄** 플레이어가 아닌 화면이고, 머무는 중이며, 아직 안 기다렸다면: 지금을 `missingSince`로 적고 1초 뒤 확인을 예약한다.

```
77      /** @param label a fixed reason such as "다른 앱" or "화면 꺼짐"; never another app's name */
78      void onLeftApp(String label, long now) {
79          if (kind != null) exit(now, label);
80      }
81
82      /** Ends the stay right away, e.g. when the user picks "나가기". */
83      void forceExit(long now, String reason) {
```

onLeftApp: 다른 앱으로 가거나 화면이 꺼지면 바로 끝낸다. 이유(label)에는 "다른 앱", "화면 꺼짐" 같은 정해진 말만 쓰고 **다른 앱 이름은 절대 넣지 않는다**.

forceExit: 사용자가 "나가기"를 눌렀을 때 바로 끝낸다.

release: 서비스가 꺼질 때 예약을 지우고 초기화한다.

```
        if (kind != null) exit(now, reason);
    }

    void release() {
        cancelPendingExit();
        kind = null;
    }
```

```
    private void exit(long endedAt, String reason) {
        String ended = kind;
        cancelPendingExit();
        kind = null;
        listener.onExited(ended, endedAt, reason);
    }

    private void cancelPendingExit() {
        missingSince = 0L;
        handler.removeCallbacks(graceCheck);
    }
}
```

exit: 실제로 끝내는 공통 함수. 종류를 따로 적어 두고 → 예약 취소 → kind 비우기 → listener에게 "나감(종류, 끝 시각, 이유)"을 알린다.

cancelPendingExit: 기다림 표시를 지우고, 예약해 둔 1초 뒤 확인을 취소한다(`removeCallbacks`).

GuardPlugin — 웹과 Java를 잇는 다리 `android/app/src/main/java/app/focuson/shorts/GuardPlugin.java (169줄)`

웹(TypeScript) 코드가 부를 수 있는 Java 함수 모음. 웹은 `await HanbakjaGuard.함수(값)`로 부르고, Java는 `call.resolve(결과)`(성공) 또는 `call.reject("이유")`(실패)로 답한다.

```
package app.focuson.shorts;

import android.content.Intent;
import android.content.pm.PackageInfo;
import android.content.pm.PackageManager;
import android.provider.Settings;
import android.text.TextUtils;
import com.getcapacitor.JSArray;
import com.getcapacitor.JSObject;
import com.getcapacitor.Plugin;
import com.getcapacitor.PluginCall;
import com.getcapacitor.PluginMethod;
import com.getcapacitor.annotation.CapacitorPlugin;
import java.util.HashSet;
import java.util.Set;
import org.json.JSONException;

@CapacitorPlugin(name = "HanbakjaGuard")
public class GuardPlugin extends Plugin {
```

3~16줄 가져오는 도구: 화면 열기(`Intent`), 앱 정보(`PackageInfo`), 폰 설정(`Settings`), Capacitor 플러그인 도구들(`JSObject`=JSON 객체, `JSArray`=JSON 목록, `PluginCall`=웹에서 온 요청 하나).

18줄 `@CapacitorPlugin(name = "HanbakjaGuard")`: 웹에서 부를 이름. 팀원 원래 앱 이름을 그대로 이어받았다.

19줄 Capacitor의 `Plugin`을 물려받는다.

```
    @PluginMethod
    public void getStatus(PluginCall call) {
        JSObject ret = new JSObject();
```

getStatus: 지금 상태를 한 번에 알려 준다. `@PluginMethod` = 웹에서 부를 수 있는 함수라는 표시.

```
23          ret.put("enabled", isServiceEnabled());
24          ret.put("pendingTarget", GuardState.pendingTarget(getContext()));
25          ret.put("pendingLevel", GuardState.pendingLevel(getContext()));
26          ret.put("pendingUsageSeconds", GuardState.pendingUsage(getContext()));
27          ret.put("watching", GuardState.activeSession(getContext()) != null);
28          ret.put("blockerEnabled", GuardState.isBlockerEnabled(getContext()));
29          ret.put("blockedUntil", GuardState.blockedUntil(getContext()));
30          ret.put("blockMinutes", GuardState.blockMinutes(getContext()));
31          ret.put("appVersion", appVersion());
32          call.resolve(ret);
33      }
```

- **23줄** 접근성 서비스가 켜졌는지.
- **24~26줄** 앱 화면이 이어받을 개입(Instagram 등 예전 방식): 대상, 단계, 사용 초.
- **27줄** 지금 쇼츠를 보는 중인지.
- **28~30줄** 개입 스위치, 잠금 끝 시각, 잠금 길이.
- **31줄** 앱 버전. **32줄** 결과를 웹으로 돌려준다.

```
35      @PluginMethod
36      public void setBlocker(PluginCall call) {
37          boolean enabled = Boolean.TRUE.equals(call.getBoolean("enabled", true));
38          GuardState.setBlockerEnabled(getContext(), enabled);
39          JSObject ret = new JSObject();
40          ret.put("blockerEnabled", GuardState.isBlockerEnabled(getContext()));
41          call.resolve(ret);
42      }
```

setBlocker: 개입 스위치 켜기/끄기. 웹이 보낸 `enabled` 값을 꺼내(없으면 true) 저장하고, 실제로 저장된 값을 돌려준다.

```
44      /**
45       * Hands the service the thresholds and the seconds the web counted itself today,
46       * and returns everything queued since the last {@link #ack}.
47       */
48      @PluginMethod
49      public void sync(PluginCall call) {
50          int level1 = call.getInt("level1Threshold", GuardState.level1Minutes(getContext()));
51          int level3 = call.getInt("level3Threshold", GuardState.level3Minutes(getContext()));
52          String webDate = call.getString("webDate", "");
53          Double webSeconds = call.getDouble("webSeconds", 0.0);
54          GuardState.setConfig(getContext(), level1, level3, webDate, webSeconds == null ? 0 : web
Seconds);
55          JSObject policy = call.getObject("policy");
56          JSArray cards = call.getArray("cards");
57          GuardState.setPolicyAndCards(
58              getContext(),
59              policy == null ? null : policy.toString(),
60              cards == null ? null : cards.toString());
61          try {
62              call.resolve(JSObject.fromJSONObject(GuardState.peekPending(getContext())));
63          } catch (JSONException e) {
64              call.reject("대기 중인 기록을 읽지 못했습니다.");
65          }
66      }
```

sync — 가장 중요한 함수 (1.5초마다 호출).

1. **50~54줄** 웹이 보낸 단계 기준(분), 오늘 날짜, 웹이 직접 잰 초를 저장한다. 없으면 이전 값을 그대로 쓴다.
2. **55~60줄** 정책표(policy)와 공부 카드(cards)를 JSON 글자로 바꿔 저장한다. 없으면 null을 넘겨 기존 것을 유지한다.
3. **61~65줄** 대기열에 쌓인 기록 전체를 돌려준다(`peekPending` = 엿보기, 이때는 지우지 않음). 읽다 오류가 나면 거절한다.

```
68      /** The web side stored these rows in SQLite; drop them from the queue. */
69      @PluginMethod
70      public void ack(PluginCall call) {
71          JSArray ids = call.getArray("ids");
72          Set<String> set = new HashSet<>();
```

ack: 웹이 "이 id들은 데이터베이스에 저장했어"라고 보낸 목록을 받아 대기열에서 지운다.

72~78줄 목록을 하나씩 꺼내 빈 id를 빼고 `HashSet`(중복 없는 모음)에 담는다.
79줄 그 id들을 대기열에서 지운다.

```
        if (ids != null) {
            for (int i = 0; i < ids.length(); i++) {
                String id = ids.optString(i, "");
                if (!id.isEmpty()) set.add(id);
            }
        }
        GuardState.ack(getContext(), set);
        call.resolve();
    }
```

이렇게 "저장 확인 후 삭제"를 하기 때문에 앱 화면이 꺼져 있어도 기록을 잃지 않는다.

```
    @PluginMethod
    public void finish(PluginCall call) {
        String outcome = call.getString("outcome", "");
        String target = call.getString("target", "");
        if (!"youtube".equals(target) && !"instagram".equals(target)) {
            call.reject("대상 앱이 없습니다.");
            return;
        }
        android.content.Context host = getActivity() != null ? getActivity() : getContext();
        if ("watch".equals(outcome)) {
            ShortsGuardService.requestWatch(getContext(), target);
            if ("instagram".equals(target) && getActivity() != null) {
                getActivity().moveTaskToBack(true);
            }
        } else if ("leave".equals(outcome)) {
            ShortsGuardService.requestLeave(host, target);
        } else if ("block".equals(outcome)) {
            int minutes = call.getInt("blockMinutes", 0);
            boolean dayEnd = Boolean.TRUE.equals(call.getBoolean("dayEnd", false));
            if (minutes > 0) {
                long until = GuardState.startBlockUntil(
                    getContext(), System.currentTimeMillis() + minutes * 60_000L, minutes, dayEn
d);

                ShortsGuardService.onBlockStarted(getContext(), until);
            }
            ShortsGuardService.requestLeave(host, target);
        } else {
            call.reject("알 수 없는 결과입니다.");
            return;
        }
        call.resolve();
    }
```

finish: 앱 화면에서 진행한 개입(Instagram, 연습 화면)의 결과를 서비스에 전달.

- **85~90줄** 결과와 대상을 꺼내고, 대상이 youtube/instagram이 아니면 거절.
- **91줄** 화면을 열 때 쓸 기준(지금 앱 화면, 없으면 앱 전체).
- **92~96줄** `watch`: 보기 허락. Instagram이면 우리 앱을 뒤로 보내 릴스로 돌아가게 한다.
- **97~98줄** `leave`: 나가기.
- **99~107줄** `block`: 잠금 분이 있으면 잠금 끝 시각을 계산해 시작하고(오늘은 끝인지도 기록), 서비스에 알린 뒤 나가기.
- **108~111줄** 그 밖의 값은 거절.

```
    /** Starts a time block from the web side without an open prompt (e.g. practice flow). */
    @PluginMethod
    public void startBlock(PluginCall call) {
        int minutes = call.getInt("minutes", 0);
        if (minutes <= 0) {
            call.reject("차단 시간이 없습니다.");
            return;
        }
```

startBlock: 개입 화면 없이 바로 잠금 (위험 시간 알림의 [1시간 잠그기] 버튼 등). 0분 이하면 거절하고, 잠금을 시작해 서비스에 알린 뒤 끝 시각을 돌려준다.

```
        long until = GuardState.startBlock(getContext(), minutes, System.currentTimeMillis());
        ShortsGuardService.onBlockStarted(getContext(), until);
        JSObject ret = new JSObject();
        ret.put("blockedUntil", until);
        call.resolve(ret);
    }
```

```
    @PluginMethod
    public void resetRecords(PluginCall call) {
        GuardState.resetRecords(getContext());
        GuardNotifier.cancelLock(getContext());
        call.resolve();
    }

    @PluginMethod
    public void openSettings(PluginCall call) {
        Intent intent = new Intent(Settings.ACTION_ACCESSIBILITY_SETTINGS);
        intent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK);
        getContext().startActivity(intent);
        call.resolve();
    }
```

resetRecords: 네이티브 쪽 기록 초기화와 잠금 알림 지우기.

openSettings: 폰의 "설정 → 접근성" 화면을 연다. `FLAG_ACTIVITY_NEW_TASK`는 앱 밖의 화면을 열 때 필요한 표시.

```
    private String appVersion() {
        try {
            PackageInfo info = getContext().getPackageManager()
                .getPackageInfo(getContext().getPackageName(), 0);
            return info.versionName == null ? "" : info.versionName;
        } catch (PackageManager.NameNotFoundException e) {
            return "";
        }
    }
```

appVersion: 설치된 앱의 버전 이름(예: "0.9.2")을 읽는다. 못 읽으면 빈 글자.

```
    private boolean isServiceEnabled() {
        String enabled = Settings.Secure.getString(
            getContext().getContentResolver(),
            Settings.Secure.ENABLED_ACCESSIBILITY_SERVICES
        );
        if (TextUtils.isEmpty(enabled)) return false;
        String id = getContext().getPackageName() + "/" + ShortsGuardService.class.getName();
        TextUtils.SimpleStringSplitter splitter = new TextUtils.SimpleStringSplitter(':');
        splitter.setString(enabled);
        while (splitter.hasNext()) {
            if (id.equalsIgnoreCase(splitter.next())) return true;
        }
        return false;
    }
}
```

isServiceEnabled: 접근성 서비스가 켜졌는지 확인.

1. **156~159줄** 폰 설정에서 "켜진 접근성 서비스 목록"을 읽는다. `:`로 이어 붙은 긴 글자다.
2. **160줄** 비어 있으면 꺼진 것.
3. **161줄** 우리 서비스의 이름 `app.focuson.shorts/app.focuson.shorts.ShortsGuardService`를 만든다.
4. **162~166줄** `:`로 잘라 하나씩 비교하고, 같은 게 있으면(대소문자 무시) 켜진 것.

GuardBandit — AI가 방식을 고르는 곳 (톰슨 샘플링) `android/app/src/main/java/app/focuson/shorts/GuardBandit.java (88줄)`

웹이 계산해서 보낸 "방식별 성공·실패 점수"로, 개입할 때 어떤 방식을 보여 줄지 고른다. 안드로이드 기능을 쓰지 않아 테스트(GuardBanditTest)로 확인한다.

> **톰슨 샘플링 쉽게:** 방식마다 "잘 먹힐 확률"을 정확히 모른다. 그래서 기록으로 만든 **범위 안에서 주사위를 한 번씩 굴려** 가장 큰 값이 나온 방식을 고른다. 성공이 많은 방식은 높은 값이 자주 나와 자주 뽑히고, 기록이 적은 방식은 값이 들쭉날쭉해서 가끔 높게 나와 다시 시도된다. 이 "범위"를 수학에서는 **Beta 분포**라고 하고, 두 숫자 [α, β](대략 성공 수 +1, 실패 수+1)로 나타낸다.

```java
package app.focuson.shorts;

import java.util.List;
import java.util.Map;
import java.util.Random;

/**
 * Thompson sampling over the arms the web side scored (lib/ai.ts banditTables).
 * Each arm has a Beta(alpha, beta) posterior; draw once from each and take the highest.
 * Arms without a record use Beta(1, 1), so new methods still get tried.
 * No Android types, so it can be unit tested with a seeded Random.
 */
final class GuardBandit {
    private GuardBandit() {}
```

3~5줄 목록, 표, 난수(무작위 숫자) 도구. **7~12줄** 설명 주석: 기록 없는 방식은 Beta(1, 1)(전혀 모름)으로 본다. **14줄** 함수만 쓰는 도구 클래스.

```java
    static String pick(List<String> arms, Map<String, double[]> posteriors, Random random) {
        return pick(arms, posteriors, random, null, 0L, 0, 1);
    }
```

짧은 버전의 **pick**: 질림 반영 없이 고른다. 아래 긴 버전을 "질림 정보 없음, 세기 0"으로 부른다.

```java
    /**
     * Same, with habituation (ai-2): an arm shown recently has its draw scaled by
     * 1 - strength * 0.5^(hours since shown / recoveryHours), so a fresh arm gets a turn.
     * Learning is unchanged; only the pick leans away from repeats for a while.
     */
    static String pick(
        List<String> arms,
        Map<String, double[]> posteriors,
        Random random,
        Map<String, Long> lastShown,
        long now,
        double strength,
        double recoveryHours
    ) {
        if (arms == null || arms.isEmpty()) return null;
        String best = arms.get(0);
        double bestDraw = -1;
        for (String arm : arms) {
            double[] ab = posteriors == null ? null : posteriors.get(arm);
            double alpha = ab == null ? 1 : Math.max(0.01, ab[0]);
```

pick — 방식 고르기. 받는 값: 고를 수 있는 방식 목록(`arms`), 방식별 [α, β] 표(`posteriors`), 난수, 방식별 마지막으로 보여 준 시각, 지금 시각, 질림 세기, 회복 시간.

1. **34줄** 고를 게 없으면 null.
2. **35~36줄** 지금까지 가장 좋은 방식과 그 값을 기억할 칸.
3. **37줄** 방식마다 반복:
4. **38~40줄** 표에서 [α, β]를 꺼낸다. 없으면 1, 1. 0 이하가 되지 않게 최소 0.01.
5. **41줄** Beta 분포에서 **한 번 뽑는다** (주사위 굴리기).
6. **42~43줄** 최근에 보여 준 방식이면 뽑은 값을 줄인다 (질림 반영).
7. **44~47줄** 지금까지 가장 크면 그 방식을 기억한다.
8. **49줄** 가장 큰 값이 나온 방식을 돌려준다.

```
            double beta = ab == null ? 1 : Math.max(0.01, ab[1]);
            double draw = beta(alpha, beta, random);
            Long shown = lastShown == null ? null : lastShown.get(arm);
            draw *= freshness(shown == null ? 0L : shown, now, strength, recoveryHours);
            if (draw > bestDraw) {
                bestDraw = draw;
                best = arm;
            }
        }
        return best;
    }
```

```
    /** 1 for never or long ago; 1 - strength right after being shown. */
    static double freshness(long shownAt, long now, double strength, double recoveryHours) {
        if (strength <= 0 || shownAt <= 0 || now < shownAt || recoveryHours <= 0) return 1;
        double hours = (now - shownAt) / 3_600_000.0;
        return 1 - Math.min(1, strength) * Math.pow(0.5, hours / recoveryHours);
    }
```

freshness — 질림 계산. 방금 보여 준 방식은 1 − 세기(0.3) = **0.7배**, 시간이 지날수록 회복된다.

54줄 세기가 0이거나, 보여 준 적 없거나, 시각이 이상하면 1(줄이지 않음).

55줄 지난 시간(시간 단위). **56줄** `1 − 세기 × 0.5^(지난 시간 ÷ 회복 시간)`: 회복 시간(6시간)마다 줄어드는 양이 절반이 된다. 예: 6시간 뒤 0.85배, 12시간 뒤 0.925배.

```
    /** Mean of the posterior, for showing what the AI has learned. */
    static double mean(double[] ab) {
        return ab == null ? 0.5 : ab[0] / (ab[0] + ab[1]);
    }
```

mean: [α, β]의 평균 α ÷ (α + β) = "멈추게 한 비율" 추정치. AI가 배운 것을 보여 줄 때 쓴다. 표가 없으면 0.5.

```
    static double beta(double alpha, double beta, Random random) {
        double x = gamma(alpha, random);
        double y = gamma(beta, random);
        return x + y == 0 ? 0.5 : x / (x + y);
    }
```

beta — Beta 분포에서 한 번 뽑기. 수학 공식: Gamma 분포에서 x(α)와 y(β)를 하나씩 뽑으면 x ÷ (x + y)가 Beta(α, β)를 따른다. 둘 다 0이면 0.5.

```
70      /** Marsaglia–Tsang; for shape < 1 uses the boost gamma(a + 1) * U^(1 / a). */
71      static double gamma(double shape, Random random) {
72          if (shape < 1) {
73              double u = random.nextDouble();
74              return gamma(shape + 1, random) * Math.pow(u, 1.0 / shape);
75          }
76          double d = shape - 1.0 / 3.0;
77          double c = 1.0 / Math.sqrt(9 * d);
78          while (true) {
79              double x = random.nextGaussian();
80              double v = 1 + c * x;
81              if (v <= 0) continue;
82              v = v * v * v;
83              double u = random.nextDouble();
84              if (u < 1 - 0.0331 * x * x * x * x) return d * v;
85              if (Math.log(u) < 0.5 * x * x + d * (1 - v + Math.log(v))) return d * v;
86          }
87      }
88  }
```

gamma — Gamma 분포에서 한 번 뽑기 (Marsaglia–Tsang 방법). 널리 쓰이는 빠른 계산법을 그대로 옮긴 부분이라 수식 자체를 이해할 필요는 없다.

- **72~75줄** 모양 값이 1보다 작으면 1을 더해 뽑은 뒤 보정한다.
- **76~77줄** 계산에 쓸 상수 d, c를 준비.
- **78~86줄** 정규분포 난수(`nextGaussian`)로 후보를 만들고, 두 조건 중 하나를 통과하면 그 값을 돌려준다. 통과 못 하면 다시 시도(`while (true)` 반복). 거의 항상 몇 번 안에 끝난다.

GuardPolicy — 정책표 읽기 `android/app/src/main/java/app/focuson/shorts/GuardPolicy.java (354줄)`

웹이 만들어 보낸 JSON 정책표를 Java가 쓰기 좋은 모양으로 바꾼다. 서비스는 개입할 때 이 표만 보고 결정한다. 값이 빠지거나 깨져도 **기본값으로 동작**하도록 만들었다.

```
1   package app.focuson.shorts;
2
3   import java.util.ArrayList;
4   import java.util.Arrays;
5   import java.util.Calendar;
6   import java.util.Collections;
7   import java.util.HashMap;
8   import java.util.HashSet;
9   import java.util.Iterator;
10  import java.util.List;
11  import java.util.Map;
12  import java.util.Set;
13  import java.util.TimeZone;
14  import org.json.JSONArray;
15  import org.json.JSONObject;
16
17  /**
18   * Policy table the service reads at the moment of an intervention. The web side builds it
19   * (rules now, the AI phase later) and hands it over through GuardPlugin.sync, so the
20   * service never computes personalisation itself. Missing fields fall back to defaults.
21   */
22  final class GuardPolicy {
23      static final int BAND_LIGHT = 1;
```

17~21줄 설명: 개인화 계산은 웹이 하고, 서비스는 받은 표만 읽는다.

23~25줄 단계 번호: 약함 1, 중간 2, 강함 3.

```
    static final int BAND_MEDIUM = 2;
    static final int BAND_STRONG = 3;

```

```
    final String version;
    final int goalMinutes;
    final double mediumAt;
    final double strongAt;
    final List<String> lightMethods;
    final List<String> mediumMethods;
    final int[] commitOptions;
    final int[] extensionLadder;
    final int overdraftMinutes;
    final int overdraftWaitSec;
    final int dayEndHour;
    /** Kinds of the time line (S1): guilt, weekly, motivate, future, yesterday, average, goal,
sleep. */
    final List<String> framings;
    /** Stats the web side computed for the wording. Negative = unknown. */
    Stats stats = Stats.none();
    /** AI phase settings and learned tables (lib/ai.ts, lib/policy.ts). */
    Ai ai = new Ai();
    /** 공부 모드: a due study card replaces the light/medium activity (normal methods otherwis
e). */
    boolean studyMode;
```

정책표의 칸들. `final`은 한 번 정하면 안 바뀌는 값.

- `version`: 표를 만든 방식 ("rule-2", "ai-1", "ai-2").
- `goalMinutes`: 하루 목표. `mediumAt / strongAt`: 목표의 몇 배부터 중간/강함 (0.5, 1.0).
- `lightMethods / mediumMethods`: 단계별 개입 방식 목록.
- `commitOptions`: 약속 선택지(3, 5, 10). `extensionLadder`: 연장할수록 줄어드는 최대 (10, 5, 3).
- `overdraftMinutes / overdraftWaitSec`: "3분만 더"의 분, 대기 초.
- `dayEndHour`: "오늘은 끝"이 풀리는 시.
- `framings`: 시간 체감 문구 종류 목록.
- `stats`: 문구에 쓸 통계. `ai`: AI 설정과 학습 표. `studyMode`: 공부 모드.

```
    /**
     * What the AI decides inside the rules: which method / line to show (Thompson
     * sampling over learned posteriors), tighter numbers in risky hours, and when to ask
     * for feedback. The band itself stays rule based.
     */
    static final class Ai {
        boolean enabled;
        boolean strict;
        /** "band:slot" → arm → {alpha, beta} */
        Map<String, Map<String, double[]>> methods = new HashMap<>();
        /** "slot" → framing kind → {alpha, beta} */
        Map<String, Map<String, double[]>> framings = new HashMap<>();
        Set<Integer> riskHours = new HashSet<>();
        int firstRiskHour = -1;
        String riskLabel = "";
        int riskCommitMax = 3;
        int riskExtensionMax = 1;
        int riskOverdraftWaitSec = 10;
        int feedbackEvery = 5;
        /** Habituation (ai-2): 0 = off. See GuardBandit.freshness. */
        double habituationStrength = 0;
        double habituationRecoveryHours = 6;

```

Ai — AI 관련 값 상자.

- `enabled`: AI 켜짐. `strict`: 엄격 모드.
- `methods`: 칸("강도:시간대:평일주말") → 방식 → [α, β]. `framings`: 문구용 같은 표.
- `riskHours`: 위험 시간 목록. `firstRiskHour`: 그중 가장 이른 시. `riskLabel`: "밤 11시" 같은 이름.
- `riskCommitMax / riskExtensionMax / riskOverdraftWaitSec`: 위험 시간에 조이는 값 (3분, 1번, 10초).
- `feedbackEvery`: "도움이 됐나요?"를 몇 번에 한 번 묻나 (5).
- `habituationStrength / RecoveryHours`: 질림 반영 세기와 회복 시간 (0이면 끔).

```
    boolean isRiskHour(int hour) {
        return enabled && riskHours.contains(hour);
    }

    /**
     * Method posteriors for this band right now: "band:slot:weekday|weekend" (ai-2),
     * falling back to "band:slot" (ai-1).
     */
    Map<String, double[]> methodsAt(int band, Calendar now) {
        String slot = timeSlot(now.get(Calendar.HOUR_OF_DAY));
        Map<String, double[]> row = methods.get(band + ":" + slot + ":" + dayType(now));
        return row != null ? row : methods.get(band + ":" + slot);
    }

    /** Framing posteriors right now: "slot:weekday|weekend" (ai-2), falling back to "slot".
*/
    Map<String, double[]> framingsAt(Calendar now) {
        String slot = timeSlot(now.get(Calendar.HOUR_OF_DAY));
        Map<String, double[]> row = framings.get(slot + ":" + dayType(now));
        return row != null ? row : framings.get(slot);
    }

    /** Before today's first risky hour: suggest keeping budget for later. */
    boolean beforeRisk(int hour) {
        return enabled && firstRiskHour >= 0 && hour < firstRiskHour && hour >= 5;
    }
}
```

- **isRiskHour**: AI가 켜져 있고 지금 시가 위험 시간 목록에 있으면 true.
- **methodsAt**: 지금 시각의 칸 이름을 만들어 표를 찾는다. 먼저 새 방식 "2:night:weekday"로 찾고, 없으면 예전 방식 "2:night"로 찾는다 (v0.9.0 정책표와도 호환).
- **framingsAt**: 문구 표를 같은 방식으로 찾는다.
- **beforeRisk**: 오늘 첫 위험 시간보다 이른 시각(새벽 5시 이후)이면 true. "지금은 3분만 쓰고 남겨 둘까요?" 추천에 쓴다.

```
/** 아침 5~11, 낮 11~17, 저녁 17~22, 밤 22~5 (same as lib/ai.ts timeSlot). */
static String timeSlot(int hour) {
    if (hour >= 5 && hour < 11) return "morning";
    if (hour >= 11 && hour < 17) return "day";
    if (hour >= 17 && hour < 22) return "evening";
    return "night";
}

/** 토·일은 weekend (same as lib/ai.ts dayType). */
static String dayType(Calendar now) {
    int day = now.get(Calendar.DAY_OF_WEEK);
    return day == Calendar.SATURDAY || day == Calendar.SUNDAY ? "weekend" : "weekday";
}
```

timeSlot: 시(0~23)를 시간대로: 5~11시 아침, 11~17시 낮, 17~22시 저녁, 나머지 밤. 웹 `lib/ai.ts`와 같은 규칙이어야 칸 이름이 맞는다.

dayType: 토요일, 일요일이면 "weekend", 아니면 "weekday".

```
/** Numbers behind the lines ("어제 이 시간엔…", "이번 주 평균보다…"). */
static final class Stats {
    /** Average of the last 7 days before today (seconds). */
    double avg7Seconds = -1;
    /** This week (Monday-based) before today, and how many days that covers. */
    double pastWeekSeconds = -1;
    int daysBeforeToday = 0;
    /** Yesterday's running total at the end of each hour 0..23, or null. */
    double[] yesterdayCumulative;
```

Stats — 문구용 통계 상자. 음수(-1)는 "모름" 뜻.

- `avg7Seconds`: 지난 7일 평균. `pastWeekSeconds / daysBeforeToday`: 이번 주(월요일부터) 어제까지 합과 날 수.
- `yesterdayCumulative`: 어제 0시~23시 각 시각까지 쌓인 시청 초 24칸.
- `wakeHour / wakeMinute`: 일어나는 시각 (수면 문구용).

```
        int wakeHour = 7;
        int wakeMinute = 0;

        static Stats none() {
            return new Stats();
        }

        /** Yesterday's total by this time of day, or -1 when there is no record. */
        double yesterdayByNow(int hour, int minute) {
            if (yesterdayCumulative == null || yesterdayCumulative.length < 24) return -1;
            double before = hour == 0 ? 0 : yesterdayCumulative[hour - 1];
            double end = yesterdayCumulative[hour];
            return before + (end - before) * (minute / 60.0);
        }

        /** Projected total for this week at today's pace, or -1. */
        double weekProjection(double todaySeconds) {
            if (pastWeekSeconds < 0) return -1;
            int days = daysBeforeToday + 1;
            return (pastWeekSeconds + todaySeconds) / days * 7;
        }
    }
```

- **yesterdayByNow**: "어제 이 시간까지 몇 초 봤나". 그 시의 앞뒤 누적값 사이를 분 비율만큼 나눠 계산한다 (선형 보간).
- **weekProjection**: "이 속도면 이번 주 총 몇 초" = (지난날 합 + 오늘) ÷ 날 수 × 7.

```
    GuardPolicy(
        String version,
        int goalMinutes,
        double mediumAt,
        double strongAt,
        List<String> lightMethods,
        List<String> mediumMethods,
        int[] commitOptions,
        int[] extensionLadder,
        int overdraftMinutes,
        int overdraftWaitSec,
        int dayEndHour,
        List<String> framings
    ) {
        this.version = version;
        this.goalMinutes = Math.max(1, goalMinutes);
        this.mediumAt = mediumAt;
        this.strongAt = strongAt;
        this.lightMethods = lightMethods;
        this.mediumMethods = mediumMethods;
        this.commitOptions = commitOptions;
        this.extensionLadder = extensionLadder;
        this.overdraftMinutes = Math.max(1, overdraftMinutes);
        this.overdraftWaitSec = Math.max(0, overdraftWaitSec);
        this.dayEndHour = Math.max(0, Math.min(23, dayEndHour));
        this.framings = framings;
    }
```

생성자: 받은 값을 칸에 넣는다. 안전장치: 목표는 최소 1분, "3분만 더"는 최소 1분, 대기는 0초 이상, 하루 끝 시는 0~23 사이로 맞춘다 (`Math.max` = 둘 중 큰 값, `Math.min` = 작은 값).

```
    static GuardPolicy defaults(int goalMinutes) {
        return new GuardPolicy(
            "rule-default",
            goalMinutes,
            0.5,
            1.0,
            Arrays.asList("pause", "framing"),
            Arrays.asList("breath", "alternative", "reflection", "card", "pause", "framing"),
            new int[] {3, 5, 10},
            new int[] {10, 5, 3},
            3,
            3,
            6,
            Arrays.asList("guilt", "yesterday", "motivate", "weekly", "future", "average", "goal", "sleep"));
    }
```

defaults: 정책표가 아직 없을 때 쓰는 기본 표. 약함 단계는 숨 고르기와 시간 문구, 중간은 6가지 전부, 약속 3/5/10분, 연장 10→5→3, 3분만 더 3초 대기, 하루 끝 6시, 시간 문구 8종류.

```
    /** Reads the table the web side stored. Anything unreadable keeps its default. */
    static GuardPolicy fromJson(String raw, int fallbackGoal) {
        GuardPolicy base = defaults(fallbackGoal);
        if (raw == null || raw.isEmpty()) return base;
        try {
            JSONObject json = new JSONObject(raw);
            JSONObject methods = json.optJSONObject("methods");
            GuardPolicy policy = new GuardPolicy(
                json.optString("version", base.version),
                json.optInt("goalMinutes", base.goalMinutes),
                json.optDouble("mediumAt", base.mediumAt),
                json.optDouble("strongAt", base.strongAt),
                strings(methods == null ? null : methods.optJSONArray("light"), base.lightMethods),
                strings(methods == null ? null : methods.optJSONArray("medium"), base.mediumMethods),
                ints(json.optJSONArray("commitOptions"), base.commitOptions),
                ints(json.optJSONArray("extensionLadder"), base.extensionLadder),
                json.optInt("overdraftMinutes", base.overdraftMinutes),
                json.optInt("overdraftWaitSec", base.overdraftWaitSec),
                json.optInt("dayEndHour", base.dayEndHour),
                strings(json.optJSONArray("framings"), base.framings));
            policy.stats = stats(json.optJSONObject("stats"));
            policy.ai = ai(json.optJSONObject("ai"));
            policy.studyMode = json.optBoolean("studyMode", false);
            return policy;
        } catch (Exception e) {
            return base;
        }
    }
```

fromJson — 웹이 보낸 글자를 정책표로.

1. **189~190줄** 기본 표를 먼저 만들고, 받은 게 없으면 그대로 돌려준다.
2. **192줄** 글자를 JSON 객체로 읽는다.
3. **194~206줄** 칸마다 `optInt("이름", 기본값)`처럼 읽는다. `opt`는 "없으면 기본값"이라는 뜻이라 빠진 칸이 있어도 멈추지 않는다.
4. **207~209줄** 통계, AI, 공부 모드를 따로 읽는다.
5. **211~213줄** 어떤 오류가 나도 기본 표로 동작한다.

```
    private static Ai ai(JSONObject json) {
        Ai ai = new Ai();
        if (json == null) return ai;
```

ai — AI 부분 읽기.

- **219~222줄** 켜짐, 엄격, 방식 표, 문구 표.

```
        ai.enabled = json.optBoolean("enabled", false);
        ai.strict = json.optBoolean("strict", false);
        ai.methods = table(json.optJSONObject("methods"));
        ai.framings = table(json.optJSONObject("framings"));
        JSONArray hours = json.optJSONArray("riskHours");
        if (hours != null) {
            for (int i = 0; i < hours.length(); i++) {
                int hour = hours.optInt(i, -1);
                if (hour < 0 || hour > 23) continue;
                ai.riskHours.add(hour);
                if (ai.firstRiskHour < 0 || hour < ai.firstRiskHour) ai.firstRiskHour = hour;
            }
        }
        ai.riskLabel = json.optString("riskLabel", "");
        JSONObject risk = json.optJSONObject("risk");
        if (risk != null) {
            ai.riskCommitMax = Math.max(1, risk.optInt("commitMax", ai.riskCommitMax));
            ai.riskExtensionMax = Math.max(0, risk.optInt("extensionMax", ai.riskExtensionMax));
            ai.riskOverdraftWaitSec = Math.max(0, risk.optInt("overdraftWaitSec", ai.riskOverdra
ftWaitSec));
        }
        ai.feedbackEvery = Math.max(1, json.optInt("feedbackEvery", ai.feedbackEvery));
        JSONObject habituation = json.optJSONObject("habituation");
        if (habituation != null) {
            ai.habituationStrength = Math.max(0, Math.min(1, habituation.optDouble("strength",
0)));
            ai.habituationRecoveryHours = Math.max(0.1, habituation.optDouble("recoveryHours",
6));
        }
        return ai;
    }
```

- **223~231줄** 위험 시간 목록: 0~23 밖의 값은 버리고, 가장 이른 시를 따로 기억한다.
- **233~238줄** 위험 시간 조이기 값 (최소값 보장).
- **239줄** 피드백 간격 (최소 1).
- **240~244줄** 질림 반영: 세기는 0~1 사이, 회복 시간은 최소 0.1시간.

```
    private static Map<String, Map<String, double[]>> table(JSONObject json) {
        Map<String, Map<String, double[]>> out = new HashMap<>();
        if (json == null) return out;
        Iterator<String> contexts = json.keys();
        while (contexts.hasNext()) {
            String context = contexts.next();
            JSONObject arms = json.optJSONObject(context);
            if (arms == null) continue;
            Map<String, double[]> row = new HashMap<>();
            Iterator<String> names = arms.keys();
            while (names.hasNext()) {
                String arm = names.next();
                JSONArray ab = arms.optJSONArray(arm);
                if (ab == null || ab.length() < 2) continue;
                row.put(arm, new double[] {ab.optDouble(0, 1), ab.optDouble(1, 1)});
            }
            out.put(context, row);
        }
```

table — 학습 표 읽기. JSON `{"2:night:weekday": {"breath": [4.2, 1.8], ...}, ...}`를 Java 표로 바꾼다. 칸마다(`contexts`), 방식마다(`names`) 반복하며 값이 두 개 이상인 것만 [α, β]로 넣는다. `Iterator`는 "하나씩 꺼내 주는 도구".

```
        return out;
    }
```

```
    private static Stats stats(JSONObject json) {
        Stats stats = new Stats();
        if (json == null) return stats;
        stats.avg7Seconds = json.optDouble("avg7Seconds", -1);
        stats.pastWeekSeconds = json.optDouble("pastWeekSeconds", -1);
        stats.daysBeforeToday = json.optInt("daysBeforeToday", 0);
        stats.wakeHour = json.optInt("wakeHour", 7);
        stats.wakeMinute = json.optInt("wakeMinute", 0);
        JSONArray cumulative = json.optJSONArray("yesterdayCumulative");
        if (cumulative != null && cumulative.length() == 24) {
            stats.yesterdayCumulative = new double[24];
            for (int i = 0; i < 24; i++) stats.yesterdayCumulative[i] = cumulative.optDouble(i,
0);
        }
        return stats;
    }
```

stats — 통계 읽기. 각 값을 읽고, 어제 누적 목록은 정확히 24칸일 때만 받아들인다.

```
    double ratio(double seconds) {
        return seconds / (goalMinutes * 60.0);
    }

    int band(double seconds) {
        double r = ratio(seconds);
        if (r >= strongAt) return BAND_STRONG;
        if (r >= mediumAt) return BAND_MEDIUM;
        return BAND_LIGHT;
    }

    double remainingSeconds(double seconds) {
        return Math.max(0, goalMinutes * 60.0 - seconds);
    }

    List<String> methodsFor(int band) {
        return band == BAND_LIGHT ? lightMethods : mediumMethods;
    }
```

- **ratio**: 오늘 사용 초 ÷ 목표 초 = 목표 대비 비율.
- **band**: 비율이 1.0 이상이면 강함, 0.5 이상이면 중간, 아니면 약함.
- **remainingSeconds**: 목표까지 남은 초 (음수면 0).
- **methodsFor**: 약함이면 약함 목록, 아니면 중간 목록.

```
    /**
     * Commitment choices for this round: within the extension ladder step and the
     * remaining budget. When less than the smallest option is left, offers the
     * remaining whole minutes (at least 1).
     */
    List<Integer> commitChoices(double remainingSeconds, int extensionIndex) {
        int cap = extensionIndex <= 0
            ? Integer.MAX_VALUE
            : extensionLadder[Math.min(extensionIndex, extensionLadder.length) - 1];
        int remainingMinutes = (int) Math.floor(remainingSeconds / 60.0);
        List<Integer> out = new ArrayList<>();
        for (int option : commitOptions) {
```

commitChoices — "몇 분 볼래요?" 선택지 만들기.

1. **310~312줄** 이번이 몇 번째 연장인지로 최대값을 정한다. 처음(0)은 제한 없음, 1번째 연장 10분, 2번째 5분, 3번째부터 3분.
2. **313줄** 남은 예산을 분으로 (내림).
3. **315~317줄** 3/5/10 중 최대값 이하이면서 남은 예산 이하인 것만 넣는다.
4. **318~321줄** 하나도 없는데 30초 이상 남았으면, 남은 분(올림, 최소 1분)을 하나 넣는다.
5. **322줄** 작은 순서로 정렬.

```
            if (option <= cap && option <= remainingMinutes) out.add(option);
        }
        if (out.isEmpty() && remainingSeconds >= 30) {
            int fallback = Math.max(1, Math.min(cap, (int) Math.ceil(remainingSeconds / 60.0)));
            out.add(fallback);
        }
        Collections.sort(out);
        return out;
    }
```

```
    /** Next local {@link #dayEndHour}:00 strictly after {@code now} ("오늘은 끝" until then). */
    long nextDayEnd(long now, TimeZone zone) {
        Calendar cal = Calendar.getInstance(zone);
        cal.setTimeInMillis(now);
        cal.set(Calendar.HOUR_OF_DAY, dayEndHour);
        cal.set(Calendar.MINUTE, 0);
        cal.set(Calendar.SECOND, 0);
        cal.set(Calendar.MILLISECOND, 0);
        if (cal.getTimeInMillis() <= now) cal.add(Calendar.DAY_OF_MONTH, 1);
        return cal.getTimeInMillis();
    }
```

nextDayEnd: "오늘은 끝"이 풀리는 시각. 오늘 그 시(기본 6시) 정각을 만들고, 이미 지났으면 하루를 더한다. 예: 밤 11시에 누르면 다음 날 6시, 새벽 3시에 누르면 같은 날 6시.

```
    private static List<String> strings(JSONArray array, List<String> fallback) {
        if (array == null || array.length() == 0) return fallback;
        List<String> out = new ArrayList<>();
        for (int i = 0; i < array.length(); i++) {
            String value = array.optString(i, "");
            if (!value.isEmpty()) out.add(value);
        }
        return out.isEmpty() ? fallback : out;
    }

    private static int[] ints(JSONArray array, int[] fallback) {
        if (array == null || array.length() == 0) return fallback;
        int[] out = new int[array.length()];
        for (int i = 0; i < array.length(); i++) out[i] = Math.max(1, array.optInt(i, 1));
        return out;
    }
}
```

strings / ints: JSON 목록을 Java 목록으로 바꾸는 도우미. 비었으면 기본값을 쓰고, 빈 글자는 빼고, 숫자는 최소 1로 맞춘다.

GuardCopy — 개입 화면 문구 모음 `android/app/src/main/java/app/focuson/shorts/GuardCopy.java (271줄)`

개입 화면에 나오는 **모든 문장**이 여기 있다. 팀이 상황별 표(S1~S9, HANDOFF.md)로 확인한 문구만 넣는다. 숫자가 필요한 문장은 숫자를 모르면 `null`을 돌려줘서 그 문장을 건너뛰게 한다. **문구를 바꾸려면 이 파일을 고친다.**

```
package app.focuson.shorts;

import java.util.Locale;
```

5~11줄 주의 주석: 문구는 상황별 표로 확인받은 뒤에만 바꾼다. 문장은 정해진 순서로 돌아가며 나오고, 숫자가 의미 없으면 건너뛴다. 이유와 성찰 답은 웹

```
 4
 5 /**
 6  * Sheet wording. Every line here was approved by the user per situation (S1–S9 in
 7  * docs/HANDOFF.md); change wording only after showing the user the new table.
 8  * Lines rotate in a fixed order (GuardState.nextInTurn); a line whose numbers are
 9  * unknown or meaningless right now returns null and is skipped.
10  * REASONS and the reflection answers must match lib/logic.ts so records line up.
11  */
12 final class GuardCopy {
13     private GuardCopy() {}
14
15     static final String[] REASONS = {
16         "심심해서",
17         "습관적으로",
18         "잠깐 보려고",
19         "스트레스 해소를 위해",
20         "특별한 이유 없이",
21         "기타"
22     };
```

(`lib/logic.ts`)과 글자가 똑같아야 기록이 맞는다.

15~22줄 `REASONS`: 쇼츠를 연 이유 6가지 (개입 화면의 "왜 열었어요?" 선택지).

```
24     static final class Alternative {
25         final String label;
26         final String guide;
27
28         Alternative(String label, String guide) {
29             this.label = label;
30             this.guide = guide;
31         }
32     }
33
34     /** All alternatives, in the order the medium band rotates through them. */
35     static final Alternative[] ALTERNATIVES = {
36         new Alternative("물 마시기", "자리에서 일어나 물 한 잔을 마시고 오세요."),
37         new Alternative("간단한 스트레칭", "어깨를 천천히 돌리고, 목을 좌우로 기울여 보세요."),
38         new Alternative("잠시 걷기", "방 안이라도 열 걸음만 걸어 보세요."),
39         new Alternative("눈 쉬기", "화면에서 눈을 떼고, 멀리 있는 한 점을 바라보세요."),
40     };
```

Alternative: 대체 행동 하나 = 짧은 이름(`label`) + 안내 문장(`guide`).

35~40줄 `ALTERNATIVES`: 물 마시기, 스트레칭, 잠시 걷기, 눈 쉬기. 중간 단계에서 이 순서로 돌아가며 나온다.

```
42     static final String REFLECTION_QUESTION = "지금 보면 뭐가 남을까요?";
43     static final String[] REFLECTION_ANSWERS = {"기분 전환", "아무것도", "할 일 미루는 중", "잘 모
르겠어요"};
44
45     static final String[] AFTER_LOCK_ANSWERS = {"쉬었어요", "공부·일", "그냥 기다림"};
46
47     static final String STRONG_TITLE = "오늘은 여기까지 해볼까요?";
48     static final String LOCKED_TITLE = "쇼츠 쉬는 중";
49     static final String PIP_NOTE = "작은 창으로 보던 쇼츠예요";
50     static final String PIP_CLOSE_GUIDE = "작은 창의 X를 눌러 닫아 주세요. 닫으면 소리가 돌아와요.";
```

- **42~43줄** 성찰 질문과 객관식 답 4개 (주관식 입력 부담을 없애려고 객관식만).
- **45줄** 잠금이 끝난 뒤 "그동안 뭐 했어요?"의 답 3개.
- **47~50줄** 강함 단계 제목, 잠금 화면 제목, 작은 창(PIP) 안내 문구 2개.

```
    // --- Study cards (approved 2026-10-13) ---
    static final String CARD_REPORT = "이 문제 이상해요";
    static final String CARD_REPORTED = "이 문제는 빼 둘게요. 다시 나오지 않아요.";
```

공부 카드 문구 (2026-10-13 확인): 문제 아래 "이 문제 이상해요" 버튼과 누른 뒤 안내.

```
    // --- AI phase lines (approved 2026-10-06, docs/HANDOFF.md) ---

    /** Sheet notice in a risky hour (A7). */
    static String aiRiskNotice(String riskLabel) {
        return "AI · " + riskLabel + "쯤은 평소 오래 보던 시간이라 조금 짧게 잡았어요";
    }

    /** Commitment step before today's risky hour (A5). */
    static String reserveHint(String riskLabel) {
        return "보통 " + riskLabel + "쯤 많이 보셔서, 지금은 3분만 쓰고 남겨 둘까요?";
    }

    static final String FEEDBACK_QUESTION = "방금 화면, 멈추는 데 도움이 됐나요?";
    static final String FEEDBACK_YES = "도움 됐어요";
    static final String FEEDBACK_NO = "아니에요";
```

AI 문구 (2026-10-06 확인).

- **aiRiskNotice**: 위험 시간에 개입 화면 맨 위에 뜨는 한 줄. 예: "AI · 밤 11시쯤은 평소 오래 보던 시간이라 조금 짧게 잡았어요". `+`는 글자 이어 붙이기.
- **reserveHint**: 위험 시간 전에 "몇 분 볼래요?" 아래 나오는 추천 문장.
- **68~70줄** "도움이 됐나요?" 질문과 두 답.

```
    /** Numbers a line may use. Negative or null = unknown, and lines that need it are skipped.
*/
    static final class Facts {
        double usage;
        double goal;
        double weekProjection = -1;
        double avg7 = -1;
        double yesterdayByNow = -1;
        /** Sleep left if the user went to bed now, or -1 outside night hours / unknown. */
        double sleepSeconds = -1;
        /** e.g. "7:10 알람 기준" or "일어나는 시각 7:00 기준" */
        String sleepBasis = "";
        int keptToday;
        double restToday;

        double remaining() {
            return Math.max(0, goal - usage);
        }
    }
```

Facts — 문장에 넣을 숫자 상자. 음수(-1)는 "모름".

- `usage` 오늘 본 초, `goal` 목표 초, `weekProjection` 이번 주 예상, `avg7` 7일 평균, `yesterdayByNow` 어제 이 시간까지.
- `sleepSeconds`: 지금 자면 잘 수 있는 초 (밤이 아니면 -1). `sleepBasis`: "7:10 알람 기준" 같은 근거 글.
- `keptToday` 오늘 약속 지킨 횟수, `restToday` 오늘 쉰 초.
- **remaining()**: 목표까지 남은 초 (음수면 0).

```
    // --- S1. Time line (entry sheet) ----------------------------------------

    static String insight(String kind, Facts f) {
        if (f.usage < 60) return null;
        switch (kind) {
            case "guilt":
                return "오늘 쇼츠에 쓴 " + minutes(f.usage) + ", 다시 돌아오지 않아요.";
            case "weekly":
                if (f.weekProjection < 600) return null;
                return "이 속도면 이번 주에 " + minutes(f.weekProjection) + "을 쇼츠에 써요.";
            case "motivate":
```

insight — 시간 체감 문구 (상황 S1). 종류(kind)마다 다른 문장을 만든다. `switch`는 "값에 따라 갈라지기".

- **92줄** 오늘 1분 미만이면 아무 문장도 안 냄.
- **guilt** "오늘 쇼츠에 쓴 N분, 다시 돌아오지 않아요."
- **weekly** 이번 주 예상이 10분 이상일 때 "이 속도면 이번 주에 N분을..."
- **motivate** 남은 시간이 1분 이상일 때 "지금 멈추면 오늘 목표 안에서..."
- **future** "자기 전의 나는 오늘 이 N분을 어떻게 생각할까요?"
- **yesterday** 어제 기록이 있고 오늘이 1분 이상 더 많을 때 비교.

```
                if (f.remaining() < 60) return null;
                return "지금 멈추면 오늘 목표 안에서 끝낼 수 있어요. (" + minutes(f.remaining()) +
" 남음)";
            case "future":
                return "자기 전의 나는 오늘 이 " + minutes(f.usage) + "을 어떻게 생각할까요?";
            case "yesterday":
                if (f.yesterdayByNow < 0 || f.usage - f.yesterdayByNow < 60) return null;
                return "어제 이 시간엔 " + minutes(f.yesterdayByNow) + "이었는데, 오늘은 벌써 "
                    + minutes(f.usage) + "이에요.";
            case "average":
                if (f.avg7 <= 0 || f.usage - f.avg7 < 60) return null;
                return "이번 주 평균보다 " + minutes(f.usage - f.avg7) + " 더 보고 있어요.";
            case "goal":
                if (f.usage >= f.goal) return null;
                return "정한 " + minutes(f.goal) + " 중 " + minutes(f.usage) + "을 썼어요.";
            case "sleep":
                return sleepLine(f);
            default:
                return null;
        }
    }
```

- **average** 7일 평균보다 1분 이상 더 볼 때.
- **goal** 아직 목표 안일 때 "정한 N분 중 M분을 썼어요."
- **sleep** 아래 sleepLine.

```
    private static String sleepLine(Facts f) {
        if (f.sleepSeconds < 600) return null;
        return "지금 자면 " + minutes(f.sleepSeconds) + " 잘 수 있어요. 10분 더 보면 "
            + minutes(f.sleepSeconds - 600) + "이에요. (" + f.sleepBasis + ")";
    }
```

sleepLine: 밤에 "지금 자면 N 잘 수 있어요. 10분 더 보면 M이에요. (근거)". 잘 수 있는 시간이 10분 미만이면 안 냄.

```
    // --- S2. Commitment over ------------------------------------------------

    static String commitmentOver(int index, Facts f) {
        switch (Math.floorMod(index, 3)) {
            case 0:
                return "약속을 지키면 오늘 스스로 멈춘 횟수가 하나 늘어요.";
            case 1:
                if (f.remaining() >= 60) {
                    return "오늘 목표까지 " + minutes(f.remaining()) + " 남았어요. 여기서 멈추면 여
유가 생겨요.";
                }
                return "약속을 지키면 오늘 스스로 멈춘 횟수가 하나 늘어요.";
            default:
                return "방금 정한 건 몇 분 전의 나예요. 그 약속, 지켜 볼까요?";
        }
    }
```

commitmentOver — 약속 시간이 끝났을 때 문장 (S2). 3개를 번갈아 쓴다 (`floorMod(index, 3)` = 3으로 나눈 나머지, 음수도 안전).

1. "약속을 지키면 오늘 스스로 멈춘 횟수가 하나 늘어요."
2. 남은 시간이 1분 이상이면 "오늘 목표까지 N 남았어요...", 아니면 1번 문장.
3. "방금 정한 건 몇 분 전의 나예요. 그 약속, 지켜 볼까요?"

```
    // --- S3. Strong band ----------------------------------------------------

    static String strong(int index, Facts f) {
        if (Math.floorMod(index, 3) == 2 && f.sleepSeconds >= 600) {
            return "지금 자면 " + minutes(f.sleepSeconds) + " 잘 수 있어요. 오늘은 여기서 마무리해
볼까요? ("
```

strong — 목표 초과 화면 본문 (S3). 세 번째 차례이고 밤이면 수면 문장, 두 번째면 "한 개만 더 보려다 30분이...", 그 밖에는 "오늘 정한 N분을 넘어 M분을 봤어요...".

```
                + f.sleepBasis + ")";
        }
        if (Math.floorMod(index, 3) == 1) {
            return "한 개만 더 보려다 30분이 지나기 쉬워요. 오늘은 이미 " + minutes(f.usage) + "이에
요.";
        }
        return "오늘 정한 " + minutes(f.goal) + "을 넘어 " + minutes(f.usage)
            + "을 봤어요. 지금 멈추거나, 잠시 쇼츠를 잠가 둘 수 있어요.";
    }
```

```
    // --- S4. "3분만 더" nudge ------------------------------------------------

    static String overdraftTitle(int count, int minutes) {
        return "오늘 " + count + "번째 " + minutes + "분이에요";
    }

    static String overdraft(int count, int extraMinutes, Facts f) {
        double overAfter = f.usage + extraMinutes * 60.0 - f.goal;
        switch (count) {
            case 1:
                return "오늘 목표를 넘어서 보는 첫 " + extraMinutes + "분이에요. 정말 필요할까요?";
            case 2:
                return "벌써 두 번째 " + extraMinutes + "분이에요. 이번 " + extraMinutes + "분까지
목표보다 "
                    + minutes(overAfter) + " 더 보게 돼요.";
            case 3:
                return "세 번째예요. 자기 전의 나는 이 시간을 어떻게 생각할까요?";
            default:
                String over = "오늘 목표보다 " + minutes(f.usage - f.goal) + " 넘게 봤어요.";
                if (f.yesterdayByNow >= 0) {
                    return over + " 어제는 이 시간까지 " + minutes(f.yesterdayByNow) + "이었어
요.";
                }
                return over + " " + count + "번째 " + extraMinutes + "분이에요.";
        }
    }
```

"3분만 더" 대기 화면 (S4).

- **overdraftTitle**: "오늘 N번째 3분이에요".
- **overdraft**: 몇 번째인지에 따라 다른 문장. 1번째 "정말 필요할까요?", 2번째 "이번까지 목표보다 N 더", 3번째 "자기 전의 나는...", 4번째부터 목표보다 넘은 시간과 어제 비교.

```
    // --- S5. Praise ----------------------------------------------------------

    static String praise(String outcome, int index, Facts f, int blockMinutes) {
        switch (outcome) {
            case "kept":
                return Math.floorMod(index, 2) == 0
                    ? "약속을 지켰어요. 이게 쌓이면 습관이 돼요."
                    : "정한 만큼만 봤어요. 오늘 " + Math.max(1, f.keptToday) + "번째예요.";
            case "day_end":
                return "오늘을 스스로 마무리했어요. 내일 아침에 고마워할 거예요.";
            case "lock":
                String length = blockText(blockMinutes);
                switch (Math.floorMod(index, 4)) {
                    case 0:
```

praise — 칭찬 문장 (S5). 결과(outcome)마다 다르다.

- **kept**(약속 지킴): 2개 번갈아. 두 번째는 "오늘 N번째예요".
- **day_end**(오늘은 끝): 한 문장.
- **lock**(잠금): 4개 번갈아. 잠금 길이(`blockText`, 예: "30분", "1시간")를 넣는다.
- **그 밖**(바로 나감): 2개 번갈아.

```
                        return "스스로 쉬는 시간을 정했어요. 그것만으로도 반은 성공이에요.";
                    case 1:
                        return "좋은 선택이에요. " + length + " 뒤에 다시 만나요.";
                    case 2:
                        return length + ", 폰 말고 다른 걸 해 볼 시간이에요.";
                    default:
                        return "멈추기로 한 건 지금의 나예요. " + length + " 동안 그 결정을 지켜 볼게요.";
                }
            default:
                return Math.floorMod(index, 2) == 0
                    ? "방금 나간 건 진짜 어려운 일이에요."
                    : "멈출 줄 아는 게 진짜 실력이에요.";
        }
    }

    // --- S6–S8 ------------------------------------------------------------------

    static String locked(long remainingMs, double restToday) {
        String line = clockFromMs(remainingMs) + " 남았어요.";
        if (restToday >= 60) line += " 지금까지 오늘 " + minutes(restToday) + " 쉬었어요.";
        return line;
    }

    static String afterLockQuestion(int blockMinutes) {
        return blockText(blockMinutes) + " 잘 쉬었어요. 그동안 뭐 했어요?";
    }

    static String resumeToast(int commitMinutes, double remainingSeconds) {
        return "약속한 " + commitMinutes + "분 중 " + minutes(Math.max(60, remainingSeconds)) + " 남았어요.";
    }

    // --- Formatting -------------------------------------------------------------

    /** Whole minutes for sentences: "23분", "2시간 40분". */
    static String minutes(double seconds) {
        long total = Math.max(1, Math.round(seconds / 60.0));
        long hours = total / 60;
        long rest = total % 60;
        if (hours > 0 && rest > 0) return hours + "시간 " + rest + "분";
        if (hours > 0) return hours + "시간";
        return total + "분";
    }

    static String blockText(int minutes) {
        return minutes >= 60 && minutes % 60 == 0 ? (minutes / 60) + "시간" : minutes + "분";
    }

    static String clock(double totalSeconds) {
        long safe = Math.max(0, (long) Math.floor(totalSeconds));
```

- **locked** (S6): 잠금 화면 본문 "N분 남았어요." + 오늘 1분 이상 쉬었으면 "지금까지 오늘 N 쉬었어요."
- **afterLockQuestion** (S7): "30분 잘 쉬었어요. 그동안 뭐 했어요?"
- **resumeToast** (S8): 약속 이어가기 안내 "약속한 5분 중 3분 남았어요."

minutes: 초를 문장용 "23분", "2시간 40분", "1시간"으로. 반올림하고 최소 1분.

blockText: 잠금 길이 표기. 60의 배수면 "1시간", 아니면 "30분".

clock: 짧은 시간 표기 "2분 30초", "45초", "1시간 5분". 내림으로 계산한다.

```
        long hours = safe / 3600;
        long minutes = (safe % 3600) / 60;
        long seconds = safe % 60;
        if (hours > 0) return hours + "시간 " + minutes + "분";
        if (minutes > 0 && seconds == 0) return minutes + "분";
        if (minutes > 0) return minutes + "분 " + seconds + "초";
        return seconds + "초";
    }

    private static String clockFromMs(long ms) {
        long total = Math.max(0, (ms + 59_999) / 60_000);
        if (total >= 60) return (total / 60) + "시간 " + (total % 60) + "분";
        return total + "분";
    }
```

clockFromMs: 밀리초를 분 단위로 올림해서 "5분", "1시간 20분".

```
    /** mm:ss or h:mm:ss for the lock countdown. */
    static String countdown(long remainingMs) {
        long total = Math.max(0, (remainingMs + 999) / 1000);
        long hours = total / 3600;
        long minutes = (total % 3600) / 60;
        long seconds = total % 60;
        if (hours > 0) return String.format(Locale.ROOT, "%d:%02d:%02d", hours, minutes, second
s);
        return String.format(Locale.ROOT, "%02d:%02d", minutes, seconds);
    }
}
```

countdown: 잠금 화면 큰 숫자 "04:59" 또는 "1:04:59" (시간이 있으면 시:분:초). `%02d`는 두 자리로 맞추기.

GuardOverlay — 개입 화면 그리기 `android/app/src/main/java/app/focuson/shorts/GuardOverlay.java (362줄)`

쇼츠 위에 아래에서 올라오는 어두운 창(바텀 시트)을 **코드로 직접** 그린다(XML 화면 파일 없이). 창 하나를 띄워 두고, GuardFlow가 단계마다 안의 내용만 바꾼다.
`TYPE_ACCESSIBILITY_OVERLAY` 창이라 "다른 앱 위에 그리기" 권한이 필요 없고, `FLAG_NOT_FOCUSABLE`이라 YouTube가 계속 "앞에 있는 앱"으로 남아 감지가 이어진다. 글자 입력칸은 일부러 없다.

```
package app.focuson.shorts;

import android.content.Context;
import android.graphics.Color;
import android.graphics.PixelFormat;
import android.graphics.Typeface;
import android.graphics.drawable.GradientDrawable;
import android.os.Handler;
import android.util.TypedValue;
import android.view.Gravity;
import android.view.View;
import android.view.ViewGroup;
import android.view.WindowManager;
import android.widget.FrameLayout;
```

3~17줄 화면 그리기 도구들: 색(`Color`), 창 관리자(`WindowManager`), 화면 부품(`View`), 겹쳐 놓는 틀(`FrameLayout`), 세로/가로로 쌓는 틀(`LinearLayout`), 글자칸(`TextView`), 둥근 배경(`GradientDrawable`).

```
import android.widget.LinearLayout;
import android.widget.TextView;
import java.util.List;

/**
 * Bottom sheet drawn over the Shorts player from the accessibility service. One sheet
 * stays up while {@link GuardFlow} swaps its steps (method, reason, commitment...).
 * TYPE_ACCESSIBILITY_OVERLAY + FLAG_NOT_FOCUSABLE keeps YouTube as the active window,
 * so detection keeps working underneath. No text input.
 */
```

```
final class GuardOverlay {
    static final long ANIM_MS = 250L;
    static final float SCRIM_ALPHA = 0.9f;

    static final int SHEET = Color.parseColor("#1C1C1E");
    static final int CARD = Color.parseColor("#2C2C2E");
    static final int CARD_ON = Color.parseColor("#3A3A3C");
    static final int TEXT = Color.parseColor("#F5F5F7");
    static final int MUTED = Color.parseColor("#A1A1A6");
    static final int GOOD = Color.parseColor("#2E7D4F");
    static final int BAD = Color.parseColor("#8E2C2C");
    private static final int TRACK = Color.parseColor("#3A3A3C");
    private static final int[] BAND_TONE = {
        Color.parseColor("#7BD88F"), Color.parseColor("#F5B754"), Color.parseColor("#FF6B6B")
    };

```

- **26줄** 열고 닫는 애니메이션 0.25초. **27줄** 뒤 배경을 90% 어둡게.
- **29~35줄** 색: 시트(아주 어두운 회색), 카드, 선택된 카드, 글자(거의 흰색), 흐린 글자, 정답(초록), 오답(빨강). `#1C1C1E` 같은 건 색 번호(16진수).
- **36~39줄** 게이지 바탕색과 단계별 색: 약함 초록, 중간 주황, 강함 빨강.

```
    interface Tick {
        /** @return true to keep ticking */
        boolean run();
    }

    private final Context context;
    private final Handler handler;
    private final WindowManager windows;

    private FrameLayout root;
    private View scrim;
    private LinearLayout sheet;
    private TextView noticeText;
    private TextView gaugeText;
    private View gaugeFill;
    private View gaugeRest;
    private TextView title;
    private TextView subtitle;
    private LinearLayout body;
    private TextView leaveButton;
    private boolean closing;
    private Runnable ticker;
```

Tick: 주기적으로 실행할 작업의 약속. true를 돌려주면 계속, false면 멈춤.

46~62줄 기억하는 것들: 창 관리자, 시트의 부품들(위 안내 줄, 게이지 글자와 막대, 제목, 부제목, 본문 칸, 아래 버튼), 닫는 중인지, 돌고 있는 타이머.

```
    GuardOverlay(Context context, Handler handler) {
        this.context = context;
        this.handler = handler;
        this.windows = (WindowManager) context.getSystemService(Context.WINDOW_SERVICE);
    }

    boolean isShowing() {
        return root != null;
    }
```

생성자: 안드로이드에서 창 관리자(WindowManager)를 얻어 둔다.

isShowing: 지금 시트가 떠 있는지 (root가 있으면 떠 있음).

```
    /** Shows the sheet if needed. Further calls only update the content. */
    void open() {
        if (root != null && !closing) return;
        dismissNow();
        build();
        attach();
    }

    /** Small line above the gauge, e.g. "작은 창으로 보던 쇼츠예요". Null hides it. */
    void notice(String text) {
        if (root == null) return;
        noticeText.setText(text == null ? "" : text);
        noticeText.setVisibility(text == null ? View.GONE : View.VISIBLE);
    }
```

open: 이미 떠 있으면 아무것도 안 하고(내용만 바꿀 수 있게), 아니면 남은 창을 정리한 뒤 새로 만들어(`build`) 화면에 붙인다(`attach`).

notice: 게이지 위 작은 안내 줄 (예: "작은 창으로 보던 쇼츠예요", AI 위험 시간 안내). null이면 숨긴다(`View.GONE`).

```
    /** Gauge row: "오늘 목표의 64% · 남은 7분" with a bar in the band's colour. */
    void gauge(double ratio, double remainingSeconds, int band) {
        if (root == null) return;
        int percent = (int) Math.round(ratio * 100);
        String rest = remainingSeconds > 0
            ? "남은 " + GuardCopy.clock(remainingSeconds)
            : "목표 " + GuardCopy.clock(-remainingSeconds) + " 넘음";
        gaugeText.setText("오늘 목표의 " + percent + "% · " + rest);
        int tone = BAND_TONE[Math.max(1, Math.min(3, band)) - 1];
        gaugeText.setTextColor(tone);
        float filled = (float) Math.max(0.02, Math.min(1.0, ratio));
        gaugeFill.setBackground(rounded(tone, dp(3), false));
        gaugeFill.setLayoutParams(new LinearLayout.LayoutParams(0, dp(6), filled));
        gaugeRest.setLayoutParams(new LinearLayout.LayoutParams(0, dp(6), 1f - filled));
    }
```

gauge — "오늘 목표의 64% · 남은 7분" 줄과 막대.

1. **92줄** 비율을 퍼센트로.
2. **93~95줄** 남았으면 "남은 7분", 넘었으면 "목표 3분 넘음".
3. **97~98줄** 단계 색으로 글자를 칠한다.
4. **99~102줄** 막대 채움 비율(최소 2%, 최대 100%)만큼 왼쪽 칸을 넓히고 나머지는 오른쪽 칸. 무게(weight)로 너비를 나누는 방식.

```
    /** Replaces the step content. Stops any running countdown. */
    void step(String titleText, String subtitleText, View content) {
        if (root == null) return;
        stopTicker();
        title.setText(titleText);
        subtitle.setText(subtitleText == null ? "" : subtitleText);
        subtitle.setVisibility(subtitleText == null || subtitleText.isEmpty() ? View.GONE : Vie
w.VISIBLE);
        body.removeAllViews();
```

step — 단계 내용 바꾸기. 돌던 타이머를 멈추고, 제목과 부제목을 바꾸고(부제목이 비면 숨김), 본문 칸을 비운 뒤 새 내용을 넣는다. GuardFlow가 단계마다 이걸 부른다.

```
        if (content != null) body.addView(content, matchWrap());
    }
```

```
    /** The bottom button, always in the same place. A null label hides it. */
    void leave(String label, Runnable action) {
        if (root == null) return;
        if (label == null) {
            leaveButton.setVisibility(View.GONE);
            leaveButton.setOnClickListener(null);
            return;
        }
        leaveButton.setVisibility(View.VISIBLE);
        leaveButton.setText(label);
        leaveButton.setOnClickListener(v -> {
            if (!closing) action.run();
        });
    }
```

leave — 아래 큰 버튼 (보통 "나가기"). 항상 같은 자리에 있다. 글자가 null이면 숨긴다. 누르면 닫히는 중이 아닐 때만 동작을 실행한다 (두 번 눌림 방지).

```
    /** Slides the sheet away, then runs {@code after} once the player is uncovered. */
    void close(Runnable after) {
        stopTicker();
        if (root == null) {
            if (after != null) after.run();
            return;
        }
        if (closing) return;
        closing = true;
        View closingRoot = root;
        scrim.animate().alpha(0f).setDuration(ANIM_MS).start();
        sheet.animate().translationY(sheet.getHeight() + dp(40)).setDuration(ANIM_MS)
            .withEndAction(() -> {
                if (root == closingRoot) dismissNow();
            })
            .start();
        if (after != null) handler.postDelayed(after, ANIM_MS);
    }
```

close — 시트를 내려 보내며 닫기.

1. 타이머를 멈추고, 떠 있지 않으면 바로 다음 일(`after`)을 한다.
2. 이미 닫는 중이면 무시.
3. **141줄** 어두운 배경을 0.25초 동안 투명하게.
4. **142~146줄** 시트를 아래로 밀어 내리고, 끝나면 창을 뗀다(그 사이 새 시트가 떴으면 건드리지 않음).
5. **147줄** 애니메이션이 끝날 즈음 다음 일을 실행한다.

```
    /** Removes the sheet without callbacks, e.g. when the user already left the player. */
    void dismissNow() {
        stopTicker();
        closing = false;
        if (root == null) return;
        View old = root;
        root = null;
        try {
            windows.removeView(old);
        } catch (RuntimeException ignored) {
            // Already detached.
        }
    }
```

dismissNow: 애니메이션 없이 바로 창을 뗀다 (사용자가 이미 쇼츠를 떠났을 때). 이미 떨어져 있어 오류가 나도 무시한다.

```
    void startTicker(long periodMs, Tick tick) {
        stopTicker();
        ticker = new Runnable() {
            @Override
            public void run() {
                if (ticker != this) return;
                if (tick.run()) handler.postDelayed(this, periodMs);
            }
        };
        handler.post(ticker);
    }

    void stopTicker() {
        if (ticker != null) handler.removeCallbacks(ticker);
        ticker = null;
    }
```

startTicker / stopTicker — 주기 작업 (5초 숨 고르기 숫자, 잠금 카운트다운 등).

정해진 간격마다 `tick.run()`을 부르고, true면 다시 예약한다. **169줄** 새 타이머로 바뀌었으면 옛 타이머는 스스로 멈춘다. 새로 시작하면 이전 것은 먼저 멈춘다.

```
    // --- Builders used by GuardFlow -----------------------------------------

    LinearLayout vertical() {
        LinearLayout layout = new LinearLayout(context);
        layout.setOrientation(LinearLayout.VERTICAL);
        return layout;
    }

    TextView text(String value, int sp, int color, boolean bold) {
        TextView view = new TextView(context);
        view.setText(value);
        view.setTextColor(color);
        view.setTextSize(TypedValue.COMPLEX_UNIT_SP, sp);
        if (bold) view.setTypeface(Typeface.DEFAULT_BOLD);
        return view;
    }

    TextView chip(String label) {
        TextView view = text(label, 15, TEXT, false);
        view.setGravity(Gravity.CENTER);
        view.setPadding(dp(8), 0, dp(8), 0);
        view.setBackground(rounded(CARD, dp(14), false));
        view.setClickable(true);
        return view;
    }
```

화면 부품 만들기 도구 (GuardFlow가 씀).

- **vertical**: 부품을 위에서 아래로 쌓는 틀.
- **text**: 글자 칸 (글자, 크기 sp, 색, 굵게).
- **chip**: 둥근 카드 모양의 선택 버튼 (이유, 분, 보기 고르기).

```
    TextView button(String label, boolean primary) {
        TextView view = text(label, 16, primary ? SHEET : TEXT, true);
        view.setGravity(Gravity.CENTER);
        GradientDrawable bg = rounded(primary ? TEXT : Color.TRANSPARENT, dp(14), false);
        if (!primary) bg.setStroke(dp(1), Color.parseColor("#48484A"));
        view.setBackground(bg);
        view.setClickable(true);
```

button: 큰 버튼. 주 버튼(`primary`)은 흰 바탕에 어두운 글자, 보조 버튼은 투명 바탕에 테두리.

```
        return view;
    }
```

```
    /** Lays views out in rows of {@code columns}, each {@code heightDp} tall. */
    LinearLayout grid(List<? extends View> items, int columns, int heightDp) {
        LinearLayout grid = vertical();
        for (int i = 0; i < items.size(); i += columns) {
            LinearLayout row = new LinearLayout(context);
            row.setOrientation(LinearLayout.HORIZONTAL);
            for (int j = i; j < i + columns; j++) {
                LinearLayout.LayoutParams lp = new LinearLayout.LayoutParams(0, dp(heightDp), 1
f);
                lp.setMargins(j == i ? 0 : dp(4), dp(4), j == i + columns - 1 ? 0 : dp(4), dp
(4));
                if (j < items.size()) {
                    row.addView(items.get(j), lp);
                } else {
                    row.addView(new View(context), lp);
                }
            }
            grid.addView(row);
        }
        return grid;
    }
```

grid — 부품을 줄 맞춰 배치. 예: 칩 6개를 2칸씩 3줄. 한 줄에 columns개씩 같은 너비로 넣고, 마지막 줄이 모자라면 빈 칸을 채워 너비를 맞춘다. 양 끝이 아닌 칸에는 좌우 여백을 준다.

```
    void tint(View view, int color) {
        if (view.getBackground() instanceof GradientDrawable) {
            ((GradientDrawable) view.getBackground()).setColor(color);
        }
    }

    GradientDrawable rounded(int color, int radius, boolean topOnly) {
        GradientDrawable drawable = new GradientDrawable();
        drawable.setColor(color);
        if (topOnly) {
            drawable.setCornerRadii(new float[] {radius, radius, radius, radius, 0, 0, 0, 0});
        } else {
            drawable.setCornerRadius(radius);
        }
        return drawable;
    }

    GradientDrawable oval(int color) {
        GradientDrawable drawable = new GradientDrawable();
        drawable.setShape(GradientDrawable.OVAL);
        drawable.setColor(color);
        return drawable;
    }
```

- **tint**: 고른 칩이나 정답·오답 칸의 색을 바꾼다.
- **rounded**: 둥근 모서리 배경. `topOnly`면 위쪽 모서리만 둥글게 (시트 모양).
- **oval**: 동그라미 배경 (호흡 따라 하기 원).

```
    View spacer(int heightDp) {
        View view = new View(context);
```

- **spacer**: 빈 간격.

```
        view.setLayoutParams(new LinearLayout.LayoutParams(1, dp(heightDp)));
        return view;
    }

    static LinearLayout.LayoutParams matchWrap() {
        return new LinearLayout.LayoutParams(
            ViewGroup.LayoutParams.MATCH_PARENT, ViewGroup.LayoutParams.WRAP_CONTENT);
    }

    LinearLayout.LayoutParams matchHeight(int heightDp) {
        return new LinearLayout.LayoutParams(ViewGroup.LayoutParams.MATCH_PARENT, dp(heightDp));
    }

    int dp(int value) {
        return Math.round(TypedValue.applyDimension(
            TypedValue.COMPLEX_UNIT_DIP, value, context.getResources().getDisplayMetrics()));
    }
```

- **matchWrap**: "가로는 꽉 채우고 세로는 내용만큼" 크기 설정.
- **matchHeight**: 가로는 꽉, 세로는 지정 높이.
- **dp**: 화면 밀도에 맞게 크기를 바꾼다. 폰마다 화소 크기가 달라서 "dp" 단위를 실제 화소로 바꿔야 같은 크기로 보인다.

```
    // --- Window ---------------------------------------------------------------

    private void build() {
        closing = false;
        root = new FrameLayout(context);
        root.setClickable(true);

        scrim = new View(context);
        scrim.setBackgroundColor(Color.BLACK);
        scrim.setAlpha(0f);
        root.addView(scrim, new FrameLayout.LayoutParams(
            ViewGroup.LayoutParams.MATCH_PARENT, ViewGroup.LayoutParams.MATCH_PARENT));

        sheet = vertical();
        sheet.setClickable(true);
        sheet.setBackground(rounded(SHEET, dp(24), true));
        int pad = dp(20);
        sheet.setPadding(pad, pad, pad, pad);

```

build — 시트 만들기 (1).

- **285~286줄** 맨 바깥 틀(root). 눌러도 뒤 YouTube로 터치가 넘어가지 않게 `setClickable(true)`.
- **288~292줄** 화면 전체를 덮는 검은 배경(scrim). 처음엔 투명.
- **294~298줄** 시트: 위쪽만 둥근 어두운 판, 안쪽 여백 20dp.

```
        noticeText = text("", 13, TEXT, true);
        noticeText.setPadding(0, 0, 0, dp(6));
        noticeText.setVisibility(View.GONE);
        sheet.addView(noticeText);
        gaugeText = text("", 13, MUTED, true);
        sheet.addView(gaugeText);
        LinearLayout bar = new LinearLayout(context);
        bar.setOrientation(LinearLayout.HORIZONTAL);
        bar.setBackground(rounded(TRACK, dp(3), false));
        gaugeFill = new View(context);
        gaugeRest = new View(context);
        bar.addView(gaugeFill, new LinearLayout.LayoutParams(0, dp(6), 0.02f));
        bar.addView(gaugeRest, new LinearLayout.LayoutParams(0, dp(6), 0.98f));
```

build (2): 위 안내 줄(처음엔 숨김), 게이지 글자, 게이지 막대(채움 칸 + 나머지 칸을 가로로).

```
        LinearLayout.LayoutParams barLp = matchWrap();
        barLp.setMargins(0, dp(6), 0, dp(12));
        sheet.addView(bar, barLp);
```

```
        title = text("", 22, TEXT, true);
        sheet.addView(title);
        subtitle = text("", 14, MUTED, false);
        subtitle.setPadding(0, dp(4), 0, 0);
        sheet.addView(subtitle);
        body = vertical();
        LinearLayout.LayoutParams bodyLp = matchWrap();
        bodyLp.setMargins(0, dp(14), 0, 0);
        sheet.addView(body, bodyLp);
        leaveButton = button("나가기", false);
        LinearLayout.LayoutParams leaveLp = matchHeight(52);
        leaveLp.setMargins(0, dp(12), 0, 0);
        sheet.addView(leaveButton, leaveLp);
```

build (3): 큰 제목(22sp, 굵게), 흐린 부제목, 단계 내용이 들어갈 본문 칸, 아래 "나가기" 버튼(높이 52dp).

```
        FrameLayout.LayoutParams sheetLp = new FrameLayout.LayoutParams(
            ViewGroup.LayoutParams.MATCH_PARENT, ViewGroup.LayoutParams.WRAP_CONTENT, Gravity.BO
TTOM);
        root.addView(sheet, sheetLp);
        root.setOnApplyWindowInsetsListener((v, insets) -> {
            sheet.setPadding(pad, pad, pad, pad + insets.getSystemWindowInsetBottom());
            return insets;
        });
    }
```

build (4): 시트를 화면 아래쪽(`Gravity.BOTTOM`)에 붙인다. **334~337줄** 폰 아래 내비게이션 바가 있으면 그 높이만큼 아래 여백을 더해 버튼이 가려지지 않게 한다.

```
    private void attach() {
        WindowManager.LayoutParams params = new WindowManager.LayoutParams(
            WindowManager.LayoutParams.MATCH_PARENT,
            WindowManager.LayoutParams.MATCH_PARENT,
            WindowManager.LayoutParams.TYPE_ACCESSIBILITY_OVERLAY,
            WindowManager.LayoutParams.FLAG_NOT_FOCUSABLE
                | WindowManager.LayoutParams.FLAG_LAYOUT_IN_SCREEN,
            PixelFormat.TRANSLUCENT);
        params.gravity = Gravity.BOTTOM;
        try {
            windows.addView(root, params);
        } catch (RuntimeException e) {
            root = null;
            return;
        }
```

attach — 창을 화면에 붙이기.

- **341~347줄** 창 설정: 화면 전체 크기, 접근성 오버레이 종류, 포커스를 가져가지 않음(YouTube가 계속 앞 앱), 화면 전체에 배치, 반투명 가능.
- **349~354줄** 창 관리자에 붙인다. 실패하면(서비스가 꺼지는 중 등) 조용히 포기.

```
        sheet.setTranslationY(dp(600));
        sheet.post(() -> {
            sheet.setTranslationY(sheet.getHeight());
            sheet.animate().translationY(0f).setDuration(ANIM_MS).start();
            scrim.animate().alpha(SCRIM_ALPHA).setDuration(ANIM_MS).start();
        });
    }
}
```

등장 애니메이션: 시트를 화면 아래로 내려 둔 뒤, 크기가 정해지면(`post`) 0.25초 동안 위로 올리고 배경을 90%까지 어둡게 한다.

GuardState — 휴대폰 쪽 메모장 `android/app/src/main/java/app/focuson/shorts/GuardState.java (704줄)`

안드로이드의 작은 메모장(SharedPreferences)에 값을 적고 읽는 함수 모음. 앱이 꺼졌다 켜져도 남아야 하는 것(대기 중인 기록, 잠금 시각, 정책표 사본, 공부 카드 사본 등)을 여기 둔다. 화면 글자, 계정, 화면 이름표는 저장하지 않는다.

> **synchronized (LOCK):** 이 파일의 거의 모든 함수가 이걸로 감싸져 있다. 서비스와 플러그인이 **동시에** 메모장을 고치면 내용이 섞일 수 있어서, "한 번에 한 쪽만 들어오게" 자물쇠를 거는 것이다. **commit()**은 바로 저장하고 끝날 때까지 기다리고, **apply()**는 나중에 저장하고 바로 넘어간다(덜 중요한 값에 사용).

```
package app.focuson.shorts;

import android.content.Context;
import android.content.SharedPreferences;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Iterator;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.TimeZone;
import java.util.UUID;
import org.json.JSONArray;
import org.json.JSONException;
import org.json.JSONObject;

/**
 * On-device guard state. Sessions, intervention results and lock attempts wait here
 * until the web side copies them into SQLite and acknowledges them. No screen text,
 * account data, or view ids.
 */
public final class GuardState {
```

3~15줄 메모장(`SharedPreferences`), 목록과 표, 고유 번호 만들기(`UUID`), JSON 도구.

17~21줄 설명: 시청 기록, 개입 결과, 잠금 중 시도는 웹이 SQLite에 저장하고 확인(ack)할 때까지 여기서 기다린다.

```
    private static final String PREFS = "hanbakja_guard";
    private static final String KEY_BLOCKER = "blocker_enabled";
    private static final String KEY_LEVEL1 = "level1_minutes";
    private static final String KEY_LEVEL3 = "level3_minutes";
    private static final String KEY_WEB_DATE = "web_seconds_date";
    private static final String KEY_WEB_SECONDS = "web_seconds";
```

메모장 칸 이름들. 메모장은 "이름 → 값" 표라서 칸마다 이름을 정해 둔다.

- `PREFS = "hanbakja_guard"`: 메모장 파일 이름 (팀원 앱에서 이어받음).
- 개입 스위치, 단계 기준(분), 웹이 잰 날짜와 초, 날짜별 사용 초, 지금 열린 시청.

```
    private static final String KEY_DAY_USAGE = "day_usage";
    private static final String KEY_ACTIVE = "active_session";
    private static final String KEY_SESSIONS = "pending_sessions";
    private static final String KEY_INTERVENTIONS = "pending_interventions";
    private static final String KEY_LOCKS = "pending_locks";
    private static final String KEY_BLOCKED_UNTIL = "blocked_until";
    private static final String KEY_BLOCK_MINUTES = "block_minutes";
    private static final String KEY_PROMPT_TARGET = "prompt_target";
    private static final String KEY_PROMPT_LEVEL = "prompt_level";
    private static final String KEY_PROMPT_USAGE = "prompt_usage";
    private static final String KEY_ALLOW_TARGET = "allow_target";
    private static final String KEY_ALLOW_UNTIL = "allow_until";
    private static final String KEY_POLICY = "policy";
    private static final String KEY_CARDS = "cards";
    private static final String KEY_ROTATE = "rotate_";
    private static final String KEY_SHOWN = "shown_";
    /** Per card: when it was last shown, and how many times in a row it was answered right. */
    private static final String KEY_CARD_LAST = "card_last_";
    private static final String KEY_CARD_STREAK = "card_streak_";
    private static final long HOUR_MS = 3_600_000L;
    private static final String KEY_LOCK_ASKED = "lock_asked_for";
    private static final String KEY_MUTED = "muted_by_us";
    private static final String KEY_BLOCK_KIND = "block_kind";
    private static final String KEY_LOCK_LOG = "lock_log";
    private static final String KEY_COUNTERS = "day_counters";
```

- 대기열 3개: `pending_sessions`(시청), `pending_interventions`(개입), `pending_locks`(잠금 중 시도).
- 잠금 끝 시각, 잠금 길이, 앱 화면이 이어받을 개입(대상·단계·초), 보기 허락.
- 정책표, 공부 카드, 차례대로 고르기 번호(`rotate_`), 마지막 표시 시각(`shown_`).
- 카드별 마지막 표시 시각(`card_last_`)과 연속 정답 수(`card_streak_`). `HOUR_MS` = 1시간의 밀리초.
- 잠금 뒤 질문을 했는지, 우리가 음소거했는지, 잠금 종류, 잠금 기록, 하루 카운터.

```
    /** The after-lock question is asked only this soon after a timed lock ended. */
    private static final long AFTER_LOCK_WINDOW_MS = 3 * 60 * 60 * 1000L;

    /** Drop the oldest queued rows past this so a never-opened app cannot grow without bound.
*/
    private static final int QUEUE_LIMIT = 2000;
    private static final int DAYS_KEPT = 3;

    private static final Object LOCK = new Object();

    private GuardState() {}

    private static SharedPreferences prefs(Context context) {
        return context.getApplicationContext().getSharedPreferences(PREFS, Context.MODE_PRIVAT
E);
    }
```

- **56줄** 잠금 뒤 "뭐 했어요?" 질문은 잠금이 끝나고 3시간 안에만.
- **59줄** 대기열 최대 2000줄. 앱을 오래 안 열어도 끝없이 커지지 않게, 넘치면 오래된 것부터 버린다.
- **60줄** 날짜별 사용 초는 최근 3일만 보관.
- **62줄** 자물쇠로 쓸 물건. **64줄** 함수만 쓰는 클래스.
- **66~68줄 prefs**: 메모장 열기. `MODE_PRIVATE` = 이 앱만 읽을 수 있음.

```
    // --- Switch and thresholds ---------------------------------------------------

    public static boolean isBlockerEnabled(Context context) {
        synchronized (LOCK) {
            return prefs(context).getBoolean(KEY_BLOCKER, true);
        }
    }

```

isBlockerEnabled: 개입 스위치 상태 (기본 켜짐).

setBlockerEnabled: 스위치 저장. 끌 때는 앱 화면이 이어받으려던 개입도 지워서 예전 방식 화면이 열리지 않게 한다.

```
    /** Off also drops a pending prompt so the Level flow does not open. */
    public static void setBlockerEnabled(Context context, boolean enabled) {
        synchronized (LOCK) {
            SharedPreferences.Editor editor = prefs(context).edit().putBoolean(KEY_BLOCKER, enab
led);
            if (!enabled) editor.putString(KEY_PROMPT_TARGET, "");
            editor.commit();
        }
    }
```

```
    /**
     * Thresholds and the seconds the web side counted itself today (practice feed,
     * migrated logs). Native sessions are counted here, not by the web.
     */
    public static void setConfig(Context context, int level1, int level3, String webDate, double
webSeconds) {
        synchronized (LOCK) {
            prefs(context).edit()
                .putInt(KEY_LEVEL1, level1)
                .putInt(KEY_LEVEL3, level3)
                .putString(KEY_WEB_DATE, webDate == null ? "" : webDate)
                .putFloat(KEY_WEB_SECONDS, (float) webSeconds)
                .commit();
        }
    }

    public static int level1Minutes(Context context) {
        synchronized (LOCK) {
            return prefs(context).getInt(KEY_LEVEL1, 10);
        }
    }

    public static int level3Minutes(Context context) {
        synchronized (LOCK) {
            return prefs(context).getInt(KEY_LEVEL3, 20);
        }
    }
```

setConfig: 웹이 sync 때 보내는 값 저장 (단계 기준 분 2개, 웹이 직접 잰 날짜와 초).

level1Minutes / level3Minutes: 저장된 기준 (기본 10분, 20분).

```
    /** Today's Shorts seconds: finished native sessions, web seconds, and the open session. */
    public static double todaySeconds(Context context, long now) {
        synchronized (LOCK) {
            SharedPreferences prefs = prefs(context);
            TimeZone zone = TimeZone.getDefault();
            String today = GuardTime.dayKey(now, zone);
            double total = readObject(prefs, KEY_DAY_USAGE).optDouble(today, 0);
            if (today.equals(prefs.getString(KEY_WEB_DATE, ""))) {
                total += prefs.getFloat(KEY_WEB_SECONDS, 0f);
            }
            JSONObject active = readObject(prefs, KEY_ACTIVE);
            if (active.has("startedAt")) {
                Double open = GuardTime.splitByDay(active.optLong("startedAt"), now, zone).get(t
```

todaySeconds — 오늘 쇼츠를 본 총 초. 세 가지를 더한다.

1. **120줄** 끝난 시청들의 오늘 합 (날짜별 사용 표에서).
2. **121~123줄** 웹이 직접 잰 초 (날짜가 오늘일 때만, 브라우저 연습 화면 등).
3. **124~128줄 지금 보고 있는** 시청의 오늘 몫 (시작부터 지금까지, 자정 넘으면 오늘 부분만).

이 값으로 단계(약함·중간·강함)를 정한다.

```
oday);
                if (open != null) total += open;
            }
            return total;
        }
    }

    // --- Policy table and cards from the web side ------------------------------

    /** Stores the policy table and study cards the web side sent with sync. */
    public static void setPolicyAndCards(Context context, String policyJson, String cardsJson) {
        synchronized (LOCK) {
            SharedPreferences.Editor editor = prefs(context).edit();
            if (policyJson != null) editor.putString(KEY_POLICY, policyJson);
            if (cardsJson != null) editor.putString(KEY_CARDS, cardsJson);
            editor.commit();
        }
    }

    static GuardPolicy policy(Context context) {
        synchronized (LOCK) {
            SharedPreferences prefs = prefs(context);
            return GuardPolicy.fromJson(prefs.getString(KEY_POLICY, ""), prefs.getInt(KEY_LEVEL
3, 20));
        }
    }

    /** Drops a card the user reported from the cached list, so it is not shown again before the
next sync. */
    public static void removeCard(Context context, String id) {
        if (id == null) return;
        synchronized (LOCK) {
            SharedPreferences prefs = prefs(context);
            JSONArray cards = readArray(prefs, KEY_CARDS);
            JSONArray kept = new JSONArray();
            for (int i = 0; i < cards.length(); i++) {
                JSONObject card = cards.optJSONObject(i);
                if (card != null && !id.equals(card.optString("id"))) kept.put(card);
            }
            prefs.edit().putString(KEY_CARDS, kept.toString()).commit();
        }
    }

    public static boolean hasCards(Context context) {
        synchronized (LOCK) {
            return readArray(prefs(context), KEY_CARDS).length() > 0;
        }
    }

    /**
     * When a card may show again (spaced review): never shown = now; last answer wrong or
```

setPolicyAndCards: 웹이 보낸 정책표와 공부 카드(JSON 글자)를 저장. null이면 그 칸은 그대로 둔다.

policy: 저장된 정책표를 읽어 GuardPolicy로 바꿔 준다 (없으면 기본 표).

removeCard: "이 문제 이상해요"를 누른 카드를 저장된 카드 목록에서 바로 뺀다. 다음 sync 전에 다시 나오지 않게 하기 위해서다. 목록을 돌며 그 id가 아닌 것만 새 목록에 담아 저장한다.

hasCards: 카드가 하나라도 있는지.

cardDueAt — 간격 복습 규칙. 카드가 다음에 나올 수 있는 시각을 계산한다.

```
     * unanswered = 1 hour after; right 1 / 2 / 3+ times in a row = 1 / 3 / 7 days after.
     */
    static long cardDueAt(long lastShown, int streak) {
        if (lastShown <= 0) return 0L;
        if (streak <= 0) return lastShown + HOUR_MS;
        if (streak == 1) return lastShown + 24 * HOUR_MS;
        if (streak == 2) return lastShown + 72 * HOUR_MS;
        return lastShown + 168 * HOUR_MS;
    }
```

- 한 번도 안 나왔으면 → 바로 (0).
- 마지막에 틀렸거나 안 풀었으면 → 1시간 뒤.
- 연속 1번 맞힘 → 1일(24시간) 뒤, 2번 → 3일 뒤, 3번 이상 → 7일 뒤.

```
    /** True when some card is due now. */
    public static boolean hasDueCard(Context context, long now) {
        synchronized (LOCK) {
            return dueCard(prefs(context), now) != null;
        }
    }

    /**
     * The first due card (the web side sends the least-shown first), or null when every card
     * is resting. Marks it shown, so leaving without answering also rests it for an hour.
     */
    public static JSONObject nextCard(Context context, long now) {
        synchronized (LOCK) {
            SharedPreferences prefs = prefs(context);
            JSONObject card = dueCard(prefs, now);
            if (card != null) prefs.edit().putLong(KEY_CARD_LAST + card.optString("id"), now).ap
ply();
            return card;
        }
    }
```

hasDueCard: 지금 나올 수 있는 카드가 있는지 (공부 모드 판단에 사용).

nextCard: 지금 나올 수 있는 첫 카드를 고르고 "지금 보여 줌"으로 표시한다. 그래서 풀지 않고 나가도 1시간은 다시 안 나온다. 웹이 덜 본 카드부터 보내므로 앞쪽 카드가 먼저 나온다.

```
    /** After an answer: a right answer extends the streak, a wrong one resets it. */
    public static void recordCardAnswer(Context context, String id, boolean correct, long now) {
        if (id == null) return;
        synchronized (LOCK) {
            SharedPreferences prefs = prefs(context);
            int streak = correct ? prefs.getInt(KEY_CARD_STREAK + id, 0) + 1 : 0;
            prefs.edit().putInt(KEY_CARD_STREAK + id, streak).putLong(KEY_CARD_LAST + id, now).a
pply();
        }
    }
```

recordCardAnswer: 답을 고른 뒤 호출. 맞히면 연속 정답 +1, 틀리면 0으로 되돌리고, 마지막 표시 시각을 지금으로 저장한다.

```
    private static JSONObject dueCard(SharedPreferences prefs, long now) {
        JSONArray cards = readArray(prefs, KEY_CARDS);
        for (int i = 0; i < cards.length(); i++) {
            JSONObject card = cards.optJSONObject(i);
            if (card == null) continue;
            String id = card.optString("id");
            long due = cardDueAt(prefs.getLong(KEY_CARD_LAST + id, 0L), prefs.getInt(KEY_CARD_ST
REAK + id, 0));
            if (due <= now) return card;
```

dueCard: 저장된 카드를 순서대로 보며, 다음 표시 시각(`cardDueAt`)이 지금 이전인 첫 카드를 돌려준다. 없으면 null (모두 쉬는 중).

```
        }
        return null;
    }
```

```
    /**
     * Round-robin pick among {@code options} for this key, so the same sheet does not show
     * every time (habituation) and each method gets data before the AI phase.
     */
    public static String nextInTurn(Context context, String key, List<String> options) {
        if (options == null || options.isEmpty()) return null;
        synchronized (LOCK) {
            SharedPreferences prefs = prefs(context);
            int index = prefs.getInt(KEY_ROTATE + key, 0);
            prefs.edit().putInt(KEY_ROTATE + key, index + 1).apply();
            return options.get(Math.floorMod(index, options.size()));
        }
    }
```

nextInTurn — 차례대로 고르기 (AI가 꺼져 있을 때). 칸 이름마다 번호를 기억해 두고, 부를 때마다 1씩 늘려 목록에서 (번호 ÷ 목록 길이의 나머지)번째를 돌려준다. 예: [숨 고르기, 문구] → 숨 고르기, 문구, 숨 고르기 …

```
    /** When each of {@code arms} was last shown (0 = never), for the bandit's habituation. */
    public static Map<String, Long> lastShown(Context context, String group, List<String> arms)
{
        Map<String, Long> out = new java.util.HashMap<>();
        if (arms == null) return out;
        synchronized (LOCK) {
            SharedPreferences prefs = prefs(context);
            for (String arm : arms) out.put(arm, prefs.getLong(KEY_SHOWN + group + "_" + arm, 0
L));
        }
        return out;
    }

    public static void markShown(Context context, String group, String arm, long now) {
        if (arm == null) return;
        synchronized (LOCK) {
            prefs(context).edit().putLong(KEY_SHOWN + group + "_" + arm, now).apply();
        }
    }
```

lastShown: 방식(또는 문구)마다 마지막으로 보여 준 시각을 표로 돌려준다 (질림 반영용). 없으면 0.

markShown: 방금 보여 준 방식의 시각을 저장한다. 칸 이름 예: `shown_method_breath`.

```
    /** Running counter for rotating lines: returns 0, 1, 2, ... for this key. */
    public static int nextIndex(Context context, String key) {
        synchronized (LOCK) {
            SharedPreferences prefs = prefs(context);
            int index = prefs.getInt(KEY_ROTATE + key, 0);
            prefs.edit().putInt(KEY_ROTATE + key, index + 1).apply();
            return index;
        }
    }
```

nextIndex: 칸 이름마다 0, 1, 2, … 를 차례로 돌려주는 계수기. 문구를 번갈아 쓸 때(칭찬, 약속 끝 문장 등) 사용.

```
    /**
     * True once per finished timed lock, within 3 hours of its end. Not after "오늘은 끝"
     * (asking "what did you do during the lock" first thing in the morning reads oddly).
     */
```

needsAfterLockQuestion — 잠금 뒤 질문을 할지. 다섯 조건이 모두 맞아야 true:

```
    public static boolean needsAfterLockQuestion(Context context, long now) {
        synchronized (LOCK) {
            SharedPreferences prefs = prefs(context);
            long until = prefs.getLong(KEY_BLOCKED_UNTIL, 0L);
            return until > 0
                && now >= until
                && now - until <= AFTER_LOCK_WINDOW_MS
                && "timed".equals(prefs.getString(KEY_BLOCK_KIND, "timed"))
                && prefs.getLong(KEY_LOCK_ASKED, 0L) != until;
        }
    }

    public static void markAfterLockAsked(Context context) {
        synchronized (LOCK) {
            SharedPreferences prefs = prefs(context);
            prefs.edit().putLong(KEY_LOCK_ASKED, prefs.getLong(KEY_BLOCKED_UNTIL, 0L)).commit();
        }
    }
```

1. 잠금 기록이 있고 **2.** 이미 끝났고 **3.** 끝난 지 3시간 이내이고 **4.** 시간 잠금이었고("오늘은 끝"이 아님. 아침에 묻는 건 어색해서) **5.** 이 잠금에 대해 아직 안 물었을 때.

markAfterLockAsked: 이 잠금에 대해 물었다고 표시 (한 번만 묻기).

```
    /** Remembers that we muted the music stream, so a restarted service can undo it. */
    public static void setMutedByUs(Context context, boolean muted) {
        synchronized (LOCK) {
            prefs(context).edit().putBoolean(KEY_MUTED, muted).commit();
        }
    }

    public static boolean mutedByUs(Context context) {
        synchronized (LOCK) {
            return prefs(context).getBoolean(KEY_MUTED, false);
        }
    }
```

setMutedByUs / mutedByUs: "우리가 음악 소리를 껐다"를 기억한다. 서비스가 갑자기 꺼졌다 켜져도 소리를 되돌릴 수 있게 하기 위한 안전장치.

```
    // --- Watch sessions ------------------------------------------------------

    public static final class ActiveSession {
        public final String target;
        public final long startedAt;
        public final long lastSeenAt;

        ActiveSession(String target, long startedAt, long lastSeenAt) {
            this.target = target;
            this.startedAt = startedAt;
            this.lastSeenAt = lastSeenAt;
        }
    }

    public static ActiveSession activeSession(Context context) {
        synchronized (LOCK) {
            JSONObject active = readObject(prefs(context), KEY_ACTIVE);
            if (!active.has("startedAt")) return null;
            return new ActiveSession(
```

ActiveSession: 지금 열린 시청 정보 상자 (대상 앱, 시작 시각, 마지막으로 쇼츠가 보인 시각).

activeSession: 메모장에서 읽어 상자로 돌려준다. 시작 시각이 없으면 null (보는 중 아님).

```
                active.optString("target"), active.optLong("startedAt"), active.optLong("lastSee
nAt"));
        }
    }
```

```
    public static void startSession(Context context, String target, long now) {
        synchronized (LOCK) {
            JSONObject active = new JSONObject();
            try {
                active.put("target", target).put("startedAt", now).put("lastSeenAt", now);
            } catch (JSONException ignored) {
                return;
            }
            prefs(context).edit().putString(KEY_ACTIVE, active.toString()).commit();
        }
    }
```

startSession: 쇼츠에 들어왔을 때 "보는 중" 기록을 만든다. 대상, 시작 시각, 마지막 본 시각(=지금)을 JSON으로 저장.

```
    /** Lets a restarted service close the session at the last time Shorts was seen. */
    public static void touchSession(Context context, long now) {
        synchronized (LOCK) {
            SharedPreferences prefs = prefs(context);
            JSONObject active = readObject(prefs, KEY_ACTIVE);
            if (!active.has("startedAt")) return;
            try {
                active.put("lastSeenAt", now);
            } catch (JSONException ignored) {
                return;
            }
            prefs.edit().putString(KEY_ACTIVE, active.toString()).apply();
        }
    }
```

touchSession: 보는 중에 1초마다 "마지막으로 쇼츠가 보인 시각"을 갱신한다. 서비스가 갑자기 꺼지면, 다시 켜질 때 이 시각까지만 시청으로 계산한다 (꺼져 있던 시간이 시청으로 잡히지 않게).

```
    /**
     * Closes the open session at {@code endedAt}, queues it for the web side and adds
     * its seconds to each local date it covers.
     * @return seconds recorded, 0 when there was no session
     */
    public static double endSession(Context context, long endedAt) {
        synchronized (LOCK) {
            SharedPreferences prefs = prefs(context);
            JSONObject active = readObject(prefs, KEY_ACTIVE);
            SharedPreferences.Editor editor = prefs.edit().putString(KEY_ACTIVE, "");
            if (!active.has("startedAt")) {
                editor.commit();
                return 0;
            }
```

endSession — 시청 끝내기 (1). 열린 시청을 비우고(`KEY_ACTIVE = ""`), 열린 게 없었으면 0을 돌려준다.

```
            long startedAt = active.optLong("startedAt");
            long end = Math.max(startedAt, endedAt);
            double seconds = (end - startedAt) / 1000.0;
            if (seconds >= 1) {
                JSONObject row = new JSONObject();
```

endSession (2): 시작~끝 초를 계산하고, **1초 이상**이면 기록 한 줄을 만든다: 고유 id(UUID), 대상, 시작, 끝, 초. 이걸 시청 대기열에 붙인다(`append`).

```
                try {
                    row.put("id", UUID.randomUUID().toString())
                        .put("target", active.optString("target"))
                        .put("startedAt", startedAt)
                        .put("endedAt", end)
                        .put("durationSec", seconds);
                } catch (JSONException ignored) {
                    editor.commit();
                    return 0;
                }
                editor.putString(KEY_SESSIONS, append(prefs, KEY_SESSIONS, row));
```

```
                JSONObject days = readObject(prefs, KEY_DAY_USAGE);
                for (Map.Entry<String, Double> part
                    : GuardTime.splitByDay(startedAt, end, TimeZone.getDefault()).entrySet()) {
                    try {
                        days.put(part.getKey(), days.optDouble(part.getKey(), 0) + part.getValue
());
                    } catch (JSONException ignored) {
                        // Keep the queued row even if the day total cannot be updated.
                    }
                }
                editor.putString(KEY_DAY_USAGE, pruneDays(days).toString());
            }
            editor.commit();
            return seconds >= 1 ? seconds : 0;
        }
    }
```

endSession (3): 날짜별 사용 표에 이 시청 초를 더한다. 자정을 넘겼으면 `splitByDay`로 나눠 각 날짜에 더한다. 오래된 날짜는 지우고(`pruneDays`) 저장한 뒤 초를 돌려준다.

```
    // --- Intervention results and lock attempts --------------------------------

    public static void addIntervention(Context context, JSONObject row) {
        synchronized (LOCK) {
            SharedPreferences prefs = prefs(context);
            prefs.edit().putString(KEY_INTERVENTIONS, append(prefs, KEY_INTERVENTIONS, row)).com
mit();
        }
    }

    public static void addLockAttempt(Context context, String target, long now, long remainingM
s) {
        synchronized (LOCK) {
            SharedPreferences prefs = prefs(context);
            JSONObject row = new JSONObject();
            try {
                row.put("id", UUID.randomUUID().toString())
                    .put("target", target)
                    .put("createdAt", now)
                    .put("remainingSec", Math.max(0, Math.round(remainingMs / 1000.0)));
            } catch (JSONException ignored) {
                return;
```

addIntervention: 개입 기록 한 줄(GuardFlow가 만든 JSON)을 개입 대기열에 붙인다.

addLockAttempt: 잠금 중에 쇼츠를 연 시도를 기록 (id, 대상, 시각, 남은 초).

```
            }
            prefs.edit().putString(KEY_LOCKS, append(prefs, KEY_LOCKS, row)).commit();
        }
    }
```

```
    /** Everything waiting for the web side. Rows stay queued until {@link #ack}. */
    public static JSONObject peekPending(Context context) {
        synchronized (LOCK) {
            SharedPreferences prefs = prefs(context);
            JSONObject out = new JSONObject();
            try {
                out.put("sessions", readArray(prefs, KEY_SESSIONS))
                    .put("interventions", readArray(prefs, KEY_INTERVENTIONS))
                    .put("lockAttempts", readArray(prefs, KEY_LOCKS));
            } catch (JSONException ignored) {
                // An empty object is still a valid answer.
            }
            return out;
        }
    }
```

peekPending: 대기열 3개를 한 덩어리로 돌려준다 `{"sessions": [...], "interventions": [...], "lockAttempts": [...]}`. 웹이 ack할 때까지 지우지 않는다.

```
    /** Drops queued rows the web side has stored. */
    public static void ack(Context context, Set<String> ids) {
        if (ids == null || ids.isEmpty()) return;
        synchronized (LOCK) {
            SharedPreferences prefs = prefs(context);
            prefs.edit()
                .putString(KEY_SESSIONS, without(prefs, KEY_SESSIONS, ids))
                .putString(KEY_INTERVENTIONS, without(prefs, KEY_INTERVENTIONS, ids))
                .putString(KEY_LOCKS, without(prefs, KEY_LOCKS, ids))
                .commit();
        }
    }
```

ack: 웹이 저장했다고 알려 준 id들을 세 대기열에서 모두 뺀다 (`without` 사용).

```
    // --- Time block (Level 3) ------------------------------------------------

    public static long blockedUntil(Context context) {
        synchronized (LOCK) {
            return prefs(context).getLong(KEY_BLOCKED_UNTIL, 0L);
        }
    }

    public static int blockMinutes(Context context) {
        synchronized (LOCK) {
            return prefs(context).getInt(KEY_BLOCK_MINUTES, 0);
        }
    }

    public static boolean isLocked(Context context, long now) {
        return now < blockedUntil(context);
    }
```

잠금 관련 읽기: 잠금 끝 시각, 잠금 길이(분), 지금 잠겨 있는지(지금 < 끝 시각).

```
473     /** @return the wall-clock end of the block */
474     public static long startBlock(Context context, int minutes, long now) {
475         return startBlockUntil(context, now + minutes * 60_000L, minutes);
476     }
477
478     /** Block until a wall-clock time, e.g. "오늘은 끝" until tomorrow morning. */
479     public static long startBlockUntil(Context context, long until, int minutes) {
480         return startBlockUntil(context, until, minutes, false);
481     }
482
483     /**
484      * @param dayEnd true for "오늘은 끝"; those hours are not counted as "쉰 시간" and are
485      *               not followed by the after-lock question
486      */
```

startBlock: "지금부터 N분" 잠금 → 끝 시각을 계산해 아래 함수로.

startBlockUntil (짧은 버전): "이 시각까지" 잠금 (시간 잠금으로 취급).

483~486줄 설명: `dayEnd`가 true("오늘은 끝")면 쉰 시간에 넣지 않고, 잠금 뒤 질문도 하지 않는다.

```
487     public static long startBlockUntil(Context context, long until, int minutes, boolean dayEnd)
{
488         long now = System.currentTimeMillis();
489         synchronized (LOCK) {
490             SharedPreferences prefs = prefs(context);
491             JSONArray log = readArray(prefs, KEY_LOCK_LOG);
492             JSONArray kept = new JSONArray();
493             for (int i = 0; i < log.length(); i++) {
494                 JSONObject row = log.optJSONObject(i);
495                 if (row != null && now - row.optLong("until") < 48 * 3600_000L) kept.put(row);
496             }
497             if (!dayEnd) {
498                 JSONObject row = new JSONObject();
499                 try {
500                     row.put("start", now).put("until", until);
501                     kept.put(row);
502                 } catch (JSONException ignored) {
503                     // The lock still applies; only the rest-time total misses it.
504                 }
505             }
506             prefs.edit()
507                 .putLong(KEY_BLOCKED_UNTIL, until)
508                 .putInt(KEY_BLOCK_MINUTES, minutes)
509                 .putString(KEY_BLOCK_KIND, dayEnd ? "day_end" : "timed")
510                 .putString(KEY_LOCK_LOG, kept.toString())
511                 .commit();
512         }
513         return until;
514     }
```

startBlockUntil — 잠금 시작.

1. **491~496줄** 잠금 기록 중 48시간 안에 끝난 것만 남긴다 (오래된 건 정리).
2. **497~505줄** 시간 잠금이면 이번 잠금(시작, 끝)을 기록에 더한다 ("오늘 쉰 시간" 계산용).
3. **506~511줄** 끝 시각, 길이, 종류("day_end" 또는 "timed"), 정리한 기록을 저장.

```
516     /** Minutes of timed locks already gone by today ("오늘 쉰 시간"), including a running one.
*/
517     public static double restSecondsToday(Context context, long now) {
518         synchronized (LOCK) {
519             JSONArray log = readArray(prefs(context), KEY_LOCK_LOG);
520             TimeZone zone = TimeZone.getDefault();
```

restSecondsToday — 오늘 쉰 시간. 시간 잠금 기록마다 (시작 ~ 끝 또는 지금 중 이른 쪽)에서 **오늘에 해당하는 부분**만 더한다. 진행 중인 잠금도 지금까지만큼 포함된다.

```
            String today = GuardTime.dayKey(now, zone);
            double total = 0;
            for (int i = 0; i < log.length(); i++) {
                JSONObject row = log.optJSONObject(i);
                if (row == null) continue;
                long start = row.optLong("start");
                long end = Math.min(now, row.optLong("until"));
                Double part = GuardTime.splitByDay(start, end, zone).get(today);
                if (part != null) total += part;
            }
            return total;
        }
    }
```

```
    // --- Per-day counters (kept promises, "3분만 더") ---------------------------

    /** Adds one to today's counter and returns the new value. */
    public static int bumpToday(Context context, String name, long now) {
        synchronized (LOCK) {
            SharedPreferences prefs = prefs(context);
            String today = GuardTime.dayKey(now, TimeZone.getDefault());
            JSONObject counters = readObject(prefs, KEY_COUNTERS);
            if (!today.equals(counters.optString("date"))) counters = new JSONObject();
            int next = counters.optInt(name, 0) + 1;
            try {
                counters.put("date", today).put(name, next);
            } catch (JSONException ignored) {
                return next;
            }
            prefs.edit().putString(KEY_COUNTERS, counters.toString()).commit();
            return next;
        }
    }

    public static int countToday(Context context, String name, long now) {
        synchronized (LOCK) {
            JSONObject counters = readObject(prefs(context), KEY_COUNTERS);
            String today = GuardTime.dayKey(now, TimeZone.getDefault());
            return today.equals(counters.optString("date")) ? counters.optInt(name, 0) : 0;
        }
    }
```

하루 카운터 (약속 지킨 횟수, "3분만 더" 횟수 등).

- **bumpToday**: 오늘 날짜가 아니면 카운터를 새로 시작하고, 그 이름의 값을 1 올려 저장한 뒤 새 값을 돌려준다.
- **countToday**: 오늘 값 읽기 (날짜가 다르면 0).

```
    // --- Prompt handed to the web flow (Level 3, Instagram) --------------------

    public static void startPrompt(Context context, String target, int level, double usageSeconds) {
        synchronized (LOCK) {
            prefs(context).edit()
                .putString(KEY_PROMPT_TARGET, target)
                .putInt(KEY_PROMPT_LEVEL, level)
                .putFloat(KEY_PROMPT_USAGE, (float) usageSeconds)
```

앱 화면이 이어받는 개입 (Instagram 릴스, 예전 방식).

- **startPrompt**: 대상, 단계, 사용 초를 적어 둔다. 웹이 getStatus로 보고 개입 화면을 연다.
- **pendingTarget / Level / Usage**: 적어 둔 값 읽기.
- **isPrompting**: 그 앱에 대한 개입이 진행 중인지. **clearPrompt**: 지우기.

```
                .commit();
        }
    }

    public static String pendingTarget(Context context) {
        synchronized (LOCK) {
            String target = prefs(context).getString(KEY_PROMPT_TARGET, "");
            return target == null ? "" : target;
        }
    }

    public static int pendingLevel(Context context) {
        synchronized (LOCK) {
            return prefs(context).getInt(KEY_PROMPT_LEVEL, 0);
        }
    }

    public static double pendingUsage(Context context) {
        synchronized (LOCK) {
            return prefs(context).getFloat(KEY_PROMPT_USAGE, 0f);
        }
    }

    public static boolean isPrompting(Context context, String target) {
        return target != null && target.equals(pendingTarget(context));
    }

    public static void clearPrompt(Context context) {
        synchronized (LOCK) {
            prefs(context).edit().putString(KEY_PROMPT_TARGET, "").commit();
        }
    }
```

```
    /** After "watch" in the web flow, the next entry to this app's player is not stopped again.
*/
    public static void allow(Context context, String target, long until) {
        synchronized (LOCK) {
            prefs(context).edit()
                .putString(KEY_ALLOW_TARGET, target)
                .putLong(KEY_ALLOW_UNTIL, until)
                .commit();
        }
    }

    public static boolean consumeAllow(Context context, String target, long now) {
        synchronized (LOCK) {
            SharedPreferences prefs = prefs(context);
            boolean ok = target.equals(prefs.getString(KEY_ALLOW_TARGET, ""))
                && now < prefs.getLong(KEY_ALLOW_UNTIL, 0L);
            if (ok) prefs.edit().putString(KEY_ALLOW_TARGET, "").commit();
            return ok;
```

allow: 앱 화면에서 "보기"를 고르면, 그 앱 플레이어에 다시 들어갈 때 한 번은 멈추지 않게 허락을 적어 둔다 (기한 포함).

consumeAllow: 허락이 있고 기한 안이면 true를 돌려주고 허락을 지운다 (한 번만 쓰기).

```
        }
    }
```

```
    /** "기록 모두 지우기": usage, queues, block and prompt. Keeps the switch and thresholds. */
    public static void resetRecords(Context context) {
        synchronized (LOCK) {
            prefs(context).edit()
                .putString(KEY_DAY_USAGE, "")
                .putString(KEY_ACTIVE, "")
                .putString(KEY_SESSIONS, "")
                .putString(KEY_INTERVENTIONS, "")
                .putString(KEY_LOCKS, "")
                .putLong(KEY_BLOCKED_UNTIL, 0L)
                .putInt(KEY_BLOCK_MINUTES, 0)
                .putString(KEY_LOCK_LOG, "")
                .putString(KEY_COUNTERS, "")
                .putString(KEY_PROMPT_TARGET, "")
                .putString(KEY_ALLOW_TARGET, "")
                .putString(KEY_WEB_DATE, "")
                .putFloat(KEY_WEB_SECONDS, 0f)
                .commit();
            // Card review state goes with the records (cards themselves stay).
            SharedPreferences prefs = prefs(context);
            SharedPreferences.Editor editor = prefs.edit();
            for (String key : prefs.getAll().keySet()) {
                if (key.startsWith(KEY_CARD_LAST) || key.startsWith(KEY_CARD_STREAK)) editor.rem
ove(key);
            }
            editor.commit();
        }
    }
```

resetRecords — "기록 모두 지우기". 사용 시간, 열린 시청, 대기열 3개, 잠금, 잠금 기록, 카운터, 진행 중 개입, 허락, 웹 초를 비운다. 개입 스위치와 기준은 남긴다.

642~648줄 카드 복습 기록(`card_last_`, `card_streak_`로 시작하는 칸)도 모두 지운다. 카드 자체는 남는다.

```
    // --- JSON helpers ------------------------------------------------------------

    private static JSONObject readObject(SharedPreferences prefs, String key) {
        String raw = prefs.getString(key, "");
        if (raw == null || raw.isEmpty()) return new JSONObject();
        try {
            return new JSONObject(raw);
        } catch (JSONException e) {
            return new JSONObject();
        }
    }

    private static JSONArray readArray(SharedPreferences prefs, String key) {
        String raw = prefs.getString(key, "");
        if (raw == null || raw.isEmpty()) return new JSONArray();
        try {
            return new JSONArray(raw);
        } catch (JSONException e) {
            return new JSONArray();
```

readObject / readArray: 메모장의 글자를 JSON 객체/목록으로 읽는다. 비어 있거나 깨져 있으면 빈 것을 돌려줘서 앱이 멈추지 않는다.

```
        }
    }
```

```
    private static String append(SharedPreferences prefs, String key, JSONObject row) {
        JSONArray array = readArray(prefs, key);
        array.put(row);
        if (array.length() <= QUEUE_LIMIT) return array.toString();
        JSONArray trimmed = new JSONArray();
        for (int i = array.length() - QUEUE_LIMIT; i < array.length(); i++) {
            trimmed.put(array.opt(i));
        }
        return trimmed.toString();
    }
```

append: 대기열에 한 줄을 붙인다. 2000줄을 넘으면 **가장 오래된 것부터 버리고** 최근 2000줄만 남긴다.

```
    private static String without(SharedPreferences prefs, String key, Set<String> ids) {
        JSONArray array = readArray(prefs, key);
        JSONArray kept = new JSONArray();
        for (int i = 0; i < array.length(); i++) {
            JSONObject row = array.optJSONObject(i);
            if (row != null && !ids.contains(row.optString("id"))) kept.put(row);
        }
        return kept.toString();
    }
```

without: 목록에서 주어진 id들을 뺀 새 목록을 만든다 (ack에 사용).

```
    private static JSONObject pruneDays(JSONObject days) {
        List<String> keys = new ArrayList<>();
        Iterator<String> it = days.keys();
        while (it.hasNext()) keys.add(it.next());
        if (keys.size() <= DAYS_KEPT) return days;
        Collections.sort(keys);
        for (int i = 0; i < keys.size() - DAYS_KEPT; i++) days.remove(keys.get(i));
        return days;
    }
}
```

pruneDays: 날짜별 사용 표에서 날짜가 3개보다 많으면 날짜순으로 정렬해 오래된 것을 지운다. 날짜가 "2026-10-13" 형식이라 글자 순서가 곧 날짜 순서다.

GuardFlow — 개입 화면의 순서 `android/app/src/main/java/app/focuson/shorts/GuardFlow.java (869줄)`

개입 화면(시트) 하나 안에서 **어떤 단계를 어떤 순서로** 보여 줄지 정한다. 실제 그리기는 GuardOverlay, 문구는 GuardCopy가 맡는다. 끝나면 한 번의 개입 기록(Rec)을 만들어 대기열에 넣는다.

람다와 Runnable: 이 파일에는 `() -> 무엇무엇` 같은 코드가 많다. "나중에 실행할 짧은 작업"을 만드는 문법이다. 예: `Runnable next = () -> reason(...)`는 "다음 단계로 이유 고르기를 해라"라는 작업을 만들어 두는 것이다. 단계 함수들은 다음에 할 일(`next`)을 받아 두었다가 자기 단계가 끝나면 실행한다. 이렇게 단계를 줄줄이 이어 붙인다.

```
package app.focuson.shorts;

import android.content.Context;
import android.os.Handler;
```

3~19줄 화면 부품, 시간(`SystemClock`: 폰이 켜진 뒤 흐른 시간, 시계를 바꿔도 영향 없음), 달력, 난수, 고유 번호, JSON 도구.

```
import android.os.SystemClock;
import android.view.Gravity;
import android.view.View;
import android.widget.FrameLayout;
import android.widget.LinearLayout;
import android.widget.TextView;
import java.util.ArrayList;
import java.util.Calendar;
import java.util.List;
import java.util.Random;
import java.util.TimeZone;
import java.util.UUID;
import org.json.JSONArray;
import org.json.JSONException;
import org.json.JSONObject;

/**
 * Order of the steps inside one sheet. Intensity (band) comes from the policy table by
 * today's share of the goal; the method shown inside a band rotates (later: picked by the
 * AI phase through the same table). Wording comes from {@link GuardCopy}.
 *
 * Entry (light / medium): [after-lock question] → method → reason → commitment → play.
 * Commitment over: [band went up → that band's activity] → kept (leave) or extend.
 * Strong: day end / lock / "3분만 더" (through a nudge screen) / leave.
 * Locked ("쇼츠 쉬는 중"): remaining time, rest so far, optionally one study card.
 */
```

21~29줄 전체 흐름 요약:

- **진입(약함·중간)**: [잠금 뒤 질문] → 방식 → 이유 → 약속 → 재생
- **약속 끝**: [단계가 올라갔으면 그 단계 활동] → 약속 지키고 나가기 또는 연장
- **강함**: 오늘은 끝 / 잠금 / "3분만 더"(대기 화면) / 나가기
- **잠금 중**: 남은 시간, 오늘 쉰 시간, 공부 카드 한 문제

```
final class GuardFlow {
    interface Host {
        /** Resume playback and start a commitment timer. */
        void onWatch(String kind, int minutes, boolean overdraft, int extensionIndex, int band);

        void onLeave(String kind);

        void onBlock(String kind, long until, int minutes, boolean dayEnd);

        void record(JSONObject row);

        /** Numbers for the wording right now. */
        GuardCopy.Facts facts();

        /** True while the Shorts being handled plays in a picture-in-picture window. */
        boolean inPip();
    }
```

Host — 이 흐름이 서비스에 부탁하는 일의 약속. ShortsGuardService가 이 약속을 지킨다.

- **onWatch**: 보기로 했으니 재생을 이어 가고 약속 타이머를 시작해 줘 (분, 3분만 더인지, 몇 번째 연장인지, 단계).
- **onLeave**: 쇼츠에서 나가게 해 줘. **onBlock**: 이 시각까지 잠가 줘.
- **record**: 이 기록을 대기열에 넣어 줘. **facts**: 문구에 쓸 지금 숫자를 줘. **inPip**: 지금 작은 창으로 보는 중인지.

```
    private static final long PRAISE_MS = 1600L;
    private static final long CARD_RESULT_MS = 1200L;
    private static final int PAUSE_SEC = 5;
    private static final int ALTERNATIVE_SEC = 30;
    private static final int SKIP_AFTER_SEC = 10;
    private static final int BREATH_CYCLES = 3;
```

시간 상수: 칭찬 표시 1.6초, 카드 결과 표시 1.2초, 숨 고르기 5초, 대체 행동 30초, 건너뛰기는 10초 뒤부터, 호흡 3번(들이마시기 4초, 참기 2초, 내쉬기 4초), 잠금 화면 자동 닫힘 5초(카드가 있으면 20초), 피드백 질문 대기 4.5초.

```
    private static final long INHALE_MS = 4000L;
    private static final long HOLD_MS = 2000L;
    private static final long EXHALE_MS = 4000L;
    private static final int LOCK_AUTO_LEAVE_SEC = 5;
    private static final int LOCK_CARD_WAIT_SEC = 20;
    private static final long FEEDBACK_WAIT_MS = 4500L;
```

```
    private final Context context;
    private final GuardOverlay overlay;
    private final Handler handler;
    private final Host host;

    /** The record for the sheet on screen; written once, on whatever ends it. */
    private Rec open;
    /** Policy of the sheet on screen (null for the lock sheet). */
    private GuardPolicy openPolicy;
    private final Random random = new Random();

    GuardFlow(Context context, GuardOverlay overlay, Handler handler, Host host) {
        this.context = context;
        this.overlay = overlay;
        this.handler = handler;
        this.host = host;
    }
```

- **62~65줄** 받아 둔 도구: 안드로이드 정보(context), 시트 그리기(overlay), 시간 예약(handler), 서비스(host).
- **68줄** `open`: 지금 화면에 떠 있는 시트의 기록. 끝나는 순간 딱 한 번 저장한다.
- **70줄** `openPolicy`: 지금 시트의 정책표. **71줄** 밴딧용 난수 생성기.
- **73~78줄** 생성자: 도구들을 기억한다.

```
    // --- Entry points ------------------------------------------------------------

    void entry(String kind, double usage, GuardPolicy policy, boolean afterLock) {
        int band = policy.band(usage);
        Rec rec = begin(kind, "entry", band, usage, policy);
        overlay.leave("나가기", () -> finishExit(rec, "exit"));
        Runnable afterMethod = () -> reason(rec, () -> commitment(rec, policy, 0, "watch"));
        Runnable start = () -> method(rec, policy, band, afterMethod);
        if (afterLock) afterLock(rec, start);
        else start.run();
    }
```

entry — 약함·중간 단계 진입 (쇼츠를 열었을 때).

1. **83줄** 오늘 사용량으로 단계를 정한다.
2. **84줄** 시트를 열고 기록을 시작한다(`begin`).
3. **85줄** 아래 버튼은 "나가기" → 누르면 칭찬 후 나가기.
4. **86줄** 방식이 끝나면 할 일: 이유 고르기 → 그다음 약속 정하기.
5. **87~89줄** 잠금이 막 끝난 뒤면 "뭐 했어요?"를 먼저 묻고, 아니면 바로 방식부터.

```
    void commitmentOver(String kind, double usage, GuardPolicy policy, int minutes, int extensionIndex, int previousBand) {
        int band = policy.band(usage);
        if (band == GuardPolicy.BAND_STRONG) {
            strong(kind, usage, policy, "정한 시간이 끝났고, 오늘 목표도 다 썼어요.", false);
            return;
        }
        Rec rec = begin(kind, "commit_end", band, usage, policy);
        rec.extensionIndex = extensionIndex;
        boolean canExtend = !policy.ai.strict
            && !(policy.ai.isRiskHour(hourNow()) && extensionIndex + 1 > policy.ai.riskExtensionMax);

        Runnable choices = () -> {
            View body = null;
```

commitmentOver — 약속한 시간이 끝났을 때.

1. **93~97줄** 지금 단계가 강함이면 강함 화면으로 ("정한 시간이 끝났고, 오늘 목표도 다 썼어요.").
2. **98~99줄** 기록을 시작하고 몇 번째 연장인지 적는다.
3. **100~101줄** 연장할 수 있는지: 엄격 모드가 아니고, 위험 시간이면 연장 횟수 제한(1번) 안일 때만.
4. **102~112줄** 선택 화면: 연장 가능하면 "조금 더 볼래요" 버튼(누르면 다시 몇 분 고르기), 제목 "정한 N분이 됐어요", 약속 끝 문장, 아래 버튼 "약속 지키고 나가기".

```
            if (canExtend) {
                TextView more = overlay.button("조금 더 볼래요", false);
                more.setOnClickListener(v -> commitment(rec, policy, extensionIndex + 1, "exten
d"));
                body = wrap(more, 52);
            }
            String line = GuardCopy.commitmentOver(GuardState.nextIndex(context, "s2"), host.fac
ts());
            overlay.step("정한 " + minutes + "분이 됐어요", line, body);
            overlay.leave("약속 지키고 나가기", () -> finishExit(rec, "kept"));
        };
        overlay.leave("약속 지키고 나가기", () -> finishExit(rec, "kept"));
        if (band > previousBand) {
            // Crossed into the medium band while watching: its activity comes first.
            method(rec, policy, band, choices);
        } else {
            choices.run();
        }
    }
```

5. **114~119줄** 보는 사이 단계가 올라갔으면(약함 → 중간) 그 단계의 활동을 먼저 하고 선택 화면으로.

```
    void strong(String kind, double usage, GuardPolicy policy, String headline, boolean afterLoc
k) {
        Rec rec = begin(kind, "strong", GuardPolicy.BAND_STRONG, usage, policy);
        overlay.leave("지금은 나가기", () -> finishExit(rec, "exit"));
        Runnable show = () -> strongChoices(rec, policy, headline);
        if (afterLock) afterLock(rec, show);
        else show.run();
    }
```

strong — 강함 단계 (목표 초과). 기록을 시작하고, 아래 버튼은 "지금은 나가기". 잠금이 막 끝난 뒤면 질문을 먼저 하고 선택 화면(`strongChoices`)을 보여준다.

```
    void locked(String kind, long until) {
        Rec rec = begin(kind, "locked", GuardPolicy.BAND_STRONG, 0, null);
        rec.recorded = true; // The lock attempt row is written by the service.
        GuardCopy.Facts facts = host.facts();
        overlay.gauge(1, 0, GuardPolicy.BAND_STRONG);
        LinearLayout box = overlay.vertical();
        TextView remaining = overlay.text(GuardCopy.countdown(until - System.currentTimeMillis
()), 48, GuardOverlay.TEXT, true);
        remaining.setGravity(Gravity.CENTER);
        remaining.setPadding(0, overlay.dp(4), 0, overlay.dp(8));
        box.addView(remaining);
        overlay.step(GuardCopy.LOCKED_TITLE, GuardCopy.locked(until - System.currentTimeMillis
(), facts.restToday), box);
        overlay.leave("나가기", () -> leaveQuietly(kind));
```

locked — 잠금 중 화면 (1).

- **131~132줄** 기록을 만들되 이미 저장된 것으로 표시 (잠금 중 시도 기록은 서비스가 따로 쓴다).
- **134줄** 게이지를 꽉 찬 빨강으로.
- **135~139줄** 큰 카운트다운 숫자 (48sp, 가운데).
- **140줄** 제목 "쇼츠 쉬는 중", 본문 "N분 남았어요. 지금까지 오늘 M 쉬었어요."
- **141줄** 아래 "나가기" 버튼.

```
        JSONObject card = GuardState.nextCard(context, System.currentTimeMillis());
        boolean[] answering = {false};
        if (card != null) {
            // Turn the urge into one question while resting (rest time keeps counting).
            Rec cardRec = new Rec(kind, "locked", GuardPolicy.BAND_STRONG, 0, "");
            cardRec.method = "card";
            TextView prompt = overlay.text("이 문제 하나만 풀고 가요.", 14, GuardOverlay.MUTED, fa
```

locked (2) — 공부 카드 한 문제. 지금 나올 수 있는 카드가 있으면 "이 문제 하나만 풀고 가요."와 카드를 붙인다. 카드를 다 풀면 기록(결과 "잠금 중 다시 열기")을 남기고 조용히 나간다. 보기를 누르기 시작하면 `answering`을 true로 바꿔 자동 닫힘을 멈춘다.

```
lse);
            prompt.setPadding(0, overlay.dp(4), 0, overlay.dp(4));
            box.addView(prompt);
            View cardView = cardView(cardRec, card, () -> {
                cardRec.outcome = "blocked_retry";
                host.record(cardRec.toJson());
                leaveQuietly(kind);
            }, () -> answering[0] = true);
            if (cardView != null) box.addView(cardView);
        }
```

`boolean[] answering = {false}`처럼 칸 하나짜리 목록을 쓰는 이유: 람다 안에서는 바깥 변수를 바꿀 수 없어서, 목록 안의 값을 바꾸는 방법을 쓴다.

```
        // Leave by itself after a few seconds; with a card, give time to read it, and once
        // an option is tapped wait for the user to finish.
        long leaveAt = SystemClock.uptimeMillis() + (card == null ? LOCK_AUTO_LEAVE_SEC : LOCK_C
ARD_WAIT_SEC) * 1000L;
        overlay.startTicker(250, () -> {
            long left = until - System.currentTimeMillis();
            remaining.setText(GuardCopy.countdown(left));
            if (left <= 0 || (!answering[0] && SystemClock.uptimeMillis() >= leaveAt)) {
                leaveQuietly(kind);
                return false;
            }
            return true;
        });
    }
```

locked (3) — 카운트다운과 자동 닫힘. 0.25초마다 남은 시간을 갱신한다. 잠금이 끝났거나, 답하는 중이 아니고 기다린 시간(5초, 카드가 있으면 20초)이 지나면 조용히 나간다.

```
    /** The user left the player while a sheet was up: record it as an exit without a choice. */
    void abandon() {
        Rec rec = open;
        open = null;
        if (rec != null && !rec.recorded) {
            rec.outcome = "exit";
            write(rec);
        }
        overlay.dismissNow();
    }
```

abandon — 시트가 떠 있는데 사용자가 쇼츠를 떠났을 때 (홈 버튼 등). 아직 저장 안 된 기록이 있으면 결과를 "나감"으로 저장하고, 시트를 바로 없앤다.

```
    // --- Steps ---------------------------------------------------------------

    private void afterLock(Rec rec, Runnable next) {
        List<TextView> chips = new ArrayList<>();
        for (String answer : GuardCopy.AFTER_LOCK_ANSWERS) {
            TextView chip = overlay.chip(answer);
            chip.setOnClickListener(v -> {
                if (rec.afterLock != null) return;
                rec.afterLock = answer;
                overlay.tint(chip, GuardOverlay.CARD_ON);
                GuardState.markAfterLockAsked(context);
                handler.postDelayed(next, 200);
            });
            chips.add(chip);
```

afterLock — "30분 잘 쉬었어요. 그동안 뭐 했어요?" 답 3개(쉬었어요/공부·일/그냥 기다림)를 칩으로 만든다. 하나를 누르면 기록에 적고, 색을 바꾸고, "물어봤음"을 저장한 뒤 0.2초 뒤 다음 단계로. 이미 답했으면 두 번째 누름은 무시.

```
        }
        overlay.step(
            GuardCopy.afterLockQuestion(GuardState.blockMinutes(context)), null, overlay.grid(ch
ips, 3, 52));
    }
```

```
    private void method(Rec rec, GuardPolicy policy, int band, Runnable next) {
        boolean cardDue = GuardState.hasDueCard(context, System.currentTimeMillis());
        if (policy.studyMode && cardDue) {
            // 공부 모드: a due card replaces the activity. When every card is resting, fall
            // through to the usual activities so the same few cards do not repeat.
            rec.method = "card";
            card(rec, policy, next);
            return;
        }
        List<String> options = new ArrayList<>();
        for (String m : policy.methodsFor(band)) {
            if ("card".equals(m) && !cardDue) continue;
            if ("framing".equals(m) && rec.usage < 60) continue;
            if (isKnownMethod(m)) options.add(m);
        }
```

method — 개입 방식 고르기 (1).

- **204~211줄 공부 모드**이고 나올 카드가 있으면 바로 공부 카드. 카드가 모두 쉬는 중이면 아래 평소 방식으로 넘어간다 (같은 카드 반복 방지).
- **212~217줄** 이 단계에서 쓸 수 있는 방식 목록을 만든다. 나올 카드가 없으면 카드를 빼고, 오늘 1분 미만이면 시간 문구를 뺀다 (보여 줄 숫자가 없어서).

```
        String picked;
        if (policy.ai.enabled) {
            // AI: Thompson sampling over what worked for this band, time of day and
            // weekday/weekend, leaning away from a method shown very recently (habituation).
            Calendar now = Calendar.getInstance();
            picked = GuardBandit.pick(
                options,
                policy.ai.methodsAt(band, now),
                random,
                GuardState.lastShown(context, "method", options),
                now.getTimeInMillis(),
                policy.ai.habituationStrength,
                policy.ai.habituationRecoveryHours);
        } else {
            picked = GuardState.nextInTurn(context, "band" + band, options);
        }
        if (picked == null) picked = "pause";
        GuardState.markShown(context, "method", picked, System.currentTimeMillis());
        rec.method = picked;
```

method (2) — 하나 고르기.

- **AI 켜짐**: 밴딧(`GuardBandit.pick`)이 이 단계·시간대·평일주말 칸의 학습 표와, 방식별 마지막 표시 시각(질림 반영)을 보고 고른다.
- **AI 꺼짐**: 단계별로 차례대로(`nextInTurn`).
- **234~236줄** 못 골랐으면 숨 고르기. 고른 방식의 표시 시각을 저장하고 기록에 적는다.

```
        switch (picked) {
            case "framing":
                framing(rec, policy, next);
                break;
            case "breath":
                breath(rec, next);
                break;
            case "alternative":
                alternative(rec, next);
```

method (3): 고른 방식의 화면 함수로 이동한다 (`switch`). 모르는 값이면 숨 고르기.

isKnownMethod: 정책표에 이상한 방식 이름이 와도 무시하도록, 아는 6가지인지 확인.

```
                break;
            case "reflection":
                reflection(rec, next);
                break;
            case "card":
                card(rec, policy, next);
                break;
            default:
                pause(rec, policy, next);
        }
    }

    private static boolean isKnownMethod(String m) {
        return "pause".equals(m) || "framing".equals(m) || "breath".equals(m)
            || "alternative".equals(m) || "reflection".equals(m) || "card".equals(m);
    }
```

```
263     /** S1 line in turn, skipping kinds whose numbers are unknown right now. */
264     private String pickInsight(Rec rec, GuardPolicy policy) {
265         GuardCopy.Facts facts = host.facts();
266         List<String> kinds = policy.framings;
267         if (policy.ai.enabled) {
268             List<String> usable = new ArrayList<>();
269             for (String kind : kinds) {
270                 if (GuardCopy.insight(kind, facts) != null) usable.add(kind);
271             }
272             Calendar now = Calendar.getInstance();
273             String kind = GuardBandit.pick(
274                 usable,
275                 policy.ai.framingsAt(now),
276                 random,
277                 GuardState.lastShown(context, "framing", usable),
278                 now.getTimeInMillis(),
279                 policy.ai.habituationStrength,
280                 policy.ai.habituationRecoveryHours);
281             if (kind == null) return null;
282             GuardState.markShown(context, "framing", kind, now.getTimeInMillis());
283             rec.framing = kind;
284             return GuardCopy.insight(kind, facts);
285         }
286         int start = GuardState.nextIndex(context, "framing");
287         for (int i = 0; i < kinds.size(); i++) {
288             String kind = kinds.get(Math.floorMod(start + i, kinds.size()));
289             String line = GuardCopy.insight(kind, facts);
290             if (line != null) {
291                 rec.framing = kind;
292                 return line;
293             }
294         }
295         return null;
296     }
```

pickInsight — 시간 체감 문구 하나 고르기.

- **AI 켜짐 (267~285줄)**: 지금 숫자로 문장을 만들 수 있는 종류만 모아 (`usable`) 밴딧으로 고르고, 표시 시각과 종류를 기록한다.
- **AI 꺼짐 (286~294줄)**: 순서 번호부터 시작해 한 바퀴 돌며, 문장이 만들어지는 첫 종류를 쓴다 (숫자를 모르는 종류는 건너뜀).
- 쓸 문장이 없으면 null.

```
298     private void pause(Rec rec, GuardPolicy policy, Runnable next) {
299         rec.verification = "waited";
300         TextView count = overlay.text(String.valueOf(PAUSE_SEC), 64, GuardOverlay.TEXT, true);
301         count.setGravity(Gravity.CENTER);
302         overlay.step("한 박자만 쉬어 볼게요", pickInsight(rec, policy), count);
303         long endAt = SystemClock.uptimeMillis() + PAUSE_SEC * 1000L;
304         overlay.startTicker(250, () -> {
305             long left = endAt - SystemClock.uptimeMillis();
306             if (left <= 0) {
307                 next.run();
308                 return false;
309             }
310             count.setText(String.valueOf((left + 999) / 1000));
311             return true;
```

pause — 5초 숨 고르기. 제목 "한 박자만 쉬어 볼게요", 부제목은 시간 문구, 가운데 큰 숫자 5. 0.25초마다 남은 초를 표시하고 0이 되면 다음 단계로. 기록에는 "기다림(waited)".

```
        });
    }
```

```
    private void framing(Rec rec, GuardPolicy policy, Runnable next) {
        String line = pickInsight(rec, policy);
        if (line == null) {
            rec.method = "pause";
            pause(rec, policy, next);
            return;
        }
        rec.verification = "seen";
        LinearLayout box = overlay.vertical();
        TextView text = overlay.text(line, 20, GuardOverlay.TEXT, true);
        text.setPadding(0, overlay.dp(4), 0, overlay.dp(16));
        box.addView(text);
        TextView go = overlay.button("확인했어요", true);
        go.setOnClickListener(v -> next.run());
        box.addView(go, overlay.matchHeight(52));
        overlay.step("잠깐, 오늘 쓴 시간이에요", null, box);
    }
```

framing — 시간 체감 문구 화면. 문장이 없으면 숨 고르기로 바꾼다. 있으면 큰 글씨(20sp)로 보여 주고 "확인했어요" 버튼을 누르면 다음으로. 기록에는 "봤음(seen)".

```
    /** Guided breathing: text and a growing / shrinking circle, no pressing. */
    private void breath(Rec rec, Runnable next) {
        LinearLayout box = overlay.vertical();
        FrameLayout stage = new FrameLayout(context);
        View circle = new View(context);
        circle.setBackground(overlay.oval(GuardOverlay.CARD_ON));
        int size = overlay.dp(120);
        stage.addView(circle, new FrameLayout.LayoutParams(size, size, Gravity.CENTER));
        TextView cue = overlay.text("", 18, GuardOverlay.TEXT, true);
        cue.setGravity(Gravity.CENTER);
        stage.addView(cue, new FrameLayout.LayoutParams(size, size, Gravity.CENTER));
        box.addView(stage, overlay.matchHeight(150));
        TextView status = overlay.text("", 14, GuardOverlay.MUTED, false);
        status.setGravity(Gravity.CENTER);
        box.addView(status);
        TextView skip = overlay.text("", 13, GuardOverlay.MUTED, false);
        skip.setGravity(Gravity.CENTER);
        skip.setPadding(0, overlay.dp(10), 0, 0);
        box.addView(skip);

        overlay.step("숨 한 번 고르고 갈게요", "원을 따라 천천히 숨 쉬어 보세요. " + BREATH_CYCLES +
"번이에요.", box);
```

breath — 호흡 따라 하기 (1): 화면 만들기. 가운데 동그라미와 그 위 안내 글("들이마셔요"), 아래 "1 / 3" 진행 표시, 그 아래 건너뛰기 글. 제목 "숨 한 번 고르고 갈게요". 누르는 동작 없이 글과 원으로만 안내한다 (테스트 피드백 반영).

```
        long cycleMs = INHALE_MS + HOLD_MS + EXHALE_MS;
        long start = SystemClock.uptimeMillis();
        boolean[] done = {false};
        skip.setOnClickListener(v -> {
            if (done[0] || SystemClock.uptimeMillis() - start < SKIP_AFTER_SEC * 1000L) return;
            done[0] = true;
            rec.verification = "skipped";
```

breath (2): 건너뛰기. 한 주기 = 4 + 2 + 4 = 10초. 건너뛰기는 시작 10초 뒤부터만 눌린다. 누르면 기록에 "건너뜀(skipped)".

```
            next.run();
        });
```

```
        overlay.startTicker(100, () -> {
            if (done[0]) return false;
            long elapsed = SystemClock.uptimeMillis() - start;
            int cycle = (int) (elapsed / cycleMs);
            if (cycle >= BREATH_CYCLES) {
                done[0] = true;
                rec.verification = "waited";
                next.run();
                return false;
            }
            long in = elapsed % cycleMs;
            float phase;
            String text;
            if (in < INHALE_MS) {
                phase = in / (float) INHALE_MS;
                text = "들이마셔요";
            } else if (in < INHALE_MS + HOLD_MS) {
                phase = 1f;
                text = "참아요";
            } else {
                phase = 1f - (in - INHALE_MS - HOLD_MS) / (float) EXHALE_MS;
                text = "내쉬어요";
            }
            float scale = 0.55f + 0.45f * phase;
            circle.setScaleX(scale);
            circle.setScaleY(scale);
            cue.setText(text);
            status.setText((cycle + 1) + " / " + BREATH_CYCLES);
            long waitLeft = SKIP_AFTER_SEC * 1000L - elapsed;
            skip.setText(waitLeft > 0 ? "건너뛰기 (" + ((waitLeft + 999) / 1000) + ")" : "건너뛰
기");
            return true;
        });
    }
```

breath (3): 0.1초마다 움직임.

1. 지난 시간으로 몇 번째 주기인지 계산. 3주기를 다 하면 "기다림"으로 다음 단계.
2. 주기 안 위치(`in`)에 따라: 0~4초 원이 커지며 "들이마셔요", 4~6초 그대로 "참아요", 6~10초 작아지며 "내쉬어요".
3. 원 크기 = 55% + 45% × 진행 정도.
4. "N / 3"과 건너뛰기 남은 초를 갱신.

```
    private void alternative(Rec rec, Runnable next) {
        List<String> labels = new ArrayList<>();
        for (GuardCopy.Alternative alt : GuardCopy.ALTERNATIVES) labels.add(alt.label);
        String label = GuardState.nextInTurn(context, "alternative", labels);
        GuardCopy.Alternative alt = GuardCopy.ALTERNATIVES[Math.max(0, labels.indexOf(label))];
        rec.alternative = alt.label;

        LinearLayout box = overlay.vertical();
        TextView count = overlay.text(String.valueOf(ALTERNATIVE_SEC), 48, GuardOverlay.TEXT, tr
ue);
        count.setGravity(Gravity.CENTER);
        box.addView(count);
        TextView did = overlay.button("했어요 (" + ALTERNATIVE_SEC + ")", true);
```

alternative — 대체 행동 (1). 4가지 중 차례대로 하나를 고르고(물, 스트레칭, 걷기, 눈 쉬기) 기록에 적는다. 화면: 큰 30초 카운트다운, "했어요 (30)" 버튼, 건너뛰기. 제목은 행동 이름, 부제목은 안내 문장.

```
        LinearLayout.LayoutParams didLp = overlay.matchHeight(52);
        didLp.setMargins(0, overlay.dp(12), 0, 0);
        box.addView(did, didLp);
        TextView skip = overlay.text("", 13, GuardOverlay.MUTED, false);
        skip.setGravity(Gravity.CENTER);
        skip.setPadding(0, overlay.dp(10), 0, 0);
        box.addView(skip);
        overlay.step(alt.label, alt.guide, box);

```

```
        long start = SystemClock.uptimeMillis();
        boolean[] done = {false};
        did.setOnClickListener(v -> {
            if (done[0] || SystemClock.uptimeMillis() - start < ALTERNATIVE_SEC * 1000L) return;
            done[0] = true;
            rec.verification = "self";
            next.run();
        });
        skip.setOnClickListener(v -> {
            if (done[0] || SystemClock.uptimeMillis() - start < SKIP_AFTER_SEC * 1000L) return;
            done[0] = true;
            rec.verification = "skipped";
            next.run();
        });
        overlay.startTicker(250, () -> {
            if (done[0]) return false;
            long elapsed = SystemClock.uptimeMillis() - start;
            long left = ALTERNATIVE_SEC * 1000L - elapsed;
            count.setText(String.valueOf(Math.max(0, (left + 999) / 1000)));
            did.setText(left > 0 ? "했어요 (" + ((left + 999) / 1000) + ")" : "했어요");
            long skipLeft = SKIP_AFTER_SEC * 1000L - elapsed;
            skip.setText(skipLeft > 0 ? "건너뛰기 (" + ((skipLeft + 999) / 1000) + ")" : "건너뛰
기");
            return true;
        });
    }
```

alternative (2). "했어요"는 30초가 지나야 눌린다 (기록: 스스로 답함 self). 건너뛰기는 10초 뒤부터 (기록: skipped). 0.25초마다 남은 초를 버튼과 글자에 표시한다.

```
    private void reflection(Rec rec, Runnable next) {
        List<TextView> chips = new ArrayList<>();
        for (String answer : GuardCopy.REFLECTION_ANSWERS) {
            TextView chip = overlay.chip(answer);
            chip.setOnClickListener(v -> {
                if (rec.reflection != null) return;
                rec.reflection = answer;
                rec.verification = "answered";
                overlay.tint(chip, GuardOverlay.CARD_ON);
                handler.postDelayed(next, 200);
            });
            chips.add(chip);
        }
        overlay.step(GuardCopy.REFLECTION_QUESTION, "솔직하게 하나만 골라 보세요.", overlay.grid(c
```

reflection — 성찰 질문. "지금 보면 뭐가 남을까요?"와 객관식 4개(2칸씩). 누르면 기록하고(answered) 0.2초 뒤 다음으로.

```
hips, 2, 52));
    }
```

```
    private void card(Rec rec, GuardPolicy policy, Runnable next) {
        JSONObject card = GuardState.nextCard(context, System.currentTimeMillis());
        View view = card == null ? null : cardView(rec, card, next, null);
        if (view == null) {
            rec.method = "pause";
            pause(rec, policy, next);
            return;
        }
        overlay.step("한 문제만 풀고 가요", card.optString("question", ""), view);
    }
```

card — 공부 카드 단계. 지금 나올 카드를 가져와 보기 화면을 만든다. 카드가 없거나 쓸 수 없으면 숨 고르기로 바꾼다. 제목 "한 문제만 풀고 가요", 부제목은 문제.

```
    /**
     * Options of one study card. After a choice: colours, then the explanation (if any)
     * with a "계속" button, or a short pause without one.
     * @return null when the card is unusable
     */
    private View cardView(Rec rec, JSONObject card, Runnable next, Runnable onAnswering) {
        JSONArray options = card.optJSONArray("options");
        if (options == null || options.length() < 2) return null;
        rec.cardId = card.optString("id", null);
        int answer = card.optInt("answer", -1);
        String explanation = card.optString("explanation", "").trim();
        LinearLayout box = overlay.vertical();
        if (onAnswering != null) {
            TextView question = overlay.text(card.optString("question", ""), 16, GuardOverlay.TE
XT, true);
            question.setPadding(0, overlay.dp(4), 0, overlay.dp(4));
            box.addView(question);
        }
        List<TextView> buttons = new ArrayList<>();
        boolean[] answered = {false};
        TextView note = overlay.text("", 14, GuardOverlay.MUTED, false);
        note.setPadding(0, overlay.dp(8), 0, 0);
        note.setVisibility(View.GONE);
        TextView go = overlay.button("계속", true);
        go.setVisibility(View.GONE);
        go.setOnClickListener(v -> next.run());
```

cardView — 카드 보기 화면 (1): 준비.

- 보기가 2개 미만이면 null (쓸 수 없는 카드).
- 카드 id, 정답 번호, 해설을 꺼낸다.
- 잠금 화면에서 쓸 때(`onAnswering`이 있을 때)는 문제를 본문에 직접 쓴다 (제목 자리가 "쇼츠 쉬는 중"이라서).
- 해설 글 칸과 "계속" 버튼을 만들어 숨겨 둔다.

```
        for (int i = 0; i < options.length(); i++) {
            int index = i;
            TextView option = overlay.chip(options.optString(i, ""));
            option.setGravity(Gravity.CENTER_VERTICAL);
            option.setPadding(overlay.dp(14), 0, overlay.dp(14), 0);
            option.setOnClickListener(v -> {
                if (answered[0]) return;
                answered[0] = true;
                if (onAnswering != null) onAnswering.run();
                rec.cardCorrect = index == answer;
```

cardView (2): 보기 버튼. 보기마다 칩을 만든다. 하나를 누르면:

1. 두 번 누름 방지, 잠금 화면이면 "답하는 중" 알림.
2. 정답 여부를 기록하고, **간격 복습 기록**(`recordCardAnswer`)을 저장.
3. 누른 보기를 초록(정답) 또는 빨강(오답)으로. 틀렸으면 정답 보기도 초록으로 보여 준다.
4. 해설이 없으면 1.2초 뒤 다음으로, 있으면 "정답이에요. / 아쉬워요. + 해설"과 "계속" 버튼을 보여 준다.

```
                rec.verification = "answered";
                GuardState.recordCardAnswer(context, rec.cardId, rec.cardCorrect, System.current
TimeMillis());
                overlay.tint(option, rec.cardCorrect ? GuardOverlay.GOOD : GuardOverlay.BAD);
                if (!rec.cardCorrect && answer >= 0 && answer < buttons.size()) {
                    overlay.tint(buttons.get(answer), GuardOverlay.GOOD);
                }
                if (explanation.isEmpty()) {
                    handler.postDelayed(next, CARD_RESULT_MS);
                } else {
                    note.setText((rec.cardCorrect ? "정답이에요. " : "아쉬워요. ") + explanation);
                    note.setVisibility(View.VISIBLE);
                    go.setVisibility(View.VISIBLE);
                }
            });
            buttons.add(option);
        }
```

```
        box.addView(overlay.grid(buttons, 1, 52));
        box.addView(note);
        LinearLayout.LayoutParams goLp = overlay.matchHeight(48);
        goLp.setMargins(0, overlay.dp(10), 0, 0);
        box.addView(go, goLp);
        // "이 문제 이상해요": drop the card here and in the app, then move on.
        TextView report = overlay.text(GuardCopy.CARD_REPORT, 13, GuardOverlay.MUTED, false);
        report.setPadding(0, overlay.dp(12), 0, overlay.dp(4));
        report.setOnClickListener(v -> {
            if (report.getTag() != null) return;
            report.setTag("reported");
            answered[0] = true;
            if (onAnswering != null) onAnswering.run();
            GuardState.removeCard(context, rec.cardId);
            host.record(cardReportRow(rec));
            go.setVisibility(View.GONE);
            note.setText(GuardCopy.CARD_REPORTED);
            note.setVisibility(View.VISIBLE);
            report.setVisibility(View.GONE);
            handler.postDelayed(next, CARD_RESULT_MS + 600);
        });
        box.addView(report);
        return box;
    }
```

cardView (3): 배치와 "이 문제 이상해요". 보기를 한 줄에 하나씩, 그 아래 해설과 계속 버튼, 맨 아래 작은 "이 문제 이상해요". 누르면 (한 번만):

1. 더 답할 수 없게 하고, 이 카드를 휴대폰 쪽 목록에서 바로 뺀다 (`removeCard`).
2. 신고 기록(`cardReportRow`)을 보내서 앱에서도 지우게 한다.
3. "이 문제는 빼 둘게요. 다시 나오지 않아요."를 보여 주고 1.8초 뒤 다음으로.

```
    private void reason(Rec rec, Runnable next) {
        List<TextView> chips = new ArrayList<>();
        for (String reason : GuardCopy.REASONS) {
            TextView chip = overlay.chip(reason);
            chip.setOnClickListener(v -> {
                if (rec.reason != null) return;
                rec.reason = reason;
                overlay.tint(chip, GuardOverlay.CARD_ON);
```

reason — "왜 쇼츠를 보려고 하나요?" 이유 6개를 2칸씩 칩으로. 누르면 기록하고 0.15초 뒤 다음(약속 정하기)으로.

```
                handler.postDelayed(next, 150);
            });
            chips.add(chip);
        }
        overlay.step("왜 쇼츠를 보려고 하나요?", null, overlay.grid(chips, 2, 56));
    }

    /** "몇 분 볼래요?" within the remaining budget and the extension ladder. */
    private void commitment(Rec rec, GuardPolicy policy, int extensionIndex, String outcome) {
        List<Integer> choices = policy.commitChoices(policy.remainingSeconds(rec.usage), extensionIndex);
        int hour = hourNow();
        boolean risky = policy.ai.isRiskHour(hour);
        if (risky && !choices.isEmpty()) {
            // AI: in a risky hour the longest choice is shorter.
            List<Integer> capped = new ArrayList<>();
            for (int c : choices) if (c <= policy.ai.riskCommitMax) capped.add(c);
            choices = capped.isEmpty() ? java.util.Collections.singletonList(choices.get(0)) : capped;
        }
        boolean reserve = !risky && extensionIndex == 0 && policy.ai.beforeRisk(hour) && !policy.ai.riskLabel.isEmpty();
        if (choices.isEmpty()) {
            rec.outcome = "budget_out";
            write(rec);
            strong(rec.kind, rec.usage, policy, "오늘 목표를 다 썼어요.", false);
            return;
        }

        List<TextView> chips = new ArrayList<>();
        for (int minutes : choices) {
            boolean suggested = reserve && minutes == choices.get(0);
            TextView chip = overlay.chip(minutes + "분" + (suggested ? " · 추천" : ""));
            if (suggested) overlay.tint(chip, GuardOverlay.CARD_ON);
            chip.setOnClickListener(v -> {
                if (rec.recorded) return;
                rec.commitMinutes = minutes;
                rec.outcome = outcome;
                write(rec);
                overlay.close(() -> host.onWatch(rec.kind, minutes, false, extensionIndex, rec.brand));
            });
            chips.add(chip);
        }
        String subtitle = reserve
            ? GuardCopy.reserveHint(policy.ai.riskLabel)
            : "오늘 남은 시간 " + GuardCopy.clock(policy.remainingSeconds(rec.usage))
                + ". 정한 시간이 되면 멈춰 드려요.";
        overlay.step(extensionIndex > 0 ? "몇 분만 더 볼래요?" : "몇 분 볼래요?", subtitle, overlay.grid(chips, 3, 56));
    }
```

commitment — "몇 분 볼래요?" (1): 선택지 정하기.

- **564줄** 남은 예산과 연장 순서로 선택지(3/5/10)를 만든다 (GuardPolicy.commitChoices).
- **565~572줄 AI 위험 시간**이면 최대 3분까지만 남긴다 (다 빠지면 가장 짧은 것 하나).
- **573줄** 위험 시간 전이고 첫 약속이면 "남겨 두기 추천"을 켠다.
- **574~579줄** 선택지가 하나도 없으면(예산 소진) 기록을 "예산 끝"으로 저장하고 강함 화면으로.

commitment (2): 화면. 분마다 칩("3분", 추천이면 "3분 · 추천" + 강조 색). 누르면 약속 분과 결과(보기/연장)를 기록하고, 시트를 닫은 뒤 서비스에 "N분 보기 시작"(`onWatch`)을 알린다. 부제목은 추천 문장 또는 "오늘 남은 시간 N. 정한 시간이 되면 멈춰 드려요." 제목은 첫 약속이면 "몇 분 볼래요?", 연장이면 "몇 분만 더 볼래요?".

```
    private void strongChoices(Rec rec, GuardPolicy policy, String headline) {
        long now = System.currentTimeMillis();
        long dayEnd = policy.nextDayEnd(now, TimeZone.getDefault());
        LinearLayout box = overlay.vertical();

        TextView end = overlay.button("오늘은 끝 · 내일 " + policy.dayEndHour + "시까지", true);
        end.setOnClickListener(v -> {
            int minutes = (int) Math.ceil((dayEnd - System.currentTimeMillis()) / 60000.0);
            rec.blockMinutes = minutes;
            praiseThen(rec, "day_end", () -> host.onBlock(rec.kind, dayEnd, minutes, true));
        });
        box.addView(end, overlay.matchHeight(56));
```

strongChoices — 강함 단계 선택 화면 (1). "오늘은 끝"이 풀리는 시각을 계산하고, 가장 큰 주 버튼 "오늘은 끝 · 내일 6시까지"를 만든다. 누르면 잠글 분을 계산해 기록하고 칭찬 후 잠금(`onBlock`, 오늘은 끝 표시).

```
        List<TextView> locks = new ArrayList<>();
        for (int minutes : new int[] {10, 20, 30, 60}) {
            TextView chip = overlay.chip(minutes == 60 ? "1시간 잠금" : minutes + "분 잠금");
            chip.setOnClickListener(v -> {
                rec.blockMinutes = minutes;
                long until = System.currentTimeMillis() + minutes * 60_000L;
                praiseThen(rec, "lock", () -> host.onBlock(rec.kind, until, minutes, false));
            });
            locks.add(chip);
        }
        LinearLayout lockGrid = overlay.grid(locks, 4, 48);
        LinearLayout.LayoutParams lockLp = GuardOverlay.matchWrap();
        lockLp.setMargins(0, overlay.dp(8), 0, 0);
        box.addView(lockGrid, lockLp);
```

strongChoices (2): 잠금 칩 4개 "10분 잠금", "20분 잠금", "30분 잠금", "1시간 잠금". 누르면 칭찬 후 그 시간만큼 잠근다.

```
        TextView more = overlay.text(policy.overdraftMinutes + "분만 더", 14, GuardOverlay.MUTED,
false);
        more.setGravity(Gravity.CENTER);
        more.setPadding(0, overlay.dp(14), 0, 0);
        more.setClickable(true);
        more.setOnClickListener(v -> {
            if (!rec.recorded) overdraftNudge(rec, policy);
        });
        if (!policy.ai.strict) box.addView(more);

        String body = GuardCopy.strong(GuardState.nextIndex(context, "s3"), host.facts());
        overlay.step(GuardCopy.STRONG_TITLE, headline == null ? body : headline + " " + body, bo
x);
    }
```

strongChoices (3): "3분만 더". 일부러 작고 흐린 글자로 만든다 (덜 눈에 띄게). 누르면 대기 화면(`overdraftNudge`)으로. **엄격 모드면 아예 붙이지 않는다.** 제목 "오늘은 여기까지 해볼까요?", 본문은 앞 문장(있으면) + 강함 문장.

```
    /** S4: every "3분만 더" goes through a line that changes with the count. */
    private void overdraftNudge(Rec rec, GuardPolicy policy) {
        int count = GuardState.countToday(context, "overdraft", System.currentTimeMillis()) + 1;
        LinearLayout box = overlay.vertical();
        TextView stop = overlay.button("여기서 그만할래요", true);
        stop.setOnClickListener(v -> finishExit(rec, "exit"));
        box.addView(stop, overlay.matchHeight(56));
        TextView watch = overlay.text("", 14, GuardOverlay.MUTED, false);
```

overdraftNudge — "3분만 더" 대기 화면 (1).

- 오늘 몇 번째 "3분만 더"인지 센다.
- 큰 주 버튼 "여기서 그만할래요" (누르면 칭찬 후 나가기).
- 작은 글자 버튼 "그래도 3분 볼래요"는 **3초(위험 시간이면 10초) 뒤에야** 눌린다. 누르면 횟수를 올리고, "3분만 더"로 기록하고, 시트를 닫고 3분 보기 시작.

```
        watch.setGravity(Gravity.CENTER);
        watch.setPadding(0, overlay.dp(14), 0, 0);
        watch.setClickable(true);
        box.addView(watch);
        int waitSec = policy.ai.isRiskHour(hourNow()) ? policy.ai.riskOverdraftWaitSec : policy.
overdraftWaitSec;
        long enableAt = SystemClock.uptimeMillis() + waitSec * 1000L;
        String label = "그래도 " + policy.overdraftMinutes + "분 볼래요";
        watch.setOnClickListener(v -> {
            if (rec.recorded || SystemClock.uptimeMillis() < enableAt) return;
            GuardState.bumpToday(context, "overdraft", System.currentTimeMillis());
            rec.outcome = "overdraft";
            rec.commitMinutes = policy.overdraftMinutes;
            write(rec);
            overlay.close(() -> host.onWatch(rec.kind, policy.overdraftMinutes, true, 0, GuardPo
licy.BAND_STRONG));
        });
        overlay.step(
            GuardCopy.overdraftTitle(count, policy.overdraftMinutes),
            GuardCopy.overdraft(count, policy.overdraftMinutes, host.facts()),
            box);
```

- 제목 "오늘 N번째 3분이에요", 본문은 횟수별 문장.

```
        overlay.startTicker(250, () -> {
            long left = enableAt - SystemClock.uptimeMillis();
            watch.setText(left > 0 ? label + " (" + ((left + 999) / 1000) + ")" : label);
            return left > 0;
        });
    }
```

overdraftNudge (2): 0.25초마다 남은 대기 초를 "그래도 3분 볼래요 (2)"처럼 표시하고, 다 기다리면 타이머를 멈춘다.

```
    // --- Endings ----------------------------------------------------------------

    private void finishExit(Rec rec, String outcome) {
        praiseThen(rec, outcome, () -> host.onLeave(rec.kind));
    }

    /** Records the choice, shows a short praise line (S5), then slides away. */
    private void praiseThen(Rec rec, String outcome, Runnable after) {
        if (rec.recorded) return;
        rec.outcome = outcome;
        write(rec);
        if ("kept".equals(outcome)) GuardState.bumpToday(context, "kept", System.currentTimeMill
is());
        GuardCopy.Facts facts = host.facts();
        String line = GuardCopy.praise(outcome, GuardState.nextIndex(context, "praise_" + outcom
e), facts, rec.blockMinutes);
        String guide = host.inPip() ? "\n" + GuardCopy.PIP_CLOSE_GUIDE : "";
        overlay.leave(null, null);
        GuardPolicy policy = openPolicy;
        boolean askFeedback = policy != null && policy.ai.enabled
            && Math.floorMod(GuardState.nextIndex(context, "feedback"), policy.ai.feedbackEvery)
== 0;
```

finishExit: 결과를 정해 칭찬 후 나가기.

praiseThen — 기록 저장 → 칭찬 → 닫기 (1).

1. 이미 저장했으면 끝 (두 번 저장 방지). 결과를 적고 저장.
2. 약속을 지켰으면 오늘 "지킨 횟수"를 올린다.
3. 칭찬 문장을 고르고, 작은 창이면 닫는 법 안내를 덧붙인다. 아래 버튼은 숨긴다.
4. **693~694줄** AI가 켜져 있고 5번에 1번 차례면 피드백 질문을 한다.
5. **695~698줄** 아니면 "잘 멈췄어요" + 칭찬을 1.6초(안내가 있으면 2.8초) 보여주고 닫는다.

```
        if (!askFeedback) {
            overlay.step("잘 멈췄어요", line + guide, null);
            handler.postDelayed(() -> overlay.close(after), PRAISE_MS + (guide.isEmpty() ? 0 : 1
200));
            return;
        }
```

```
        // Every few times: was this sheet any help? (reward signal for the AI phase)
        boolean[] done = {false};
        Runnable finish = () -> {
            if (done[0]) return;
            done[0] = true;
            overlay.close(after);
        };
        List<TextView> chips = new ArrayList<>();
        for (String answer : new String[] {GuardCopy.FEEDBACK_YES, GuardCopy.FEEDBACK_NO}) {
            TextView chip = overlay.chip(answer);
            chip.setOnClickListener(v -> {
                if (done[0]) return;
                overlay.tint(chip, GuardOverlay.CARD_ON);
                host.record(feedbackRow(rec, GuardCopy.FEEDBACK_YES.equals(answer) ? "helpful" :
"not"));
                handler.postDelayed(finish, 300);
            });
            chips.add(chip);
        }
        overlay.step("잘 멈췄어요", line + guide + "\n\n" + GuardCopy.FEEDBACK_QUESTION, overlay.
grid(chips, 2, 48));
        handler.postDelayed(finish, FEEDBACK_WAIT_MS);
    }
```

praiseThen (2) — "방금 화면, 멈추는 데 도움이 됐나요?" 칩 2개(도움 됐어요/아니에요). 누르면 피드백 기록을 보내고 0.3초 뒤 닫는다. 아무것도 안 눌러도 4.5초 뒤 저절로 닫힌다 (`done`으로 두 번 닫힘 방지).

```
    private static JSONObject feedbackRow(Rec rec, String feedback) {
        JSONObject row = new JSONObject();
        try {
            row.put("id", UUID.randomUUID().toString())
                .put("createdAt", System.currentTimeMillis())
                .put("target", rec.kind)
                .put("level", rec.band)
                .put("stage", "feedback")
                .put("refId", rec.id)
                .put("feedback", feedback)
                .put("reason", "선택하지 않음")
                .put("interventionType", "feedback")
                .put("outcome", "feedback")
                .put("usageSeconds", 0);
        } catch (JSONException ignored) {
            // Plain values only.
        }
        return row;
    }
```

feedbackRow: 피드백 기록 한 줄. 단계 "feedback", 어느 개입에 대한 답인지 (`refId` = 그 개입의 id), 답("helpful"/"not"). 웹이 이걸 받아 해당 개입 기록에 붙인다.

```
    private static JSONObject cardReportRow(Rec rec) {
        JSONObject row = new JSONObject();
        try {
            row.put("id", UUID.randomUUID().toString())
                .put("createdAt", System.currentTimeMillis())
                .put("target", rec.kind)
                .put("level", rec.band)
                .put("stage", "card_report")
                .put("cardId", rec.cardId)
                .put("reason", "선택하지 않음")
                .put("interventionType", "card")
                .put("outcome", "card_report")
                .put("usageSeconds", 0);
        } catch (JSONException ignored) {
            // Plain values only.
        }
        return row;
    }

    private static int hourNow() {
        return Calendar.getInstance().get(Calendar.HOUR_OF_DAY);
    }

    private void leaveQuietly(String kind) {
        overlay.close(() -> host.onLeave(kind));
    }
```

cardReportRow: "이상한 문제" 신고 기록 한 줄 (단계 "card_report", 카드 id). 웹이 받아서 카드를 지운다.

hourNow: 지금 몇 시(0~23). **leaveQuietly**: 칭찬 없이 시트를 닫고 나가기.

```
    // --- Records -----------------------------------------------------------

    private Rec begin(String kind, String stage, int band, double usage, GuardPolicy policy) {
        if (open != null && !open.recorded) {
            open.outcome = "replaced";
            write(open);
        }
        overlay.open();
        openPolicy = policy;
        String notice = host.inPip() ? GuardCopy.PIP_NOTE : null;
        if (notice == null && policy != null && policy.ai.isRiskHour(hourNow()) && !policy.ai.riskLabel.isEmpty()) {
            notice = GuardCopy.aiRiskNotice(policy.ai.riskLabel);
        }
        overlay.notice(notice);
        if (policy != null) overlay.gauge(policy.ratio(usage), policy.remainingSeconds(usage), band);
        Rec rec = new Rec(kind, stage, band, usage, policy == null ? "" : policy.version);
        open = rec;
        return rec;
    }
```

begin — 시트 열고 기록 시작.

1. 이전 시트 기록이 저장 안 된 채 남아 있으면 "대체됨(replaced)"으로 저장한다 (기록이 사라지지 않게).
2. 시트를 열고 정책표를 기억한다.
3. 위 안내 줄: 작은 창이면 "작은 창으로 보던 쇼츠예요", 아니고 AI 위험 시간이면 AI 안내.
4. 게이지를 그리고, 새 기록(Rec)을 만들어 `open`에 둔다.

```
    private void write(Rec rec) {
        if (rec.recorded) return;
        rec.recorded = true;
        host.record(rec.toJson());
    }
```

write: 기록을 **딱 한 번만** 저장한다 (`recorded` 표시). 서비스를 통해 대기열로 간다.

```
    /** One sheet's worth of data for the interventions table (AI training material). */
    static final class Rec {
        final String id = UUID.randomUUID().toString();
        final long createdAt = System.currentTimeMillis();
        final String kind;
        final String stage;
        final int band;
        final double usage;
        final String policyVersion;
        String method;
        String framing;
        String verification;
        String reason;
        String reflection;
        String afterLock;
        String alternative;
        String cardId;
        Boolean cardCorrect;
        int commitMinutes;
        int blockMinutes;
        int extensionIndex;
        String outcome;
        boolean recorded;

        Rec(String kind, String stage, int band, double usage, String policyVersion) {
            this.kind = kind;
            this.stage = stage;
            this.band = band;
            this.usage = usage;
            this.policyVersion = policyVersion;
        }
```

Rec — 개입 한 번의 기록 상자 (AI 학습 재료). 만들 때 고유 id와 시각이 자동으로 정해진다.

담는 것: 대상 앱, 단계(entry/commit_end/strong/locked), 강도, 사용 초, 정책 버전, 방식, 문구 종류, 참여 확인 방법, 이유, 성찰 답, 잠금 뒤 답, 대체 행동, 카드 id와 정답 여부, 약속 분, 잠금 분, 연장 순서, 결과, 저장 여부.

```
        JSONObject toJson() {
            JSONObject row = new JSONObject();
            try {
                String type = method != null && ("entry".equals(stage) || "locked".equals(stage)) ? method : stage;
                row.put("id", id)
                    .put("createdAt", createdAt)
                    .put("target", kind)
                    .put("level", band)
                    .put("band", band)
                    .put("stage", stage)
                    .put("reason", reason == null ? "선택하지 않음" : reason)
                    .put("interventionType", type)
```

toJson: 기록 상자를 JSON 한 줄로 바꾼다. 개입 종류(`interventionType`)는 진입이나 잠금 화면이면 고른 방식, 아니면 단계 이름. 값이 없으면 JSON의 null로 넣는다(`opt`). 이유가 없으면 "선택하지 않음", 결과가 없으면 "exit".

```
                    .put("method", opt(method))
                    .put("framing", opt(framing))
                    .put("verification", opt(verification))
                    .put("reflection", opt(reflection))
                    .put("afterLock", opt(afterLock))
                    .put("alternativeAction", opt(alternative))
                    .put("cardId", opt(cardId))
                    .put("cardCorrect", cardCorrect == null ? JSONObject.NULL : cardCorrect)
                    .put("commitMinutes", commitMinutes)
                    .put("blockMinutes", blockMinutes)
                    .put("extensionIndex", extensionIndex)
                    .put("outcome", outcome == null ? "exit" : outcome)
                    .put("usageSeconds", usage)
                    .put("policyVersion", policyVersion);
            } catch (JSONException ignored) {
                // Fields are plain values; nothing to recover.
            }
            return row;
        }

        private static Object opt(String value) {
            return value == null ? JSONObject.NULL : value;
        }
    }
```

```
    private View wrap(View view, int heightDp) {
        LinearLayout box = overlay.vertical();
        box.addView(view, overlay.matchHeight(heightDp));
        return box;
    }
}
```

wrap: 버튼 하나를 정해진 높이의 상자에 담는다 ("조금 더 볼래요" 버튼용).

ShortsGuardService — 앱의 심장 (접근성 서비스) `android/app/src/main/java/app/focuson/shorts/ShortsGuardService.java (935줄)`

안드로이드가 백그라운드에서 계속 살려 두는 **접근성 서비스** 본체. 화면이 바뀔 때마다 신호를 받아 쇼츠인지 판단하고, 영상을 멈추고, 개입 화면을 띄우고, 약속 타이머와 잠금을 관리한다. 사용자가 폰 설정에서 직접 켜야 동작한다.

```
package app.focuson.shorts;

import android.accessibilityservice.AccessibilityService;
import android.app.AlarmManager;
import android.content.Context;
import android.content.Intent;
import android.content.pm.ApplicationInfo;
import android.media.AudioAttributes;
import android.media.AudioFocusRequest;
import android.media.AudioManager;
import android.net.Uri;
```

3~30줄 가져오는 도구: 접근성 서비스, 알람 정보(`AlarmManager`, 수면 문구용), 소리 제어(`AudioManager`, `AudioFocusRequest`), 화면 꺼짐 확인(`PowerManager`), 로그(`Log`), 화면 구조 정보(`AccessibilityNodeInfo`, `AccessibilityWindowInfo`), 키보드 목록(`InputMethodManager`), 짧은 안내 팝업(`Toast`), 약한 참조(`WeakReference`).

```
12 import android.os.Build;
13 import android.os.Handler;
14 import android.os.Looper;
15 import android.os.PowerManager;
16 import android.os.SystemClock;
17 import android.util.Log;
18 import android.view.accessibility.AccessibilityEvent;
19 import android.view.accessibility.AccessibilityNodeInfo;
20 import android.view.accessibility.AccessibilityWindowInfo;
21 import android.view.inputmethod.InputMethodInfo;
22 import android.view.inputmethod.InputMethodManager;
23 import android.widget.Toast;
24 import java.util.Calendar;
25 import java.lang.ref.WeakReference;
26 import java.util.ArrayList;
27 import java.util.HashSet;
28 import java.util.List;
29 import java.util.Set;
30 import org.json.JSONObject;
31
```

```
32 /**
33  * Notices the YouTube Shorts player and the Instagram Reels player.
34  * - Locked: lock sheet over the player, then leave.
35  * - YouTube: pause (audio focus + mute + media key), then a bottom sheet over the player
36  *   whose steps come from {@link GuardFlow} and the policy table. A chosen watch time
37  *   ("몇 분 볼래요?") is timed; when it is over the sheet comes back.
38  * - Instagram: the web flow in ShortsAI, as before.
39  * Watch time is recorded per stay with real start and end times.
40  * Resource ids and package names only. Screen text is not read or stored.
41  */
42 public class ShortsGuardService extends AccessibilityService {
43     /** adb logcat -s ShortsAIGuard. Debuggable builds only; view ids and states, no screen tex
t. */
44     static final String TAG = "ShortsAIGuard";
```

32~41줄 하는 일 요약: 잠금 중이면 잠금 화면, YouTube면 멈추고 시트(단계는 GuardFlow와 정책표가 정함), 고른 시청 시간이 끝나면 다시 시트, Instagram은 예전처럼 앱 화면. 시청 시간은 실제 시작·끝 시각으로 기록. 화면 이름표와 패키지 이름만 보고 화면 글자는 읽지 않는다.

42줄 `extends AccessibilityService`: 안드로이드 접근성 서비스를 물려받는다.

44줄 로그 꼬리표 "ShortsAIGuard". PC에서 `adb logcat -s ShortsAIGuard`로 볼 수 있다. 디버그 앱에서만 출력하고 화면 글자는 쓰지 않는다.

```
46     private static final long EVALUATE_DELAY_MS = 150L;
47     private static final long LEAVE_CHECK_MS = 800L;
48     /** After "watch" in the web flow, the player must be reopened within this window. */
49     private static final long ALLOW_WINDOW_MS = 15_000L;
50     /** Leaving during a commitment and coming back within this keeps the commitment. */
51     private static final long RESUME_WINDOW_MS = 10 * 60_000L;
52     /** After "나가기" from a PIP window that stays open, ask once more after this. */
53     private static final long PIP_LEAVE_WAIT_MS = 60_000L;
54     /** Time for YouTube to come to the front before checking whether the PIP expanded. */
55     private static final long PIP_EXPAND_CHECK_MS = 900L;
56     /** An alarm this far ahead counts as tomorrow's wake-up for the sleep line. */
57     private static final long ALARM_HORIZON_MS = 12 * 3600_000L;
```

시간 상수

- **46줄** 화면 신호를 0.15초 모았다가 한 번에 판단.
- **47줄** 나가기(뒤로 가기) 0.8초 뒤 아직 쇼츠면 홈으로.
- **49줄** 앱 화면에서 "보기"를 고르면 15초 안에 쇼츠를 다시 열어야 허락이 유효.
- **51줄** 약속 중에 나갔다가 **10분 안에** 돌아오면 약속 이어가기.
- **53줄** 작은 창에서 나가기 후 창이 남아 있으면 60초 뒤 한 번 더 묻기.
- **55줄** 작은 창 펼치기 시도 후 0.9초 뒤 결과 확인.
- **57줄** 12시간 안에 울리는 알람이 있으면 "일어나는 시각"으로 사용.

```
    private static WeakReference<ShortsGuardService> instance = new WeakReference<>(null);

    private final Handler mainHandler = new Handler(Looper.getMainLooper());
    private ShortsSessionTracker tracker;
    private GuardOverlay overlay;
    /** Windows that do not mean "left the player": system UI and keyboards. */
    private Set<String> passThroughPackages = new HashSet<>();
    private boolean destroyed;
    private boolean closing;
    private boolean evaluatePending;
    private int missedPromptChecks;
    private String lastVerdictLog;

    /** Set while "나가기" is in progress, to tell a failed BACK from coming back in. */
    private String leavingKind;
    private boolean sawOtherSinceLeave;

    private GuardFlow flow;

```

- **59줄** 지금 돌고 있는 서비스를 가리키는 약한 참조. 플러그인이 서비스에 일을 시킬 때 쓴다. "약한" 참조라 서비스가 꺼지면 붙잡지 않고 사라진다.
- **61줄** 메인 스레드(화면을 다루는 줄)의 Handler.
- **62~63줄** "머문 한 번" 정리기, 시트 그리기 도구.
- **65줄** 떠도 "쇼츠를 떠났다"로 보지 않을 앱: 상단 알림창(시스템 UI), 키보드.
- **66~70줄** 서비스가 꺼졌는지, 앱 화면 개입을 닫는 중인지, 판단 예약이 걸렸는지, 응답 없는 개입 확인 횟수, 마지막 판별 로그(같은 로그 반복 방지).
- **73~74줄** 나가기 진행 중인 플레이어, 그 사이 다른 화면을 봤는지 (뒤로 가기 실패와 다시 들어옴을 구분).
- **76줄** 개입 화면 순서 담당(GuardFlow).

```
    /** Running watch-time commitment ("몇 분 볼래요?"). */
    private String commitKind;
    private int commitMinutes;
    private int commitExtension;
    private boolean commitOverdraft;
    private int commitBand;
    private long commitEndsAt;

    /** A commitment cut short by leaving; resumed if the user is back within 10 minutes. */
    private String pausedKind;
    private long pausedRemainingMs;
    private long pausedAt;
```

약속 타이머 상태 ("몇 분 볼래요?"로 정한 시간): 대상 앱, 분, 몇 번째 연장, "3분만 더"인지, 단계, 끝나는 시각.

멈춘 약속: 약속 중에 나가면 남은 시간을 기억해 두고(대상, 남은 밀리초, 멈춘 시각), 10분 안에 돌아오면 이어 간다.

```
    /** Shorts went to a picture-in-picture window and is still playing there. */
    private boolean inPip;
    /**
     * "나가기" was chosen while Shorts played in PIP and the window is still open.
     * One PIP episode (until the window closes or the stay ends) gets at most one expand
     * attempt and one second ask; nothing here repeats on a timer.
     */
    private boolean pipLeaving;
    private long pipLeftAt;
    private boolean pipExpandTried;
    private boolean pipReasked;
    private boolean pipKeptWatching;

    private AudioFocusRequest focusRequest;
    private boolean holdingFocus;
    private final AudioManager.OnAudioFocusChangeListener focusListener = change -> { };
```

작은 창(PIP) 상태

- `inPip`: 쇼츠가 작은 창으로 계속 재생 중.
- `pipLeaving`: 작은 창에서 "나가기"를 골랐는데 창이 아직 열려 있음.
- **93~97줄 원칙**: 작은 창 한 번(창이 닫히거나 시청이 끝날 때까지)마다 펼치기 시도 최대 1번, 다시 묻기 최대 1번. **타이머로 반복하는 것은 없다** (무한 반복 금지).
- 나가기 고른 시각, 펼치기 시도했는지, 다시 물었는지, "계속 봄" 기록을 남겼는지.

104~106줄 오디오 포커스 요청 정보, 포커스를 갖고 있는지, 포커스 변화 알림(아무것도 안 함).

```
    private final Runnable commitTimer = this::onCommitmentOver;

    private final Runnable evaluateRunnable = () -> {
        evaluatePending = false;
        evaluate();
    };

    private final Runnable tick = new Runnable() {
        @Override
        public void run() {
            if (destroyed) return;
            try {
                tick();
            } finally {
                if (!destroyed) mainHandler.postDelayed(this, 1000);
            }
        }
    };
```

- **108줄** 약속 타이머가 끝나면 `onCommitmentOver` 실행. `this::함수`는 "이 함수를 작업으로" 쓰는 짧은 문법.
- **110~113줄** 예약된 판단 작업: 예약 표시를 지우고 `evaluate`.
- **115~125줄** 1초마다 반복 작업(`tick`). 오류가 나도(`finally`) 다음 1초를 다시 예약해서 멈추지 않게 한다. 서비스가 꺼지면 멈춤.

```
    private final Runnable leaveCheck = () -> {
        String kind = leavingKind;
        if (kind == null) return;
        if (!sawOtherSinceLeave && kind.equals(classifyActiveWindow())) {
            log("나가기: BACK 후에도 플레이어 → HOME");
            performGlobalAction(GLOBAL_ACTION_HOME);
            // Keep ignoring the player until HOME has taken effect.
            mainHandler.postDelayed(this::clearLeaving, LEAVE_CHECK_MS);
            return;
        }
        clearLeaving();
    };

    private void clearLeaving() {
        leavingKind = null;
        restoreAudio(false);
    }
```

leaveCheck — 나가기 0.8초 뒤 확인. 그 사이 다른 화면을 본 적이 없고 여전히 쇼츠 플레이어면 뒤로 가기가 안 먹힌 것이므로 **홈 버튼**을 누른다. 홈이 적용될 때까지 0.8초 더 기다린 뒤 정리한다.

clearLeaving: 나가기 상태를 지우고 소리를 원래대로.

```
    // --- Called from GuardPlugin (web flow results) ----------------------------

    static void requestLeave(Context context, String target) {
        log(context, "나가기 요청: " + target);
        GuardState.clearPrompt(context);
        if ("youtube".equals(target)) return;
        if (context instanceof android.app.Activity) {
            ((android.app.Activity) context).moveTaskToBack(true);
        }
        ShortsGuardService service = instance.get();
        if (service != null) service.leaveReelsThenReturn();
    }
```

requestLeave (플러그인이 부름, 앱 화면 개입의 "나가기"): 진행 중 개입을 지우고, YouTube면 할 일 없음(시트 방식이라). Instagram이면 우리 앱을 뒤로 보내고 서비스에 "릴스에서 나가서 앱으로 돌아오기"를 시킨다.

```
158     static void requestWatch(Context context, String target) {
159         log(context, "시청 허용: " + target);
160         GuardState.clearPrompt(context);
161         GuardState.allow(context, target, System.currentTimeMillis() + ALLOW_WINDOW_MS);
162         if ("youtube".equals(target)) openYoutubeShorts(context);
163     }
164
165     static void onBlockStarted(Context context, long blockedUntil) {
166         log(context, "시간 차단 시작: " + GuardCopy.countdown(blockedUntil - System.currentTimeMi
llis()));
167         GuardNotifier.showLock(context, blockedUntil);
168     }
```

requestWatch: 앱 화면에서 "보기"를 골랐을 때. 개입을 지우고 15초짜리 보기 허락을 적고, YouTube면 쇼츠를 연다.

onBlockStarted: 잠금이 시작되면 로그를 남기고 잠금 알림(카운트다운)을 띄운다.

```
170     // --- Lifecycle ------------------------------------------------------------
171
172     @Override
173     protected void onServiceConnected() {
174         super.onServiceConnected();
175         destroyed = false;
176         instance = new WeakReference<>(this);
177         passThroughPackages = buildPassThroughPackages();
178         tracker = new ShortsSessionTracker(mainHandler, this::classifyActiveWindow, trackerListe
ner);
179         overlay = new GuardOverlay(this, mainHandler);
180         flow = new GuardFlow(this, overlay, mainHandler, flowHost);
181         // A previous run may have died with the music stream muted.
182         restoreAudio(false);
183
184         GuardState.ActiveSession stale = GuardState.activeSession(this);
185         if (stale != null) {
186             double seconds = GuardState.endSession(this, stale.lastSeenAt);
187             log("이전 세션 정리: " + stale.target + " " + Math.round(seconds) + "초");
188         }
189         long until = GuardState.blockedUntil(this);
190         if (until > System.currentTimeMillis()) GuardNotifier.showLock(this, until);
191
192         mainHandler.removeCallbacks(tick);
193         mainHandler.postDelayed(tick, 1000);
194         log("서비스 연결됨");
195     }
```

onServiceConnected — 서비스가 켜질 때 한 번.

1. 꺼짐 표시를 풀고 자기 자신을 등록한다.
2. 시스템 UI·키보드 목록을 만들고, 정리기·시트·흐름 도구를 만든다.
3. **182줄** 이전에 갑자기 꺼지면서 소리를 꺼 둔 채였을 수 있으니 소리를 되돌린다.
4. **184~188줄** 이전에 열려 있던 시청이 있으면 **마지막으로 쇼츠가 보인 시각**에 닫는다.
5. **189~190줄** 잠금 중이면 알림을 다시 띄운다.
6. **192~193줄** 1초 반복 작업을 시작한다.

```
197     @Override
198     public boolean onUnbind(Intent intent) {
199         cleanUp();
200         return super.onUnbind(intent);
201     }
202
203     @Override
204     public void onDestroy() {
205         cleanUp();
206         super.onDestroy();
```

onUnbind / onDestroy: 서비스가 꺼질 때 `cleanUp`.

cleanUp: 한 번만 실행. 예약된 작업을 모두 지우고, 시트를 없애고, 소리를 되돌리고, 정리기를 정리하고, 열린 시청을 지금 시각으로 닫고, 등록을 지운다.

onInterrupt: 안드로이드가 요구하는 함수인데 할 일이 없어 비워 둔다.

```
    }

    private void cleanUp() {
        if (destroyed) return;
        destroyed = true;
        mainHandler.removeCallbacksAndMessages(null);
        if (overlay != null) overlay.dismissNow();
        restoreAudio(false);
        if (tracker != null) tracker.release();
        GuardState.endSession(this, System.currentTimeMillis());
        ShortsGuardService current = instance.get();
        if (current == this) instance = new WeakReference<>(null);
    }

    @Override
    public void onInterrupt() {}
```

```
    // --- Events and samples ----------------------------------------------------

    @Override
    public void onAccessibilityEvent(AccessibilityEvent event) {
        if (event == null || destroyed) return;
        CharSequence packageChars = event.getPackageName();
        if (packageChars == null) return;
        String packageName = packageChars.toString();
        if (GuardScreens.isWatchedPackage(packageName)) {
            scheduleEvaluate();
            return;
        }
        if (passThroughPackages.contains(packageName)) return;
        // Another app's window (or a toast) changed. Check which app is really in front
        // before ending the stay, so a toast over Shorts does not end it.
        if (event.getEventType() == AccessibilityEvent.TYPE_WINDOW_STATE_CHANGED
            && tracker.currentKind() != null) {
            scheduleEvaluate();
        }
    }
```

onAccessibilityEvent — 화면 신호가 올 때마다.

- 신호가 없거나 서비스가 꺼졌으면 무시. 앱 이름(패키지)을 꺼낸다.
- **232~235줄** YouTube나 Instagram이면 판단을 예약한다.
- **236줄** 상단 알림창이나 키보드면 무시.
- **237~242줄** 다른 앱의 창이 바뀌었고 쇼츠를 보는 중이었으면 판단을 예약한다. 바로 끝내지 않는 이유: 쇼츠 위에 잠깐 뜬 토스트(작은 안내)까지 "다른 앱"으로 보면 안 되기 때문에, 실제로 앞에 있는 앱을 다시 확인한다.

```
    /** Folds a burst of events into one check. */
    private void scheduleEvaluate() {
        if (evaluatePending) return;
        evaluatePending = true;
        mainHandler.postDelayed(evaluateRunnable, EVALUATE_DELAY_MS);
    }
```

scheduleEvaluate: 신호가 한꺼번에 많이 와도 0.15초 뒤 **한 번만** 판단하도록 모은다.

```
    /** Samples the window in front: a watched app's screen, or another app that ends the stay.
*/
    private void evaluate() {
        AccessibilityNodeInfo root = getRootInActiveWindow();
        if (root == null) return;
        try {
```

evaluate — 지금 앞 화면 판단. 앞 창의 맨 위 부품(root)을 얻어:

- YouTube/Instagram이면 작은 창 아님으로 두고, 쇼츠인지 판별(`classifyNode`)해 `sample`로 넘긴다.
- 다른 앱이고 쇼츠를 보던 중이면 `leftOrPip` (작은 창으로 계속 보는지 확인).

```
            CharSequence packageChars = root.getPackageName();
            if (packageChars == null) return;
            String packageName = packageChars.toString();
            if (GuardScreens.isWatchedPackage(packageName)) {
                inPip = false;
                sample(classifyNode(root), packageName);
            } else if (!passThroughPackages.contains(packageName) && tracker.currentKind() != nu
ll) {

                leftOrPip();
            }
        } finally {
            root.recycle();
        }
    }
```

- **266~268줄** 다 쓴 화면 정보는 `recycle`로 돌려준다 (메모리 절약).

```
    /** Once a second while on a player or recording: screen off and other apps end the stay. */
    private void tick() {
        checkPipLeave(System.currentTimeMillis());
        String current = tracker == null ? null : tracker.currentKind();
        if (current == null && GuardState.activeSession(this) == null) return;
        long now = System.currentTimeMillis();
        PowerManager power = (PowerManager) getSystemService(Context.POWER_SERVICE);
        if (power != null && !power.isInteractive()) {
            if (current != null) tracker.onLeftApp("화면 꺼짐", now);
            else GuardState.endSession(this, now);
            return;
        }
        String packageName = activePackage();
        if (packageName == null || passThroughPackages.contains(packageName)) return;
        if (GuardScreens.isWatchedPackage(packageName)) {
            evaluate();
        } else if (current != null) {
            leftOrPip();
        } else {
            GuardState.endSession(this, now);
        }
        if (tracker.currentKind() != null) GuardState.touchSession(this, now);
    }
```

tick — 1초마다.

1. 작은 창 나가기 대기 확인.
2. 쇼츠를 보지도 않고 열린 시청도 없으면 할 일 없음.
3. **277~282줄** 화면이 꺼졌으면 "화면 꺼짐"으로 끝낸다.
4. **283~291줄** 앞에 있는 앱 확인: YouTube/Instagram이면 판단, 다른 앱이면 떠남 또는 작은 창 확인, 열린 시청만 남아 있으면 닫는다.
5. **292줄** 보는 중이면 "마지막으로 보인 시각"을 갱신 (갑자기 꺼져도 정확히 닫기 위해).

```
    /**
     * Another app is in front. If the YouTube Shorts we were tracking went to a
     * picture-in-picture window, the stay goes on; otherwise it ends. Only the PIP
     * window's package is checked; other windows are not opened or logged.
     */
    private void leftOrPip() {
        if ("youtube".equals(tracker.currentKind()) && youtubePipVisible()) {
            if (!inPip) log("PIP: 작은 창으로 계속 시청");
            inPip = true;
            tracker.onSample("youtube", System.currentTimeMillis());
            return;
        }
```

leftOrPip — 다른 앱이 앞에 왔을 때. YouTube 쇼츠를 보던 중이고 YouTube 작은 창이 떠 있으면 **계속 보는 중**으로 처리한다(시청 시간 계속). 아니면 "다른 앱"으로 끝낸다. 작은 창의 앱 이름만 확인하고 다른 창은 열어 보지 않는다.

```
        inPip = false;
        tracker.onLeftApp("다른 앱", System.currentTimeMillis());
    }
```

```
    private boolean youtubePipVisible() {
        if (Build.VERSION.SDK_INT < Build.VERSION_CODES.O) return false;
        List<AccessibilityWindowInfo> windows;
        try {
            windows = getWindows();
        } catch (RuntimeException e) {
            return false;
        }
        for (AccessibilityWindowInfo window : windows) {
            if (!window.isInPictureInPictureMode()) continue;
            AccessibilityNodeInfo root = window.getRoot();
            if (root == null) continue;
            try {
                CharSequence pkg = root.getPackageName();
                if (pkg != null && GuardScreens.YOUTUBE.equals(pkg.toString())) return true;
            } finally {
                root.recycle();
            }
        }
        return false;
    }
```

youtubePipVisible — YouTube 작은 창이 떠 있나. 안드로이드 8 이상에서 화면의 창 목록을 받아, 작은 창(PIP) 모드인 창의 앱 이름이 YouTube인지만 본다. 오류가 나면 false.

```
    /** While waiting for the user to close a PIP window after "나가기". Called once a second. */
    private void checkPipLeave(long now) {
        if (!pipLeaving) return;
        if (!youtubePipVisible()) {
            pipLeaving = false;
            restoreAudio(false);
            log("PIP 닫힘: 소리 되돌림");
            return;
        }
        if (!pipReasked && now - pipLeftAt >= PIP_LEAVE_WAIT_MS && "youtube".equals(tracker.curr
entKind())) {
            pipReasked = true;
            pipLeaving = false;
            log("PIP 1분: 한 번만 다시 묻기");
            pausePlayback();
            GuardPolicy policy = GuardState.policy(this);
            double usage = GuardState.todaySeconds(this, now);
            if (policy.band(usage) == GuardPolicy.BAND_STRONG) {
                flow.strong("youtube", usage, policy, null, false);
            } else {
                flow.entry("youtube", usage, policy, false);
            }
        }
    }
```

checkPipLeave — 작은 창 "나가기" 뒤 1초마다.

- 작은 창이 닫혔으면 대기를 끝내고 소리를 되돌린다.
- **342~354줄** 60초가 지나도 계속 열려 있고 아직 다시 묻지 않았다면, **딱 한 번** 영상을 멈추고 단계에 맞는 개입 화면(강함 또는 진입)을 다시 띄운다.

```
    /** One try per PIP episode: bring YouTube to the front, which usually expands the window.
*/
    private void afterPipExpand(String kind) {
        if (destroyed || !pipLeaving) return;
        if (youtubePipVisible()) {
            pipStillOpen(kind);
            return;
        }
        pipLeaving = false;
        inPip = false;
        if (kind.equals(classifyActiveWindow())) {
            log("PIP 펼침: BACK으로 나가기");
            leavePlayer(kind);
        } else {
            log("PIP 사라짐");
            restoreAudio(false);
        }
    }
```

afterPipExpand — 펼치기 시도 0.9초 뒤. 아직 작은 창이면 `pipStillOpen`. 펼쳐졌으면 작은 창 상태를 끄고, 쇼츠 화면이면 뒤로 가기로 나가고, 아니면 소리만 되돌린다.

```
    /**
     * The PIP window stayed open after "나가기": ask the user to close it, and if they keep
     * watching, the time still counts (a session runs) and is recorded once as kept watching.
     */
    private void pipStillOpen(String kind) {
        log("PIP 계속 열림: 닫기 안내, 계속 보면 시청으로 기록");
        Toast.makeText(this, GuardCopy.PIP_CLOSE_GUIDE, Toast.LENGTH_LONG).show();
        if (GuardState.activeSession(this) == null) startWatch(kind);
        if (pipKeptWatching) return;
        pipKeptWatching = true;
        long now = System.currentTimeMillis();
        GuardPolicy policy = GuardState.policy(this);
        double usage = GuardState.todaySeconds(this, now);
        JSONObject row = new JSONObject();
        try {
            row.put("id", java.util.UUID.randomUUID().toString())
                .put("createdAt", now)
                .put("target", kind)
                .put("level", policy.band(usage))
                .put("band", policy.band(usage))
                .put("stage", "pip")
                .put("reason", "선택하지 않음")
                .put("interventionType", "pip")
                .put("outcome", usage >= policy.goalMinutes * 60.0 ? "overdraft" : "watch")
                .put("usageSeconds", usage)
                .put("policyVersion", policy.version);
        } catch (org.json.JSONException ignored) {
            return;
        }
        GuardState.addIntervention(this, row);
    }
```

pipStillOpen — 작은 창이 계속 열려 있을 때.

1. "작은 창의 X를 눌러 닫아 주세요. 닫으면 소리가 돌아와요." 안내(토스트).
2. 열린 시청이 없으면 시작한다 (계속 보면 시간이 쌓이게).
3. 이번 작은 창에서 처음이면 "계속 봄" 기록을 한 번 남긴다: 단계 "pip", 결과는 목표를 넘었으면 "3분만 더(overdraft)", 아니면 "보기(watch)".

```
    private void endPipEpisode() {
        pipLeaving = false;
        pipExpandTried = false;
        pipReasked = false;
        pipKeptWatching = false;
    }
```

endPipEpisode: 작은 창 한 번의 상태(나가기 대기, 펼치기 시도, 다시 묻기, 계속 봄 기록)를 모두 초기화.

```
    private void sample(String kind, String packageName) {
        if (leavingKind != null) {
            if (kind == null) {
                sawOtherSinceLeave = true;
            } else if (!sawOtherSinceLeave) {
                return; // BACK has not taken effect yet; not a new entry.
            }
        }
        if (kind == null) {
            noteAwayFromPrompt(packageName);
        } else {
            missedPromptChecks = 0;
        }
        tracker.onSample(kind, System.currentTimeMillis());
    }
```

sample — 판별 결과 넘기기. 나가기 진행 중이면: 쇼츠가 아닌 화면을 봤다고 표시하고, 아직 쇼츠 화면이면 뒤로 가기가 덜 적용된 것이라 무시한다 (새 진입으로 착각 방지). 그 외에는 정리기(`tracker.onSample`)에 넘긴다.

```
    /** A prompt the user never answered is dropped once they browse the app normally. */
    private void noteAwayFromPrompt(String packageName) {
        if (closing) return;
        String kind = GuardScreens.YOUTUBE.equals(packageName) ? "youtube" : "instagram";
        if (!GuardState.isPrompting(this, kind)) return;
        missedPromptChecks += 1;
        if (missedPromptChecks < 2) return;
        missedPromptChecks = 0;
        GuardState.clearPrompt(this);
        log("응답 없는 개입 정리: " + kind);
    }
```

noteAwayFromPrompt: 앱 화면 개입(Instagram)에 답하지 않고 그 앱을 그냥 둘러보면, 2번 확인 후 그 개입을 지운다.

```
    // --- Stay start / end ------------------------------------------------------

    private final ShortsSessionTracker.Listener trackerListener = new ShortsSessionTracker.Listener() {
        @Override
        public void onEntered(String kind) {
            log("▶ 진입: " + kind);
            leavingKind = null;
            decide(kind);
        }

        @Override
        public void onExited(String kind, long endedAt, String reason) {
            double seconds = GuardState.endSession(ShortsGuardService.this, endedAt);
            log("■ 종료: " + kind + " (" + reason + ")" + (seconds > 0 ? " 시청 " + Math.round(seconds) + "초" : ""));
```

trackerListener — 정리기의 알림을 받는 곳.

- **onEntered** (쇼츠에 들어옴): 나가기 상태를 지우고 `decide`.
- **onExited** (나감): 시청을 닫고(시간 기록), 약속 타이머를 멈춰 두고(10분 이어가기용), 작은 창 상태를 초기화. 시트가 떠 있었으면 "답 없이 나감"으로 기록하고(`abandon`) 소리를 되돌린다.

```
            pauseCommitment(kind);
            boolean wasPipLeaving = pipLeaving;
            endPipEpisode();
            inPip = false;
            if (overlay != null && overlay.isShowing()) {
                flow.abandon();
                if (leavingKind == null) restoreAudio(false);
            } else if (wasPipLeaving) {
                restoreAudio(false);
            }
        }
    };
```

```
    private void decide(String kind) {
        if (closing) return;
        long now = System.currentTimeMillis();
        if (GuardState.isLocked(this, now)) {
            showLock(kind, now);
            return;
        }
        if (GuardState.consumeAllow(this, kind, now) || !GuardState.isBlockerEnabled(this)) {
            startWatch(kind);
            return;
        }
        if (resumeCommitment(kind, now)) return;
        GuardPolicy policy = GuardState.policy(this);
        double usage = GuardState.todaySeconds(this, now);
        int band = policy.band(usage);
        boolean afterLock = GuardState.needsAfterLockQuestion(this, now);
        log("개입 판단: " + kind + " 강도 " + band + " 오늘 " + Math.round(usage) + "초 (" + polic
y.version + ")");
        if ("instagram".equals(kind)) {
            GuardState.startPrompt(this, kind, band, usage);
            bringSelf(this);
            return;
        }
        pausePlayback();
        if (band == GuardPolicy.BAND_STRONG) {
            flow.strong(kind, usage, policy, null, afterLock);
        } else {
            flow.entry(kind, usage, policy, afterLock);
        }
    }
```

decide — 쇼츠에 들어왔을 때 무엇을 할지. 순서대로 확인한다.

1. **472~475줄** 잠금 중이면 → 잠금 화면.
2. **476~479줄** 보기 허락이 있거나 개입 스위치가 꺼져 있으면 → 개입 없이 시청 기록만.
3. **480줄** 10분 안에 돌아온 약속이 있으면 → 이어서 보기.
4. **481~485줄** 정책표, 오늘 사용량, 단계, 잠금 뒤 질문 필요 여부를 정한다.
5. **486~490줄** Instagram이면 → 앱 화면 개입(예전 방식).
6. **491~496줄** YouTube면 → **영상 멈춤** 후 강함이면 강함 화면, 아니면 진입 화면.

```
    private final GuardFlow.Host flowHost = new GuardFlow.Host() {
        @Override
        public void onWatch(String kind, int minutes, boolean overdraft, int extensionIndex, int
band) {
            log("시청: " + minutes + "분" + (overdraft ? " (목표 초과)" : extensionIndex > 0 ? "
(연장 " + extensionIndex + ")" : ""));
            if (!kind.equals(tracker.currentKind())) {
```

flowHost — GuardFlow의 부탁을 실제로 처리.

onWatch (보기로 함): 그 사이 쇼츠를 떠났으면 소리만 되돌리고 끝. 아니면 소리를 되돌려 재생을 이어 가고, 시청 기록이 없으면 시작하고, 약속 정보(분, 연장, 3분만 더, 단계)를 저장한 뒤 **약속 타이머**를 건다.

```
                restoreAudio(false);
                return;
            }
            restoreAudio(true);
            if (GuardState.activeSession(ShortsGuardService.this) == null) startWatch(kind);
            commitKind = kind;
            commitMinutes = minutes;
            commitExtension = extensionIndex;
            commitOverdraft = overdraft;
            commitBand = band;
            scheduleCommitment(minutes * 60_000L);
        }
```

```
        @Override
        public void onLeave(String kind) {
            log("나가기 선택");
            leavePlayer(kind);
        }

        @Override
        public void onBlock(String kind, long until, int minutes, boolean dayEnd) {
            GuardState.startBlockUntil(ShortsGuardService.this, until, minutes, dayEnd);
            onBlockStarted(ShortsGuardService.this, until);
            leavePlayer(kind);
        }

        @Override
        public void record(JSONObject row) {
            GuardState.addIntervention(ShortsGuardService.this, row);
        }

        @Override
        public GuardCopy.Facts facts() {
            return buildFacts(System.currentTimeMillis());
        }

        @Override
        public boolean inPip() {
            return inPip;
        }
    };
```

- **onLeave**: 나가기.
- **onBlock**: 잠금을 시작하고 알림을 띄운 뒤 나가기.
- **record**: 기록을 개입 대기열에 넣는다.
- **facts**: 지금 숫자를 만든다(`buildFacts`).
- **inPip**: 작은 창으로 보는 중인지.

```
    /** Numbers for the sheet wording: today, the goal, the web side's stats, and sleep left. */
    private GuardCopy.Facts buildFacts(long now) {
        GuardPolicy policy = GuardState.policy(this);
        GuardCopy.Facts f = new GuardCopy.Facts();
        f.usage = GuardState.todaySeconds(this, now);
        f.goal = policy.goalMinutes * 60.0;
        f.weekProjection = policy.stats.weekProjection(f.usage);
        f.avg7 = policy.stats.avg7Seconds;
        Calendar cal = Calendar.getInstance();
```

buildFacts — 문구에 넣을 숫자 만들기 (1). 오늘 사용 초, 목표 초, 이번 주 예상, 7일 평균, 어제 이 시간까지, 오늘 약속 지킨 횟수, 오늘 쉰 초.

```
        cal.setTimeInMillis(now);
        int hour = cal.get(Calendar.HOUR_OF_DAY);
        f.yesterdayByNow = policy.stats.yesterdayByNow(hour, cal.get(Calendar.MINUTE));
        f.keptToday = GuardState.countToday(this, "kept", now);
        f.restToday = GuardState.restSecondsToday(this, now);
```

```
        if (hour >= 22 || hour < 4) {
            long wake = -1;
            AlarmManager alarms = (AlarmManager) getSystemService(Context.ALARM_SERVICE);
            AlarmManager.AlarmClockInfo next = alarms == null ? null : alarms.getNextAlarmClock
();
            if (next != null && next.getTriggerTime() > now && next.getTriggerTime() - now <= AL
ARM_HORIZON_MS) {
                wake = next.getTriggerTime();
                Calendar at = Calendar.getInstance();
                at.setTimeInMillis(wake);
                f.sleepBasis = String.format(java.util.Locale.ROOT, "%d:%02d 알람 기준",
                    at.get(Calendar.HOUR_OF_DAY), at.get(Calendar.MINUTE));
            } else {
                Calendar at = Calendar.getInstance();
                at.setTimeInMillis(now);
                at.set(Calendar.HOUR_OF_DAY, policy.stats.wakeHour);
                at.set(Calendar.MINUTE, policy.stats.wakeMinute);
                at.set(Calendar.SECOND, 0);
                if (at.getTimeInMillis() <= now) at.add(Calendar.DAY_OF_MONTH, 1);
                wake = at.getTimeInMillis();
                f.sleepBasis = String.format(java.util.Locale.ROOT, "일어나는 시각 %d:%02d 기준",
                    policy.stats.wakeHour, policy.stats.wakeMinute);
            }
            f.sleepSeconds = (wake - now) / 1000.0;
        }
        return f;
    }
```

buildFacts (2) — 밤(22시~새벽 4시)에만 수면 숫자.

- **562~569줄** 폰에 **12시간 안에 울리는 알람**이 있으면 그 시각을 기상 시각으로 쓰고 근거 "7:10 알람 기준".
- **570~579줄** 없으면 설정의 "일어나는 시각"을 다음 그 시각으로 계산하고 근거 "일어나는 시각 7:00 기준".
- **581줄** 지금 자면 잘 수 있는 초 = 기상 시각 − 지금.

```
    private void scheduleCommitment(long delayMs) {
        commitEndsAt = SystemClock.elapsedRealtime() + delayMs;
        mainHandler.removeCallbacks(commitTimer);
        mainHandler.postDelayed(commitTimer, delayMs);
    }

    /** The user left during a commitment: keep what is left for 10 minutes. */
    private void pauseCommitment(String kind) {
        if (commitKind != null && commitKind.equals(kind)) {
            pausedKind = kind;
            pausedRemainingMs = Math.max(0, commitEndsAt - SystemClock.elapsedRealtime());
            pausedAt = System.currentTimeMillis();
        }
        cancelCommitment();
    }
```

scheduleCommitment: 약속 끝 시각을 기억하고 그 시간 뒤에 `onCommitmentOver`를 예약한다 (이전 예약은 지움). `elapsedRealtime`은 폰 시계를 바꿔도 영향 없는 시간.

pauseCommitment: 약속 중에 나가면 남은 밀리초와 멈춘 시각을 기억하고 타이머를 끈다.

```
    /** Back within 10 minutes with time left on the commitment: no sheet, carry on. */
    private boolean resumeCommitment(String kind, long now) {
        String paused = pausedKind;
        pausedKind = null;
        if (paused == null || !paused.equals(kind) || now - pausedAt > RESUME_WINDOW_MS
            || pausedRemainingMs < 5_000) {
            return false;
        }
        commitKind = kind;
        startWatch(kind);
        scheduleCommitment(pausedRemainingMs);
        log("약속 이어가기: 남은 " + Math.round(pausedRemainingMs / 1000.0) + "초");
        Toast.makeText(this, GuardCopy.resumeToast(commitMinutes, pausedRemainingMs / 1000.0), T
oast.LENGTH_SHORT)
            .show();
        return true;
    }
```

resumeCommitment — 약속 이어가기. 멈춘 약속이 같은 앱이고, 10분 안에 돌아왔고, 5초 이상 남았으면: 시청 기록을 시작하고 남은 시간으로 타이머를 다시 걸고 "약속한 5분 중 3분 남았어요." 안내를 띄운다. 개입 화면 없이 바로 이어 본다.

```
    /** The chosen watch time is over: stop and ask again. */
    private void onCommitmentOver() {
        String kind = commitKind;
        commitKind = null;
        if (kind == null || destroyed || !kind.equals(tracker.currentKind())) return;
        GuardPolicy policy = GuardState.policy(this);
        double usage = GuardState.todaySeconds(this, System.currentTimeMillis());
        log("약속 시간 끝: " + commitMinutes + "분");
        pausePlayback();
        if (commitOverdraft) {
            flow.strong(kind, usage, policy, policy.overdraftMinutes + "분이 끝났어요.", false);
        } else {
            flow.commitmentOver(kind, usage, policy, commitMinutes, commitExtension, commitBan
d);
        }
    }

    private void cancelCommitment() {
        commitKind = null;
        mainHandler.removeCallbacks(commitTimer);
    }
```

onCommitmentOver — 약속 시간이 끝났을 때. 그 사이 쇼츠를 떠났거나 서비스가 꺼졌으면 무시. 아니면 영상을 멈추고, "3분만 더"였으면 강함 화면("3분이 끝났어요."), 보통 약속이면 약속 끝 화면을 띄운다.

cancelCommitment: 약속 정보를 지우고 타이머를 끈다.

```
    private void startWatch(String kind) {
        GuardState.startSession(this, kind, System.currentTimeMillis());
        log("시청 기록 시작: " + kind);
    }

    private void showLock(String kind, long now) {
        long until = GuardState.blockedUntil(this);
        GuardState.addLockAttempt(this, kind, now, until - now);
        log("잠금 중 진입: 남은 " + GuardCopy.countdown(until - now));
        pausePlayback();
```

startWatch: 시청 기록 시작 (지금 시각으로 열린 시청을 만든다).

showLock: 잠금 중 진입. 잠금 중 시도를 기록하고, 영상을 멈추고, 잠금 화면을 띄운다.

```
        flow.locked(kind, until);
    }
```

```
    /** BACK, then HOME if the player is still there 0.8 s later. The stay ends right away. */
    private void leavePlayer(String kind) {
        if (inPip) {
            // BACK would hit the app in front, and another app's PIP cannot be closed for
            // the user. Keep the stay (time keeps counting), keep the sound off, and try once
            // to expand the window by bringing YouTube to the front.
            pipLeaving = true;
            pipLeftAt = System.currentTimeMillis();
            if (!pipExpandTried) {
                pipExpandTried = true;
                log("나가기: PIP → 큰 화면으로 펼치기 1회 시도");
                openYoutubeFront(this);
                mainHandler.postDelayed(() -> afterPipExpand(kind), PIP_EXPAND_CHECK_MS);
            } else {
                pipStillOpen(kind);
            }
            return;
        }
        log("나가기: BACK");
        leavingKind = kind;
        sawOtherSinceLeave = false;
        tracker.forceExit(System.currentTimeMillis(), "나가기");
        performGlobalAction(GLOBAL_ACTION_BACK);
        mainHandler.removeCallbacks(leaveCheck);
        mainHandler.postDelayed(leaveCheck, LEAVE_CHECK_MS);
    }
```

leavePlayer — 쇼츠에서 나가기.

- **655~670줄 작은 창이면**: 뒤로 가기를 누르면 앞에 있는 다른 앱이 반응하고, 다른 앱의 작은 창은 우리가 닫을 수 없다. 그래서 시청은 계속 세고 소리는 끈 채로, **딱 한 번** YouTube를 앞으로 불러 창을 펼쳐 본다. 이미 시도했으면 닫기 안내로.
- **671~677줄 보통이면**: 나가기 상태를 표시하고, 시청을 바로 끝내고 (`forceExit`), **뒤로 가기**를 누르고, 0.8초 뒤 확인(`leaveCheck`)을 예약한다.

```
    // --- Instagram: leave the Reels player, then return to ShortsAI ------------

    private void stepAway(String kind, int attempt) {
        if (destroyed) {
            closing = false;
            return;
        }
        String packageName = activePackage();
        boolean stillPlayer = kind.equals(classifyActiveWindow());
        int limit = 3;
        if (packageName == null && attempt < limit + 2) {
            mainHandler.postDelayed(() -> stepAway(kind, attempt + 1), 200);
            return;
        }
        if (stillPlayer && attempt < limit) {
            log("BACK " + (attempt + 1) + "회");
            performGlobalAction(GLOBAL_ACTION_BACK);
            mainHandler.postDelayed(() -> stepAway(kind, attempt + 1), 280);
            return;
        }
        log("앱 열기 (BACK " + attempt + "회 후)");
```

stepAway — Instagram 릴스에서 나가기 (예전 방식). 앞 앱을 아직 모르면 0.2초씩 다시 확인(최대 5번). 아직 릴스 플레이어면 뒤로 가기를 0.28초 간격으로 최대 3번 누른다. 그다음 우리 앱(ShortsAI)을 연다.

leaveReelsThenReturn: 0.35초 뒤 stepAway 시작.

```
        bringSelf(this);
        closing = false;
    }

    private void leaveReelsThenReturn() {
        mainHandler.postDelayed(() -> stepAway("instagram", 0), 350);
    }
```

```
    // --- Media keys and apps ------------------------------------------------

    /**
     * Stops what plays under the sheet: transient audio focus (players pause on losing it
     * and usually resume when it comes back) plus muting the music stream until the sheet
     * is gone. Media keys are not used: they did not reach Shorts on a Galaxy, and a play
     * key could start another app's paused media instead.
     */
    private void pausePlayback() {
        AudioManager audio = (AudioManager) getSystemService(Context.AUDIO_SERVICE);
        if (audio == null) return;
        requestFocus(audio);
        try {
            if (!GuardState.mutedByUs(this) && !audio.isStreamMute(AudioManager.STREAM_MUSIC)) {
                audio.adjustStreamVolume(AudioManager.STREAM_MUSIC, AudioManager.ADJUST_MUTE,
0);
                GuardState.setMutedByUs(this, true);
            }
        } catch (RuntimeException ignored) {
            // Some modes (e.g. Do Not Disturb policies) refuse; focus still applies.
        }
    }
```

pausePlayback — 영상 멈추기.

1. **오디오 포커스 요청**: "잠깐 소리 좀 쓸게요"라고 안드로이드에 요청하면, 대부분의 동영상 앱은 스스로 멈춘다.
2. **음악 소리 음소거**: 확실히 하려고, 이미 꺼져 있지 않으면 음악 소리를 끄고 "우리가 껐다"를 기록한다. 방해 금지 모드 등에서 거절되면 무시한다.

711~716줄 미디어 키(재생/정지 신호)를 쓰지 않는 이유: 갤럭시에서 쇼츠에 전달되지 않았고, 재생 키가 다른 앱의 멈춘 음악을 틀 위험이 있다.

```
    /**
     * Undoes {@link #pausePlayback}. Giving focus back lets the player resume on its own;
     * {@code resume} only marks the intent in the log.
     */
    private void restoreAudio(boolean resume) {
        AudioManager audio = (AudioManager) getSystemService(Context.AUDIO_SERVICE);
        if (audio == null) return;
        if (GuardState.mutedByUs(this)) {
            try {
                audio.adjustStreamVolume(AudioManager.STREAM_MUSIC, AudioManager.ADJUST_UNMUTE,
0);
            } catch (RuntimeException ignored) {
                // Nothing else to do; the flag is cleared so we do not keep retrying.
            }
            GuardState.setMutedByUs(this, false);
        }
        abandonFocus(audio);
        if (resume) log("재생 이어가기 (오디오 포커스 반환)");
    }
```

restoreAudio — 소리 되돌리기. 우리가 껐으면 음소거를 풀고 기록을 지운다(실패해도 다시 시도하지 않음). 그리고 오디오 포커스를 돌려주면 동영상 앱이 보통 스스로 다시 재생된다.

```
    private void requestFocus(AudioManager audio) {
        if (holdingFocus) return;
        int result;
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            focusRequest = new AudioFocusRequest.Builder(AudioManager.AUDIOFOCUS_GAIN_TRANSIENT)
                .setAudioAttributes(new AudioAttributes.Builder()
                    .setUsage(AudioAttributes.USAGE_ASSISTANCE_SONIFICATION)
                    .setContentType(AudioAttributes.CONTENT_TYPE_SONIFICATION)
                    .build())
                .setOnAudioFocusChangeListener(focusListener)
                .build();
            result = audio.requestAudioFocus(focusRequest);
        } else {
            result = audio.requestAudioFocus(
                focusListener, AudioManager.STREAM_MUSIC, AudioManager.AUDIOFOCUS_GAIN_TRANSIEN
T);
        }
        holdingFocus = result == AudioManager.AUDIOFOCUS_REQUEST_GRANTED;
        log("오디오 포커스 요청: " + (holdingFocus ? "받음" : "거절"));
    }

    private void abandonFocus(AudioManager audio) {
        if (!holdingFocus) return;
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O && focusRequest != null) {
            audio.abandonAudioFocusRequest(focusRequest);
        } else {
            audio.abandonAudioFocus(focusListener);
        }
        holdingFocus = false;
    }
```

requestFocus: 이미 갖고 있으면 그대로. 안드로이드 8 이상은 새 방식(`AudioFocusRequest`, "잠깐(TRANSIENT)" + "알림음 용도")으로, 그 이하는 예전 방식으로 요청한다. 받았는지 기억하고 로그를 남긴다.

abandonFocus: 갖고 있으면 같은 방식으로 돌려준다.

```
    private static void bringSelf(Context context) {
        Intent intent = new Intent(context, MainActivity.class);
        intent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK
            | Intent.FLAG_ACTIVITY_SINGLE_TOP
            | Intent.FLAG_ACTIVITY_REORDER_TO_FRONT);
        context.startActivity(intent);
    }

    /** Like tapping the YouTube icon: brings its task to the front (a PIP usually expands). */
    private static void openYoutubeFront(Context context) {
        Intent launch = context.getPackageManager().getLaunchIntentForPackage(GuardScreens.YOUTU
BE);
        if (launch == null) return;
        launch.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK);
        try {
            context.startActivity(launch);
        } catch (RuntimeException ignored) {
            // The fallback (close guide, time recorded) still applies.
```

bringSelf: 우리 앱 화면을 앞으로 연다 (이미 열려 있으면 그 화면을 앞으로).

openYoutubeFront: YouTube 아이콘을 누른 것처럼 YouTube를 앞으로 불러온다. 작은 창은 보통 이때 펼쳐진다. 실패해도 닫기 안내가 대신한다.

```
        }
    }
```

```
    private static void openYoutubeShorts(Context context) {
        Intent intent = new Intent(Intent.ACTION_VIEW, Uri.parse("https://www.youtube.com/shorts"));
        intent.setPackage(GuardScreens.YOUTUBE);
        intent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK);
        try {
            context.startActivity(intent);
        } catch (RuntimeException ignored) {
            Intent launch = context.getPackageManager().getLaunchIntentForPackage(GuardScreens.YOUTUBE);
            if (launch != null) {
                launch.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK);
                context.startActivity(launch);
            }
        }
    }

    private Set<String> buildPassThroughPackages() {
        Set<String> packages = new HashSet<>();
        packages.add("com.android.systemui");
        InputMethodManager imm = (InputMethodManager) getSystemService(INPUT_METHOD_SERVICE);
        if (imm != null) {
            for (InputMethodInfo info : imm.getEnabledInputMethodList()) packages.add(info.getPackageName());
        }
        return packages;
    }
```

openYoutubeShorts: youtube.com/shorts 주소를 YouTube 앱으로 연다. 실패하면 YouTube를 그냥 연다.

buildPassThroughPackages: "떠난 것으로 보지 않을 앱" 목록 = 시스템 UI + 폰에 켜진 모든 키보드 앱.

```
    // --- Classification -------------------------------------------------------

    private String classifyActiveWindow() {
        AccessibilityNodeInfo root = getRootInActiveWindow();
        if (root == null) return null;
        try {
            return classifyNode(root);
        } finally {
            root.recycle();
        }
    }

    private String activePackage() {
        AccessibilityNodeInfo root = getRootInActiveWindow();
        if (root == null) return null;
        try {
            CharSequence packageChars = root.getPackageName();
            return packageChars == null ? null : packageChars.toString();
        } finally {
            root.recycle();
```

- **classifyActiveWindow**: 지금 앞 창이 무슨 플레이어인지 ("youtube"/"instagram"/null).
- **activePackage**: 지금 앞 창의 앱 이름.
- **classifyNode**: 감시 대상 앱이 아니면 null, YouTube면 `classifyYoutube`, Instagram이면 이름표를 모아(`inspect`) 판별.

```
        }
    }

    private String classifyNode(AccessibilityNodeInfo node) {
        CharSequence packageChars = node.getPackageName();
        if (packageChars == null) return null;
        String packageName = packageChars.toString();
        if (!GuardScreens.isWatchedPackage(packageName)) return null;
        if (GuardScreens.YOUTUBE.equals(packageName)) return classifyYoutube(node);
        return GuardScreens.classify(packageName, inspect(node));
    }
```

```
    /** Looks up only the known ids instead of walking the tree. */
    private String classifyYoutube(AccessibilityNodeInfo root) {
        List<String> visible = new ArrayList<>();
        collectVisible(root, GuardScreens.YOUTUBE_SHORTS_IDS, visible);
        if (!visible.isEmpty()) {
            collectVisible(root, GuardScreens.YOUTUBE_LONGFORM_IDS, visible);
        }
        String kind = GuardScreens.classify(GuardScreens.YOUTUBE, visible);
        if (isDebuggable(this)) {
            String line = "[판별] 쇼츠=" + (kind != null)
                + " 쇼츠ID=" + leaf(GuardScreens.firstExact(visible, GuardScreens.YOUTUBE_SHORTS_
IDS))
                + " 롱폼ID=" + leaf(GuardScreens.firstExact(visible, GuardScreens.YOUTUBE_LONGFOR
M_IDS));
            if (!line.equals(lastVerdictLog)) {
                lastVerdictLog = line;
                log(line);
            }
        }
        return kind;
    }
```

classifyYoutube — YouTube 판별. 화면 전체를 뒤지지 않고 **알고 있는 이름표만 찾아본다** (빠르고 다른 정보를 안 봄).

1. 쇼츠 이름표 2개가 보이는지 찾는다.
2. 하나라도 보이면 일반 영상 이름표 5개도 찾는다.
3. GuardScreens 규칙으로 판정.
4. 디버그 앱이면 "[판별] 쇼츠=true 쇼츠ID=reel_recycler 롱폼ID=-" 같은 로그를 남긴다 (같은 내용이면 반복하지 않음).

```
    /** Adds each id that has at least one node visible to the user. Hidden back-stack pages do
not count. */
    private static void collectVisible(AccessibilityNodeInfo root, String[] ids, List<String> ou
t) {
        for (String id : ids) {
            List<AccessibilityNodeInfo> nodes = root.findAccessibilityNodeInfosByViewId(id);
            if (nodes == null) continue;
            boolean visible = false;
            for (AccessibilityNodeInfo node : nodes) {
                if (node == null) continue;
                if (node.isVisibleToUser()) visible = true;
                node.recycle();
            }
            if (visible) out.add(id);
        }
    }

```

collectVisible: 이름표마다 그 이름표를 가진 부품을 찾고, **사용자에게 실제로 보이는 것**이 하나라도 있으면 목록에 넣는다. 뒤에 숨어 있는 이전 화면의 부품은 세지 않는다.

leaf: 로그용으로 이름표의 앞부분(접두어)을 떼어 짧게 만든다. 없으면 "-".

```
    private static String leaf(String id) {
        if (id == null) return "-";
        return id.startsWith(GuardScreens.YOUTUBE_ID) ? id.substring(GuardScreens.YOUTUBE_ID.len
gth()) : id;
    }
```

```
    /**
     * Instagram only. Resource ids only. Does not call getText or read password fields.
     */
    private static GuardScreens.Hit inspect(AccessibilityNodeInfo root) {
        List<String> ids = new ArrayList<>();
        int[] count = new int[] {0};
        walk(root, ids, 0, count);
        return new GuardScreens.Hit(ids, false, false, false);
    }

    private static void walk(AccessibilityNodeInfo node, List<String> ids, int depth, int[] coun
t) {
        if (node == null || depth > 30 || count[0] > 800) return;
        count[0] += 1;
        String id = node.getViewIdResourceName();
        if (id != null) ids.add(id);
        if (GuardScreens.classify(GuardScreens.INSTAGRAM, ids) != null) return;
        int children = node.getChildCount();
        for (int i = 0; i < children && count[0] <= 800; i++) {
            AccessibilityNodeInfo child = node.getChild(i);
            if (child == null) continue;
            walk(child, ids, depth + 1, count);
            child.recycle();
        }
    }
```

inspect / walk — Instagram 판별용 이름표 모으기. 화면 부품을 위에서부터 차례로 내려가며(재귀) 이름표만 모은다. 글자는 읽지 않는다. 깊이 30, 부품 800개까지만 보고, 릴스 표시를 찾으면 바로 멈춘다 (빠르게).

```
    // --- Logging ---------------------------------------------------------------

    private static boolean isDebuggable(Context context) {
        return (context.getApplicationInfo().flags & ApplicationInfo.FLAG_DEBUGGABLE) != 0;
    }

    private static void log(Context context, String message) {
        if (isDebuggable(context)) Log.d(TAG, message);
    }

    private void log(String message) {
        log(this, message);
    }
}
```

로그: 디버그 앱인지 확인(`FLAG_DEBUGGABLE`)하고, 디버그일 때만 "ShortsAIGuard" 꼬리표로 로그를 남긴다. 정식 앱에서는 아무것도 기록하지 않는다.

3부. 안드로이드 설정 파일 (XML, Gradle)

Java 코드가 동작하려면 "이 앱은 무엇이고, 어떤 권한이 필요하고, 어떻게 빌드하는지"를 적은 설정 파일이 필요하다. 안드로이드 설정은 **XML**(태그로 감싼 글), 빌드 설정은 **Gradle**(Groovy라는 언어) 파일로 쓴다. Capacitor가 만든 파일은 거의 그대로 쓰고, 우리가 고친 곳만 강조해서 설명한다.

> **XML 읽는 법:** `<이름 속성="값"> ... </이름>` 또는 `<이름 ... />`. 태그 안에 태그가 들어가며 나무 구조를 이룬다. `<!-- -->`는 주석. `@string/app_name`은 "strings.xml의 app_name 값을 써라"는 뜻.

AndroidManifest.xml — 앱 등록증 `android/app/src/main/AndroidManifest.xml (65줄)`

안드로이드가 앱을 설치할 때 가장 먼저 읽는 파일. 앱 이름, 화면, 서비스, 권한을 적는다.

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3      xmlns:tools="http://schemas.android.com/tools">
4
5      <application
6          android:allowBackup="true"
7          android:icon="@mipmap/ic_launcher"
8          android:label="@string/app_name"
9          android:roundIcon="@mipmap/ic_launcher_round"
10         android:supportsRtl="true"
11         android:theme="@style/AppTheme">
```

1~3줄 XML 선언과 이름 공간(안드로이드 속성, 빌드 도구 속성).

5~11줄 `application`: 앱 전체 설정. 백업 허용, 아이콘, 이름("ShortsAI"), 둥근 아이콘, 오른쪽→왼쪽 언어 지원, 기본 테마.

```
13         <activity
14             android:configChanges="orientation|keyboardHidden|keyboard|screenSize|locale|smallestScreenSize|screenLayout|uiMode|navigation|density"
15             android:name=".MainActivity"
16             android:label="@string/title_activity_main"
17             android:theme="@style/AppTheme.NoActionBarLaunch"
18             android:launchMode="singleTask"
19             android:exported="true">
20
21             <intent-filter>
22                 <action android:name="android.intent.action.MAIN" />
23                 <category android:name="android.intent.category.LAUNCHER" />
24             </intent-filter>
25
26         </activity>
```

activity (화면) MainActivity: 앱을 열면 나오는 화면.

- **14줄** 화면 회전, 키보드, 화면 크기, 다크 모드 변경 등이 있어도 화면을 새로 만들지 않는다 (웹 화면 상태 유지).
- **17줄** 열 때 스플래시(시작 화면) 테마.
- **18줄** `singleTask`: 앱 화면은 하나만 (여러 번 열어도 같은 화면).
- **19줄** `exported="true"`: 홈 화면 아이콘에서 열 수 있게 공개.
- **21~24줄** `MAIN + LAUNCHER`: "홈 화면에 아이콘을 만들어라"는 표시.

```
28         <service
29             android:name=".ShortsGuardService"
30             android:exported="true"
31             android:label="@string/app_name"
```

service ShortsGuardService — 접근성 서비스 등록.

```
32              android:permission="android.permission.BIND_ACCESSIBILITY_SERVICE">
33              <intent-filter>
34                  <action android:name="android.accessibilityservice.AccessibilityService" />
35              </intent-filter>
36              <meta-data
37                  android:name="android.accessibilityservice"
38                  android:resource="@xml/accessibility_service_config" />
39          </service>
```

- **32줄** `BIND_ACCESSIBILITY_SERVICE` 권한: **안드로이드 시스템만** 이 서비스에 연결할 수 있게 한다 (다른 앱이 함부로 부르지 못하게).
- **33~35줄** "나는 접근성 서비스다"라는 표시.
- **36~38줄** 접근성 서비스의 세부 설정 파일 (`accessibility_service_config.xml`) 위치.

```
41          <provider
42              android:name="androidx.core.content.FileProvider"
43              android:authorities="${applicationId}.fileprovider"
44              android:exported="false"
45              android:grantUriPermissions="true">
46              <meta-data
47                  android:name="android.support.FILE_PROVIDER_PATHS"
48                  android:resource="@xml/file_paths"></meta-data>
49          </provider>
50      </application>
```

provider FileProvider: Capacitor 기본 설정. 앱 파일을 다른 앱과 안전하게 공유할 때 쓰는 장치 (우리 기능에서는 쓰지 않음).

```
52      <!-- Permissions -->
53
54      <uses-permission android:name="android.permission.INTERNET" />
55      <!-- 리포트 알림과 잠금 카운트다운. 안드로이드 13 이상은 앱에서 허용을 묻는다. -->
56      <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
57      <!-- SQLite 플러그인의 암호화 잠금 해제용 권한. 쓰지 않으므로 뺀다. -->
58      <uses-permission android:name="android.permission.USE_BIOMETRIC" tools:node="remove" />
59      <uses-permission android:name="android.permission.USE_FINGERPRINT" tools:node="remove" />
```

권한

- **54줄** 인터넷: Capacitor 기본 (웹 화면 로딩 구조상 필요). 우리 앱은 외부 서버로 기록을 보내지 않는다.
- **56줄** 알림 보내기: 매일 리포트와 잠금 카운트다운. 안드로이드 13 이상은 앱에서 허용을 묻는다.
- **58~59줄 생체 인증, 지문 권한 제거**: SQLite 플러그인이 자동으로 넣는 권한인데 쓰지 않으므로 `tools:node="remove"`로 뺐다 (필요 없는 권한 최소화).

```
61      <queries>
62          <package android:name="com.google.android.youtube" />
63          <package android:name="com.instagram.android" />
64      </queries>
65  </manifest>
```

queries: 안드로이드 11부터는 다른 앱이 설치됐는지 보려면 미리 적어야 한다. YouTube와 Instagram만 적었다 (YouTube 열기 등에 필요).

accessibility_service_config.xml — 접근성 서비스 설정 `android/app/src/main/res/xml/accessibility_service_config.xml (17줄)`

접근성 서비스가 **무엇을 받고 무엇을 할 수 있는지** 정한다. 개인정보와 직결되는 파일이라 꼭 필요한 것만 켰다.

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <!--
3    No packageNames filter: window changes from other apps end a Shorts stay.
4    The service ignores everything except YouTube / Instagram ids and package names.
5    flagRetrieveInteractiveWindows: only to see whether a YouTube picture-in-picture
6    window is on screen; other windows are not opened.
7  -->
```

설명 주석: 특정 앱만 보도록 거르지 않는 이유는, 다른 앱으로 넘어간 것을 알아야 쇼츠 시청을 끝낼 수 있기 때문이다. 서비스는 YouTube/Instagram의 이름표와 앱 이름 외에는 모두 무시한다. `flagRetrieveInteractiveWindows`는 YouTube 작은 창이 떠 있는지 보는 데만 쓰고, 다른 창은 열어 보지 않는다.

```xml
<accessibility-service xmlns:android="http://schemas.android.com/apk/res/android"
    android:accessibilityEventTypes="typeWindowStateChanged|typeWindowContentChanged"
    android:accessibilityFeedbackType="feedbackGeneric"
    android:accessibilityFlags="flagReportViewIds|flagRetrieveInteractiveWindows"
    android:canRetrieveWindowContent="true"
    android:canPerformGestures="false"
    android:description="@string/accessibility_service_description"
    android:isAccessibilityTool="true"
    android:notificationTimeout="100"
    android:settingsActivity="app.focuson.shorts.MainActivity" />
```

- **9줄** 받을 신호: 창이 바뀜, 창 내용이 바뀜 (두 가지만).
- **10줄** 피드백 종류: 일반 (소리나 진동을 내는 서비스가 아님).
- **11줄** `flagReportViewIds`: 화면 부품의 **이름표(뷰 ID)**를 받는다 (쇼츠 판별의 핵심). `flagRetrieveInteractiveWindows`: 떠 있는 창 목록 (작은 창 확인).
- **12줄** 창 내용(부품 구조)을 읽을 수 있음. **13줄** 화면 터치 흉내는 **쓰지 않음**.
- **14줄** 설정 화면에 보일 설명 (strings.xml).
- **15줄** 접근성 도구로 표시. **16줄** 같은 신호는 0.1초 간격으로 묶어서 받음.
- **17줄** 접근성 설정에서 "설정"을 누르면 우리 앱이 열림.

strings.xml — 앱 글자 모음 `android/app/src/main/res/values/strings.xml (8줄)`

```xml
<?xml version='1.0' encoding='utf-8'?>
<resources>
    <string name="app_name">ShortsAI</string>
    <string name="title_activity_main">ShortsAI</string>
    <string name="package_name">app.focuson.shorts</string>
    <string name="custom_url_scheme">app.focuson.shorts</string>
    <string name="accessibility_service_description">YouTube Shorts 플레이어와 Instagram Reels 플레이어가 열렸는지만 화면 ID로 확인합니다. 쇼츠가 열리면 영상을 멈추고 그 위에 ShortsAI의 질문을 띄웁니다. 다른 앱으로 넘어간 것은 앱 이름만으로 판단하고, 쇼츠를 작은 창(PIP)으로 계속 보는지는 화면에 떠 있는 YouTube 작은 창만 확인합니다. 화면의 글, 메시지, 비밀번호, 계정은 읽거나 저장하거나 보내지 않습니다.</string>
</resources>
```

- **3~4줄** 앱 이름과 화면 제목 "ShortsAI".
- **5~6줄** 패키지 이름(앱 고유 주소) `app.focuson.shorts`.
- **7줄 접근성 서비스 설명**: 폰의 "설정 → 접근성 → ShortsAI"에 보이는 문장. 무엇을 확인하고, 무엇을 하고, 화면 글·메시지·비밀번호·계정은 읽지도 저장하지도 보내지도 않는다는 약속을 적었다.

styles.xml — 앱 테마 `android/app/src/main/res/values/styles.xml (22줄)`

```xml
<?xml version="1.0" encoding="utf-8"?>
<resources>

    <!-- Base application theme. -->
    <style name="AppTheme" parent="Theme.AppCompat.Light.DarkActionBar">
        <!-- Customize your theme here. -->
        <item name="colorPrimary">@color/colorPrimary</item>
        <item name="colorPrimaryDark">@color/colorPrimaryDark</item>
        <item name="colorAccent">@color/colorAccent</item>
    </style>

    <style name="AppTheme.NoActionBar" parent="Theme.AppCompat.DayNight.NoActionBar">
        <item name="windowActionBar">false</item>
        <item name="windowNoTitle">true</item>
        <item name="android:background">@null</item>
    </style>
```

Capacitor 기본 테마 설정. 기본 테마(색), 상단 바 없는 테마, 앱을 열 때 잠깐 보이는 스플래시 테마. 실제 화면 디자인은 웹(CSS)에서 하므로 여기서는 거의 고치지 않았다.

```
17
18
19      <style name="AppTheme.NoActionBarLaunch" parent="Theme.SplashScreen">
20          <item name="android:background">@drawable/splash</item>
21      </style>
22  </resources>
```

activity_main.xml — 앱 화면 틀 `android/app/src/main/res/layout/activity_main.xml (12줄)`

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <androidx.coordinatorlayout.widget.CoordinatorLayout xmlns:android="http://schemas.android.com/apk/res/android"
3      xmlns:app="http://schemas.android.com/apk/res-auto"
4      xmlns:tools="http://schemas.android.com/tools"
5      android:layout_width="match_parent"
6      android:layout_height="match_parent"
7      tools:context=".MainActivity">
8
9      <WebView
10         android:layout_width="match_parent"
11         android:layout_height="match_parent" />
12 </androidx.coordinatorlayout.widget.CoordinatorLayout>
```

화면 전체를 웹 화면(WebView) 하나로 채우는 틀. Capacitor 기본 파일이다. 실제 내용(홈, 통계, 설정)은 모두 이 WebView 안에서 웹으로 그려진다.

file_paths.xml — 파일 공유 경로 `android/app/src/main/res/xml/file_paths.xml (5줄)`

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <paths xmlns:android="http://schemas.android.com/apk/res/android">
3      <external-path name="my_images" path="." />
4      <cache-path name="my_cache_images" path="." />
5  </paths>
```

Capacitor 기본 파일. 위 FileProvider가 공유할 수 있는 폴더(외부 저장소, 임시 폴더)를 정한다. 우리 기능에서는 쓰지 않는다.

ic_launcher_background.xml — 아이콘 배경색 `android/app/src/main/res/values/ic_launcher_background.xml (4줄)`

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <resources>
3      <color name="ic_launcher_background">#162030</color>
4  </resources>
```

앱 아이콘 뒤 배경색 `#162030`(짙은 남색). 아이콘 그림은 `scripts/make-icons.py`로 만들었다.

app/build.gradle — 앱 빌드 설정 `android/app/build.gradle (77줄)`

APK를 어떻게 만들지 정한다. **버전을 package.json 하나에서 가져오게** 하고, **정식 서명**을 할 수 있게 고쳤다.

```
1  import groovy.json.JsonSlurper
2
3  apply plugin: 'com.android.application'
4
5  // One version for web and APK: package.json "version" (e.g. 0.7.0 -> versionCode 700).
6  def appVersionName = new JsonSlurper().parse(rootProject.file('../package.json')).version
7  def appVersionCode = appVersionName.tokenize('-')[0].tokenize('.').collect { it as int }
8      .with { it[0] * 10000 + it[1] * 100 + it[2] }
```

1줄 JSON 읽기 도구. **3줄** "이건 안드로이드 앱이다".

5~8줄 버전 자동 계산: `package.json`의 `"version": "0.9.2"`를 읽어 버전 이름으로 쓰고, 버전 번호(versionCode)는 0×10000 + 9×100 + 2 = **902**로 계산한다. 그래서 버전을 한 곳(package.json)만 바꾸면 웹과 APK가 같이 바뀐다. 번호가 커야 폰이 "업데이트"로 인식한다.

```
10 // Release signing. android/keystore.properties and the .jks stay out of git.
11 def keystoreFile = rootProject.file('keystore.properties')
12 def keystoreProps = new Properties()
13 if (keystoreFile.exists()) keystoreFile.withInputStream { keystoreProps.load(it) }
```

정식 서명 준비: `android/keystore.properties` 파일이 있으면 그 안의 서명 키 정보를 읽는다. 이 파일과 키(.jks)는 비밀이라 git에 올리지 않는다(.gitignore).

```
15 android {
16     namespace = "app.focuson.shorts"
17     compileSdk = rootProject.ext.compileSdkVersion
18     defaultConfig {
19         applicationId "app.focuson.shorts"
20         minSdkVersion rootProject.ext.minSdkVersion
21         targetSdkVersion rootProject.ext.targetSdkVersion
22         versionCode appVersionCode
23         versionName appVersionName
24         testInstrumentationRunner "androidx.test.runner.AndroidJUnitRunner"
25         aaptOptions {
26              // Files and dirs to omit from the packaged assets dir, modified to accommodate mod
ern web apps.
27              // Default: https://android.googlesource.com/platform/frameworks/base/+/282e181b58c
f72b6ca770dc7ca5f91f135444502/tools/aapt/AaptAssets.cpp#61
28             ignoreAssetsPattern = '!.svn:!.git:!.ds_store:!*.scc:.*:!CVS:!thumbs.db:!picasa.in
i:!*~'
29         }
30     }
```

- **16줄** 코드 묶음 이름. **17줄** 빌드할 안드로이드 버전(36).
- **19줄 앱 고유 주소** `app.focuson.shorts` (팀원 앱과 다름. 그래서 둘 다 설치 가능).
- **20~21줄** 최소 안드로이드 7.0(24), 대상 16(36).
- **22~23줄** 위에서 계산한 버전 번호와 이름.
- **24줄** 기기 테스트 실행 도구. **25~29줄** 웹 파일을 넣을 때 빼는 파일 규칙(Capacitor 기본).

```
31     signingConfigs {
32         if (keystoreFile.exists()) {
33             release {
34                 storeFile rootProject.file(keystoreProps['storeFile'])
35                 storePassword keystoreProps['storePassword']
36                 keyAlias keystoreProps['keyAlias']
37                 keyPassword keystoreProps['keyPassword']
38             }
39         }
40     }
41     buildTypes {
```

signingConfigs / buildTypes: keystore.properties가 있을 때만 release(정식) 서명을 설정한다. 키 파일 위치, 비밀번호, 별칭을 그 파일에서 읽는다(값은 이 파일에 없음). `minifyEnabled false`: 코드 압축은 하지 않는다.

```
        release {
            if (keystoreFile.exists()) signingConfig signingConfigs.release
            minifyEnabled false
            proguardFiles getDefaultProguardFile('proguard-android.txt'), 'proguard-rules.pro'
        }
    }
}
```

```
repositories {
    flatDir{
        dirs '../capacitor-cordova-android-plugins/src/main/libs', 'libs'
    }
}

dependencies {
    implementation fileTree(include: ['*.jar'], dir: 'libs')
    implementation "androidx.appcompat:appcompat:$androidxAppCompatVersion"
    implementation "androidx.coordinatorlayout:coordinatorlayout:$androidxCoordinatorLayoutVersion"
    implementation "androidx.core:core-splashscreen:$coreSplashScreenVersion"
    implementation project(':capacitor-android')
    testImplementation "junit:junit:$junitVersion"
    androidTestImplementation "androidx.test.ext:junit:$androidxJunitVersion"
    androidTestImplementation "androidx.test.espresso:espresso-core:$androidxEspressoCoreVersion"
    implementation project(':capacitor-cordova-android-plugins')
}
```

repositories / dependencies: 가져다 쓰는 라이브러리. AndroidX(호환 도구), 스플래시 화면, Capacitor 본체, JUnit(테스트), Cordova 플러그인 호환 묶음.

```
apply from: 'capacitor.build.gradle'

try {
    def servicesJSON = file('google-services.json')
    if (servicesJSON.text) {
        apply plugin: 'com.google.gms.google-services'
    }
} catch(Exception e) {
    logger.info("google-services.json not found, google-services plugin not applied. Push Notifications won't work")
}
```

68줄 Capacitor가 만든 추가 설정(capacitor.build.gradle)을 불러온다.

70~77줄 Google 서비스 설정 파일이 있으면 푸시 알림 도구를 켠다 (Capacitor 기본. 우리는 이 파일이 없어서 꺼진 채로 동작한다. 알림은 서버 없는 로컬 알림만 사용).

capacitor.build.gradle — Capacitor 자동 생성 `android/app/capacitor.build.gradle (20줄)`

```
// DO NOT EDIT THIS FILE! IT IS GENERATED EACH TIME "capacitor update" IS RUN

android {
  compileOptions {
      sourceCompatibility JavaVersion.VERSION_21
      targetCompatibility JavaVersion.VERSION_21
  }
```

1줄 "고치지 마시오, Capacitor가 다시 만든다".

3~8줄 Java 21 버전으로 컴파일. **11~15줄** 사용하는 플러그인 2개: SQLite(데이터베이스), 로컬 알림.

```
}

apply from: "../capacitor-cordova-android-plugins/cordova.variables.gradle"
dependencies {
    implementation project(':capacitor-community-sqlite')
    implementation project(':capacitor-local-notifications')

}


if (hasProperty('postBuildExtras')) {
  postBuildExtras()
}
```

build.gradle (최상위) — 전체 빌드 설정 `android/build.gradle (29줄)`

```
// Top-level build file where you can add configuration options common to all sub-projects/modules.

buildscript {

    repositories {
        google()
        mavenCentral()
    }
    dependencies {
        classpath 'com.android.tools.build:gradle:8.13.0'
        classpath 'com.google.gms:google-services:4.4.4'

        // NOTE: Do not place your application dependencies here; they belong
        // in the individual module build.gradle files
    }
}

apply from: "variables.gradle"

allprojects {
    repositories {
        google()
        mavenCentral()
    }
}

task clean(type: Delete) {
    delete rootProject.buildDir
}
```

5~16줄 빌드 도구 자체의 버전: 안드로이드 Gradle 플러그인 **8.13.0**, Google 서비스 4.4.4. 라이브러리 저장소는 Google, Maven Central.

18줄 버전 숫자 모음(variables.gradle)을 불러온다. **27~29줄** `clean` 명령: 빌드 결과 폴더 지우기.

variables.gradle — 버전 숫자 모음 `android/variables.gradle (16줄)`

```
ext {
    minSdkVersion = 24
    compileSdkVersion = 36
    targetSdkVersion = 36
    androidxActivityVersion = '1.11.0'
    androidxAppCompatVersion = '1.7.1'
    androidxCoordinatorLayoutVersion = '1.3.0'
    androidxCoreVersion = '1.17.0'
    androidxFragmentVersion = '1.8.9'
    coreSplashScreenVersion = '1.2.0'
    androidxWebkitVersion = '1.14.0'
    junitVersion = '4.13.2'
    androidxJunitVersion = '1.3.0'
    androidxEspressoCoreVersion = '3.7.0'
    cordovaAndroidVersion = '14.0.1'
}
```

여러 파일에서 같이 쓰는 숫자: 최소 안드로이드 24(7.0), 빌드·대상 36(16), AndroidX 라이브러리 버전들, JUnit 4.13.2. Capacitor 기본값.

settings.gradle — 프로젝트 구성 `android/settings.gradle (5줄)`

```
include ':app'
include ':capacitor-cordova-android-plugins'
project(':capacitor-cordova-android-plugins').projectDir = new File('./capacitor-cordova-android-plugins/')

apply from: 'capacitor.settings.gradle'
```

빌드에 포함할 부분: 우리 앱(`:app`), Cordova 플러그인 호환 묶음, 그리고 Capacitor가 만든 목록(capacitor.settings.gradle).

capacitor.settings.gradle — Capacitor 플러그인 위치 `android/capacitor.settings.gradle (9줄)`

```
// DO NOT EDIT THIS FILE! IT IS GENERATED EACH TIME "capacitor update" IS RUN
include ':capacitor-android'
project(':capacitor-android').projectDir = new File('../node_modules/@capacitor/android/capacitor')

include ':capacitor-community-sqlite'
project(':capacitor-community-sqlite').projectDir = new File('../node_modules/@capacitor-community/sqlite/android')

include ':capacitor-local-notifications'
project(':capacitor-local-notifications').projectDir = new File('../node_modules/@capacitor/local-notifications/android')
```

자동 생성 파일. Capacitor 본체, SQLite 플러그인, 로컬 알림 플러그인의 안드로이드 코드가 `node_modules`(웹 라이브러리 폴더) 안 어디에 있는지 알려 준다. 그래서 `npm install`을 먼저 해야 안드로이드 빌드가 된다.

gradle.properties — 빌드 도구 옵션 `android/gradle.properties (22줄)`

```
# Project-wide Gradle settings.

# IDE (e.g. Android Studio) users:
# Gradle settings configured through the IDE *will override*
# any settings specified in this file.

# For more details on how to configure your build environment visit
# http://www.gradle.org/docs/current/userguide/build_environment.html

# Specifies the JVM arguments used for the daemon process.
# The setting is particularly useful for tweaking memory settings.
org.gradle.jvmargs=-Xmx1536m

# When configured, Gradle will run in incubating parallel mode.
# This option should only be used with decoupled projects. More details, visit
# http://www.gradle.org/docs/current/userguide/multi_project_builds.html#sec:decoupled_projects
# org.gradle.parallel=true

# AndroidX package structure to make it clearer which packages are bundled with the
# Android operating system, and which are packaged with your app's APK
# https://developer.android.com/topic/libraries/support-library/androidx-rn
android.useAndroidX=true
```

Gradle이 쓸 메모리(최대 1.5GB)와 AndroidX 사용 설정. 나머지는 설명 주석. Capacitor 기본값.

proguard-rules.pro — 코드 압축 규칙 `android/app/proguard-rules.pro (21줄)`

```
# Add project specific ProGuard rules here.
# You can control the set of applied configuration files using the
# proguardFiles setting in build.gradle.
#
# For more details, see
#   http://developer.android.com/guide/developing/tools/proguard.html

# If your project uses WebView with JS, uncomment the following
# and specify the fully qualified class name to the JavaScript interface
# class:
#-keepclassmembers class fqcn.of.javascript.interface.for.webview {
#   public *;
#}

# Uncomment this to preserve the line number information for
# debugging stack traces.
#-keepattributes SourceFile,LineNumberTable

# If you keep the line number information, uncomment this to
```

코드 압축·난독화(ProGuard) 규칙 파일. 모두 주석이고 실제 규칙은 없다. 우리 앱은 압축을 끈 상태(`minifyEnabled false`)라 쓰이지 않는다.

```
# hide the original source file name.
#-renamesourcefileattribute SourceFile
```

4부. 앱 화면 쪽 계산과 저장 (lib/, TypeScript)

`lib/` 폴더는 화면 없이 **계산, 저장, AI**만 하는 코드다. 화면 부품(components)이 여기 함수를 가져다 쓴다. TypeScript는 JavaScript에 "자료 모양 검사"를 더한 언어라, 코드를 실행하기 전에 "숫자 자리에 글자를 넣었다" 같은 실수를 잡아 준다.

TypeScript 읽는 법:

- `export type 이름 = {...}`: 데이터 모양 정의. 실제 실행되는 코드가 아니라 "이런 모양이어야 한다"는 약속.
- `a: number` = a는 숫자, `string` 글자, `boolean` 참/거짓, `A | B` = A 또는 B, `X[]` = X의 목록, `Record<K, V>` = K→V 표.
- `이름?: 타입` = 있어도 되고 없어도 되는 칸. `null` = 값 없음.
- `export function 이름(인자: 타입): 결과타입 { ... }` = 다른 파일에서도 쓸 수 있는 함수.
- `const x = ...` 바꾸지 않는 값, `let x = ...` 바꿀 수 있는 값.
- `(a) => a + 1` = 짧은 함수. `목록.map(f)` = 각각 바꾸기, `.filter(f)` = 조건에 맞는 것만, `.reduce` = 하나로 합치기.
- `{ ...obj, x: 1 }` = obj를 복사하면서 x만 바꾸기. `a ?? b` = a가 없으면 b. `a?.b` = a가 있을 때만 b.

types.ts — 데이터 모양 정의 `lib/types.ts (284줄)`

앱이 다루는 모든 데이터의 **모양(type)**을 정의한다. 실행되는 코드는 없고, 다른 파일들이 이 모양을 지키는지 TypeScript가 검사한다.

```
1  export type Level = 1 | 2 | 3;
2
3  /**
4   * 개입 결과.
5   * exit = 나가기, kept = 약속 시간 지키고 나가기, day-end = 오늘은 끝, timed-block = 시간 잠금,
6   * watch = 보기, extend = 약속 연장, overdraft = 목표 초과 "3분만 더",
7   * budget-out = 약속할 시간이 남지 않아 강함 단계로 넘어감, interrupted = 다른 시트로 바뀜.
8   */
9  export type Outcome =
10   | "exit"
11   | "watch"
12   | "force-quit"
13   | "timed-block"
14   | "blocked-retry"
15   | "kept"
16   | "extend"
17   | "day-end"
18   | "overdraft"
19   | "budget-out"
20   | "interrupted";
```

1줄 `Level`: 예전 단계 번호 1·2·3.

3~20줄 `Outcome` — 개입 결과 종류 11가지: 나가기(exit), 보기(watch), 강제 종료(force-quit, 예전), 시간 잠금(timed-block), 잠금 중 다시 열기(blocked-retry), 약속 지킴(kept), 연장(extend), 오늘은 끝(day-end), 3분만 더(overdraft), 예산 끝(budget-out), 다른 시트로 바뀜(interrupted).

```ts
/**
 * level3Threshold = 하루 목표(분). 강도는 목표 대비 비율로 정한다 (50% 중간, 100% 강함).
 * level1Threshold는 목표의 절반으로 자동 계산한다 (이전 버전 호환용).
 */
export type Settings = {
  level1Threshold: number;
  level2Threshold: number;
  level3Threshold: number;
  notificationTime: string;
  notifyEnabled: boolean;
  /** 목표를 줄여 갈 최종 목표(분). AI 단계에서 쓴다. */
  finalGoal: number;
  /** "오늘은 끝" 잠금이 풀리는 시각(0-23시). */
  dayEndHour: number;
  /** 일어나는 시각 HH:MM. 밤의 "잠과 비교" 문구에 쓴다. 다음 알람이 있으면 알람을 먼저 쓴다. */
  wakeTime: string;
  /** AI 개인화. 끄면 규칙대로 동작한다 (AI 전후 비교용). */
  aiEnabled: boolean;
  /** 엄격 모드: 연장과 "3분만 더"가 없다. 사용자가 직접 켠다. */
  strictMode: boolean;
  /**
   * 밴딧 학습 개선 (v0.9.1): 칸 사이 공유, 최근 가중치, 세밀한 보상, 평일/주말, 질림 반영.
   * 끄면 v0.9.0과 같은 계산. 화면에는 없고, 전후 비교가 필요할 때 쓴다.
   */
  banditV2: boolean;
  /** 공부 모드: 약함·중간 단계의 개입을 공부 카드로만 한다 (카드가 없으면 평소대로). */
  studyMode: boolean;
  /** 주간 목표 제안을 이미 답한 주 (월요일 날짜). */
  goalWeekAnswered: string;
};
```

Settings — 설정 모양.

- `level1/2/3Threshold`: 단계 기준(분). `level3Threshold`가 곧 **하루 목표**. level1은 목표의 절반으로 자동 계산 (예전 버전 호환).
- `notificationTime / notifyEnabled`: 매일 리포트 알림 시각("21:00")과 켜기.
- `finalGoal`: 줄여 갈 최종 목표. `dayEndHour`: "오늘은 끝" 풀리는 시. `wakeTime`: 일어나는 시각("07:00").
- `aiEnabled`: AI 개인화. `strictMode`: 엄격 모드. `banditV2`: AI 학습 개선 (v0.9.1, 화면에 없음). `studyMode`: 공부 모드.
- `goalWeekAnswered`: 주간 목표 제안에 이미 답한 주.

```ts
export type SessionTarget = "youtube" | "instagram" | "practice";

/**
 * 계획서 UsageLog. 한 번의 시청 세션. duration 단위는 초.
 * date는 시작한 날짜이고, 자정을 넘긴 세션은 secondsOnDate()가 날짜별로 나눈다.
 * source: native = 접근성 서비스가 잰 실제 쇼츠, web = 연습 피드·이관한 기록.
 */
export type UsageLog = {
  usageId: string;
  date: string;
  startTime: string;
  endTime: string;
  duration: number;
  clipCount: number;
  target: SessionTarget;
  source: "native" | "web";
};
```

SessionTarget: 시청 대상 — youtube, instagram, practice(브라우저 연습 화면).

UsageLog — 시청 한 번. 고유 id, 날짜(시작한 날), 시작·끝 시각(ISO 글자, 예: "2026-10-13T12:00:00.000Z"), 길이(초), 영상 수, 대상, 출처(native = 접근성 서비스가 잰 실제 쇼츠, web = 연습·이관 기록).

```ts
/**
 * 개입 화면에서 보여준 방식. 웹 흐름: pause / alternative / warning.
 * 화면 위 시트: pause, framing(시간 바꿔 말하기), breath, alternative, reflection, card,
 * commit_end(약속 시간 끝), strong(마무리 선택), locked.
 */
export type InterventionType =
  | "pause"
  | "alternative"
  | "warning"
  | "framing"
  | "breath"
  | "reflection"
  | "card"
  | "commit_end"
  | "strong"
  | "locked"
  | "pip";

/** 강도: 1 가벼움(목표 50% 미만), 2 중간(100% 미만), 3 강함(목표 초과). */
export type Band = 1 | 2 | 3;
```

InterventionType: 개입 방식 이름 11가지. 웹 예전 흐름(pause/alternative/warning)과 시트 방식(framing, breath, reflection, card), 단계 이름(commit_end, strong, locked), 작은 창(pip).

Band: 강도 1 약함, 2 중간, 3 강함.

```ts
/** 계획서 InterventionLog. date는 기기 로컬 날짜. */
export type InterventionLog = {
  interventionId: string;
  timestamp: string;
  date: string;
  level: Level;
  reason: string;
  reasonNote: string;
  alternativeAction: string | null;
  alternativeCompleted: boolean;
  reEntered: boolean;
  blocked: boolean;
  blockDuration: number;
  usageSeconds: number;
  outcome: Outcome;
  /** 기기 로컬 시각(0-23)과 요일(0=일). AI 학습용. */
  hour: number;
  weekday: number;
  target: SessionTarget | null;
  interventionType: InterventionType;
  /** 나간 뒤 30분 안에 다시 쇼츠를 열었는지. 30분이 지나기 전이거나 시청을 고른 경우 null. */
  reenteredWithin30: boolean | null;
  /** 도움이 됐는지. AI 단계에서 쓴다. */
  feedback: "helpful" | "not" | null;
  /** 아래는 v0.8.0 시트 기록 (AI 학습 재료). 웹 흐름에서는 대부분 null. */
  band: Band | null;
  /** entry / commit_end / strong / web */
  stage: string | null;
  /** 시간 바꿔 말하기 종류: shorts / walk / book / sleep / study */
```

InterventionLog — 개입 한 번의 기록 (AI 학습 재료).

- 기본: id, 시각, 날짜, 단계, 이유, 이유 메모, 대체 행동과 완료 여부, 재진입, 잠금 여부와 길이, 사용 초, 결과.
- **107~109줄** 시(0~23)와 요일(0 = 일요일): AI가 시간대·요일별로 배우는 데 쓴다.
- **112~113줄** `reenteredWithin30`: 나간 뒤 30분 안에 다시 쇼츠를 열었는지. 아직 30분이 안 지났거나 보기를 골랐으면 null.
- **115줄** `feedback`: "도움이 됐나요?" 답.
- **116~130줄** 시트 기록: 강도, 단계(entry 등), 문구 종류, 참여 확인 방법, 성찰 답, 잠금 뒤 답, 약속 분, 카드 id와 정답 여부, 연장 순서, 정책 버전.

```
  framing: string | null;
  /** 참여 확인: verified / self / answered / waited / seen / skipped */
  verification: string | null;
  reflection: string | null;
  afterLock: string | null;
  commitMinutes: number;
  cardId: string | null;
  cardCorrect: boolean | null;
  extensionIndex: number;
  policyVersion: string | null;
};
```

```ts
/** 내 공부 카드. 직접 입력하거나 ChatGPT·Gemini 결과를 붙여넣어 만든다. */
export type StudyCard = {
  id: string;
  question: string;
  options: string[];
  /** 정답 보기의 0부터 시작하는 번호 */
  answer: number;
  createdAt: string;
  source: "manual" | "import";
  shown: number;
  correct: number;
  /** 풀이 뒤에 보여 줄 해설. 없으면 "". */
  explanation: string;
};

/** 시간 차단 중에 쇼츠를 연 기록. */
export type LockAttempt = {
  id: string;
  createdAt: string;
  date: string;
  remainingSec: number;
  target: SessionTarget | null;
};

export type ActiveSession = {
  usageId: string;
  date: string;
  startTime: string;
  duration: number;
  clipCount: number;
  interventionId: string;
  levelAtStart: Level;
  lastTickAt: number;
};

export type BlockState = {
  until: string | null;
  minutes: number;
};
```

- **StudyCard**: 공부 카드 — id, 문제, 보기 목록, 정답 번호(0부터), 만든 시각, 출처(직접/가져오기), 나온 횟수, 맞힌 횟수, 해설.
- **LockAttempt**: 잠금 중 쇼츠를 연 기록 — id, 시각, 날짜, 남은 초, 대상.
- **ActiveSession**: 브라우저 연습 화면에서 보는 중인 시청.
- **BlockState**: 잠금 상태 (끝 시각, 분).

```ts
export type AppData = {
  settings: Settings;
  usageLogs: UsageLog[];
  interventions: InterventionLog[];
  lockAttempts: LockAttempt[];
  cards: StudyCard[];
  activeSession: ActiveSession | null;
  block: BlockState;
  lastNotifiedDate: string | null;
};
```

AppData — 앱 데이터 전체. 설정, 시청 기록 목록, 개입 기록 목록, 잠금 시도 목록, 공부 카드 목록, 보는 중인 시청, 잠금 상태, 마지막 리포트 알림 날짜. 화면들은 이 한 덩어리를 보고 그린다.

```ts
export type DailyStats = {
  date: string;
  usageSeconds: number;
  clipCount: number;
  interventionCount: number;
  levelCounts: Record<Level, number>;
  topReason: string | null;
  reentryCount: number;
  blockCount: number;
  forceQuitCount: number;
  alternativeCompletedCount: number;
  lockAttemptCount: number;
  /** 스스로 멈춘 횟수: 나가기, 약속 지키기, 오늘은 끝, 시간 잠금 */
  selfStopCount: number;
  keptCount: number;
  dayEndCount: number;
  overdraftCount: number;
  extendCount: number;
  cardsSolved: number;
  cardsCorrect: number;
  /** 시간 잠금으로 쉰 초 ("오늘은 끝"은 빼고). */
  restSeconds: number;
  /** 이 날까지 잠금이나 "오늘은 끝"을 고른 날이 이어진 수. */
  lockStreak: number;
};
```

DailyStats — 하루 통계 (리포트와 통계 화면용): 사용 초, 영상 수, 개입 수, 단계별 수, 가장 많은 이유, 재진입 수, 잠금 수, 강제 종료 수, 대체 행동 완료 수, 잠금 중 시도 수, **스스로 멈춘 횟수**, 약속 지킨 수, 오늘은 끝 수, 3분만 더 수, 연장 수, 푼 카드와 맞힌 카드, 쉰 초, 연속 잠금 날 수.

```ts
/** 화면에서 넘기는 개입 결과. 나머지 필드는 기록할 때 채운다. */
export type InterventionInput = Omit<
  InterventionLog,
  | "interventionId"
  | "timestamp"
  | "date"
  | "hour"
  | "weekday"
  | "target"
  | "interventionType"
  | "reenteredWithin30"
  | "feedback"
  | "band"
  | "stage"
  | "framing"
  | "verification"
  | "reflection"
  | "afterLock"
  | "commitMinutes"
  | "cardId"
  | "cardCorrect"
  | "extensionIndex"
  | "policyVersion"
> & { target?: SessionTarget | null };
```

InterventionInput: 앱 화면(Instagram 등)에서 개입 결과를 넘길 때의 모양. `Omit<A, "x" | "y">`는 "A에서 x, y 칸을 뺀 모양"이라는 뜻. 시각, 날짜, 시트 전용 칸들은 기록할 때 자동으로 채우므로 뺐다.

```ts
/** 접근성 서비스가 쌓아 둔 기록. GuardPlugin.sync()가 넘겨준다. 시각은 epoch ms. */
export type NativeSession = {
  id: string;
  target: string;
  startedAt: number;
  endedAt: number;
  durationSec: number;
};

export type NativeIntervention = {
  id: string;
  createdAt: number;
  target: string;
  level: number;
  reason: string;
  interventionType: string;
  alternativeAction: string | null;
  outcome: string;
  usageSeconds: number;
  band?: number;
  stage?: string;
  method?: string | null;
  framing?: string | null;
  verification?: string | null;
  reflection?: string | null;
  afterLock?: string | null;
  cardId?: string | null;
  cardCorrect?: boolean | null;
  commitMinutes?: number;
  blockMinutes?: number;
  extensionIndex?: number;
  policyVersion?: string;
  /** stage "card_report": 사용자가 "이상한 문제"로 신고한 카드는 cardId에 담긴다. */
  /** stage "feedback": answer to "도움이 됐나요?" for the row refId */
  refId?: string;
  feedback?: string;
};

export type NativeLockAttempt = {
  id: string;
  createdAt: number;
  remainingSec: number;
  target: string;
};

export type NativeBatch = {
  sessions: NativeSession[];
  interventions: NativeIntervention[];
  lockAttempts: NativeLockAttempt[];
};
```

휴대폰 쪽(Java)에서 넘어오는 기록 모양 (`GuardPlugin.sync`의 결과). 시각은 1970년부터의 밀리초 숫자.

- **NativeSession**: 시청 (id, 대상, 시작, 끝, 초).
- **NativeIntervention**: 개입 (Java의 `Rec.toJson`과 같은 칸들). 신고 기록은 단계 "card_report"에 카드 id, 피드백 기록은 단계 "feedback"에 `refId`와 답.
- **NativeLockAttempt**: 잠금 시도.
- **NativeBatch**: 세 목록을 묶은 한 덩어리.

storage.ts — 기본값과 안전하게 읽기 `lib/storage.ts (349줄)`

설정 기본값을 정하고, 저장된 데이터를 **한 줄씩 검사하며** 읽는다. 일부 기록이 깨져 있어도 그 줄만 버리고 나머지는 살린다 (예전 버전은 하나만 틀려도 전체를 버렸다).

```ts
import { asTarget, finalizeSession, todayKey } from "./logic.ts";
import type {
  ActiveSession,
  AppData,
  BlockState,
  InterventionLog,
  InterventionType,
  LockAttempt,
  Outcome,
  Settings,
  StudyCard,
  UsageLog,
} from "./types.ts";

/** 브라우저(웹) 저장 키. 안드로이드는 SQLite를 쓰고, 첫 실행 때 이 키의 기록을 옮긴다. */
export const STORAGE_KEY = "hanbakja.v1";
export const MIGRATED_KEY = "hanbakja.v1.migrated";
```

1~13줄 다른 파일에서 함수와 모양을 가져온다 (`import`). `import type`은 모양만 가져온다는 뜻.

15~17줄 브라우저 저장소의 키 이름 `hanbakja.v1`(팀원 앱에서 이어받음)과, 안드로이드로 옮긴 뒤 붙이는 이름.

```ts
export function defaultSettings(): Settings {
  return {
    level1Threshold: 10,
    level2Threshold: 20,
    level3Threshold: 20,
    notificationTime: "21:00",
    notifyEnabled: true,
    finalGoal: 10,
    dayEndHour: 6,
    wakeTime: "07:00",
    aiEnabled: true,
    strictMode: false,
    banditV2: true,
    studyMode: false,
    goalWeekAnswered: "",
  };
}
```

defaultSettings — 처음 설치했을 때의 설정. 하루 목표 **20분**(중간 단계 시작 10분), 리포트 알림 21:00 켜짐, 최종 목표 10분, "오늘은 끝" 6시, 일어나는 시각 7:00, AI 켜짐, 엄격 모드 꺼짐, 학습 개선 켜짐, 공부 모드 꺼짐. **기본 목표를 바꾸려면 여기를 고친다.**

```ts
/** 하루 목표에서 나머지 기준을 만든다. level1(중간 강도 시작)은 목표의 절반. */
export function settingsFromGoal(
  goal: number,
  rest: Pick<
    Settings,
    | "notificationTime"
    | "notifyEnabled"
    | "finalGoal"
    | "dayEndHour"
```

settingsFromGoal: 하루 목표 하나로 단계 기준을 만든다. 중간 시작 = 목표의 절반(최소 1분), 나머지 두 기준 = 목표. 나머지 설정은 그대로 붙인다(`...rest`). `Pick<Settings, ...>`는 "Settings에서 이 칸들만".

```
    | "wakeTime"
    | "aiEnabled"
    | "strictMode"
    | "banditV2"
    | "studyMode"
    | "goalWeekAnswered"
  >,
): Settings {
  return {
    level1Threshold: Math.max(1, Math.round(goal / 2)),
    level2Threshold: goal,
    level3Threshold: goal,
    ...rest,
  };
}
```

```
export function defaultData(): AppData {
  return {
    settings: defaultSettings(),
    usageLogs: [],
    interventions: [],
    lockAttempts: [],
    cards: [],
    activeSession: null,
    block: { until: null, minutes: 0 },
    lastNotifiedDate: null,
  };
}
```

defaultData: 빈 앱 데이터 (기본 설정, 빈 목록들, 잠금 없음).

```
function isLevel(value: unknown): value is 1 | 2 | 3 {
  return value === 1 || value === 2 || value === 3;
}

const OUTCOMES: Outcome[] = [
  "exit",
  "watch",
  "force-quit",
  "timed-block",
  "blocked-retry",
  "kept",
  "extend",
  "day-end",
  "overdraft",
  "budget-out",
  "interrupted",
];
const TYPES: InterventionType[] = [
  "pause",
  "alternative",
  "warning",
  "framing",
```

- **isLevel**: 값이 1, 2, 3 중 하나인지.
- **OUTCOMES / TYPES**: 허용하는 결과와 방식 이름 목록. 저장된 값이 이 목록에 없으면 버리거나 기본값으로 바꾼다.
- **text**: 빈 글자가 아닌 글자면 그대로, 아니면 null.
- **count**: 정상 숫자면 0 이상 정수로, 아니면 0.

```
  "breath",
  "reflection",
  "card",
  "commit_end",
  "strong",
  "locked",
  "pip",
];

const text = (value: unknown): string | null =>
  typeof value === "string" && value !== "" ? value : null;
const count = (value: unknown): number =>
  typeof value === "number" && Number.isFinite(value) ? Math.max(0, Math.round(value)) : 0;
```

```
/** 한 줄씩 읽는다. 형식이 틀린 줄만 버리고 나머지는 살린다. */
export function readUsageLog(value: unknown): UsageLog | null {
  if (!value || typeof value !== "object") return null;
  const log = value as Partial<UsageLog>;
  if (
    typeof log.usageId !== "string" ||
    typeof log.startTime !== "string" ||
    typeof log.duration !== "number" ||
    !Number.isFinite(log.duration)
  ) {
    return null;
  }
  const start = new Date(log.startTime);
  if (Number.isNaN(start.getTime())) return null;
  return {
    usageId: log.usageId,
    date: typeof log.date === "string" ? log.date : todayKey(start),
    startTime: log.startTime,
    endTime: typeof log.endTime === "string" ? log.endTime : log.startTime,
    duration: Math.max(0, log.duration),
    clipCount: typeof log.clipCount === "number" ? log.clipCount : 1,
    target: asTarget(log.target) ?? "practice",
    source: log.source === "native" ? "native" : "web",
  };
}
```

readUsageLog — 시청 기록 한 줄 읽기. id·시작 시각·길이가 없거나 시각이 날짜로 안 읽히면 버린다(null). 나머지 칸은 없으면 기본값으로 채운다 (날짜는 시작 시각에서, 끝 시각은 시작과 같게, 영상 수 1, 대상 practice, 출처 web).

```
export function readIntervention(value: unknown): InterventionLog | null {
  if (!value || typeof value !== "object") return null;
  const log = value as Partial<InterventionLog>;
  if (
    typeof log.interventionId !== "string" ||
    typeof log.timestamp !== "string" ||
    !isLevel(log.level) ||
    typeof log.reason !== "string"
  ) {
    return null;
  }
```

readIntervention — 개입 기록 한 줄 읽기 (1). id, 시각, 단계(1~3), 이유가 없으면 버린다. 시각이 날짜로 안 읽혀도 버린다.

```
  const at = new Date(log.timestamp);
  if (Number.isNaN(at.getTime())) return null;
  const reEntered = Boolean(log.reEntered);
  const level = log.level;
  return {
    interventionId: log.interventionId,
    timestamp: log.timestamp,
    date: typeof log.date === "string" ? log.date : todayKey(at),
    level,
    reason: log.reason,
    reasonNote: typeof log.reasonNote === "string" ? log.reasonNote : "",
    alternativeAction: typeof log.alternativeAction === "string" ? log.alternativeAction : null,
    alternativeCompleted: Boolean(log.alternativeCompleted),
    reEntered,
    blocked: Boolean(log.blocked),
    blockDuration: typeof log.blockDuration === "number" ? log.blockDuration : 0,
    usageSeconds: typeof log.usageSeconds === "number" ? log.usageSeconds : 0,
    outcome: OUTCOMES.includes(log.outcome as Outcome)
      ? (log.outcome as Outcome)
      : reEntered
        ? "watch"
        : "exit",
    hour: typeof log.hour === "number" ? log.hour : at.getHours(),
    weekday: typeof log.weekday === "number" ? log.weekday : at.getDay(),
    target: asTarget(log.target),
    interventionType: TYPES.includes(log.interventionType as InterventionType)
      ? (log.interventionType as InterventionType)
      : level === 1
        ? "pause"
        : level === 2
          ? "alternative"
          : "warning",
    reenteredWithin30:
      typeof log.reenteredWithin30 === "boolean" ? log.reenteredWithin30 : null,
    feedback: log.feedback === "helpful" || log.feedback === "not" ? log.feedback : null,
    band: isLevel(log.band) ? log.band : null,
    stage: text(log.stage),
    framing: text(log.framing),
    verification: text(log.verification),
    reflection: text(log.reflection),
    afterLock: text(log.afterLock),
    commitMinutes: count(log.commitMinutes),
    cardId: text(log.cardId),
    cardCorrect: typeof log.cardCorrect === "boolean" ? log.cardCorrect : null,
    extensionIndex: count(log.extensionIndex),
    policyVersion: text(log.policyVersion),
  };
}
```

readIntervention (2): 칸마다 형식을 검사해 채운다.

- 결과가 목록에 없으면 재진입이면 "보기", 아니면 "나가기"로 (예전 기록 호환).
- 시·요일이 없으면 시각에서 계산.
- 방식이 목록에 없으면 단계에 따라 예전 방식 이름으로.
- 재진입·피드백·강도·카드 정답 등은 맞는 형식일 때만, 아니면 null.

```ts
export function readCard(value: unknown): StudyCard | null {
  if (!value || typeof value !== "object") return null;
  const raw = value as Partial<StudyCard>;
  if (typeof raw.id !== "string" || typeof raw.question !== "string" || !raw.question.trim()) {
    return null;
  }
  if (!Array.isArray(raw.options)) return null;
  const options = raw.options.filter((o): o is string => typeof o === "string" && o.trim() !==
"");
  if (options.length < 2 || options.length !== raw.options.length) return null;
  const answer = typeof raw.answer === "number" ? raw.answer : -1;
  if (!Number.isInteger(answer) || answer < 0 || answer >= options.length) return null;
  return {
    id: raw.id,
    question: raw.question.trim(),
    options: options.map((o) => o.trim()),
    answer,
    createdAt: typeof raw.createdAt === "string" ? raw.createdAt : new Date().toISOString(),
    source: raw.source === "import" ? "import" : "manual",
    shown: count(raw.shown),
    correct: count(raw.correct),
    explanation: typeof raw.explanation === "string" ? raw.explanation.trim().slice(0, 500) :
"",
  };
}
```

readCard — 공부 카드 읽기. id와 문제가 있어야 하고, 보기가 2개 이상이며 빈 보기가 없어야 하고, 정답 번호가 보기 범위 안이어야 한다. 해설은 500자까지.

```ts
export function readLockAttempt(value: unknown): LockAttempt | null {
  if (!value || typeof value !== "object") return null;
  const row = value as Partial<LockAttempt>;
  if (typeof row.id !== "string" || typeof row.createdAt !== "string") return null;
  const at = new Date(row.createdAt);
  if (Number.isNaN(at.getTime())) return null;
  return {
    id: row.id,
    createdAt: row.createdAt,
    date: typeof row.date === "string" ? row.date : todayKey(at),
    remainingSec: typeof row.remainingSec === "number" ? row.remainingSec : 0,
    target: asTarget(row.target),
  };
}
```

readLockAttempt: 잠금 시도 한 줄 읽기 (id와 시각 필수).

```ts
export function readSettings(value: unknown): Settings {
  const defaults = defaultSettings();
  if (!value || typeof value !== "object") return defaults;
  const raw = value as Partial<Settings>;
  const level3 = Number(raw.level3Threshold ?? defaults.level3Threshold);
  const notificationTime =
    typeof raw.notificationTime === "string" && /^\d{2}:\d{2}$/.test(raw.notificationTime)
      ? raw.notificationTime
      : defaults.notificationTime;
  const rest = {
    notificationTime,
    notifyEnabled: raw.notifyEnabled !== false,
    finalGoal: count(raw.finalGoal) || defaults.finalGoal,
    dayEndHour:
      typeof raw.dayEndHour === "number" && raw.dayEndHour >= 0 && raw.dayEndHour <= 12
        ? Math.round(raw.dayEndHour)
        : defaults.dayEndHour,
    wakeTime:
      typeof raw.wakeTime === "string" && /^\d{2}:\d{2}$/.test(raw.wakeTime)
        ? raw.wakeTime
        : defaults.wakeTime,
    aiEnabled: raw.aiEnabled !== false,
    strictMode: raw.strictMode === true,
    banditV2: raw.banditV2 !== false,
    studyMode: raw.studyMode === true,
    goalWeekAnswered: typeof raw.goalWeekAnswered === "string" ? raw.goalWeekAnswered : "",
  };
  const goal =
    Number.isFinite(level3) && level3 >= 1 && level3 <= 360 ? Math.round(level3) : defaults.level3Threshold;
  const settings = settingsFromGoal(goal, rest);
  return { ...settings, finalGoal: Math.min(settings.finalGoal, goal) };
}
```

readSettings — 설정 읽기. 칸마다 검사하고 이상하면 기본값을 쓴다.

- 목표는 1~360분 사이 정수, 알림·기상 시각은 "HH:MM" 형식 (`/^\d{2}:\d{2}$/`는 "숫자 2개:숫자 2개"를 뜻하는 정규식).
- "오늘은 끝" 시는 0~12.
- 켜기/끄기 값은 기본이 켜짐인 것은 `!== false`, 꺼짐인 것은 `=== true`로 읽는다 (값이 없을 때 기본값이 되게).
- **266줄** 최종 목표가 하루 목표보다 크면 하루 목표로 맞춘다.

```ts
export function readBlock(value: unknown): BlockState {
  if (!value || typeof value !== "object") return { until: null, minutes: 0 };
  const raw = value as Partial<BlockState>;
  return {
    until: typeof raw.until === "string" ? raw.until : null,
    minutes: typeof raw.minutes === "number" ? raw.minutes : 0,
  };
}

export function readSession(value: unknown): ActiveSession | null {
  if (!value || typeof value !== "object") return null;
  const raw = value as Partial<ActiveSession>;
  if (
    typeof raw.usageId !== "string" ||
    typeof raw.date !== "string" ||
    typeof raw.startTime !== "string" ||
```

readBlock: 잠금 상태 읽기. **readSession**: 브라우저 연습 화면의 보는 중인 시청 읽기 (필수 칸 검사).

```
    typeof raw.duration !== "number" ||
    typeof raw.clipCount !== "number" ||
    !isLevel(raw.levelAtStart)
  ) {
    return null;
  }
  return {
    usageId: raw.usageId,
    date: raw.date,
    startTime: raw.startTime,
    duration: raw.duration,
    clipCount: raw.clipCount,
    interventionId: typeof raw.interventionId === "string" ? raw.interventionId : "",
    levelAtStart: raw.levelAtStart,
    lastTickAt: typeof raw.lastTickAt === "number" ? raw.lastTickAt : Date.now(),
  };
}
```

```
function readAll<T>(value: unknown, read: (row: unknown) => T | null): T[] {
  if (!Array.isArray(value)) return [];
  return value.map(read).filter((row): row is T => row !== null);
}

/** 전체 문서가 객체가 아니면 null. 줄 단위 오류는 그 줄만 버린다. */
export function normalizeData(value: unknown): AppData | null {
  if (!value || typeof value !== "object") return null;
  const raw = value as Partial<AppData>;
  if (!Array.isArray(raw.usageLogs) || !Array.isArray(raw.interventions)) return null;
  return {
    settings: readSettings(raw.settings),
    usageLogs: readAll(raw.usageLogs, readUsageLog),
    interventions: readAll(raw.interventions, readIntervention),
    lockAttempts: readAll(raw.lockAttempts, readLockAttempt),
    cards: readAll(raw.cards, readCard),
    activeSession: readSession(raw.activeSession),
    block: readBlock(raw.block),
    lastNotifiedDate: typeof raw.lastNotifiedDate === "string" ? raw.lastNotifiedDate : null,
  };
}
```

readAll: 목록의 각 줄을 읽고 실패한 줄(null)만 뺀다. `<T>`는 "어떤 모양이든" 쓸 수 있는 함수라는 뜻(제네릭).

normalizeData — 앱 데이터 전체 읽기. 전체가 객체가 아니거나 핵심 목록이 없으면 null(손상). 아니면 칸마다 위 함수들로 읽는다.

```
/** 브라우저 저장소에서 읽는다. 안드로이드 이관에도 쓴다. */
export function readLocalStorage(key = STORAGE_KEY): { data: AppData | null; corrupt: boolean }
{
  const raw = localStorage.getItem(key);
  if (!raw) return { data: null, corrupt: false };
  try {
    const data = normalizeData(JSON.parse(raw));
    return data ? { data, corrupt: false } : { data: null, corrupt: true };
  } catch {
    return { data: null, corrupt: true };
  }
```

- **readLocalStorage**: 브라우저 저장소에서 글자를 꺼내 JSON으로 읽고 검사한다. 없으면 null, 깨졌으면 "손상" 표시.
- **loadData**: 브라우저용 불러오기. 손상이면 빈 데이터 + 손상 표시, 아니면 남아 있던 연습 시청을 마감(`finalizeSession`)해서 돌려준다.
- **saveData / clearData**: 브라우저 저장소에 전체 저장 / 지우기.

```
}

export function loadData(): { data: AppData; corrupt: boolean } {
  const { data, corrupt } = readLocalStorage();
  if (corrupt) return { data: defaultData(), corrupt: true };
  return { data: data ? finalizeSession(data) : defaultData(), corrupt: false };
}

export function saveData(data: AppData): void {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(data));
}

export function clearData(): void {
  localStorage.removeItem(STORAGE_KEY);
}
```

logic.ts — 핵심 계산 모음 `lib/logic.ts (867줄)`

날짜, 단계, 시간 문구, 통계, 리포트 문장, 휴대폰 기록 변환, 30분 재진입 판정 등 **앱의 계산 대부분**이 여기 있다. 화면 없이 계산만 하므로 테스트(`logic.test.ts`)로 검증한다.

```
import type {
  AppData,
  Band,
  DailyStats,
  InterventionInput,
  InterventionLog,
  InterventionType,
  Level,
  LockAttempt,
  NativeBatch,
  SessionTarget,
  Settings,
  UsageLog,
} from "./types.ts";

export const REASONS: { id: string; hint: string }[] = [
  { id: "심심해서", hint: "특별한 목적 없이 시간을 보내려고" },
  { id: "습관적으로", hint: "별다른 생각 없이 반복해서" },
  { id: "잠깐 보려고", hint: "짧은 시간만 이용하려고" },
  { id: "스트레스 해소를 위해", hint: "기분 전환이나 휴식을 위해" },
  { id: "특별한 이유 없이", hint: "명확한 실행 목적이 없어서" },
  { id: "기타", hint: "위 항목에 해당하지 않는 경우" },
];
```

1~14줄 데이터 모양을 가져온다.

16~23줄 `REASONS`: 쇼츠를 연 이유 6가지와 설명 (앱 화면 개입용). Java의 `GuardCopy.REASONS`와 글자가 같아야 기록이 맞는다.

```ts
export type Alternative = {
  id: string;
  label: string;
  seconds: number;
  guide: string;
};

export const ALTERNATIVES: Alternative[] = [
  {
    id: "water",
    label: "물 마시기",
    seconds: 20,
    guide: "자리에서 일어나 물 한 잔을 마시고 오세요.",
  },
  {
    id: "stretch",
    label: "간단한 스트레칭",
    seconds: 30,
    guide: "어깨를 천천히 돌리고, 목을 좌우로 기울여 보세요.",
  },
  {
    id: "read",
    label: "5분 독서",
    seconds: 60,
    guide:
      "손에 잡히는 글에서 한 단락만 읽으세요. 브라우저 안내 시간은 1분입니다.",
  },
  {
    id: "walk",
    label: "잠시 걷기",
    seconds: 45,
    guide: "방 안이라도 열 걸음만 걸어 보세요.",
  },
  {
    id: "eyes",
    label: "눈 쉬기",
    seconds: 20,
    guide: "화면에서 눈을 떼고, 멀리 있는 한 점을 바라보세요.",
  },
  {
    id: "breath",
    label: "호흡하기",
    seconds: 30,
    guide: "넷을 들이쉬고, 넷을 멈추고, 여섯을 내쉬세요.",
  },
];
```

Alternative / ALTERNATIVES: 대체 행동 6가지 (앱 화면 개입·연습 화면용). 이름, 기다릴 초, 안내 문장. 시트(Java)는 이 중 4가지(물, 스트레칭, 걷기, 눈 쉬기)를 쓴다.

```ts
const REASON_ALT: Record<string, string> = {
  심심해서: "잠시 걷기",
  습관적으로: "호흡하기",
```

REASON_ALT: 이유별 추천 대체 행동 (예: 심심해서 → 잠시 걷기, 스트레스 → 스트레칭).

```
  "잠깐 보려고": "눈 쉬기",
  "스트레스 해소를 위해": "간단한 스트레칭",
  "특별한 이유 없이": "물 마시기",
  기타: "호흡하기",
};
```

```
export const LEVEL_SUMMARY: Record<Level, string> = {
  1: "가벼움 · 이유와 시청 약속",
  2: "중간 · 멈추는 활동 하나 거치기",
  3: "강함 · 오늘 마무리 선택",
};

export const BAND_LABEL: Record<Band, string> = {
  1: "가벼움",
  2: "중간",
  3: "강함",
};

export const TARGET_LABEL: Record<SessionTarget, string> = {
  youtube: "YouTube Shorts",
  instagram: "Instagram Reels",
  practice: "연습 피드",
};

export const OUTCOME_LABEL = {
  exit: "나가기",
  watch: "시청함",
  "force-quit": "강제 종료",
  "timed-block": "시간 잠금",
  "blocked-retry": "차단 중 다시 열기",
  kept: "약속 지킴",
  extend: "약속 연장",
  "day-end": "오늘은 끝",
  overdraft: "목표 초과 3분",
  "budget-out": "목표 소진",
  interrupted: "중단",
} as const;

/** 스스로 멈춘 결과. 리포트 첫 줄 "스스로 멈춘 횟수"에 센다. */
export const SELF_STOP_OUTCOMES = ["exit", "kept", "day-end", "timed-block", "force-quit"] as co
nst;

/** 시청으로 이어진 결과. 30분 재진입 판정에서 뺀다. */
const WATCH_OUTCOMES = ["watch", "extend", "overdraft", "budget-out", "interrupted"];

export const BLOCK_MINUTES = [10, 20, 30, 60] as const;
```

화면에 쓰는 이름표들

- `LEVEL_SUMMARY / BAND_LABEL`: 단계 설명과 이름 (가벼움, 중간, 강함).
- `TARGET_LABEL`: 대상 이름 (YouTube Shorts, Instagram Reels, 연습 피드).
- `OUTCOME_LABEL`: 결과 이름 (나가기, 시청함, 약속 지킴 ...). `as const`는 "이 값들은 고정"이라는 표시.
- **114줄** `SELF_STOP_OUTCOMES`: **"스스로 멈춘 횟수"**에 세는 결과 = 나가기, 약속 지킴, 오늘은 끝, 시간 잠금, 강제 종료.
- **117줄** `WATCH_OUTCOMES`: 계속 본 결과 (재진입 판정에서 뺀다).
- **119줄** 잠금 선택지 10, 20, 30, 60분.

```
export function todayKey(date = new Date()): string {
  const y = date.getFullYear();
  const m = String(date.getMonth() + 1).padStart(2, "0");
  const d = String(date.getDate()).padStart(2, "0");
```

- **todayKey**: 날짜를 "2026-10-13"으로 (Java의 dayKey와 같은 모양). `padStart(2, "0")`은 두 자리로 맞추기. 백틱(`` `...${x}...` ``)은 글자 안에 값을 끼워 넣는 문법.

```
  return `${y}-${m}-${d}`;
}

export function formatKoreanDate(dateKey: string): string {
  const [y, m, d] = dateKey.split("-").map(Number);
  const dt = new Date(y, (m || 1) - 1, d || 1);
  const week = ["일", "월", "화", "수", "목", "금", "토"][dt.getDay()];
  return `${y}년 ${m}월 ${d}일 (${week})`;
}

export function formatShortDate(dateKey: string, today = todayKey()): string {
  if (dateKey === today) return "오늘";
  const [, m, d] = dateKey.split("-");
  return `${Number(m)}/${Number(d)}`;
}

export function formatClock(totalSeconds: number): string {
  const safe = Math.max(0, Math.floor(totalSeconds));
  const hours = Math.floor(safe / 3600);
  const minutes = Math.floor((safe % 3600) / 60);
  const seconds = safe % 60;
  if (hours > 0) return `${hours}시간 ${minutes}분`;
  if (minutes > 0 && seconds === 0) return `${minutes}분`;
  if (minutes > 0) return `${minutes}분 ${seconds}초`;
  return `${seconds}초`;
}

export function formatReportDuration(totalSeconds: number): string {
  return formatClock(totalSeconds);
}

export function formatTime(iso: string): string {
  const date = new Date(iso);
  if (Number.isNaN(date.getTime())) return "--:--";
  return date.toLocaleTimeString("ko-KR", {
    hour: "2-digit",
    minute: "2-digit",
  });
}
```

- **formatKoreanDate**: "2026년 10월 13일 (화)".
- **formatShortDate**: 오늘이면 "오늘", 아니면 "10/13".
- **formatClock / formatReportDuration**: 초를 "1시간 5분", "2분 30초", "45초"로.
- **formatTime**: 시각을 "오후 09:30"처럼 (한국식 표기).

```
/** 오늘 목표(분) 대비 사용 비율. */
export function goalRatio(seconds: number, settings: Settings): number {
  return seconds / (Math.max(1, settings.level3Threshold) * 60);
}

/** 강도: 목표의 50% 미만 가벼움, 100% 미만 중간, 이상 강함. 서비스의 GuardPolicy.band와 같다. */
export function levelForUsage(seconds: number, settings: Settings): Level {
  const ratio = goalRatio(seconds, settings);
  if (ratio < 0.5) return 1;
  if (ratio < 1) return 2;
```

goalRatio: 오늘 사용 초 ÷ 목표 초.

levelForUsage: 비율이 0.5 미만이면 1(약함), 1 미만이면 2(중간), 아니면 3(강함). Java의 `GuardPolicy.band`와 같은 규칙.

```
  return 3;
}
```

```
/**
 * 시트의 시간 체감 문구 종류 (S1). 사용자 확인을 거친 문구다. 바꿀 때는 먼저 상황별 표로 확인받는다.
 * android GuardCopy.insight()와 같은 문장, 같은 조건을 쓴다.
 */
export const FRAMINGS = [
  "guilt",
  "yesterday",
  "motivate",
  "weekly",
  "future",
  "average",
  "goal",
  "sleep",
] as const;

export type InsightFacts = {
  usage: number;
  goal: number;
  weekProjection: number;
  avg7: number;
  yesterdayByNow: number;
  sleepSeconds: number;
  sleepBasis: string;
};

/** 문장용 분 표시: "23분", "2시간 40분". */
export function minutesText(seconds: number): string {
  const total = Math.max(1, Math.round(seconds / 60));
  const hours = Math.floor(total / 60);
  const rest = total % 60;
  if (hours > 0 && rest > 0) return `${hours}시간 ${rest}분`;
  if (hours > 0) return `${hours}시간`;
  return `${total}분`;
}
```

- **FRAMINGS**: 시간 체감 문구 종류 8개 (Java와 같은 순서).
- **InsightFacts**: 문구에 넣을 숫자 모양 (Java의 Facts와 같음).
- **minutesText**: 문장용 "23분", "2시간 40분" (Java의 GuardCopy.minutes와 같음).

```
/** 숫자가 없거나 의미가 없으면 null (그 문구는 건너뛴다). */
export function insightLine(kind: string, f: InsightFacts): string | null {
  if (f.usage < 60) return null;
  const remaining = Math.max(0, f.goal - f.usage);
  switch (kind) {
    case "guilt":
      return `오늘 쇼츠에 쓴 ${minutesText(f.usage)}, 다시 돌아오지 않아요.`;
    case "weekly":
      return f.weekProjection < 600
        ? null
        : `이 속도면 이번 주에 ${minutesText(f.weekProjection)}을 쇼츠에 써요.`;
    case "motivate":
      return remaining < 60
```

insightLine — 시간 체감 문구 (홈 화면용). Java의 `GuardCopy.insight`와 **같은 문장, 같은 조건**으로 만든다. 그래서 홈 화면 한 줄과 개입 화면 문구가 같다. 숫자가 없거나 의미 없으면 null(그 종류는 건너뜀).

```
        ? null
        : `지금 멈추면 오늘 목표 안에서 끝낼 수 있어요. (${minutesText(remaining)} 남음)`;
    case "future":
      return `자기 전의 나는 오늘 이 ${minutesText(f.usage)}을 어떻게 생각할까요?`;
    case "yesterday":
      return f.yesterdayByNow < 0 || f.usage - f.yesterdayByNow < 60
        ? null
        : `어제 이 시간엔 ${minutesText(f.yesterdayByNow)}이었는데, 오늘은 벌써 ${minutesText(f.usage)}이에요.`;
    case "average":
      return f.avg7 <= 0 || f.usage - f.avg7 < 60
        ? null
        : `이번 주 평균보다 ${minutesText(f.usage - f.avg7)} 더 보고 있어요.`;
    case "goal":
      return f.usage >= f.goal ? null : `정한 ${minutesText(f.goal)} 중 ${minutesText(f.usage)}을 썼어요.`;
    case "sleep":
      return f.sleepSeconds < 600
        ? null
        : `지금 자면 ${minutesText(f.sleepSeconds)} 잘 수 있어요. 10분 더 보면 ${minutesText(f.sleepSeconds - 600)}이에요. (${f.sleepBasis})`;
    default:
      return null;
  }
}
```

```
/** 웹(홈 화면)에서 쓸 문구 재료. 서비스는 같은 값을 정책표 stats로 받아 직접 계산한다. */
export function insightFacts(data: AppData, now = new Date()): InsightFacts {
  const today = todayKey(now);
  const stats = usageStats(data, now);
  const usage = usageSecondsOn(data, today);
  const hour = now.getHours();
  let yesterdayByNow = -1;
  if (stats.yesterdayCumulative) {
    const before = hour === 0 ? 0 : stats.yesterdayCumulative[hour - 1];
    const end = stats.yesterdayCumulative[hour];
    yesterdayByNow = before + (end - before) * (now.getMinutes() / 60);
  }
  let sleepSeconds = -1;
  let sleepBasis = "";
  if (hour >= 22 || hour < 4) {
    const [wh, wm] = data.settings.wakeTime.split(":").map(Number);
    const wake = new Date(now);
    wake.setHours(wh, wm, 0, 0);
    if (wake.getTime() <= now.getTime()) wake.setDate(wake.getDate() + 1);
    sleepSeconds = (wake.getTime() - now.getTime()) / 1000;
    sleepBasis = `일어나는 시각 ${wh}:${String(wm).padStart(2, "0")} 기준`;
  }
  return {
    usage,
    goal: data.settings.level3Threshold * 60,
```

insightFacts — 홈 화면용 문구 숫자 만들기.

- 오늘 사용 초와 통계(`usageStats`)를 구한다.
- **256~260줄** 어제 이 시간까지: 어제 시간대별 누적값을 분 비율로 나눠 계산.
- **263~270줄** 밤(22~4시)이면 설정의 일어나는 시각까지 남은 초 (웹은 폰 알람을 못 읽어서 설정값만 쓴다).
- **274~277줄** 이번 주 예상 = (이번 주 지난날 합 + 오늘) ÷ 날 수 × 7.

```
    weekProjection:
      stats.pastWeekSeconds < 0
        ? -1
        : ((stats.pastWeekSeconds + usage) / (stats.daysBeforeToday + 1)) * 7,
    avg7: stats.avg7Seconds,
    yesterdayByNow,
    sleepSeconds,
    sleepBasis,
  };
}
```

```
/**
 * 문구용 통계: 지난 7일 평균, 이번 주(월요일 시작) 오늘 전까지 합, 어제 시간대별 누적.
 * 기록이 없는 값은 -1 / null.
 */
export function usageStats(
  data: AppData,
  now = new Date(),
): {
  avg7Seconds: number;
  pastWeekSeconds: number;
  daysBeforeToday: number;
  yesterdayCumulative: number[] | null;
} {
  const days = lastDates(8, now).slice(0, 7); // 오늘 전 7일
  const firstRecord = data.usageLogs.reduce(
    (min, log) => (log.date < min ? log.date : min),
    todayKey(now),
  );
```

usageStats — 문구용 통계 (1). 오늘 전 7일 날짜 목록을 만들고, 가장 오래된 기록 날짜를 찾는다 (기록이 시작되기 전 날은 평균에 넣지 않기 위해).

```
  const counted = days.filter((d) => d >= firstRecord);
  const avg7Seconds =
    counted.length === 0
      ? -1
      : counted.reduce((sum, d) => sum + usageSecondsOn(data, d), 0) / counted.length;
  const weekday = (now.getDay() + 6) % 7; // 월요일 = 0
  const weekDays = weekday === 0 ? [] : lastDates(weekday + 1, now).slice(0, weekday);
  const pastWeekSeconds =
    counted.length === 0 ? -1 : weekDays.reduce((sum, d) => sum + usageSecondsOn(data, d), 0);
  const yesterday = days[days.length - 1];
  const hours = hourlyUsage(data, yesterday);
  const hasYesterday = yesterday >= firstRecord && hours.some((v) => v > 0);
  let running = 0;
  const yesterdayCumulative = hasYesterday
    ? hours.map((v) => {
        running += v;
        return running;
      })
    : null;
  return { avg7Seconds, pastWeekSeconds, daysBeforeToday: weekday, yesterdayCumulative };
}
```

usageStats (2)

- **avg7Seconds**: 기록이 있는 날들만의 평균. 없으면 -1.
- **308~311줄** 이번 주(월요일 시작)의 오늘 전 날들 합. `(요일 + 6) % 7`로 월요일을 0으로 바꾼다.
- **312~321줄** 어제 시간대별 시청을 0시부터 차례로 더한 누적 24칸 (어제 기록이 없으면 null).

```ts
/** 시간 잠금으로 쉰 초 (그 날짜 몫, 지금까지 지난 만큼). "오늘은 끝"은 쉰 시간에 넣지 않는다. */
export function restSecondsOn(data: AppData, date: string, now = Date.now()): number {
  const [dayStart, dayEnd] = dayRange(date);
  return data.interventions
    .filter((log) => log.outcome === "timed-block" && log.blockDuration > 0)
    .reduce((sum, log) => {
      const start = new Date(log.timestamp).getTime();
      const end = Math.min(now, start + log.blockDuration * 60_000);
      const overlap = Math.min(end, dayEnd) - Math.max(start, dayStart);
      return overlap > 0 ? sum + overlap / 1000 : sum;
    }, 0);
}
```

restSecondsOn — 그 날 쉰 시간. 결과가 "시간 잠금"인 기록마다 (시작 ~ 시작+잠금 분, 지금보다 늦으면 지금까지) 중 그 날짜에 겹치는 초를 더한다. "오늘은 끝"은 쉰 시간에 넣지 않는다.

```ts
/** 이 날까지 잠금이나 "오늘은 끝"을 고른 날이 연속된 수. 이 날에 없으면 전날부터 센다. */
export function lockStreak(data: AppData, date: string): number {
  const lockDays = new Set(
    data.interventions
      .filter((log) => log.outcome === "timed-block" || log.outcome === "day-end")
      .map((log) => log.date),
  );
  const [y, m, d] = date.split("-").map(Number);
  const cursor = new Date(y, m - 1, d);
  if (!lockDays.has(todayKey(cursor))) cursor.setDate(cursor.getDate() - 1);
  let streak = 0;
  while (lockDays.has(todayKey(cursor))) {
    streak += 1;
    cursor.setDate(cursor.getDate() - 1);
  }
  return streak;
}
```

lockStreak — 잠금 연속 날 수. 시간 잠금이나 오늘은 끝을 고른 날짜를 모으고, 그 날(없으면 전날)부터 하루씩 거꾸로 가며 연속된 날을 센다.

```ts
/** 오늘 시간대별(0~23시) 시청 초. 홈의 하루 막대에 쓴다. */
export function hourlyUsage(data: AppData, date: string): number[] {
  const hours = new Array<number>(24).fill(0);
  const [dayStart] = dayRange(date);
  for (const log of data.usageLogs) {
    const start = new Date(log.startTime).getTime();
    const end = new Date(log.endTime).getTime();
    if (!Number.isFinite(start) || !Number.isFinite(end) || end <= start) continue;
    const scale = log.duration / ((end - start) / 1000);
    for (let h = 0; h < 24; h += 1) {
      const from = dayStart + h * 3_600_000;
      const to = from + 3_600_000;
      const overlap = Math.min(end, to) - Math.max(start, from);
      if (overlap > 0) hours[h] += (overlap / 1000) * scale;
    }
  }
  return hours;
}
```

hourlyUsage — 시간대별(0~23시) 시청 초 (홈의 하루 막대). 시청 기록마다 각 시간 칸과 겹치는 부분을 더한다. `scale`은 연습 화면처럼 실제 본 시간이 시계 시간보다 짧을 때 비율을 맞추는 값.

```ts
/** "오늘은 끝": 다음 {dayEndHour}시까지 남은 분. */
export function minutesUntilDayEnd(dayEndHour: number, now = new Date()): number {
  const end = new Date(now);
  end.setHours(dayEndHour, 0, 0, 0);
  if (end.getTime() <= now.getTime()) end.setDate(end.getDate() + 1);
  return Math.ceil((end.getTime() - now.getTime()) / 60_000);
}

/** 로컬 날짜의 시작·끝 시각(ms). */
export function dayRange(date: string): [number, number] {
  const [y, m, d] = date.split("-").map(Number);
  const start = new Date(y, (m || 1) - 1, d || 1).getTime();
  const end = new Date(y, (m || 1) - 1, (d || 1) + 1).getTime();
  return [start, end];
}
```

minutesUntilDayEnd: "오늘은 끝"이 풀릴 때까지 남은 분 (다음 그 시 정각까지).

dayRange: 날짜의 시작(0시)과 끝(다음 날 0시)을 밀리초로.

```ts
/**
 * 세션 중 그 날짜에 속한 초. 자정을 넘긴 세션은 걸친 시간 비율대로 나눈다.
 * 연습 피드처럼 duration이 벽시계 시간보다 짧으면 같은 비율로 줄인다.
 */
export function secondsOnDate(log: UsageLog, date: string): number {
  const start = new Date(log.startTime).getTime();
  const end = new Date(log.endTime).getTime();
  if (!Number.isFinite(start) || !Number.isFinite(end) || end <= start) {
    return log.date === date ? log.duration : 0;
  }
  const [dayStart, dayEnd] = dayRange(date);
  const overlap = Math.min(end, dayEnd) - Math.max(start, dayStart);
  if (overlap <= 0) return 0;
  return log.duration * (overlap / (end - start));
}

export function usageSecondsOn(data: AppData, date: string): number {
  return data.usageLogs.reduce((sum, log) => sum + secondsOnDate(log, date), 0);
}

/** 웹이 직접 잰 초(연습 피드, 이관한 기록). 네이티브 세션은 서비스가 따로 센다. */
export function webSecondsOn(data: AppData, date: string): number {
  return data.usageLogs
    .filter((log) => log.source !== "native")
    .reduce((sum, log) => sum + secondsOnDate(log, date), 0);
}
```

- **secondsOnDate**: 시청 한 번 중 **그 날짜에 속한 초**. 자정을 넘긴 시청은 걸친 시간 비율대로 나눈다.
- **usageSecondsOn**: 그 날짜 전체 시청 초 = 모든 기록의 그 날 몫의 합.
- **webSecondsOn**: 웹이 직접 잰 초만 (연습 화면, 이관 기록). 휴대폰이 잰 초는 휴대폰이 따로 세므로 sync 때 이것만 넘긴다.

```ts
export function nextLevelGap(
  seconds: number,
  settings: Settings,
): { label: string; remainSeconds: number } | null {
  const goal = settings.level3Threshold * 60;
  if (seconds < goal / 2) {
    return { label: "중간 강도까지", remainSeconds: goal / 2 - seconds };
  }
```

- **nextLevelGap**: 다음 단계까지 남은 시간 ("중간 강도까지 N", "오늘 목표까지 N").
- **recommendAlternative**: 이유에 맞는 대체 행동 (없으면 호흡하기).
- **isBlocked / blockRemainingSeconds**: 지금 잠겨 있는지, 남은 초.
- **blockUntilFromMinutes**: 지금 + N분의 시각 (ISO 글자).

```
  if (seconds < goal) {
    return { label: "오늘 목표까지", remainSeconds: goal - seconds };
  }
  return null;
}

export function recommendAlternative(reason: string): Alternative {
  const label = REASON_ALT[reason] ?? "호흡하기";
  return ALTERNATIVES.find((item) => item.label === label) ?? ALTERNATIVES[5];
}

export function isBlocked(block: AppData["block"], now = Date.now()): boolean {
  if (!block.until) return false;
  return new Date(block.until).getTime() > now;
}

export function blockRemainingSeconds(
  block: AppData["block"],
  now = Date.now(),
): number {
  if (!block.until) return 0;
  return Math.max(0, (new Date(block.until).getTime() - now) / 1000);
}

export function blockUntilFromMinutes(minutes: number, now = Date.now()): string {
  const realMs = minutes * 60 * 1000;
  return new Date(now + realMs).toISOString();
}
```

```
export function isReportDue(
  notificationTime: string,
  now = new Date(),
): boolean {
  const match = /^(\d{2}):(\d{2})$/.exec(notificationTime);
  if (!match) return false;
  const target = new Date(now);
  target.setHours(Number(match[1]), Number(match[2]), 0, 0);
  return now.getTime() >= target.getTime();
}

export function normalizeClock(value: string): string | null {
  const match = /^(\d{2}):(\d{2})(?::\d{2})?$/.exec(value.trim());
  if (!match) return null;
  const hour = Number(match[1]);
  const minute = Number(match[2]);
  if (hour > 23 || minute > 59) return null;
  return `${match[1]}:${match[2]}`;
}
```

isReportDue: 리포트 알림 시각이 지났는지.

normalizeClock: "21:00" 또는 "21:00:00" 형식인지 검사하고 "21:00"으로 맞춘다. 24시 이상이나 60분 이상이면 null.

```
export function validateSettings(input: {
  goal: number;
```

validateSettings — 설정 저장 전 검사. 문제가 있으면 안내 문장, 없으면 null.

```
  finalGoal: number;
  dayEndHour: number;
  notificationTime: string;
  wakeTime?: string;
}): string | null {
  if (input.wakeTime !== undefined && !normalizeClock(input.wakeTime)) {
    return "일어나는 시각을 다시 확인해 주세요.";
  }
  if (!Number.isInteger(input.goal) || input.goal < 1 || input.goal > 360) {
    return "하루 목표는 1분에서 360분 사이의 정수여야 합니다.";
  }
  if (!Number.isInteger(input.finalGoal) || input.finalGoal < 1 || input.finalGoal > input.goal)
{
    return "최종 목표는 1분 이상, 하루 목표 이하의 정수여야 합니다.";
  }
  if (!Number.isInteger(input.dayEndHour) || input.dayEndHour < 0 || input.dayEndHour > 12) {
    return "\"오늘은 끝\"이 풀리는 시각은 0시에서 12시 사이여야 합니다.";
  }
  if (!normalizeClock(input.notificationTime)) {
    return "알림 시각을 다시 확인해 주세요.";
  }
  return null;
}
```

- 일어나는 시각 형식, 하루 목표 1~360분 정수, 최종 목표 1분 이상 하루 목표 이하, "오늘은 끝" 0~12시, 알림 시각 형식.

```
export function lastDates(count: number, from = new Date()): string[] {
  const dates: string[] = [];
  for (let i = count - 1; i >= 0; i -= 1) {
    const day = new Date(from);
    day.setDate(from.getDate() - i);
    dates.push(todayKey(day));
  }
  return dates;
}
```

lastDates: 오늘(또는 주어진 날)까지 최근 N일의 날짜 목록 (오래된 날부터).

```
export function statsForDate(data: AppData, date: string): DailyStats {
  const usage = data.usageLogs.filter((log) => log.date === date);
  const usageSeconds = usageSecondsOn(data, date);
  const interventions = data.interventions.filter((log) => log.date === date);
  const levelCounts: Record<Level, number> = { 1: 0, 2: 0, 3: 0 };
  const reasonCounts = new Map<string, { count: number; latest: number }>();

  interventions.forEach((log, index) => {
    levelCounts[log.level] += 1;
    if (!log.reason || log.reason === "선택하지 않음" || log.outcome === "blocked-retry") {
      return;
    }
    const prev = reasonCounts.get(log.reason) ?? { count: 0, latest: -1 };
    reasonCounts.set(log.reason, { count: prev.count + 1, latest: index });
  });

  let topReason: string | null = null;
```

statsForDate — 하루 통계 (1). 그 날의 시청과 개입을 모으고, 단계별 개수를 센다. 이유는 "선택하지 않음"과 잠금 중 다시 열기를 빼고 세서, **가장 많이 고른 이유**를 찾는다 (같으면 더 최근 것).

```
  let best = { count: 0, latest: -1 };
  for (const [reason, info] of reasonCounts) {
    if (info.count > best.count || (info.count === best.count && info.latest > best.latest)) {
      best = info;
      topReason = reason;
    }
  }
```

```
  return {
    date,
    usageSeconds,
    clipCount: usage.reduce((sum, log) => sum + log.clipCount, 0),
    interventionCount: interventions.length,
    levelCounts,
    topReason,
    reentryCount: interventions.filter((log) => log.level === 2 && log.reEntered)
      .length,
    blockCount: interventions.filter((log) => log.outcome === "timed-block").length,
    forceQuitCount: interventions.filter((log) => log.outcome === "force-quit")
      .length,
    alternativeCompletedCount: interventions.filter((log) => log.alternativeCompleted)
      .length,
    lockAttemptCount: data.lockAttempts.filter((item) => item.date === date).length,
    selfStopCount: interventions.filter((log) =>
      (SELF_STOP_OUTCOMES as readonly string[]).includes(log.outcome),
    ).length,
    keptCount: interventions.filter((log) => log.outcome === "kept").length,
    dayEndCount: interventions.filter((log) => log.outcome === "day-end").length,
    overdraftCount: interventions.filter((log) => log.outcome === "overdraft").length,
    extendCount: interventions.filter((log) => log.outcome === "extend").length,
    cardsSolved: interventions.filter((log) => log.cardCorrect !== null).length,
    cardsCorrect: interventions.filter((log) => log.cardCorrect === true).length,
    restSeconds: restSecondsOn(data, date),
    lockStreak: lockStreak(data, date),
  };
}
```

statsForDate (2): DailyStats 모양으로 채운다 — 사용 초, 영상 수, 개입 수, 재진입, 잠금, 강제 종료, 대체 행동 완료, 잠금 중 시도, **스스로 멈춘 횟수**, 약속 지킴, 오늘은 끝, 3분만 더, 연장, 푼 카드(정답 여부가 있는 개입)와 맞힌 카드, 쉰 초, 연속 잠금.

```
/** 리포트 문장. 정해진 문장 틀에 숫자만 채운다 (사용자 글이나 생성 문장은 넣지 않는다). */
export function reportLines(stats: DailyStats): string[] {
  const lines = [
    `스스로 멈춘 횟수 : ${stats.selfStopCount}번`,
    `총 사용시간 : ${formatReportDuration(stats.usageSeconds)}`,
    `실행 횟수 : ${stats.clipCount}회`,
    `개입 횟수 : ${stats.interventionCount}회`,
    `약속 지킴 : ${stats.keptCount}번 · 연장 : ${stats.extendCount}번`,
    `가장 많이 선택한 이유 : ${stats.topReason ?? "없음"}`,
    `시간 잠금 : ${stats.blockCount}회`,
    `잠금 중 시도 : ${stats.lockAttemptCount}회`,
  ];
  if (stats.restSeconds >= 60) lines.push(`오늘 쉰 시간 : ${minutesText(stats.restSeconds)}`);
  if (stats.lockStreak > 1) lines.push(`잠금 연속 기록 : ${stats.lockStreak}일`);
```

reportLines — 리포트 문장. 정해진 문장 틀에 숫자만 채운다 (AI가 만든 문장이나 사용자 글은 넣지 않는다). 맨 앞은 "스스로 멈춘 횟수". 쉰 시간(1분 이상), 연속 잠금(2일 이상), 오늘은 끝, 목표 초과, 푼 문제는 해당할 때만 덧붙인다.

```
  if (stats.dayEndCount > 0) lines.push("오늘은 스스로 마무리했어요");
  if (stats.overdraftCount > 0) lines.push(`목표 초과 : ${stats.overdraftCount}번`);
  if (stats.cardsSolved > 0) {
    lines.push(`쇼츠 대신 푼 문제 : ${stats.cardsSolved}개 (정답 ${stats.cardsCorrect}개)`);
  }
  return lines;
}
```

```
export function applyFlush(
  data: AppData,
  visible: boolean,
  now = Date.now(),
): AppData {
  const session = data.activeSession;
  if (!session) return data;
  const elapsedMs = Math.min(2000, Math.max(0, now - session.lastTickAt));
  const addSeconds = visible ? elapsedMs / 1000 : 0;
  const duration = session.duration + addSeconds;
  const activeSession = { ...session, duration, lastTickAt: now };
  const usageLogs = data.usageLogs.map((log) =>
    log.usageId === session.usageId
      ? {
          ...log,
          duration,
          clipCount: session.clipCount,
          endTime: new Date(now).toISOString(),
        }
      : log,
  );
  return { ...data, activeSession, usageLogs };
}
```

applyFlush: 브라우저 연습 화면에서 보는 중인 시청 시간을 늘린다. 화면이 보일 때만 지난 시간(최대 2초)을 더하고, 시청 기록의 끝 시각을 갱신한다.

```
export function finalizeSession(data: AppData, now = Date.now()): AppData {
  if (!data.activeSession) return data;
  const session = data.activeSession;
  const hasLog = data.usageLogs.some((log) => log.usageId === session.usageId);
  const usageLogs = hasLog
    ? data.usageLogs.map((log) =>
        log.usageId === session.usageId
          ? {
              ...log,
              duration: session.duration,
              clipCount: session.clipCount,
              endTime: new Date(now).toISOString(),
            }
          : log,
      )
    : [
        ...data.usageLogs,
        {
          usageId: session.usageId,
```

finalizeSession: 보는 중이던 연습 시청을 마감한다. 기록이 있으면 길이와 끝 시각을 고치고, 없으면 새 기록으로 더한 뒤 "보는 중"을 비운다.

```
          date: session.date,
          startTime: session.startTime,
          endTime: new Date(now).toISOString(),
          duration: session.duration,
          clipCount: session.clipCount,
          target: "practice" as const,
          source: "web" as const,
        },
      ];
  return { ...data, usageLogs, activeSession: null };
}
```

```
export function withClip(data: AppData): AppData {
  if (!data.activeSession) return data;
  const clipCount = data.activeSession.clipCount + 1;
  const activeSession = { ...data.activeSession, clipCount };
  const usageLogs = data.usageLogs.map((log) =>
    log.usageId === activeSession.usageId ? { ...log, clipCount } : log,
  );
  return { ...data, activeSession, usageLogs };
}

export function reasonLabel(log: Pick<InterventionLog, "reason" | "reasonNote">): string {
  if (log.reason === "기타" && log.reasonNote.trim()) {
    return `기타 · ${log.reasonNote.trim()}`;
  }
  return log.reason || "기록 없음";
}
```

withClip: 연습 화면에서 다음 영상으로 넘기면 영상 수 +1.

reasonLabel: 이유가 "기타"이고 메모가 있으면 "기타 · 메모", 없으면 이유 그대로.

```
const INTERVENTION_TYPE: Record<Level, InterventionType> = {
  1: "pause",
  2: "alternative",
  3: "warning",
};

const REENTRY_WINDOW_MS = 30 * 60 * 1000;

/** 화면에서 받은 개입 결과에 시각·요일·종류를 채운다. */
export function buildIntervention(
  input: InterventionInput,
  id: string,
  now = new Date(),
): InterventionLog {
  return {
    ...input,
    interventionId: id,
    timestamp: now.toISOString(),
    date: todayKey(now),
    hour: now.getHours(),
    weekday: now.getDay(),
    target: input.target ?? null,
```

buildIntervention — 앱 화면 개입 기록 만들기. 화면에서 받은 결과에 id, 시각, 날짜, 시, 요일, 단계별 예전 방식 이름, 단계 "web"을 채우고, 시트 전용 칸은 비운다. `REENTRY_WINDOW_MS` = 30분.

```
    interventionType: INTERVENTION_TYPE[input.level],
    reenteredWithin30: null,
    feedback: null,
    band: input.level,
    stage: "web",
    framing: null,
    verification: null,
    reflection: null,
    afterLock: null,
    commitMinutes: 0,
    cardId: null,
    cardCorrect: null,
    extensionIndex: 0,
    policyVersion: null,
  };
}
```

```
const NATIVE_OUTCOME: Record<string, InterventionLog["outcome"]> = {
  watch: "watch",
  exit: "exit",
  kept: "kept",
  extend: "extend",
  day_end: "day-end",
  lock: "timed-block",
  overdraft: "overdraft",
  budget_out: "budget-out",
  replaced: "interrupted",
};

const SHEET_TYPES: InterventionLog["interventionType"][] = [
  "pause",
  "alternative",
  "warning",
  "framing",
  "breath",
  "reflection",
  "card",
  "commit_end",
  "strong",
  "locked",
  "pip",
];

function textOrNull(value: unknown): string | null {
  return typeof value === "string" && value !== "" ? value : null;
}

export function asTarget(value: unknown): SessionTarget | null {
  return value === "youtube" || value === "instagram" || value === "practice" ? value : null;
}

```

- **NATIVE_OUTCOME**: 휴대폰 결과 이름을 웹 이름으로 바꾸는 표 (예: `day_end` → `day-end`, `lock` → `timed-block`, `replaced` → `interrupted`).
- **SHEET_TYPES**: 허용하는 방식 이름.
- **textOrNull / asTarget / asLevel**: 값 검사 도우미 (빈 글자는 null, 대상은 세 가지만, 단계는 1~3).

```
function asLevel(value: number): Level {
  return value === 2 ? 2 : value === 3 ? 3 : 1;
}
```

```
/** 네이티브가 넘긴 기록을 앱 데이터 형태로 바꾼다. 이미 있는 id는 건너뛴다. */
export function convertNativeBatch(
  data: AppData,
  batch: NativeBatch,
): {
  usageLogs: UsageLog[];
  interventions: InterventionLog[];
  lockAttempts: LockAttempt[];
  feedback: { refId: string; feedback: "helpful" | "not" }[];
  /** 시트에서 "이상한 문제"로 신고된 카드 id */
  reports: string[];
} {
  const haveSession = new Set(data.usageLogs.map((log) => log.usageId));
  const haveIntervention = new Set(data.interventions.map((log) => log.interventionId));
  const haveLock = new Set(data.lockAttempts.map((item) => item.id));

```

convertNativeBatch — 휴대폰 기록을 웹 데이터로 바꾸기 (1). 이미 저장한 id 목록을 만들어 둔다 (같은 기록이 두 번 들어오지 않게).

```
  const usageLogs: UsageLog[] = (batch.sessions ?? [])
    .filter((row) => row.id && !haveSession.has(row.id) && row.endedAt > row.startedAt)
    .map((row) => ({
      usageId: row.id,
      date: todayKey(new Date(row.startedAt)),
      startTime: new Date(row.startedAt).toISOString(),
      endTime: new Date(row.endedAt).toISOString(),
      duration: Math.max(0, row.durationSec),
      clipCount: 1,
      target: asTarget(row.target) ?? "youtube",
      source: "native" as const,
    }));
```

(2) 시청 기록: 새 id이고 끝이 시작보다 뒤인 것만, 웹 모양으로 (날짜는 시작한 날, 출처 native).

```
  const feedback = (batch.interventions ?? [])
    .filter((row) => row.stage === "feedback" && row.refId)
    .map((row) => ({
      refId: row.refId as string,
      feedback: row.feedback === "helpful" ? ("helpful" as const) : ("not" as const),
    }));

  const reports = (batch.interventions ?? [])
    .filter((row) => row.stage === "card_report" && row.cardId)
    .map((row) => row.cardId as string);
```

(3) 피드백과 신고: 단계가 "feedback"인 줄은 **(어느 개입, 답)** 목록으로, "card_report"인 줄은 **신고된 카드 id** 목록으로 따로 모은다. 이 둘은 개입 기록이 아니다.

```
  const interventions: InterventionLog[] = (batch.interventions ?? [])
    .filter(
      (row) =>
        row.id && row.stage !== "feedback" && row.stage !== "card_report" && !haveIntervention.has(row.id),
    )
    .map((row) => {
```

(4) 개입 기록: 피드백·신고를 뺀 새 개입만 바꾼다.

- 결과 이름 변환, 계속 본 결과인지, 잠금 분, 방식 이름 검사.
- 이유가 없으면 "선택하지 않음". 대체 행동은 "스스로 했어요"나 확인됐을 때 완료로.

```
      const at = new Date(row.createdAt);
      const level = asLevel(row.level);
      const outcome = NATIVE_OUTCOME[row.outcome] ?? "exit";
      const watching = WATCH_OUTCOMES.includes(outcome);
      const blockMinutes = Math.max(0, Math.round(row.blockMinutes ?? 0));
      const type = SHEET_TYPES.find((t) => t === row.interventionType) ?? INTERVENTION_TYPE[leve
l];
      return {
        interventionId: row.id,
        timestamp: at.toISOString(),
        date: todayKey(at),
        level,
        reason: row.reason || "선택하지 않음",
        reasonNote: "",
        alternativeAction: row.alternativeAction ?? null,
        alternativeCompleted: row.verification === "self" || row.verification === "verified",
        reEntered: watching,
        blocked: outcome === "timed-block" || outcome === "day-end",
        blockDuration: blockMinutes,
        usageSeconds: Math.max(0, row.usageSeconds),
        outcome,
        hour: at.getHours(),
        weekday: at.getDay(),
        target: asTarget(row.target),
        interventionType: type,
        reenteredWithin30: null,
        feedback: null,
        band: row.band === 1 || row.band === 2 || row.band === 3 ? row.band : level,
        stage: textOrNull(row.stage),
        framing: textOrNull(row.framing),
        verification: textOrNull(row.verification),
        reflection: textOrNull(row.reflection),
        afterLock: textOrNull(row.afterLock),
        commitMinutes: Math.max(0, Math.round(row.commitMinutes ?? 0)),
        cardId: textOrNull(row.cardId),
        cardCorrect: typeof row.cardCorrect === "boolean" ? row.cardCorrect : null,
        extensionIndex: Math.max(0, Math.round(row.extensionIndex ?? 0)),
        policyVersion: textOrNull(row.policyVersion),
      };
    });
```

- 잠금(시간 잠금, 오늘은 끝)이면 `blocked`.
- 시·요일은 기록 시각에서 계산. 30분 재진입과 피드백은 나중에 채운다(null).
- 시트 칸들(강도, 단계, 문구, 참여 확인, 성찰, 잠금 뒤 답, 약속 분, 카드, 연장, 정책 버전)은 형식을 검사해 옮긴다.

```
  const lockAttempts: LockAttempt[] = (batch.lockAttempts ?? [])
    .filter((row) => row.id && !haveLock.has(row.id))
    .map((row) => {
      const at = new Date(row.createdAt);
      return {
        id: row.id,
        createdAt: at.toISOString(),
        date: todayKey(at),
        remainingSec: Math.max(0, Math.round(row.remainingSec)),
        target: asTarget(row.target),
```

(5) 잠금 시도를 바꾸고, 다섯 가지(시청, 개입, 잠금 시도, 피드백, 신고)를 돌려준다.

```
      };
    });

  return { usageLogs, interventions, lockAttempts, feedback, reports };
}
```

```
/**
 * 나가기·종료·차단 뒤 30분이 지난 개입에 재진입 여부를 채운다.
 * 재진입 = 그 사이 새 시청 세션, 새 개입, 또는 잠금 중 시도가 있었는지.
 * 시청을 고른 개입은 대상이 아니라 null로 둔다.
 */
export function settleReentry(
  data: AppData,
  now = Date.now(),
): { data: AppData; changed: InterventionLog[] } {
  const changed: InterventionLog[] = [];
  // 새로 들어온 순간만 센다. 약속 시간 끝 시트는 같은 시청 안이라 뺀다.
  const starts = [
    ...data.usageLogs.filter((log) => log.target !== "practice").map((log) => log.startTime),
    ...data.interventions.filter((log) => log.stage !== "commit_end").map((log) => log.timestamp),
    ...data.lockAttempts.map((item) => item.createdAt),
  ]
    .map((iso) => new Date(iso).getTime())
    .filter((ms) => Number.isFinite(ms));

  const interventions = data.interventions.map((log) => {
    if (log.reenteredWithin30 !== null || WATCH_OUTCOMES.includes(log.outcome)) return log;
    const at = new Date(log.timestamp).getTime();
    if (!Number.isFinite(at) || now - at < REENTRY_WINDOW_MS) return log;
    const reentered = starts.some((ms) => ms > at && ms <= at + REENTRY_WINDOW_MS);
    const next = { ...log, reenteredWithin30: reentered };
    changed.push(next);
    return next;
  });
  if (changed.length === 0) return { data, changed };
  return { data: { ...data, interventions }, changed };
}
```

settleReentry — 30분 재진입 판정. 나가기·잠금 같은 개입 뒤 **30분 안에 다시 쇼츠를 열었는지**를 채운다. AI 점수의 재료다.

1. **848~854줄** "새로 들어온 순간" 목록: 시청 시작 시각, 개입 시각(약속 끝 화면은 같은 시청이라 뺌), 잠금 중 시도 시각.
2. **856~864줄** 아직 판정 안 됐고 계속 본 결과가 아닌 개입 중, 30분이 지난 것만: 그 뒤 30분 안에 "새로 들어온 순간"이 있으면 true.
3. 바뀐 것이 있으면 새 데이터와 바뀐 목록을 돌려준다 (바뀐 것만 다시 저장하기 위해).

ai.ts — AI 개인화 계산 `lib/ai.ts (368줄)`

AI의 "학습" 부분. 지난 기록을 보고 ① 방식·문구별 점수표(밴딧 표), ② 위험 시간, ③ 주간 목표 제안을 계산한다. 모두 폰 안에서 계산하고 서버나 유료 AI를 쓰지 않는다. 실제로 "고르는" 일은 Java의 GuardBandit이 이 표를 받아 한다.

밴딧 용어: 팔(arm) = 고를 수 있는 선택지 (숨 고르기, 호흡 …). **칸(context)** = 상황 (강도 × 시간대 × 평일/주말). **보상(reward)** = 그 선택이 얼마나 잘 먹혔는지 0~1 점수. **[α, β]** = 점수를 모은 결과 (대략 1+성공 합, 1+실패 합).

```
/**
 * AI 개인화 (폰 안에서만 계산, 서버·유료 API 없음).
 *
 * 1. 방식·문구 고르기: 문맥(강도 × 시간대 × 평일/주말)별 톰슨 샘플링 밴딧.
 *    여기서는 지난 기록으로 각 팔(arm)의 Beta(α, β)를 계산해 정책표에 넣고,
 *    실제 뽑기는 개입 순간 서비스(GuardBandit.java)가 한다.
 * 2. 위험 시간대: 시간대별 평균 시청과 연장·초과 횟수로 "평소 오래 보는 시간"을 찾는다.
 * 3. 주간 목표 제안: 지난주 목표를 지킨 날이 5일 이상이면 2~5분 줄이자고 제안 (최종 목표에서 멈춤).
 *
 * 학습 개선(v0.9.1, 설정 banditV2, 정책 버전 ai-2). 끄면 v0.9.0(ai-1)과 같은 계산.
 *   - 칸 사이 공유: 기록이 적은 칸은 비슷한 칸의 경험으로 시작한다 (한 사람 안의 계층 사전분포).
 *   - 최근 가중치: 오래된 기록일수록 무게가 작다 (반감기 7일).
 *   - 세밀한 보상: 개입 뒤 30분 동안 실제로 더 본 시간으로 점수를 매긴다.
 *   - 평일/주말 구분: 칸을 나누되, 빈 주말 칸은 평일 같은 시간대에서 가장 많이 빌려 온다.
 *   - 질림 반영: 최근에 나온 방식은 잠깐 덜 고른다 (서비스가 뽑을 때 적용, 정책표의 habituation).
 *
 * 원칙: 강도 단계는 규칙 그대로. AI는 방식·문구·세부 수치·제안만 바꾼다. 잠금은 사람이 확정.
 */
import { hourlyUsage, lastDates, todayKey, usageSecondsOn } from "./logic.ts";
import type { AppData, InterventionLog, UsageLog } from "./types.ts";

/** 밴딧이 보는 기록 기간 */
export const BANDIT_WINDOW_DAYS = 30;
/**
 * 칸 사이 공유: 다른 칸 경험을 최대 이만큼의 기록 무게로 빌려 온다.
 * 4면 자기 칸 기록이 몇 개만 쌓여도 자기 기록 쪽으로 기운다 (빈 칸만 많이 돕는다).
 */
export const BANDIT_SHARE_K = 4;
/** 최근 가중치: 이 날수가 지난 기록은 무게가 절반 (2배 지나면 4분의 1). */
export const BANDIT_HALF_LIFE_DAYS = 7;
/**
 * 빌려 올 때 칸이 얼마나 비슷한지. 다른 점마다 곱한다.
 * 평일↔주말만 다르면 0.8 (가장 비슷), 시간대가 다르면 0.5, 강도가 다르면 0.5.
 */
export const SIMILARITY = { day: 0.8, slot: 0.5, band: 0.5 } as const;
/** 세밀한 보상: 개입 뒤 이 시간 동안 본 쇼츠를 센다. */
export const REWARD_WINDOW_MINUTES = 30;
/** 세밀한 보상: 개입 뒤 이만큼 이상 봤으면 0점. 3분 약속을 지키면 0.8점. */
export const REWARD_WATCH_CAP_MINUTES = 15;
/**
 * 질림 반영 (서비스가 뽑을 때): 방금 나온 방식은 뽑은 값이 strength만큼 줄고,
 * recoveryHours마다 그 감소가 절반씩 회복된다.
 */
export const HABITUATION = { strength: 0.3, recoveryHours: 6 } as const;
/** 방식 밴딧의 강도: 가벼움 1, 중간 2. 강함 단계는 방식을 고르지 않는다. */
```

1~18줄 파일 설명: 무엇을 계산하는지, v0.9.1 학습 개선 5가지(칸 사이 공유, 최근 가중치, 세밀한 보상, 평일/주말, 질림 반영), 원칙(강도 단계는 규칙 그대로, AI는 방식·문구·세부 수치·제안만, 잠금은 사람이 확정).

19~20줄 logic.ts의 함수와 데이터 모양을 가져온다.

AI 수치 (조정하려면 여기를 고친다)

- `BANDIT_WINDOW_DAYS = 30`: 최근 30일 기록만 본다.
- `BANDIT_SHARE_K = 4`: 다른 칸에서 빌려 오는 양은 최대 기록 4개만큼.
- `BANDIT_HALF_LIFE_DAYS = 7`: 7일 지난 기록은 무게 절반.
- `SIMILARITY`: 칸이 얼마나 비슷한지. 평일↔주말만 다르면 0.8, 시간대가 다르면 0.5, 강도가 다르면 0.5 (다른 점마다 곱함).
- `REWARD_WINDOW_MINUTES = 30`, `REWARD_WATCH_CAP_MINUTES = 15`: 개입 뒤 30분 동안 본 시간으로 점수, 15분 이상 보면 0점.
- `HABITUATION`: 질림 반영 세기 0.3, 6시간마다 절반 회복.
- `METHOD_BANDS = [1, 2]`: 방식을 고르는 건 약함·중간 단계만.
- `RISK_WINDOW_DAYS = 14`, `MIN_DAYS_FOR_RISK = 5`: 위험 시간은 최근 14일, 기록 5일 이상일 때.

```
const METHOD_BANDS = [1, 2];
/** 위험 시간대·평균을 계산할 기간 */
export const RISK_WINDOW_DAYS = 14;
/** 위험 시간대를 찾으려면 이만큼의 날 기록이 있어야 한다 */
export const MIN_DAYS_FOR_RISK = 5;
```

```
export type TimeSlot = "morning" | "day" | "evening" | "night";
export type DayType = "weekday" | "weekend";

const SLOTS: TimeSlot[] = ["morning", "day", "evening", "night"];
const DAY_TYPES: DayType[] = ["weekday", "weekend"];

/** 아침 5~11시, 낮 11~17시, 저녁 17~22시, 밤 22~5시 */
export function timeSlot(hour: number): TimeSlot {
  if (hour >= 5 && hour < 11) return "morning";
  if (hour >= 11 && hour < 17) return "day";
  if (hour >= 17 && hour < 22) return "evening";
  return "night";
}

/** 토·일은 주말 (기기 로컬 날짜 기준, 서비스의 GuardPolicy.dayType과 같다). */
export function dayType(date: Date): DayType {
  const d = date.getDay();
  return d === 0 || d === 6 ? "weekend" : "weekday";
}
```

- **TimeSlot / DayType**: 시간대 4개, 평일/주말.
- **timeSlot**: 시를 시간대로 (5~11 아침, 11~17 낮, 17~22 저녁, 나머지 밤). Java와 같은 규칙.
- **dayType**: 토·일이면 주말 (`getDay()`는 일요일 0, 토요일 6).

```
const STOP_OUTCOMES = new Set(["exit", "kept", "day-end", "timed-block", "force-quit"]);

/**
 * v0.9.0 보상: 스스로 멈췄으면 1, 그 뒤 30분 안에 다시 열었으면 0.5, 계속 봤으면 0.
 * "도움 안 됐어요" 피드백이 있으면 절반.
 */
export function reward(log: InterventionLog): number {
  let r = STOP_OUTCOMES.has(log.outcome) ? 1 : 0;
  if (r > 0 && log.reenteredWithin30 === true) r = 0.5;
  if (log.feedback === "not") r *= 0.5;
  return r;
}
```

reward — v0.9.0 점수 (학습 개선을 끄면 사용). 스스로 멈췄으면 1, 그 뒤 30분 안에 다시 열었으면 0.5, 계속 봤으면 0. "도움 안 됐어요"면 절반.

```
/** [start, end) 구간에 겹치는 쇼츠 시청 초. 연습 기록은 뺀다. */
export function watchedSecondsBetween(sessions: UsageLog[], start: number, end: number): number
{
  let total = 0;
  for (const s of sessions) {
    if (s.target === "practice") continue;
    const from = Math.max(start, new Date(s.startTime).getTime());
    const to = Math.min(end, new Date(s.endTime).getTime());
    if (to > from) total += (to - from) / 1000;
  }
  return total;
}
```

watchedSecondsBetween: 정해진 시간 구간에 겹치는 쇼츠 시청 초의 합. 연습 화면 기록은 뺀다.

```ts
/**
 * 세밀한 보상 (v0.9.1): 개입 뒤 30분 동안 실제로 더 본 시간으로 매긴다.
 *   1 − (더 본 분 ÷ 15), 0~1. 바로 나가면 1, 3분 약속을 지키면 0.8, 15분 넘게 보면 0.
 * 다시 들어와 본 시간도 함께 세므로 "30분 안 재방문" 규칙을 자연스럽게 포함한다.
 * "도움 안 됐어요" 피드백이 있으면 절반.
 */
export function gradedReward(log: InterventionLog, sessions: UsageLog[]): number {
  const at = new Date(log.timestamp).getTime();
  const watched = watchedSecondsBetween(sessions, at, at + REWARD_WINDOW_MINUTES * 60_000);
  let r = Math.max(0, 1 - watched / (REWARD_WATCH_CAP_MINUTES * 60));
  if (log.feedback === "not") r *= 0.5;
  return r;
}
```

gradedReward — 세밀한 점수 (v0.9.1). 개입 뒤 30분 동안 **실제로 더 본 시간**으로 매긴다: `1 − (더 본 분 ÷ 15)`, 0~1.

- 바로 나감 → 1점. 3분 약속을 지킴 → 0.8점. 나갔다가 다시 와서 6분 → 0.6점. 15분 넘게 → 0점.
- 다시 들어와 본 시간도 세므로 "30분 안 재방문" 규칙이 자연스럽게 들어 있다.
- "도움 안 됐어요"면 절반.

```ts
/** 칸 → 선택지 → [α, β]. 서비스로 넘기는 정책표 형식. */
export type BetaTable = Record<string, Record<string, [number, number]>>;
/** 칸 → 선택지 → [성공, 실패]. 그 칸에서 직접 쌓인 기록만 (최근 가중치 반영). */
export type CountTable = Record<string, Record<string, [number, number]>>;

function addCount(table: CountTable, context: string, arm: string, r: number, weight: number) {
  const row = (table[context] ??= {});
  const [s, f] = row[arm] ?? [0, 0];
  row[arm] = [s + weight * r, f + weight * (1 - r)];
}
```

- **BetaTable**: 칸 → 선택지 → [α, β] (정책표로 Java에 넘기는 모양).
- **CountTable**: 칸 → 선택지 → [성공 합, 실패 합] (그 칸에서 직접 쌓인 것만).
- **addCount**: 점수 r을 무게만큼 성공 쪽에 r, 실패 쪽에 1−r 더한다. `??=`는 "없으면 새로 만들어 넣기".

```ts
/** 자기 기록만으로 만든 Beta(1 + 성공, 1 + 실패). v0.9.0 계산과 같다. */
function ownPosteriors(counts: CountTable): BetaTable {
  const out: BetaTable = {};
  for (const [context, arms] of Object.entries(counts)) {
    out[context] = {};
    for (const [arm, [s, f]] of Object.entries(arms)) out[context][arm] = [1 + s, 1 + f];
  }
  return out;
}
```

ownPosteriors: 자기 칸 기록만으로 [1 + 성공, 1 + 실패]를 만든다 (v0.9.0 방식). 1을 더하는 건 "아무것도 모를 때 50:50에서 시작"하는 뜻.

```ts
/**
 * 두 칸이 얼마나 비슷한지 (0~1). 칸 이름은 "강도:시간대:평일주말" 또는 "시간대:평일주말".
 * 앞쪽부터 맞춰 보므로 두 형식 모두 쓸 수 있다.
 */
export function similarity(a: string, b: string): number {
  const pa = a.split(":");
  const pb = b.split(":");
  const withBand = pa.length === 3;
  const [bandA, slotA, dayA] = withBand ? pa : ["", pa[0], pa[1]];
  const [bandB, slotB, dayB] = withBand ? pb : ["", pb[0], pb[1]];
  let w = 1;
  if (bandA !== bandB) w *= SIMILARITY.band;
  if (slotA !== slotB) w *= SIMILARITY.slot;
  if (dayA !== dayB) w *= SIMILARITY.day;
  return w;
}
```

similarity — 두 칸이 얼마나 비슷한지. 칸 이름을 `:`로 잘라 강도, 시간대, 평일주말을 비교하고, 다른 것마다 비슷한 정도(0.5, 0.5, 0.8)를 곱한다. 예: "2:night:weekday"와 "2:night:weekend" → 0.8, "2:night:weekday"와 "1:morning:weekend" → 0.5 × 0.5 × 0.8 = 0.2. 문구 칸(강도 없음)도 같은 함수로 계산한다.

```
/**
 * 칸 사이 공유 (한 사람 안의 계층 사전분포).
 * 칸 c, 선택지 a: 다른 칸들의 성공 S, 실패 F를 비슷한 정도로 곱해 더한다 (c 자신은 빼서 이중 계산 방지).
 *   n = S + F, p = (S + 1) / (n + 2), k = min(K, n)
 *   α = 1 + k·p + (c의 성공), β = 1 + k·(1 − p) + (c의 실패)
 * 다른 칸 기록이 없으면 k = 0이라 자기 기록만 쓴 것과 같다.
 */
function sharedPosteriors(counts: CountTable, contexts: string[], k: number): BetaTable {
  const arms = new Set<string>();
  for (const row of Object.values(counts)) for (const arm of Object.keys(row)) arms.add(arm);
  const out: BetaTable = {};
  const all = [...new Set([...contexts, ...Object.keys(counts)])];
  for (const context of all) {
    for (const arm of arms) {
      let otherS = 0;
      let otherF = 0;
      for (const [other, row] of Object.entries(counts)) {
        if (other === context || !row[arm]) continue;
        const w = similarity(context, other);
        otherS += w * row[arm][0];
        otherF += w * row[arm][1];
      }
      const [s, f] = counts[context]?.[arm] ?? [0, 0];
      const n = otherS + otherF;
      const p = (otherS + 1) / (n + 2);
      const borrow = Math.min(k, n);
      (out[context] ??= {})[arm] = [1 + borrow * p + s, 1 + borrow * (1 - p) + f];
    }
  }
  return out;
}
```

sharedPosteriors — 칸 사이 공유 (한 사람 안의 계층 사전분포). 칸 c, 선택지 a마다:

1. **163~170줄 다른 칸들**의 성공 S, 실패 F를 비슷한 정도를 곱해 더한다 (c 자신은 빼서 두 번 세지 않음).
2. **172~174줄** n = S + F, 다른 칸 성공률 p = (S + 1) ÷ (n + 2), 빌려 오는 양 k = min(4, n).
3. **175줄** α = 1 + k·p + (c의 성공), β = 1 + k·(1 − p) + (c의 실패).

뜻: 기록이 적은 칸은 다른 칸 경험(최대 기록 4개만큼)으로 시작하고, 그 칸 기록이 쌓이면 자기 기록이 판단을 주도한다. 다른 칸 기록이 없으면 k = 0이라 자기 기록만 쓴 것과 같다. 기록이 없는 빈 칸도 표에 넣어서(`contexts`), Java가 찾을 수 있게 한다.

```
export type BanditOptions = {
  /** 학습 개선(공유, 최근 가중치, 세밀한 보상, 평일/주말). 기본은 설정(banditV2)을 따른다. */
  v2?: boolean;
  k?: number;
  halfLifeDays?: number;
};

/**
 * 지난 30일 시트 기록으로 Beta 사후분포를 만든다.
 * 학습 개선을 끄면 v0.9.0과 같다: 칸은 "강도:시간대"·"시간대", 사전분포 Beta(1, 1),
 * 모든 기록 같은 무게, 보상은 reward(), 기록 없는 팔은 표에 없음 (서비스가 Beta(1, 1)로 본다).
 * 켜면 칸은 "강도:시간대:평일주말"·"시간대:평일주말", 보상은 gradedReward(), 최근 가중치와 공유를 쓴다.
 * ownMethods, ownFramings는 그 칸에서 직접 쌓인 [성공, 실패]로, AI 패널 설명용이다.
 */
export function banditTables(
  data: AppData,
```

BanditOptions: 테스트에서 학습 개선 켜기/끄기, K, 반감기를 바꿔 볼 수 있게 하는 옵션.

banditTables 시작: 돌려주는 것 — 방식 표, 문구 표(둘 다 [α, β]), 칸별 자기 기록(AI 패널 설명용), 기록 수, 학습 개선 여부.

```
  now = new Date(),
  options: BanditOptions = {},
): {
  methods: BetaTable;
  framings: BetaTable;
  ownMethods: CountTable;
  ownFramings: CountTable;
  samples: number;
  v2: boolean;
} {
```

```
  const v2 = options.v2 ?? data.settings.banditV2 !== false;
  const k = options.k ?? BANDIT_SHARE_K;
  const halfLife = options.halfLifeDays ?? BANDIT_HALF_LIFE_DAYS;
  const nowMs = now.getTime();
  const since = nowMs - BANDIT_WINDOW_DAYS * 86_400_000;
  // 세밀한 보상에 쓸 시청 기록 (창 안의 것만)
  const sessions = v2
    ? data.usageLogs.filter((s) => new Date(s.endTime).getTime() >= since)
    : [];
  const ownMethods: CountTable = {};
  const ownFramings: CountTable = {};
  let samples = 0;
```

banditTables (1) 준비. 학습 개선 사용 여부(설정 `banditV2`), K, 반감기, 지금 시각, 30일 전 시각. 학습 개선이면 세밀한 점수에 쓸 최근 시청 기록을 모은다.

```
  for (const log of data.interventions) {
    if (log.stage !== "entry" && log.stage !== "commit_end") continue;
    const at = new Date(log.timestamp);
    const atMs = at.getTime();
    if (atMs < since || atMs > nowMs) continue;
    let r: number;
    if (v2) {
      // 개입 뒤 30분이 지나야 점수가 정해진다.
      if (nowMs - atMs < REWARD_WINDOW_MINUTES * 60_000) continue;
      r = gradedReward(log, sessions);
    } else {
      // 시청 결과가 아직 정해지지 않은 최근 30분 기록은 재진입 판정 전이라 뺀다.
      if (STOP_OUTCOMES.has(log.outcome) && log.reenteredWithin30 === null) continue;
      r = reward(log);
    }
    const weight = v2 ? Math.pow(0.5, (nowMs - atMs) / 86_400_000 / halfLife) : 1;
    const slot = timeSlot(log.hour);
    const band = log.band ?? log.level;
    const suffix = v2 ? `:${dayType(at)}` : "";
    if (log.interventionType && log.stage === "entry") {
      addCount(ownMethods, `${band}:${slot}${suffix}`, log.interventionType, r, weight);
      samples += 1;
    }
    if (log.framing) addCount(ownFramings, `${slot}${suffix}`, log.framing, r, weight);
  }
```

banditTables (2) 기록마다 점수 매기기.

1. **220줄** 진입(entry)과 약속 끝(commit_end) 기록만 본다.
2. **223줄** 30일 안 기록만.
3. **225~233줄** 점수: 학습 개선이면 개입 뒤 30분이 지난 것만 세밀한 점수로, 아니면 재진입 판정이 끝난 것만 예전 점수로.
4. **234줄** 무게: 학습 개선이면 `0.5^(지난 날 ÷ 7)` (최근일수록 큼), 아니면 1.
5. **235~237줄** 칸 이름: 강도:시간대(:평일주말).
6. **238~242줄** 진입 기록이면 **방식** 칸에 점수를 더하고, 문구를 보여 줬으면 **문구** 칸(시간대:평일주말)에도 더한다.

```
  if (!v2) {
    return {
      methods: ownPosteriors(ownMethods),
      framings: ownPosteriors(ownFramings),
      ownMethods,
      ownFramings,
      samples,
      v2,
    };
  }
  const framingContexts = SLOTS.flatMap((slot) => DAY_TYPES.map((day) => `${slot}:${day}`));
  const methodContexts = METHOD_BANDS.flatMap((band) => framingContexts.map((c) => `${band}:${c}
`));
  return {
    methods: sharedPosteriors(ownMethods, methodContexts, k),
    framings: sharedPosteriors(ownFramings, framingContexts, k),
    ownMethods,
    ownFramings,
    samples,
    v2,
  };
}
```

banditTables (3) 표 만들기. 학습 개선을 끄면 자기 기록만으로(v0.9.0과 똑같이). 켜면 모든 칸 이름(문구 8칸, 방식 16칸)을 만들고 **칸 사이 공유**로 표를 만든다.

```
/** 지난 14일 중 기록이 있는 날 수 */
export function recordedDays(data: AppData, now = new Date()): number {
  const days = new Set(data.usageLogs.map((log) => log.date));
  return lastDates(RISK_WINDOW_DAYS, now).filter((d) => days.has(d) && d !== todayKey(now)).leng
th;
}

export type RiskInfo = {
  /** 평소 오래 보는 시간 (0~23시), 가장 이른 시간부터 */
  hours: number[];
  /** 그 시간대 평균 시청 초 (하루 기준) */
  avgSeconds: number;
  /** 연장·초과를 고른 비율 (그 시간대 개입 중) */
  extendRate: number;
};
```

recordedDays: 최근 14일(오늘 빼고) 중 기록이 있는 날 수 (AI 패널 표시용).

RiskInfo: 위험 시간 결과 모양 — 시간 목록, 그 시간 평균 시청 초, 그 시간 연장·초과 비율.

```
/**
 * 위험 시간대: 지난 14일(오늘 제외) 시간대별 평균 시청이 5분 이상이고,
 * 하루 평균 시간대 시청의 1.5배 이상인 시간 중 상위 3개.
 * 기록이 5일보다 적으면 없음.
 */
export function riskHours(data: AppData, now = new Date()): RiskInfo | null {
  const days = lastDates(RISK_WINDOW_DAYS + 1, now).slice(0, RISK_WINDOW_DAYS);
  const recorded = new Set(data.usageLogs.map((log) => log.date));
  const usedDays = days.filter((d) => recorded.has(d));
  if (usedDays.length < MIN_DAYS_FOR_RISK) return null;
  const totals = new Array<number>(24).fill(0);
  for (const day of usedDays) {
```

riskHours — 평소 오래 보는 시간 찾기 (1).

1. 최근 14일(오늘 제외) 중 기록이 있는 날만 쓴다. 5일 미만이면 없음.
2. 그 날들의 시간대별 시청을 더해 하루 평균을 낸다 (24칸).
3. 시청이 있는 시간들의 평균(mean)을 구한다.

```
    hourlyUsage(data, day).forEach((v, h) => {
      totals[h] += v;
    });
  }
  const avg = totals.map((v) => v / usedDays.length);
  const active = avg.filter((v) => v > 0);
  if (active.length === 0) return null;
  const mean = active.reduce((s, v) => s + v, 0) / active.length;
```

```
  const hours = avg
    .map((v, h) => ({ v, h }))
    .filter(({ v }) => v >= 300 && v >= mean * 1.5)
    .sort((a, b) => b.v - a.v)
    .slice(0, 3)
    .map(({ h }) => h)
    .sort((a, b) => a - b);
  if (hours.length === 0) return null;
  const since = now.getTime() - RISK_WINDOW_DAYS * 86_400_000;
  const inRisk = data.interventions.filter(
    (log) => hours.includes(log.hour) && new Date(log.timestamp).getTime() >= since,
  );
  const extend = inRisk.filter((log) => log.outcome === "extend" || log.outcome === "overdraf
t").length;
  return {
    hours,
    avgSeconds: hours.reduce((s, h) => s + avg[h], 0) / hours.length,
    extendRate: inRisk.length === 0 ? 0 : extend / inRisk.length,
  };
}
```

riskHours (2)

1. **301~307줄** 평균 시청이 **5분(300초) 이상이고 전체 평균의 1.5배 이상**인 시간을 많은 순으로 최대 3개 고른 뒤 시간 순으로 정렬한다.
2. **309~313줄** 그 시간대 개입 중 연장·3분만 더를 고른 비율.
3. 시간 목록, 평균 초, 연장 비율을 돌려준다.

```
/** "밤 11시", "저녁 9시", "오후 2시" 같은 말 */
export function hourLabel(hour: number): string {
  if (hour === 0) return "밤 12시";
  if (hour < 5) return `새벽 ${hour}시`;
  if (hour < 12) return `오전 ${hour}시`;
  if (hour === 12) return "낮 12시";
  if (hour < 18) return `오후 ${hour - 12}시`;
  if (hour < 21) return `저녁 ${hour - 12}시`;
  return `밤 ${hour - 12}시`;
}

/** 이번 주 월요일 날짜 (주간 제안을 한 주에 한 번만 보이려고) */
export function weekKey(now = new Date()): string {
  const monday = new Date(now);
  monday.setDate(now.getDate() - ((now.getDay() + 6) % 7));
  return todayKey(monday);
}
```

hourLabel: 시를 말로 — 0시 "밤 12시", 1~4시 "새벽 N시", 5~11시 "오전 N시", 12시 "낮 12시", 13~17시 "오후 N시", 18~20시 "저녁 N시", 21~23시 "밤 N시".

weekKey: 이번 주 월요일 날짜 (주간 제안을 한 주에 한 번만 보여 주려고).

```
export type GoalSuggestion = {
  week: string;
  current: number;
```

goalSuggestion — 주간 목표 줄이기 제안.

1. 지난주(월~일) 날짜를 구하고, 기록이 있으면서 목표 안에서 끝낸 날을 센다.

```
  suggested: number;
  keptDays: number;
  /** true면 줄이기 제안, false면 최종 목표에 닿아 유지 */
  reduce: boolean;
};

/**
 * 주간 목표 제안. 지난주(월~일) 중 기록이 있는 날 가운데 목표 안에서 끝낸 날이 5일 이상이면
 * 목표를 2~5분 줄이자고 제안한다. 최종 목표(바닥 5분)에 닿으면 유지 단계로 본다.
 * 지난주를 못 지켰으면 제안하지 않는다 (실패 뒤에 더 조이지 않는다).
 */
export function goalSuggestion(data: AppData, now = new Date()): GoalSuggestion | null {
  const week = weekKey(now);
  const [y, m, d] = week.split("-").map(Number);
  const lastSunday = new Date(y, m - 1, d - 1);
  const lastWeek = lastDates(7, lastSunday);
  const recorded = new Set(data.usageLogs.map((log) => log.date));
  const goal = data.settings.level3Threshold;
  const keptDays = lastWeek.filter(
    (day) => recorded.has(day) && usageSecondsOn(data, day) <= goal * 60,
  ).length;
  if (keptDays < 5) return null;
  const floor = Math.max(5, Math.min(data.settings.finalGoal, goal));
  if (goal <= floor) return { week, current: goal, suggested: goal, keptDays, reduce: false };
  const step = goal >= 30 ? 5 : goal >= 15 ? 3 : 2;
  return { week, current: goal, suggested: Math.max(floor, goal - step), keptDays, reduce: true
};
}
```

2. 5일 미만이면 제안하지 않는다 (**못 지킨 주에는 더 조이지 않는다**).
3. 바닥 = 최종 목표와 5분 중 큰 값 (단, 목표보다 크지 않게). 이미 바닥이면 "유지" 제안.
4. 줄이는 폭: 목표 30분 이상이면 5분, 15분 이상이면 3분, 아니면 2분. 바닥 아래로는 내리지 않는다.

policy.ts — 정책표 만들기와 공부 카드 가져오기 `lib/policy.ts (266줄)`

휴대폰 쪽(Java)에 넘길 **정책표**를 만들고, 공부 카드를 넘길 목록과 **카드 가져오기 검사**를 담당한다.

```
import { banditTables, HABITUATION, hourLabel, riskHours, type BetaTable } from "./ai.ts";
import { FRAMINGS, usageStats } from "./logic.ts";
import type { AppData, Settings, StudyCard } from "./types.ts";

/**
 * 개입 정책표. 서비스(GuardPolicy.java)는 개입 순간 이 표만 읽고 결정한다.
 * v0.8.0은 규칙으로 채운다. AI 단계에서는 같은 형식을 AI가 채운다
 * (방식·문구 순서, 시간대별 약속 최대치 등). 형식을 바꾸면 GuardPolicy.fromJson도 맞춘다.
 */
```

1~3줄 AI 계산 함수, 문구 종류와 통계 함수, 데이터 모양을 가져온다.

5~9줄 설명: 서비스(Java GuardPolicy)는 개입 순간 이 표만 읽는다. 규칙이든 AI든 같은 모양으로 채운다. 모양을 바꾸면 Java 쪽 읽기도 같이 고쳐야 한다.

```ts
export type Policy = {
  version: string;
  goalMinutes: number;
  /** 목표 대비 비율: 이 이상이면 중간 / 강함 */
  mediumAt: number;
  strongAt: number;
  /** 강도별로 번갈아 보여줄 방식 */
  methods: { light: string[]; medium: string[] };
  commitOptions: number[];
  /** 연장할 때마다 줄어드는 최대 약속 시간 */
  extensionLadder: number[];
  overdraftMinutes: number;
  overdraftWaitSec: number;
  dayEndHour: number;
  /** 시간 체감 문구(S1) 종류와 순서 */
  framings: string[];
  /**
   * 공부 모드: 쉬는 중이 아닌 카드가 있으면 약함·중간 활동 대신 공부 카드.
   * 모든 카드가 쉬는 중이거나 카드가 없으면 서비스가 평소 활동으로 넘어간다.
   */
  studyMode: boolean;
  /** 문구에 쓰는 통계. 서비스가 지금 시각 기준으로 마무리 계산을 한다. */
  stats: {
    avg7Seconds: number;
    pastWeekSeconds: number;
    daysBeforeToday: number;
    yesterdayCumulative: number[] | null;
    wakeHour: number;
    wakeMinute: number;
  };
```

Policy — 정책표 모양 (1). 버전, 하루 목표, 중간·강함 기준 비율, 단계별 방식 목록, 약속 선택지와 연장 사다리, "3분만 더" 분과 대기 초, "오늘은 끝" 시, 시간 문구 종류, 공부 모드, 문구용 통계(7일 평균, 이번 주 합과 날 수, 어제 누적, 일어나는 시·분).

```ts
  /** AI 단계. enabled가 false면 서비스는 규칙대로 (번갈아) 동작한다. */
  ai: {
    enabled: boolean;
    /** 엄격 모드: 연장과 "3분만 더"를 없앤다. AI와 상관없이 적용. */
    strict: boolean;
    /**
     * 방식별 Beta(α, β). 서비스가 톰슨 샘플링으로 뽑는다.
     * 칸 이름: ai-2는 "강도:시간대:평일주말", ai-1은 "강도:시간대" (서비스는 둘 다 찾아본다).
     */
    methods: BetaTable;
    /** 문구 종류별 Beta(α, β). 칸 이름: ai-2는 "시간대:평일주말", ai-1은 "시간대". */
    framings: BetaTable;
    /**
     * 질림 반영: 최근에 나온 선택지는 뽑은 값을 (1 − strength·0.5^(지난 시간/recoveryHours))배.
     * ai-1에서는 strength 0 (반영 안 함).
     */
    habituation: { strength: number; recoveryHours: number };
    /** 평소 오래 보는 시간 (0~23시). 이 시간엔 세부 수치를 조인다. */
    riskHours: number[];
```

Policy (2) — AI 부분. 켜짐, 엄격 모드, 방식 표와 문구 표([α, β], 칸 이름 형식 설명), 질림 반영 값, 위험 시간 목록과 이름, 위험 시간 조이기 값, 피드백 간격.

```
    /** 문구에 쓸 위험 시간 이름, 예: "밤 11시" */
    riskLabel: string;
    risk: { commitMax: number; extensionMax: number; overdraftWaitSec: number };
    /** 칭찬 화면에서 피드백을 묻는 간격 (N번에 1번) */
    feedbackEvery: number;
  };
};
```

```
export const RULE_POLICY_VERSION = "rule-2";
export const AI_POLICY_VERSION = "ai-1";
/** AI + 밴딧 학습 개선 (v0.9.1). 기록의 policyVersion으로 전후를 나눠 본다. */
export const AI_SHARED_POLICY_VERSION = "ai-2";
```

정책 버전 이름: AI 꺼짐 "rule-2", AI 켜짐 "ai-1"(v0.9.0 계산), AI + 학습 개선 "ai-2". 개입 기록마다 이 버전이 남아서 나중에 효과를 비교할 수 있다.

```
export function buildPolicy(settings: Settings, data?: AppData, now = new Date()): Policy {
  const stats = data
    ? usageStats(data, now)
    : { avg7Seconds: -1, pastWeekSeconds: -1, daysBeforeToday: 0, yesterdayCumulative: null };
  const [wakeHour, wakeMinute] = settings.wakeTime.split(":").map(Number);
  const aiOn = settings.aiEnabled && data !== undefined;
  const tables = aiOn ? banditTables(data, now) : { methods: {}, framings: {} };
  const risk = aiOn ? riskHours(data, now) : null;
```

buildPolicy (1) — 재료 준비. 데이터가 있으면 통계를 계산(없으면 모름 -1), 일어나는 시각을 시·분으로 나누고, AI가 켜져 있으면 밴딧 표(`banditTables`)와 위험 시간(`riskHours`)을 계산한다.

```
  return {
    version: !aiOn
      ? RULE_POLICY_VERSION
      : settings.banditV2
        ? AI_SHARED_POLICY_VERSION
        : AI_POLICY_VERSION,
    goalMinutes: settings.level3Threshold,
    mediumAt: 0.5,
    strongAt: 1,
    methods: {
      light: ["pause", "framing"],
      medium: ["breath", "alternative", "reflection", "card", "pause", "framing"],
    },
    studyMode: settings.studyMode,
    commitOptions: [3, 5, 10],
    extensionLadder: [10, 5, 3],
    overdraftMinutes: 3,
    overdraftWaitSec: 3,
    dayEndHour: settings.dayEndHour,
    framings: [...FRAMINGS],
    stats: { ...stats, wakeHour, wakeMinute },
```

buildPolicy (2) — 규칙 부분.

- 버전: AI 꺼짐이면 rule-2, 학습 개선이면 ai-2, 아니면 ai-1.
- 하루 목표 = 설정의 목표, **중간 0.5, 강함 1.0** (목표의 50%, 100%).
- 약함 단계 방식: 숨 고르기, 시간 문구. 중간: 호흡, 대체 행동, 성찰, 카드, 숨 고르기, 시간 문구.
- 공부 모드, 약속 3/5/10분, 연장 10→5→3, "3분만 더" 3분·3초 대기, "오늘은 끝" 시, 문구 종류, 통계.

단계 기준이나 약속 선택지를 바꾸려면 여기를 고친다.

```
    ai: {
      enabled: aiOn,
      strict: settings.strictMode,
      methods: tables.methods,
      framings: tables.framings,
      habituation:
        aiOn && settings.banditV2 ? { ...HABITUATION } : { strength: 0, recoveryHours: HABITUATI
ON.recoveryHours },
```

buildPolicy (3) — AI 부분. AI 켜짐, 엄격 모드, 방식·문구 표, 질림 반영(학습 개선일 때만 세기 0.3, 아니면 0), 위험 시간과 첫 위험 시간 이름("밤 11시"), 위험 시간 조이기(약속 최대 3분, 연장 1번, 대기 10초), 피드백 5번에 1번.

```
      riskHours: risk?.hours ?? [],
      riskLabel: risk ? hourLabel(risk.hours[0]) : "",
      risk: { commitMax: 3, extensionMax: 1, overdraftWaitSec: 10 },
      feedbackEvery: 5,
    },
  };
}
```

```
/** 서비스에 넘길 공부 카드: 덜 본 카드부터 최대 {limit}장. */
export function cardsForService(
  data: AppData,
  limit = 60,
): { id: string; question: string; options: string[]; answer: number; explanation: string }[] {
  return [...data.cards]
    .sort((a, b) => a.shown - b.shown || a.createdAt.localeCompare(b.createdAt))
    .slice(0, limit)
    .map((card) => ({
      id: card.id,
      question: card.question,
      options: card.options,
      answer: card.answer,
      explanation: card.explanation,
    }));
}
```

cardsForService — 휴대폰에 넘길 공부 카드. 덜 본 카드부터(같으면 먼저 만든 것부터) 최대 60장을 골라, 필요한 칸(id, 문제, 보기, 정답, 해설)만 넘긴다. 휴대폰은 이 순서대로 "지금 나올 수 있는" 첫 카드를 쓴다.

```
/** 붙여넣기 가져오기용 질문 문장. ChatGPT·Gemini 무료 채팅에 자료와 함께 붙여 넣는다. */
export const CARD_PROMPT = `아래 자료(붙여 넣은 글이나 첨부한 파일)로 객관식 문제 5개를 만들어 줘.
범위: (비워 두면 전체. 예: 3장, 1~10쪽)
반드시 아래 JSON 형식만 코드 블록 하나로 출력하고, 그 밖의 말은 쓰지 마.
[{"question":"문제","options":["보기1","보기2","보기3","보기4"],"answer":1,"explanation":"정답인 이유를 한두 문장으로"}]
- answer는 정답 보기의 번호(1~4)야.
- explanation에는 왜 그게 정답인지 짧게 설명해 줘.
- 따옴표는 곧은 따옴표(")만 써 줘.
- 숫자나 연도를 외우는 문제보다, 개념을 이해했는지 묻는 문제로 만들어 줘.
- 다 만든 뒤 정답이 자료에 실제로 나오는지 다시 확인하고, 애매하거나 자료에 없는 문제는 빼 줘.

자료:
`;
```

CARD_PROMPT — 카드 만들기 질문 문장. 사용자가 "질문 문장 복사"로 복사해서 ChatGPT·Gemini에 붙여 넣는 글이다. 요청 내용: 첨부한 자료로 객관식 5개, 범위 칸, **정해진 JSON 형식만** 출력, 정답은 1~4번, 해설 한두 문장, 곧은 따옴표, 숫자·연도 암기 대신 개념 문제, 다 만든 뒤 자료에 실제로 있는지 스스로 확인. 백틱(`)으로 감싼 여러 줄 글이다.

```
/** 비교용: 띄어쓰기·문장부호·대소문자를 지운다. */
export function normalizeCardText(text: string): string {
  return text.toLowerCase().replace(/[\s.,!?·'"“”‘’()[\]{}:;~\-]/g, "");
}

/** "보기1", "...", "-" 처럼 내용 없는 보기 */
const JUNK_OPTION = /^(보기\s*\d+|선택지\s*\d+|option\s*\d+|\.{2,}|…+|-+|\?+|_+)$/i;
```

normalizeCardText: 비교용으로 소문자로 바꾸고 띄어쓰기·문장부호를 모두 지운다. 예: "인플레이션이란 ?" → "인플레이션이란". 정규식 `[\s.,!?…]`는 "공백이나 이 기호들 중 하나"를 뜻하고, `g`는 "전부".

JUNK_OPTION: 내용 없는 보기 모양 — "보기1", "선택지 2", "option 3", "…", "...", "---", "???", "___". `^...$`는 "글 전체가 이 모양", `i`는 대소문자 무시.

```
/**
 * 해설이 정답이 아닌 보기를 가리키는 것 같으면 true.
 * 해설에 보기 글자가 하나 이상 나오는데 정답 보기는 없을 때만 의심한다
```

explanationLooksWrong — 해설이 정답과 어긋나 보이는지. 해설에 보기 글자(2자 이상)가 들어 있는지 보고, **하나 이상 나오는데 그중 정답 보기가 없으면**

```
 * (정답을 풀어 쓴 해설은 보기 글자가 안 나오므로 의심하지 않는다).
 */
export function explanationLooksWrong(card: Pick<StudyCard, "options" | "answer" | "explanation">): boolean {
  const explanation = normalizeCardText(card.explanation);
  if (!explanation) return false;
  const mentioned = card.options
    .map((o, i) => ({ i, text: normalizeCardText(o) }))
    .filter((o) => o.text.length >= 2 && explanation.includes(o.text));
  return mentioned.length > 0 && !mentioned.some((o) => o.i === card.answer);
}
```

의심한다. 정답을 풀어 쓴 해설은 보기 글자가 안 나오므로 의심하지 않는다. 예: 정답 "4배"인데 해설이 "2.6배 올랐어요"면 의심.

```
/**
 * 붙여넣은 글에서 카드를 꺼낸다. 앞뒤 설명이나 코드 블록 표시가 섞여 있어도
 * 첫 [ 부터 마지막 ] 까지를 읽는다.
 * 자동 검사: 형식이 틀리거나, 보기가 비었거나 겹치거나, 내용 없는 보기("보기1", "...")가 있으면 건너뛴다.
 * 이미 있는 카드(또는 같은 묶음 안)와 문제가 같으면 duplicates로 센다.
 * 해설이 다른 보기를 가리키는 것 같으면 가져오되 suspectIds에 넣는다 (가져온 뒤 확인 화면에 표시).
 */
export function parseCardImport(
  text: string,
  now = new Date(),
  makeId: () => string = () => crypto.randomUUID(),
  existing: Pick<StudyCard, "question">[] = [],
): { cards: StudyCard[]; skipped: number; duplicates: number; suspectIds: string[]; error: string | null } {
  const start = text.indexOf("[");
  const end = text.lastIndexOf("]");
  const none = { cards: [], skipped: 0, duplicates: 0, suspectIds: [] };
  if (start < 0 || end <= start) {
    return { ...none, error: "[ ] 로 묶인 JSON을 찾지 못했어요." };
  }
```

parseCardImport — 붙여 넣은 글에서 카드 꺼내기 (1). 받는 것: 붙여 넣은 글, 지금 시각, id 만드는 함수, 이미 있는 카드(중복 확인용). 글에서 **첫 `[`부터 마지막 `]`까지**만 잘라 쓴다 (AI가 앞뒤에 인사말이나 코드 블록 표시를 붙여도 괜찮게). 없으면 안내 문장.

```
  // ChatGPT·Gemini가 흔히 섞는 것: 둥근 따옴표, 끝에 남은 쉼표.
  const cleaned = text
    .slice(start, end + 1)
    .replace(/[“”„‟]/g, '"')
    .replace(/[‘’]/g, "'")
    .replace(/,\s*([\]}])/g, "$1");
  let parsed: unknown;
  try {
    parsed = JSON.parse(cleaned);
  } catch {
    return { ...none, error: "JSON 형식이 맞지 않아요. 결과를 그대로 다시 붙여넣어 주세요." };
  }
  if (!Array.isArray(parsed)) {
    return { ...none, error: "문제 목록(배열)이 아니에요." };
  }
```

(2) 흔한 실수 고치고 읽기. 둥근 따옴표(“ ” ‘ ’)를 곧은 따옴표로, 목록 끝에 남은 쉼표를 지운다. 그다음 JSON으로 읽고, 실패하거나 목록이 아니면 안내 문장을 돌려준다.

```
  const cards: StudyCard[] = [];
  const suspectIds: string[] = [];
  const seen = new Set(existing.map((card) => normalizeCardText(card.question)));
  let skipped = 0;
  let duplicates = 0;
  for (const item of parsed) {
    const raw = item as { question?: unknown; options?: unknown; answer?: unknown; explanation?: unknown };
    const options = Array.isArray(raw.options)
      ? raw.options
          .map((o) => (typeof o === "number" ? String(o) : o))
          .filter((o): o is string => typeof o === "string" && o.trim() !== "")
          .map((o) => o.trim())
      : [];
    // 정답이 "2", "2번", 또는 보기 글자 그대로 와도 읽는다.
    let answer = Number(String(raw.answer ?? "").replace(/[^0-9]/g, "") || NaN);
    if (!Number.isInteger(answer) && typeof raw.answer === "string") {
      const at = options.indexOf(raw.answer.trim());
      if (at >= 0) answer = at + 1;
    }
```

(3) 문제마다 준비. 결과 목록, 의심 목록, 이미 있는 문제(정리한 글자)를 담은 집합, 건너뛴 수, 중복 수. 문제마다 보기를 정리하고(숫자 보기는 글자로, 빈 보기는 뺌), 정답을 읽는다: "2", "2번"처럼 숫자만 뽑고, 숫자가 없으면 보기 글자와 같은 번호를 찾는다.

```
    const question = typeof raw.question === "string" ? raw.question.trim() : "";
    const optionKeys = options.map(normalizeCardText);
    if (
      question.length < 4 ||
      options.length < 2 ||
      options.length > 6 ||
      options.some((o) => JUNK_OPTION.test(o)) ||
      optionKeys.some((key) => !key) ||
      new Set(optionKeys).size !== optionKeys.length ||
      !Number.isInteger(answer) ||
      answer < 1 ||
      answer > options.length ||
      question.length > 300
    ) {
      skipped += 1;
      continue;
    }
```

(4) 자동 검사 — 하나라도 걸리면 건너뜀. 문제 4자 미만 또는 300자 초과, 보기 2개 미만 또는 6개 초과, 내용 없는 보기, 정리하니 빈 보기, **똑같은 보기 중복**(집합 크기가 다르면 중복), 정답 번호가 범위 밖.

```
    const key = normalizeCardText(question);
    if (seen.has(key)) {
      duplicates += 1;
      continue;
    }
    seen.add(key);
    const card: StudyCard = {
      id: makeId(),
      question,
      options,
      answer: answer - 1,
      createdAt: now.toISOString(),
```

(5) 중복 거르기와 카드 만들기. 정리한 문제 글자가 이미 있거나 같은 묶음에 나왔으면 중복으로 세고 넘어간다. 아니면 카드를 만든다 (정답은 0부터 세도록 −1, 출처 "가져오기", 해설 500자까지). 해설이 어긋나 보이면 의심 목록에 넣는다.

```
      source: "import",
      shown: 0,
      correct: 0,
      explanation: typeof raw.explanation === "string" ? raw.explanation.trim().slice(0, 500) :
"",
    };
    if (explanationLooksWrong(card)) suspectIds.push(card.id);
    cards.push(card);
  }
```

```
  return {
    cards,
    skipped,
    duplicates,
    suspectIds,
    error: cards.length === 0 ? "가져올 수 있는 문제가 없어요." : null,
  };
}
```

(6) 결과: 가져온 카드, 건너뛴 수, 중복 수, 의심 카드 id, 하나도 없으면 안내 문장. 화면(cards-panel)이 이걸로 알림 문장과 확인 목록을 만든다.

db.ts — 폰 안 데이터베이스 (SQLite) `lib/db.ts (440줄)`

안드로이드에서 **진짜 기록 보관소**. 브라우저 저장소(localStorage)는 폰이 지울 수 있어서 SQLite 데이터베이스에 저장한다. 표 5개: sessions(시청), interventions(개입), lock_attempts(잠금 중 시도), cards(공부 카드), settings(설정 등 이름→값).

SQL 읽는 법: `CREATE TABLE 표 (칸 종류, ...)` 표 만들기 · `INSERT OR REPLACE INTO 표 (...) VALUES (?, ...)` 넣기(같은 id면 덮어쓰기) · `SELECT * FROM 표` 읽기 · `DELETE FROM 표 WHERE 조건` 지우기 · `?` 자리에 값이 안전하게 들어간다. 종류: TEXT 글자, INTEGER 정수, REAL 소수. `PRIMARY KEY` = 줄마다 다른 고유 값.

```
import {
  CapacitorSQLite,
  SQLiteConnection,
  type SQLiteDBConnection,
} from "@capacitor-community/sqlite";
import {
  defaultData,
  MIGRATED_KEY,
  readBlock,
  readCard,
  readIntervention,
  readLocalStorage,
  readLockAttempt,
  readSession,
  readSettings,
  readUsageLog,
  STORAGE_KEY,
} from "./storage.ts";
import type {
```

1~5줄 SQLite 플러그인(Capacitor 커뮤니티 제작)을 가져온다. **6~26줄** 줄 읽기 함수들과 데이터 모양.

```
  AppData,
  InterventionLog,
  LockAttempt,
  Settings,
  StudyCard,
  UsageLog,
} from "./types.ts";

/**
 * 안드로이드 저장소. localStorage는 OS가 지울 수 있어 기록은 SQLite에 둔다.
 * 표: sessions, interventions, lock_attempts, cards, settings(키-값).
 */
```

```
const DB_NAME = "shortsai";
const SCHEMA = `
CREATE TABLE IF NOT EXISTS sessions (
  id TEXT PRIMARY KEY NOT NULL,
  target TEXT NOT NULL,
  source TEXT NOT NULL,
  date TEXT NOT NULL,
  started_at TEXT NOT NULL,
  ended_at TEXT NOT NULL,
  duration_sec REAL NOT NULL,
  clip_count INTEGER NOT NULL DEFAULT 1
);
CREATE INDEX IF NOT EXISTS sessions_started ON sessions (started_at);
```

데이터베이스 이름 "shortsai"와 표 만들기 SQL (없을 때만 만든다 `IF NOT EXISTS`).

sessions 표: id, 대상, 출처, 날짜, 시작·끝 시각, 초, 영상 수. **44줄** 시작 시각으로 빨리 찾을 수 있게 색인(INDEX).

```
CREATE TABLE IF NOT EXISTS interventions (
  id TEXT PRIMARY KEY NOT NULL,
  created_at TEXT NOT NULL,
  date TEXT NOT NULL,
  hour INTEGER NOT NULL,
  weekday INTEGER NOT NULL,
  target TEXT,
  level INTEGER NOT NULL,
  reason TEXT NOT NULL,
  reason_note TEXT NOT NULL DEFAULT '',
  intervention_type TEXT NOT NULL,
  alternative_action TEXT,
  alternative_completed INTEGER NOT NULL DEFAULT 0,
  outcome TEXT NOT NULL,
  re_entered INTEGER NOT NULL DEFAULT 0,
  blocked INTEGER NOT NULL DEFAULT 0,
  block_minutes INTEGER NOT NULL DEFAULT 0,
  usage_seconds REAL NOT NULL DEFAULT 0,
  reentered_within_30min INTEGER,
  feedback TEXT
);
CREATE INDEX IF NOT EXISTS interventions_created ON interventions (created_at);
```

interventions 표 (기본 칸): id, 시각, 날짜, 시, 요일, 대상, 단계, 이유, 이유 메모, 방식, 대체 행동과 완료(0/1), 결과, 재진입, 잠금 여부와 분, 사용 초, 30분 재진입(비어 있을 수 있음), 피드백. 시각 색인.

```
CREATE TABLE IF NOT EXISTS lock_attempts (
  id TEXT PRIMARY KEY NOT NULL,
  created_at TEXT NOT NULL,
  date TEXT NOT NULL,
  remaining_sec INTEGER NOT NULL,
  target TEXT
);
CREATE TABLE IF NOT EXISTS settings (
  key TEXT PRIMARY KEY NOT NULL,
  value TEXT NOT NULL
);
CREATE TABLE IF NOT EXISTS cards (
  id TEXT PRIMARY KEY NOT NULL,
  question TEXT NOT NULL,
  options TEXT NOT NULL,
  answer INTEGER NOT NULL,
  created_at TEXT NOT NULL,
  source TEXT NOT NULL,
  shown INTEGER NOT NULL DEFAULT 0,
  correct INTEGER NOT NULL DEFAULT 0,
  explanation TEXT NOT NULL DEFAULT ''
);
`;
```

lock_attempts 표: id, 시각, 날짜, 남은 초, 대상.

settings 표: 이름(key)과 값(value, JSON 글자). 설정, 잠금 상태 등을 이름별로 한 줄씩.

cards 표: id, 문제, 보기(JSON 글자), 정답 번호, 만든 시각, 출처, 나온 횟수, 맞힌 횟수, 해설.

```
/**
 * v0.8.0에서 interventions에 더한 칸. 이전 버전에서 만든 표에는 없으므로 열 때 추가한다.
 * (AI 학습 재료: 강도, 단계, 보여준 문구, 참여 확인, 성찰, 약속, 카드, 정책표 버전)
 */
const INTERVENTION_COLUMNS: [string, string][] = [
  ["band", "INTEGER"],
  ["stage", "TEXT"],
  ["framing", "TEXT"],
  ["verification", "TEXT"],
  ["reflection", "TEXT"],
  ["after_lock", "TEXT"],
  ["commit_minutes", "INTEGER NOT NULL DEFAULT 0"],
  ["card_id", "TEXT"],
  ["card_correct", "INTEGER"],
  ["extension_index", "INTEGER NOT NULL DEFAULT 0"],
  ["policy_version", "TEXT"],
];
```

INTERVENTION_COLUMNS — 나중에 추가한 칸들. v0.8.0에서 개입 표에 AI 학습 재료 칸(강도, 단계, 문구, 참여 확인, 성찰, 잠금 뒤 답, 약속 분, 카드 id와 정답, 연장 순서, 정책 버전)을 더했다. 예전 버전에서 만든 표에는 이 칸이 없으므로 열 때 추가한다 (업데이트해도 기록이 사라지지 않게).

```
type Statement = { statement: string; values: unknown[] };

const sqlite = new SQLiteConnection(CapacitorSQLite);
let opening: Promise<SQLiteDBConnection> | null = null;
```

Statement: SQL 한 문장과 값들. **sqlite**: 플러그인 연결 도구. **opening**: 여는 중인 연결 (여러 번 열지 않게 기억).

```
async function connect(): Promise<SQLiteDBConnection> {
  const consistent = (await sqlite.checkConnectionsConsistency()).result;
  const exists = (await sqlite.isConnection(DB_NAME, false)).result;
  const db =
```

connect — 데이터베이스 열기.

1. 이미 열린 연결이 있고 상태가 정상이면 다시 쓰고, 아니면 새로 만든다 (암호화 없음, 버전 1).

```
    consistent && exists
      ? await sqlite.retrieveConnection(DB_NAME, false)
      : await sqlite.createConnection(DB_NAME, false, "no-encryption", 1, false);
  await db.open();
  await db.execute(SCHEMA);
  const info = await db.query("PRAGMA table_info(interventions)");
  const have = new Set(((info.values ?? []) as { name?: string }[]).map((row) => row.name));
  for (const [name, type] of INTERVENTION_COLUMNS) {
    if (!have.has(name)) await db.execute(`ALTER TABLE interventions ADD COLUMN ${name} ${type};
`);
  }
  // v0.8.1: card explanation
  const cardInfo = await db.query("PRAGMA table_info(cards)");
  const cardCols = new Set(((cardInfo.values ?? []) as { name?: string }[]).map((row) => row.nam
e));
  if (!cardCols.has("explanation")) {
    await db.execute("ALTER TABLE cards ADD COLUMN explanation TEXT NOT NULL DEFAULT '';");
  }
  return db;
}
```

2. 열고, 표를 만든다(있으면 그대로).
3. **123~127줄** 개입 표의 현재 칸 목록(`PRAGMA table_info`)을 읽어 없는 칸을 `ALTER TABLE ... ADD COLUMN`으로 추가한다.
4. **128~133줄** 카드 표에 해설 칸(v0.8.1)이 없으면 추가한다.

```
function database(): Promise<SQLiteDBConnection> {
  if (!opening) {
    opening = connect().catch((error) => {
      opening = null;
      throw error;
    });
  }
  return opening;
}
```

database: 연결을 한 번만 연다. 여는 중이면 같은 약속(Promise)을 돌려주고, 실패하면 다음에 다시 시도할 수 있게 지운다. `Promise`는 "나중에 결과가 올 약속", `async/await`는 그 결과를 기다리는 문법.

```
// --- Row mapping -----------------------------------------------------------

const bit = (value: boolean) => (value ? 1 : 0);

function sessionRow(log: UsageLog): Statement {
  return {
    statement:
      "INSERT OR REPLACE INTO sessions (id, target, source, date, started_at, ended_at, duration
_sec, clip_count) VALUES (?, ?, ?, ?, ?, ?, ?, ?)",
    values: [
      log.usageId,
      log.target,
      log.source,
      log.date,
      log.startTime,
      log.endTime,
      log.duration,
      log.clipCount,
    ],
```

bit: 참/거짓을 1/0으로 (SQLite에는 참/거짓 종류가 없어서).

sessionRow: 시청 기록 하나를 "넣기 또는 덮어쓰기" SQL로 만든다.

```
  };
}
```

```
function interventionRow(log: InterventionLog): Statement {
  return {
    statement: `INSERT OR REPLACE INTO interventions (id, created_at, date, hour, weekday, target, level, reason, reason_note,
      intervention_type, alternative_action, alternative_completed, outcome, re_entered, blocked, block_minutes,
      usage_seconds, reentered_within_30min, feedback, band, stage, framing, verification, reflection, after_lock,
      commit_minutes, card_id, card_correct, extension_index, policy_version)
      VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)`,
    values: [
      log.interventionId,
      log.timestamp,
      log.date,
      log.hour,
      log.weekday,
      log.target,
      log.level,
      log.reason,
      log.reasonNote,
      log.interventionType,
      log.alternativeAction,
      bit(log.alternativeCompleted),
      log.outcome,
      bit(log.reEntered),
      bit(log.blocked),
      log.blockDuration,
      log.usageSeconds,
      log.reenteredWithin30 === null ? null : bit(log.reenteredWithin30),
      log.feedback,
      log.band,
      log.stage,
      log.framing,
      log.verification,
      log.reflection,
      log.afterLock,
      log.commitMinutes,
      log.cardId,
      log.cardCorrect === null ? null : bit(log.cardCorrect),
      log.extensionIndex,
      log.policyVersion,
    ],
  };
}
```

interventionRow: 개입 기록을 30칸짜리 SQL로 만든다. 참/거짓은 1/0으로, 비어 있을 수 있는 값(30분 재진입, 카드 정답)은 null 그대로.

```
function cardRow(card: StudyCard): Statement {
  return {
```

- **cardRow**: 카드를 SQL로 (보기 목록은 JSON 글자로).

```
    statement:
      "INSERT OR REPLACE INTO cards (id, question, options, answer, created_at, source, shown, c
orrect, explanation) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)",
    values: [
      card.id,
      card.question,
      JSON.stringify(card.options),
      card.answer,
      card.createdAt,
      card.source,
      card.shown,
      card.correct,
      card.explanation,
    ],
  };
}

function lockRow(row: LockAttempt): Statement {
  return {
    statement:
      "INSERT OR REPLACE INTO lock_attempts (id, created_at, date, remaining_sec, target) VALUES
(?, ?, ?, ?, ?)",
    values: [row.id, row.createdAt, row.date, row.remainingSec, row.target],
  };
}

function settingRow(key: string, value: unknown): Statement {
  return {
    statement: "INSERT OR REPLACE INTO settings (key, value) VALUES (?, ?)",
    values: [key, JSON.stringify(value)],
  };
}
```

- **lockRow**: 잠금 시도를 SQL로.
- **settingRow**: 이름과 값(JSON 글자)을 settings 표에 넣는 SQL.

```
type Row = Record<string, unknown>;

function fromSessionRow(row: Row): UsageLog | null {
  return readUsageLog({
    usageId: row.id,
    target: row.target,
    source: row.source,
    date: row.date,
    startTime: row.started_at,
    endTime: row.ended_at,
    duration: Number(row.duration_sec),
    clipCount: Number(row.clip_count),
  });
}

function fromInterventionRow(row: Row): InterventionLog | null {
  const within = row.reentered_within_30min;
  return readIntervention({
```

표의 한 줄을 앱 데이터로 되돌리기. `fromSessionRow`, `fromInterventionRow`: 칸 이름(밑줄 이름, 예: `started_at`)을 앱 이름(예: `startTime`)으로 바꾸고, 1/0을 참/거짓으로, 숫자 글자를 숫자로 바꾼 뒤 storage.ts의 검사 함수로 읽는다(깨진 줄은 null).

```
    interventionId: row.id,
    timestamp: row.created_at,
    date: row.date,
    hour: Number(row.hour),
    weekday: Number(row.weekday),
    target: row.target,
    level: Number(row.level),
    reason: row.reason,
    reasonNote: row.reason_note,
    interventionType: row.intervention_type,
    alternativeAction: row.alternative_action ?? null,
    alternativeCompleted: Number(row.alternative_completed) === 1,
    outcome: row.outcome,
    reEntered: Number(row.re_entered) === 1,
    blocked: Number(row.blocked) === 1,
    blockDuration: Number(row.block_minutes),
    usageSeconds: Number(row.usage_seconds),
    reenteredWithin30: within === null || within === undefined ? null : Number(within) === 1,
    feedback: row.feedback ?? null,
    band: row.band === null || row.band === undefined ? null : Number(row.band),
    stage: row.stage ?? null,
    framing: row.framing ?? null,
    verification: row.verification ?? null,
    reflection: row.reflection ?? null,
    afterLock: row.after_lock ?? null,
    commitMinutes: Number(row.commit_minutes ?? 0),
    cardId: row.card_id ?? null,
    cardCorrect:
      row.card_correct === null || row.card_correct === undefined ? null : Number(row.card_corre
ct) === 1,
    extensionIndex: Number(row.extension_index ?? 0),
    policyVersion: row.policy_version ?? null,
  });
}
```

```
function fromCardRow(row: Row): StudyCard | null {
  return readCard({
    id: row.id,
    question: row.question,
    options: parse(row.options),
    answer: Number(row.answer),
    createdAt: row.created_at,
    source: row.source,
    shown: Number(row.shown),
    correct: Number(row.correct),
    explanation: row.explanation ?? "",
  });
}

function fromLockRow(row: Row): LockAttempt | null {
  return readLockAttempt({
```

fromCardRow / fromLockRow: 카드(보기 JSON 글자를 목록으로)와 잠금 시도 되돌리기.

parse: JSON 글자를 읽고, 실패하면 null.

```
    id: row.id,
    createdAt: row.created_at,
    date: row.date,
    remainingSec: Number(row.remaining_sec),
    target: row.target,
  });
}

function parse(value: unknown): unknown {
  if (typeof value !== "string") return null;
  try {
    return JSON.parse(value);
  } catch {
    return null;
  }
}
```

```
// --- Public API ----------------------------------------------------------

/** 저장소 변경 한 건. 화면 상태를 바꿀 때 함께 만든다. */
export type Op =
  | { kind: "session"; row: UsageLog }
  | { kind: "intervention"; row: InterventionLog }
  | { kind: "lock"; row: LockAttempt }
  | { kind: "card"; row: StudyCard }
  | { kind: "deleteCard"; id: string }
  | { kind: "settings"; settings: Settings }
  | { kind: "block"; block: AppData["block"] }
  | { kind: "activeSession"; session: AppData["activeSession"] }
  | { kind: "lastNotifiedDate"; date: string | null }
  | { kind: "clear" };
```

Op — 저장 작업 한 건의 모양. 시청, 개입, 잠금 시도, 카드 넣기, 카드 지우기, 설정, 잠금 상태, 연습 시청, 마지막 알림 날짜, 전체 초기화. 화면이 데이터를 바꿀 때 이 작업 목록을 같이 만들어 저장한다.

```
function statementsFor(op: Op): Statement[] {
  switch (op.kind) {
    case "session":
      return [sessionRow(op.row)];
    case "intervention":
      return [interventionRow(op.row)];
    case "lock":
      return [lockRow(op.row)];
    case "card":
      return [cardRow(op.row)];
    case "deleteCard":
      return [{ statement: "DELETE FROM cards WHERE id = ?", values: [op.id] }];
    case "settings":
      return [settingRow("settings", op.settings)];
    case "block":
      return [settingRow("block", op.block)];
    case "activeSession":
      return [settingRow("activeSession", op.session)];
    case "lastNotifiedDate":
```

statementsFor: 작업 한 건을 SQL 문장으로 바꾼다. **초기화(clear)**는 시청·개입·잠금 시도를 모두 지우고, 카드는 남기되 나온 횟수와 맞힌 횟수만 0으로, 설정 표는 "옮김 완료" 표시만 남기고 지운다.

```
      return [settingRow("lastNotifiedDate", op.date)];
    case "clear":
      return [
        { statement: "DELETE FROM sessions", values: [] },
        { statement: "DELETE FROM interventions", values: [] },
        { statement: "DELETE FROM lock_attempts", values: [] },
        { statement: "UPDATE cards SET shown = 0, correct = 0", values: [] },
        { statement: "DELETE FROM settings WHERE key != 'migrated'", values: [] },
      ];
  }
}
```

```
export async function applyOps(ops: Op[]): Promise<void> {
  const set = ops.flatMap(statementsFor);
  if (set.length === 0) return;
  const db = await database();
  await db.executeSet(set as { statement: string; values: unknown[] }[], true);
}
```

applyOps — 저장하기. 작업들을 SQL로 바꿔 **한 번에(트랜잭션)** 실행한다. 중간에 실패하면 전부 취소되어 반만 저장되는 일이 없다.

```
/** 모든 기록을 읽는다. 형식이 틀린 줄은 건너뛴다. */
export async function loadAll(): Promise<AppData> {
  const db = await database();
  const [sessions, interventions, locks, cards, settings] = await Promise.all([
    db.query("SELECT * FROM sessions ORDER BY started_at"),
    db.query("SELECT * FROM interventions ORDER BY created_at"),
    db.query("SELECT * FROM lock_attempts ORDER BY created_at"),
    db.query("SELECT * FROM cards ORDER BY created_at"),
    db.query("SELECT key, value FROM settings"),
  ]);
  const meta = new Map<string, unknown>();
  for (const row of (settings.values ?? []) as Row[]) {
    meta.set(String(row.key), parse(row.value));
  }
  const keep = <T,>(rows: unknown[] | undefined, read: (row: Row) => T | null) =>
    ((rows ?? []) as Row[]).map(read).filter((row): row is T => row !== null);
  const base = defaultData();
  return {
    settings: meta.has("settings") ? readSettings(meta.get("settings")) : base.settings,
    usageLogs: keep(sessions.values, fromSessionRow),
    interventions: keep(interventions.values, fromInterventionRow),
    lockAttempts: keep(locks.values, fromLockRow),
    cards: keep(cards.values, fromCardRow),
    activeSession: readSession(meta.get("activeSession")),
    block: readBlock(meta.get("block")),
    lastNotifiedDate:
      typeof meta.get("lastNotifiedDate") === "string"
        ? (meta.get("lastNotifiedDate") as string)
        : null,
  };
}
```

loadAll — 전체 불러오기. 다섯 표를 동시에 읽는다(`Promise.all`). settings 표는 이름→값 표로 만들고, 각 표의 줄을 앱 데이터로 되돌리며 깨진 줄은 건너뛴다. 설정이 없으면 기본 설정.

```ts
/**
 * 첫 실행 때 localStorage 기록을 SQLite로 옮긴다. 옮긴 뒤 원본은
 * 다른 키로 남겨 두고(되돌릴 수 있게) 다시 옮기지 않는다.
 */
export async function migrateFromLocalStorage(): Promise<number> {
  const db = await database();
  const done = await db.query("SELECT value FROM settings WHERE key = 'migrated'");
  if ((done.values ?? []).length > 0) return 0;
  const { data } = readLocalStorage();
  const ops: Op[] = [];
  if (data) {
    ops.push({ kind: "settings", settings: data.settings });
    ops.push({ kind: "block", block: data.block });
    ops.push({ kind: "lastNotifiedDate", date: data.lastNotifiedDate });
    for (const row of data.usageLogs) ops.push({ kind: "session", row });
    for (const row of data.interventions) ops.push({ kind: "intervention", row });
    for (const row of data.lockAttempts) ops.push({ kind: "lock", row });
    for (const row of data.cards) ops.push({ kind: "card", row });
  }
  const set = [...ops.flatMap(statementsFor), settingRow("migrated", new Date().toISOString())];
  await db.executeSet(set as { statement: string; values: unknown[] }[], true);
  if (data) {
    const raw = localStorage.getItem(STORAGE_KEY);
    if (raw) localStorage.setItem(MIGRATED_KEY, raw);
    localStorage.removeItem(STORAGE_KEY);
  }
  return data ? data.usageLogs.length + data.interventions.length : 0;
}
```

migrateFromLocalStorage — 첫 실행 때 옮기기. 예전(브라우저 저장소)에 기록이 있으면 SQLite로 옮긴다.

1. 이미 옮겼으면("migrated" 표시) 아무것도 안 한다.
2. 브라우저 저장소를 읽어 모든 기록을 저장 작업으로 만들고, "옮김 완료" 표시와 함께 한 번에 저장한다.
3. 원본은 다른 이름으로 남겨 두고(되돌릴 수 있게) 원래 키는 지운다.
4. 옮긴 기록 수를 돌려준다.

persist.ts — 저장 방식 고르기 `lib/persist.ts (81줄)`

폰(안드로이드)이면 SQLite, 브라우저면 localStorage로 저장하도록 **저장 방식을 하나로 감싼다**. 화면 코드는 어느 쪽인지 몰라도 같은 방식으로 저장할 수 있다.

```ts
import type { Op } from "./db";
import { isNativeAndroid } from "./guard";
import { finalizeSession } from "./logic";
import { defaultData, loadData, saveData } from "./storage";
import type { AppData } from "./types";

export type { Op } from "./db";

/** 기록 저장소. 안드로이드는 SQLite, 브라우저는 localStorage. */
export type Backend = {
  kind: "sqlite" | "local";
  load(): Promise<{ data: AppData; corrupt: boolean; migrated: number }>;
  /** next = 바뀐 뒤 전체 상태, ops = SQLite에 쓸 변경. 순서대로 저장한다. */
```

1~7줄 가져오기. `import type`은 모양만, `export type ... from`은 다른 파일의 모양을 그대로 다시 공개.

9~15줄 Backend — 저장소의 약속: 종류("sqlite"/"local"), 불러오기(`load`: 데이터, 손상 여부, 옮긴 수), 저장하기(`apply`: 바뀐 전체 상태와 SQL 작업 목록).

```
  apply(next: AppData, ops: Op[]): Promise<void>;
};
```

```
const localBackend: Backend = {
  kind: "local",
  async load() {
    const loaded = loadData();
    return { ...loaded, migrated: 0 };
  },
  async apply(next) {
    saveData(next);
  },
};
```

localBackend (브라우저용): 불러오기는 storage.ts의 `loadData`, 저장은 **전체를** 브라우저 저장소에 다시 쓴다.

```
function sqliteBackend(): Backend {
  let chain: Promise<void> = Promise.resolve();
  return {
    kind: "sqlite",
    async load() {
      const db = await import("./db");
      const migrated = await db.migrateFromLocalStorage();
      const loaded = await db.loadAll();
      // 연습 피드가 열린 채 앱이 닫혔으면 그 세션을 마감한다.
      const data = finalizeSession(loaded);
      if (loaded.activeSession) {
        const closed = data.usageLogs.find((log) => log.usageId === loaded.activeSession?.usageId);
        await db.applyOps([
          ...(closed ? [{ kind: "session" as const, row: closed }] : []),
          { kind: "activeSession", session: null },
        ]);
      }
      return { data, corrupt: false, migrated };
    },
```

sqliteBackend (폰용) — 불러오기. db.ts를 필요할 때만 불러오고(`await import`, 브라우저에서는 SQLite 코드를 안 불러오게), 예전 기록을 옮긴 뒤 전부 읽는다. 연습 화면이 열린 채 앱이 꺼졌으면 그 시청을 마감하고 저장한다.

```
    apply(_next, ops) {
      const run = chain.then(async () => {
        const db = await import("./db");
        await db.applyOps(ops);
      });
      // 한 번 실패해도 다음 저장은 이어서 시도한다.
      chain = run.catch(() => undefined);
      return run;
    },
  };
}
```

sqliteBackend — 저장하기. 저장 작업을 **줄 세워 순서대로**(`chain`) 실행한다. 앞 저장이 끝나야 다음 저장을 해서 순서가 섞이지 않는다. 한 번 실패해도 다음 저장은 계속 시도한다.

```
let chosen: Backend | null = null;

export function backend(): Backend {
  if (!chosen) chosen = isNativeAndroid() ? sqliteBackend() : localBackend;
```

backend: 처음 부를 때 폰이면 SQLite, 아니면 브라우저 방식을 골라 기억한다.

```
  return chosen;
}
```

```
/** SQLite를 열지 못하면 localStorage로 돌아간다. 데이터는 잃지 않지만 안내한다. */
export async function loadWithFallback(): Promise<{
  data: AppData;
  corrupt: boolean;
  migrated: number;
  fallback: boolean;
}> {
  const first = backend();
  try {
    return { ...(await first.load()), fallback: false };
  } catch {
    if (first.kind === "local") return { data: defaultData(), corrupt: true, migrated: 0, fallback: false };
    chosen = localBackend;
    return { ...(await localBackend.load()), fallback: true };
  }
}
```

loadWithFallback — 불러오기 (안전장치 포함). SQLite를 열지 못하면 브라우저 저장소로 돌아가고 "대신 저장 중" 표시(`fallback`)를 한다. 화면은 이 표시를 보고 안내 문구를 보여 준다. 브라우저 방식마저 실패하면 빈 데이터 + 손상 표시.

notify.ts — 알림 (리포트, 위험 시간) `lib/notify.ts (160줄)`

서버 없이 **폰이 직접 보내는 알림**(로컬 알림)을 다룬다. 매일 리포트 알림과 AI 위험 시간 알림, 알림 버튼을 눌렀을 때의 처리.

```
import { LocalNotifications } from "@capacitor/local-notifications";
import { isNativeAndroid } from "./guard";
import type { Settings } from "./types";

/** 매일 리포트 알림. 앱이 꺼져 있어도 OS가 보낸다. 통계는 알림을 눌러 앱이 열릴 때 계산한다. */
const REPORT_ID = 2101;
const PREVIEW_ID = 2102;
const RISK_ID = 2103;
const RISK_ACTIONS = "risk-lock";
export const REPORT_ROUTE = "/report";
/** 위험 시간 알림의 버튼으로 잠그는 시간 */
export const RISK_LOCK_MINUTES = 60;

export type NotifyPermission = "granted" | "denied" | "default" | "unsupported";

function mapPermission(state: string): NotifyPermission {
  if (state === "granted") return "granted";
  if (state === "denied") return "denied";
  return "default";
}
```

6~9줄 알림 번호: 리포트 2101, 미리 보기 2102, 위험 시간 2103, 위험 알림 버튼 묶음 이름. **10줄** 알림을 누르면 갈 주소 /report. **12줄** [1시간 잠그기]의 60분.

14~20줄 알림 권한 상태 모양(허용/거부/아직/지원 안 함)과 플러그인 값을 이 모양으로 바꾸는 함수.

```
export async function checkNotifyPermission(): Promise<NotifyPermission> {
  if (!isNativeAndroid()) {
```

checkNotifyPermission: 알림 권한 확인. 브라우저면 브라우저 알림 권한, 폰이면 플러그인으로.

```
    return typeof Notification === "undefined" ? "unsupported" : Notification.permission;
  }
  try {
    return mapPermission((await LocalNotifications.checkPermissions()).display);
  } catch {
    return "unsupported";
  }
}

export async function requestNotifyPermission(): Promise<NotifyPermission> {
  if (!isNativeAndroid()) {
    if (typeof Notification === "undefined") return "unsupported";
    return Notification.requestPermission();
  }
  try {
    return mapPermission((await LocalNotifications.requestPermissions()).display);
  } catch {
    return "unsupported";
  }
}
```

requestNotifyPermission: 알림 권한 요청 (안드로이드 13 이상은 사용자에게 묻는 창이 뜬다).

```ts
/**
 * 설정 시각에 매일 반복 알림을 다시 예약한다. 끄면 예약을 지운다.
 * @returns 안내 문구, 문제가 없으면 null
 */
export async function scheduleDailyReport(settings: Settings): Promise<string | null> {
  if (!isNativeAndroid()) return null;
  try {
    await LocalNotifications.cancel({ notifications: [{ id: REPORT_ID }] });
    if (!settings.notifyEnabled) return null;
    if ((await checkNotifyPermission()) !== "granted") {
      return "알림 권한이 없어 리포트 알림을 예약하지 못했습니다.";
    }
    const [hour, minute] = settings.notificationTime.split(":").map(Number);
    await LocalNotifications.schedule({
      notifications: [
        {
          id: REPORT_ID,
          title: "오늘 리포트가 준비됐어요",
          body: "눌러서 오늘 쇼츠 사용을 확인해 보세요.",
          schedule: { on: { hour, minute }, allowWhileIdle: true },
          smallIcon: "ic_stat_shortsai",
          extra: { route: REPORT_ROUTE },
        },
      ],
    });
    return null;
  } catch {
    return "리포트 알림을 예약하지 못했습니다.";
  }
}
```

scheduleDailyReport — 매일 리포트 알림 예약. 기존 예약을 지우고, 꺼져 있거나 권한이 없으면 안내 문장을 돌려준다. 아니면 설정 시각(예: 21:00)에 **매일 반복**(`on: { hour, minute }`) 알림을 예약한다. 폰이 절전 중이어도 보내고(`allowWhileIdle`), 누르면 리포트 화면으로.

```ts
/** 바로 한 번 보내 본다. */
export async function previewReport(): Promise<string | null> {
  if (!isNativeAndroid()) return "unsupported";
  if ((await checkNotifyPermission()) !== "granted") return "알림 권한이 없습니다. 먼저 허용해 주세요.";
  try {
    await LocalNotifications.schedule({
      notifications: [
        {
          id: PREVIEW_ID,
          title: "오늘 리포트가 준비됐어요",
          body: "눌러서 오늘 쇼츠 사용을 확인해 보세요.",
          schedule: { at: new Date(Date.now() + 1500), allowWhileIdle: true },
          smallIcon: "ic_stat_shortsai",
          extra: { route: REPORT_ROUTE },
        },
      ],
    });
    return null;
```

previewReport: 설정 화면의 "알림 미리 보기" — 1.5초 뒤 한 번 보내 본다.

```
  } catch {
    return "알림을 보내지 못했습니다.";
  }
}
```

```
/**
 * AI: 평소 오래 보던 시간 10분 전에 매일 알림. [1시간 잠그기]를 누르면 잠근다.
 * 위험 시간이 없거나 AI가 꺼져 있으면 예약을 지운다. 문구는 사용자 확인 완료 (docs/HANDOFF.md).
 */
export async function scheduleRiskNotice(firstRiskHour: number | null, label: string): Promise<void> {
  if (!isNativeAndroid()) return;
  try {
    await LocalNotifications.cancel({ notifications: [{ id: RISK_ID }] });
    if (firstRiskHour === null || (await checkNotifyPermission()) !== "granted") return;
    await LocalNotifications.registerActionTypes({
      types: [
        {
          id: RISK_ACTIONS,
          actions: [
            { id: "lock", title: `${RISK_LOCK_MINUTES / 60}시간 잠그기` },
            { id: "skip", title: "괜찮아요" },
          ],
        },
      ],
    });
    const hour = (firstRiskHour + 23) % 24;
    await LocalNotifications.schedule({
      notifications: [
        {
          id: RISK_ID,
          title: "곧 평소 오래 보던 시간이에요",
          body: `오늘은 ${label}부터 쇼츠를 잠가 둘까요?`,
          schedule: { on: { hour, minute: 50 }, allowWhileIdle: true },
          smallIcon: "ic_stat_shortsai",
          actionTypeId: RISK_ACTIONS,
          extra: { kind: "risk" },
        },
      ],
    });
  } catch {
    // 알림을 못 걸어도 시트 개입은 그대로 동작한다.
  }
}
```

scheduleRiskNotice — AI 위험 시간 알림.

1. 기존 예약을 지우고, 위험 시간이 없거나 권한이 없으면 끝.
2. **108~118줄** 알림 버튼 2개를 등록: [1시간 잠그기], [괜찮아요].
3. **119줄** 위험 시간 바로 전 시 (예: 23시 → 22시). 0시면 23시.
4. **120~132줄** 그 시 50분(= 10분 전)에 매일 알림 "곧 평소 오래 보던 시간이에요 / 오늘은 밤 11시부터 쇼츠를 잠가 둘까요?".
5. 실패해도 개입 화면은 그대로 동작하므로 조용히 넘어간다.

```
/**
 * 알림을 눌렀을 때. 리포트 알림은 리포트로, 위험 시간 알림의 [잠그기]는 잠금.
 * 반환값으로 구독을 끊는다.
 */
export function onNotificationAction(handlers: {
  open: (route: string) => void;
```

onNotificationAction — 알림을 눌렀을 때. 위험 시간 알림이면 [1시간 잠그기]는 60분 잠금, 그 밖에는 홈으로. 리포트 알림이면 리포트 화면으로. 돌려주는 함수를 부르면 알림 듣기를 멈춘다(화면이 사라질 때 정리용).

```
  lock: (minutes: number) => void;
}): () => void {
  if (!isNativeAndroid()) return () => undefined;
  const handle = LocalNotifications.addListener("localNotificationActionPerformed", (action) =>
{
    const extra = action.notification.extra ?? {};
    if (extra.kind === "risk") {
      if (action.actionId === "lock") handlers.lock(RISK_LOCK_MINUTES);
      else handlers.open("/");
      return;
    }
    const route = extra.route;
    handlers.open(typeof route === "string" ? route : REPORT_ROUTE);
  });
  return () => {
    void handle.then((h) => h.remove());
  };
}
```

guard.ts — 플러그인 연결 정의 `lib/guard.ts (68줄)`

웹에서 Java 플러그인 `HanbakjaGuard`를 부를 때 쓰는 **함수 목록과 주고받는 값의 모양**. Java의 GuardPlugin.java와 짝을 이룬다.

```
import { Capacitor, registerPlugin } from "@capacitor/core";
import type { Policy } from "./policy";
import type { NativeBatch } from "./types";

export type GuardTarget = "youtube" | "instagram";
export type GuardOutcome = "watch" | "leave" | "block";

export type GuardStatus = {
  enabled: boolean;
  /** 웹 흐름이 이어받을 개입 (Level 3, Instagram). 없으면 "". */
  pendingTarget: string;
  /** 서비스가 감지 순간 정한 Level과 오늘 누적 초. 0이면 없음. */
  pendingLevel: number;
  pendingUsageSeconds: number;
  watching: boolean;
  blockerEnabled: boolean;
  /** 시간 차단이 끝나는 시각(epoch ms). 0이면 없음. */
  blockedUntil: number;
  blockMinutes: number;
  appVersion: string;
};
```

5~6줄 대상(youtube/instagram)과 앱 화면 개입 결과(보기/나가기/잠금).

GuardStatus: getStatus의 결과 모양 — 접근성 켜짐, 앱 화면이 이어받을 개입(대상, 단계, 사용 초), 보는 중, 개입 스위치, 잠금 끝 시각과 분, 앱 버전.

```
export type GuardSyncInput = {
  level1Threshold: number;
  level3Threshold: number;
```

GuardSyncInput: sync 때 보내는 값 — 단계 기준(분), 웹이 잰 날짜와 초, **정책표**, **공부 카드**.

```
  /** 웹이 직접 잰 오늘 초(연습 피드 등). 네이티브 세션은 서비스가 센다. */
  webDate: string;
  webSeconds: number;
  /** 개입 정책표 (lib/policy.ts). 서비스는 이 표만 읽고 결정한다. */
  policy: Policy;
  /** 시트에 낼 공부 카드 */
  cards: { id: string; question: string; options: string[]; answer: number; explanation: string
}[];
};
```

```
export interface HanbakjaGuardPlugin {
  getStatus(): Promise<GuardStatus>;
  setBlocker(options: { enabled: boolean }): Promise<void>;
  /** 설정을 넘기고, 서비스가 쌓아 둔 세션·개입·잠금 시도를 받는다. */
  sync(options: GuardSyncInput): Promise<NativeBatch>;
  /** SQLite에 저장한 줄을 서비스 대기열에서 지운다. */
  ack(options: { ids: string[] }): Promise<void>;
  finish(options: {
    outcome: GuardOutcome;
    target: GuardTarget;
    blockMinutes?: number;
    /** "오늘은 끝": 쉰 시간에 넣지 않고, 잠금 뒤 질문도 하지 않는다. */
    dayEnd?: boolean;
  }): Promise<void>;
  startBlock(options: { minutes: number }): Promise<{ blockedUntil: number }>;
  resetRecords(): Promise<void>;
  openSettings(): Promise<void>;
}
```

HanbakjaGuardPlugin — 플러그인 함수 목록 (Java와 이름이 같아야 한다): getStatus, setBlocker, sync(기록 받기), ack(저장 확인), finish(앱 화면 개입 결과, 오늘은 끝 여부), startBlock, resetRecords, openSettings. `interface`는 "이런 함수들이 있다"는 약속.

```
export const HanbakjaGuard = registerPlugin<HanbakjaGuardPlugin>("HanbakjaGuard", {
  web: () => import("./guard-web").then((mod) => new mod.HanbakjaGuardWeb()),
});

export function isNativeAndroid(): boolean {
  return Capacitor.isNativePlatform() && Capacitor.getPlatform() === "android";
}

export function isGuardTarget(value: string): value is GuardTarget {
  return value === "youtube" || value === "instagram";
}

export function guardSourceLabel(target: GuardTarget): string {
  return target === "instagram" ? "Instagram Reels" : "YouTube Shorts";
}
```

54~56줄 플러그인 등록. 폰이면 Java를, **브라우저면** guard-web.ts의 가짜 버전을 쓴다.

- **isNativeAndroid**: 지금 안드로이드 앱 안에서 돌고 있는지 (브라우저면 false).
- **isGuardTarget**: 값이 youtube/instagram인지.
- **guardSourceLabel**: 화면에 쓸 이름 "YouTube Shorts"/"Instagram Reels".

guard-web.ts — 브라우저용 가짜 플러그인 `lib/guard-web.ts (46줄)`

브라우저(미리 보기, 개발 중 확인)에는 접근성 서비스가 없으므로, 같은 함수들을 **아무 일도 안 하는 버전**으로 만든다. 개입 스위치 상태만 브라우저 저장소에 기억한다.

```
import { WebPlugin } from "@capacitor/core";
import type { HanbakjaGuardPlugin } from "./guard";

const BLOCKER_KEY = "hanbakja.blocker";

function readBlocker(): boolean {
  if (typeof localStorage === "undefined") return true;
  return localStorage.getItem(BLOCKER_KEY) !== "0";
}

/** 브라우저에는 접근성 서비스가 없다. 개입 스위치만 기억한다. */
```

4줄 스위치 저장 키. **6~9줄** 스위치 읽기 (저장된 값이 "0"이 아니면 켜짐).

```
export class HanbakjaGuardWeb extends WebPlugin implements HanbakjaGuardPlugin {
  async getStatus() {
    return {
      enabled: false,
      pendingTarget: "",
      pendingLevel: 0,
      pendingUsageSeconds: 0,
      watching: false,
      blockerEnabled: readBlocker(),
      blockedUntil: 0,
      blockMinutes: 0,
      appVersion: "",
    };
  }

  async setBlocker(options: { enabled: boolean }) {
    localStorage.setItem(BLOCKER_KEY, options.enabled ? "1" : "0");
  }

  async sync() {
    return { sessions: [], interventions: [], lockAttempts: [] };
  }

  async ack() {}

  async finish() {}

  async startBlock() {
    return { blockedUntil: 0 };
  }

  async resetRecords() {}

```

HanbakjaGuardWeb: getStatus는 "접근성 꺼짐, 보는 중 아님, 스위치 상태"를 돌려주고, setBlocker는 스위치를 저장한다. 나머지(sync, ack, finish, startBlock 등)는 빈 결과를 돌려준다. `implements`는 "위 약속(interface)을 지킨다"는 뜻.

```
  async openSettings() {}
}
```

app-info.ts — 앱 정보 `lib/app-info.ts (8줄)`

```
/** package.json "version". APK versionName과 같다. */
export const APP_VERSION = process.env.NEXT_PUBLIC_APP_VERSION ?? "";

/**
 * APK를 올린 GitHub Releases 주소. 비워 두면 설정 화면에 링크 없이 안내만 보인다.
 * 예: "https://github.com/<아이디>/ShortsAI/releases/latest"
 */
export const RELEASES_URL = "";
```

APP_VERSION: 빌드할 때 package.json의 버전이 들어온다 (next.config.ts가 넣어 줌). 설정 화면 "버전 0.9.2"에 쓰인다.

RELEASES_URL: APK를 받을 GitHub Releases 주소. **지금은 비어 있어서** 설정 화면에 링크 없이 안내만 보인다. 정식 배포 때 채우면 된다.

clips.ts — 연습 화면 문구 `lib/clips.ts (67줄)`

```
export type Clip = {
  id: string;
  title: string;
  caption: string;
  from: string;
  to: string;
};

```

Clip: 브라우저 연습 화면에서 쓰는 가짜 영상 한 장의 모양 — id, 제목, 설명 문장, 배경 그라데이션 색 두 개.

```
/** 실제 영상 대신 쓰는 원본 문구. YouTube 콘텐츠가 아니다. */
export const CLIPS: Clip[] = [
  {
    id: "signal",
    title: "신호등 앞에서",
    caption: "파란불이 오기 전에, 엄지가 한 번 더 올라간다.",
    from: "#3d4a3c",
    to: "#171512",
  },
  {
    id: "kettle",
    title: "물 끓는 소리",
    caption: "주전자가 울릴 때까지, 화면은 이미 세 개를 넘겼다.",
    from: "#6a5344",
    to: "#241910",
  },
  {
    id: "window",
    title: "창가의 2초",
    caption: "버스가 멈춘 사이, 밖은 그대로고 피드만 바뀐다.",
    from: "#3e5360",
    to: "#14181c",
```

CLIPS — 가짜 영상 8장. 실제 YouTube 영상이 아니라 직접 쓴 짧은 문장이다 ("신호등 앞에서", "불 끄기 전" 등). 접근성 서비스가 없는 브라우저에서 개입 흐름을 체험해 보는 데 쓴다.

```
  },
  {
    id: "desk",
    title: "덮어 둔 노트",
    caption: "한 줄만 쓰려고 앉았는데, 쇼츠가 먼저 열렸다.",
    from: "#5c4638",
    to: "#1c1612",
  },
  {
    id: "night",
    title: "불 끄기 전",
    caption: "다섯 개만 보자고 한 게, 열두 개째다.",
    from: "#3a3450",
    to: "#14131a",
  },
  {
    id: "bowl",
    title: "식기 전의 한 입",
    caption: "숟가락을 든 손이, 어느새 폰을 집는다.",
    from: "#6b4a32",
    to: "#22180f",
  },
  {
    id: "rain",
    title: "유리창의 비",
    caption: "밖이 젖는 동안, 눈은 다음 영상만 찾는다.",
    from: "#2f4652",
    to: "#101418",
  },
  {
    id: "shoes",
    title: "현관의 신발",
    caption: "나가려다 말고, 신발을 신은 채로 스크롤한다.",
    from: "#4a4036",
    to: "#181410",
  },
];
```

utils.ts — 화면 도우미 `lib/utils.ts (1줄)`

```
export { cn } from "cn"
```

`cn`이라는 작은 도구를 그대로 내보낸다. 화면 부품의 `className`(모양 이름)을 조건에 따라 합칠 때 쓴다. 예: `cn("rounded", 선택됨 && "bg-primary")`.

5부. 화면 부품 (components/, React + TSX)

`components/` 폴더는 사용자가 보는 **화면**을 만드는 코드다. React라는 도구로, 화면을 작은 부품(컴포넌트)으로 나눠 만들고 조립한다. 파일 확장자 `.tsx`는 "TypeScript + 화면 태그(HTML 비슷한 것)"라는 뜻이다.

React 읽는 법:

- `function Home() { return <div>...</div> }` = 화면 부품 하나. 돌려주는 태그가 화면에 그려진다.
- `<Button onClick={f}>` = 부품에 값(props)을 넘긴다. 중괄호 `{}` 안은 TypeScript 코드.
- `const [x, setX] = useState(처음값)` = 부품이 기억하는 값. `setX`로 바꾸면 화면이 다시 그려진다.
- `useEffect(() => {...}, [a, b])` = 화면이 뜬 뒤, 또는 a·b가 바뀔 때 실행할 일 (타이머, 저장소 읽기 등). 돌려주는 함수는 "정리할 일".
- `useRef` = 바뀌어도 화면을 다시 그리지 않는 기억 칸. `useCallback`, `useMemo` = 함수나 계산 결과를 재사용해서 쓸데없는 다시 그리기를 줄인다.
- `{조건 && <태그/>}` = 조건이 참일 때만 보여 주기. `{목록.map(x => <li key={x.id}>...)}` = 목록을 여러 줄로.
- `className="rounded bg-primary p-4"` = Tailwind 모양 이름 (둥근 모서리, 주 색 배경, 안쪽 여백).
- `"use client"` = 이 파일은 브라우저(폰 화면)에서 실행된다는 표시.

store.tsx — 앱 전체 기록 창고 `components/store.tsx (585줄)`

앱의 모든 기록(설정, 시청, 개입, 잠금, 카드)을 **한 곳에서 들고 있고**, 바꿀 때마다 저장소(SQLite나 브라우저)에 저장하는 부품. 다른 화면들은 `useStore()`로 여기서 데이터와 기능을 꺼내 쓴다. React의 "Context(공유 상자)" 방식이다.

```
1  "use client";
2
3  import {
4    createContext,
5    useCallback,
6    useContext,
7    useEffect,
8    useMemo,
9    useRef,
10   useState,
11 } from "react";
12 import { HanbakjaGuard, isNativeAndroid } from "@/lib/guard";
13 import {
14   applyFlush,
15   blockUntilFromMinutes,
16   buildIntervention,
17   convertNativeBatch,
```

1줄 브라우저에서 실행되는 파일. **3~11줄** React 기능 가져오기. **12~43줄** 플러그인(guard), 계산(logic), 알림(notify), 저장(persist, storage), 데이터 모양(types)을 가져온다.

```
  finalizeSession,
  isReportDue,
  reportLines,
  settleReentry,
  statsForDate,
  todayKey,
  withClip,
} from "@/lib/logic";
import {
  checkNotifyPermission,
  previewReport,
  requestNotifyPermission,
  scheduleDailyReport,
  type NotifyPermission,
} from "@/lib/notify";
import { backend, loadWithFallback, type Op } from "@/lib/persist";
import { clearData, defaultData } from "@/lib/storage";
import type {
  AppData,
  InterventionInput,
  Level,
  NativeBatch,
  SessionTarget,
  Settings,
  StudyCard,
} from "@/lib/types";

```

```
const SAVE_ERROR = "기록을 저장하지 못했습니다. 저장 공간이 가득 찼을 수 있습니다.";
const FALLBACK_ERROR =
  "기기 데이터베이스를 열지 못해 임시 저장소에 기록합니다. 앱을 다시 열어 주세요.";

```

화면에 보여 줄 오류 문구 2개: 저장 실패, 기기 DB를 못 열어 임시 저장소를 쓰는 중.

```
type StoreContextValue = {
  ready: boolean;
  corrupt: boolean;
  saveError: string | null;
  data: AppData | null;
  storageKind: "sqlite" | "local";
  permission: NotifyPermission;
  /** 예약 알림을 걸지 못했을 때 안내. */
  notifyIssue: string | null;
  getData: () => AppData | null;
  reload: () => void;
  reset: () => Promise<void>;
  dismissSaveError: () => void;
  saveSettings: (settings: Settings) => void;
  recordIntervention: (input: InterventionInput) => string;
  recordLockAttempt: (target: SessionTarget | null, remainingSec: number) => void;
  startWatching: (level: Level, interventionId: string) => void;
  flushWatch: (visible: boolean) => void;
```

StoreContextValue — 창고가 내주는 것 목록.

- **값**: 준비됨(ready), 손상(corrupt), 저장 오류, 데이터, 저장 방식, 알림 권한, 알림 예약 문제.
- **기능**: 다시 불러오기, 초기화, 설정 저장, 개입 기록, 잠금 중 열기 기록, 연습 시청 시작/갱신/다음 영상/끝, 잠금 시작, 서비스 잠금 따라가기, 서비스 기록 합치기(mergeNative), 리포트 봄 표시, 카드 추가/삭제, 알림 권한 요청, 알림 미리 보기 등.

```
  advanceClip: () => void;
  stopWatching: () => void;
  beginBlock: (minutes: number) => void;
  /** 서비스 쪽 차단이 더 길면 화면에도 반영한다. */
  adoptBlock: (untilMs: number, minutes: number) => void;
  /** 서비스 기록을 저장하고, 저장한(또는 이미 있던) id를 돌려준다. 그 뒤에 ack 한다. */
  mergeNative: (batch: NativeBatch) => Promise<string[]>;
  markReportSeen: () => void;
  addCards: (cards: StudyCard[]) => void;
  deleteCard: (id: string) => void;
  cancelDetach: () => void;
  scheduleDetach: () => void;
  requestNotificationPermission: () => Promise<string | null>;
  previewNotification: () => Promise<string | null>;
};
```

```
const StoreContext = createContext<StoreContextValue | null>(null);

type Change = { next: AppData; ops: Op[] };

/** 연습 피드 세션 한 줄과 열린 세션 상태를 저장하는 변경. */
function practiceOps(next: AppData, usageId: string | undefined): Op[] {
  const row = usageId ? next.usageLogs.find((log) => log.usageId === usageId) : undefined;
  return [
    ...(row ? [{ kind: "session" as const, row }] : []),
    { kind: "activeSession" as const, session: next.activeSession },
  ];
}
```

83줄 공유 상자를 만든다 (처음엔 비어 있음).

85줄 Change: "바뀐 뒤 전체 데이터 + 저장소에 보낼 작업 목록" 한 쌍.

practiceOps: 연습 시청 한 줄(있으면)과 열린 시청 상태를 저장하는 작업 목록을 만든다. `as const`는 "글자 그대로의 종류"로 고정하는 표시.

```
export function StoreProvider({ children }: { children: React.ReactNode }) {
  const [ready, setReady] = useState(false);
  const [corrupt, setCorrupt] = useState(false);
  const [saveError, setSaveError] = useState<string | null>(null);
  const [data, setData] = useState<AppData | null>(null);
  const [storageKind, setStorageKind] = useState<"sqlite" | "local">("local");
  const [permission, setPermission] = useState<NotifyPermission>("default");
  const [notifyIssue, setNotifyIssue] = useState<string | null>(null);
  const dataRef = useRef<AppData | null>(null);
  const detachRef = useRef<number | null>(null);
```

StoreProvider 시작 — 앱 전체를 감싸는 부품. 기억하는 값들: 준비됨, 손상, 저장 오류, 데이터, 저장 방식, 알림 권한, 알림 문제.

104줄 dataRef: 데이터의 **최신 값**을 바로 꺼내기 위한 칸 (state는 다음 그리기 때 바뀌므로, 연달아 바꿀 때 최신 값을 놓치지 않으려고). **105줄** 연습 화면 닫기 타이머 번호.

```
  /** 상태와 저장소를 함께 바꾼다. 저장이 끝나면 resolve. */
  const mutate = useCallback((recipe: (current: AppData) => Change | null) => {
    const current = dataRef.current;
    if (!current) return Promise.resolve();
    const change = recipe(current);
    if (!change) return Promise.resolve();
    dataRef.current = change.next;
    setData(change.next);
    return backend()
      .apply(change.next, change.ops)
      .then(
```

mutate — 모든 변경의 입구.

1. 최신 데이터를 꺼내고, 없으면 끝.
2. `recipe`(바꾸는 방법)로 바뀐 데이터와 작업 목록을 얻는다. null이면 바꿀 것 없음.
3. 최신 칸과 화면 상태를 바꾸고(화면이 즉시 반영),
4. 저장소에 저장한다. 성공하면 오류 문구 지우기, 실패하면 오류 문구를 띄우고 실패를 전달.

```
      () => setSaveError(null),
      (error: unknown) => {
        setSaveError(SAVE_ERROR);
        throw error;
      },
    );
}, []);
```

```
const fire = useCallback(
  (recipe: (current: AppData) => Change | null) => {
    void mutate(recipe).catch(() => undefined);
  },
  [mutate],
);
```

fire: mutate를 부르되 끝날 때까지 기다리지 않는 버전 ("쏘고 잊기"). 실패는 이미 오류 문구로 보이므로 여기선 무시.

```
const load = useCallback(async () => {
  try {
    const loaded = await loadWithFallback();
    dataRef.current = loaded.corrupt ? null : loaded.data;
    setCorrupt(loaded.corrupt);
    setData(loaded.corrupt ? null : loaded.data);
    setStorageKind(backend().kind);
    setSaveError(loaded.fallback ? FALLBACK_ERROR : null);
  } catch {
    setCorrupt(true);
    setData(null);
  } finally {
    setReady(true);
    setPermission(await checkNotifyPermission());
  }
}, []);
```

load — 처음 불러오기. persist.ts의 loadWithFallback으로 데이터를 읽는다. 손상이면 데이터 없이 손상 표시. 임시 저장소로 돌아갔으면 안내 문구. 끝나면 준비됨 표시, 알림 권한 확인.

```
useEffect(() => {
  // 저장소는 마운트 뒤에 읽어 서버 렌더와 첫 화면이 같게 한다.
  // eslint-disable-next-line react-hooks/set-state-in-effect
  void load();
}, [load]);

const reload = useCallback(() => {
  setReady(false);
  void load();
}, [load]);
```

150~154줄 화면이 처음 뜬 뒤 한 번 불러온다. (화면을 먼저 그리고 저장소를 읽어야 미리 만든 HTML과 첫 화면이 같다. `eslint-disable` 줄은 검사 도구 경고를 이 줄만 끄는 표시.)

reload: 준비됨을 끄고 다시 불러온다 ("다시 시도" 버튼용).

```
// 안드로이드: 설정이 바뀌거나 앱을 열 때 매일 리포트 알림을 다시 예약한다.
const settingsKey = data
  ? `${data.settings.notificationTime}|${data.settings.notifyEnabled}|${permission}`
  : "";
useEffect(() => {
  if (!ready || !isNativeAndroid() || !dataRef.current) return;
  void scheduleDailyReport(dataRef.current.settings).then(setNotifyIssue);
}, [ready, settingsKey]);
```

안드로이드 매일 리포트 알림 다시 예약. 알림 시각, 켜짐, 권한을 하나로 합친 글자(`settingsKey`)가 바뀌거나 앱을 열 때만 다시 예약한다. 결과 문구(문제 있으면)를 저장해 설정 화면에 보여 준다.

```
  const reset = useCallback(async () => {
    // 공부 카드는 사용자가 만든 내용이라 남기고, 푼 횟수만 지운다.
    const fresh = {
      ...defaultData(),
      cards: (dataRef.current?.cards ?? []).map((card) => ({ ...card, shown: 0, correct: 0 })),
    };
    if (backend().kind === "local") clearData();
    dataRef.current = dataRef.current ?? fresh;
    await mutate(() => ({
      next: fresh,
      ops: [{ kind: "clear" }, { kind: "settings", settings: fresh.settings }],
    })).catch(() => undefined);
    if (isNativeAndroid()) await HanbakjaGuard.resetRecords().catch(() => undefined);
    setCorrupt(false);
  }, [mutate]);
```

reset — 기록 초기화.

- 공부 카드는 사용자가 만든 것이라 **남기고**, 본 횟수와 정답 수만 0으로.
- 브라우저 저장소면 지우고, 저장소에 "전체 지우기 + 기본 설정" 작업을 보낸다.
- 폰이면 Java 쪽 기록(대기열, 상태)도 지운다(resetRecords). 손상 표시도 끈다.

```
  const dismissSaveError = useCallback(() => setSaveError(null), []);

  const saveSettings = useCallback(
    (settings: Settings) => {
      fire((current) => ({
        next: { ...current, settings },
        ops: [{ kind: "settings", settings }],
      }));
    },
    [fire],
  );

  const recordIntervention = useCallback(
    (input: InterventionInput) => {
      const id = crypto.randomUUID();
      const row = buildIntervention(input, id);
      fire((current) => ({
        next: { ...current, interventions: [...current.interventions, row] },
        ops: [{ kind: "intervention", row }],
      }));
      return id;
    },
    [fire],
  );
```

dismissSaveError: 오류 문구 닫기.

saveSettings: 설정을 바꾸고 저장.

recordIntervention: 개입 기록 한 줄을 만들어(새 id) 저장하고 그 id를 돌려준다. `crypto.randomUUID()`는 겹치지 않는 긴 이름표를 만든다.

```
  const recordLockAttempt = useCallback(
    (target: SessionTarget | null, remainingSec: number) => {
      const now = new Date();
      const row = {
        id: crypto.randomUUID(),
        createdAt: now.toISOString(),
        date: todayKey(now),
        remainingSec: Math.max(0, Math.round(remainingSec)),
        target,
      };
```

recordLockAttempt: 잠금 중에 쇼츠를 열려고 한 기록 (시각, 날짜, 남은 초, 대상) 저장.

```
    fire((current) => ({
      next: { ...current, lockAttempts: [...current.lockAttempts, row] },
      ops: [{ kind: "lock", row }],
    }));
  },
  [fire],
);
```

```
const startWatching = useCallback(
  (level: Level, interventionId: string) => {
    const now = Date.now();
    fire((current) => {
      const previous = current.activeSession?.usageId;
      const settled = finalizeSession(current, now);
      const usageId = crypto.randomUUID();
      const startTime = new Date(now).toISOString();
      const date = todayKey(new Date(now));
      const row = {
        usageId,
        date,
        startTime,
        endTime: startTime,
        duration: 0,
        clipCount: 1,
        target: "practice" as const,
        source: "web" as const,
      };
      const next: AppData = {
        ...settled,
        activeSession: {
          usageId,
          date,
          startTime,
          duration: 0,
          clipCount: 1,
          interventionId,
          levelAtStart: level,
          lastTickAt: now,
        },
        usageLogs: [...settled.usageLogs, row],
      };
      const closed = previous ? practiceOps(settled, previous).slice(0, -1) : [];
      return { next, ops: [...closed, ...practiceOps(next, usageId)] };
    });
  },
  [fire],
);
```

startWatching — 연습 시청 시작 (브라우저 연습 화면).

1. 이미 열린 시청이 있으면 먼저 마감한다(finalizeSession).
2. 새 시청 id, 시작 시각, 날짜로 기록 한 줄(0초, 영상 1개, 연습, 웹)을 만든다.
3. 열린 시청 상태(activeSession)에 어느 개입에서 시작했는지, 시작 단계, 마지막 갱신 시각을 넣는다.
4. **262~263줄** 이전 시청을 닫는 작업 + 새 시청 저장 작업을 함께 보낸다. (`slice(0, -1)`은 이전 것의 "열린 상태" 작업은 빼서 새 것만 남기려고.)

```
const flushWatch = useCallback(
  (visible: boolean) => {
    fire((current) => {
```

flushWatch: 연습 시청 시간을 지금까지로 갱신 (화면이 보일 때만 시간 더하기). **advanceClip**: 다음 영상으로 넘길 때 영상 수 +1.

```
      if (!current.activeSession) return null;
      const next = applyFlush(current, visible);
      return { next, ops: practiceOps(next, current.activeSession.usageId) };
    });
  },
  [fire],
);

const advanceClip = useCallback(() => {
  fire((current) => {
    if (!current.activeSession) return null;
    const next = withClip(current);
    return { next, ops: practiceOps(next, current.activeSession.usageId) };
  });
}, [fire]);
```

```
const cancelDetach = useCallback(() => {
  if (detachRef.current !== null) {
    window.clearTimeout(detachRef.current);
    detachRef.current = null;
  }
}, []);

const closePractice = useCallback(
  (flush: boolean) => {
    fire((current) => {
      if (!current.activeSession) return null;
      const visible =
        typeof document === "undefined" || document.visibilityState === "visible";
      const flushed = flush ? applyFlush(current, visible) : current;
      const next = finalizeSession(flushed);
      return { next, ops: practiceOps(next, current.activeSession.usageId) };
    });
  },
  [fire],
);
```

cancelDetach: 예약한 "연습 화면 닫기"를 취소.

closePractice: 열린 시청을 마감한다. `flush`가 참이면 마지막 시간까지 더한 뒤 마감.

```
const stopWatching = useCallback(() => {
  cancelDetach();
  closePractice(true);
}, [cancelDetach, closePractice]);

const scheduleDetach = useCallback(() => {
  cancelDetach();
  flushWatch(typeof document === "undefined" || document.visibilityState === "visible");
  detachRef.current = window.setTimeout(() => {
    detachRef.current = null;
    closePractice(false);
  }, 400);
}, [cancelDetach, closePractice, flushWatch]);
```

stopWatching: 바로 마감.

scheduleDetach: 연습 화면을 떠날 때 시간을 갱신하고 **0.4초 뒤**에 마감한다. 그 사이 다시 돌아오면(화면 전환 중 잠깐 떠나는 경우) 취소되어 시청이 끊기지 않는다.

```
  const beginBlock = useCallback(
    (minutes: number) => {
      const block = { minutes, until: blockUntilFromMinutes(minutes) };
      fire((current) => ({ next: { ...current, block }, ops: [{ kind: "block", block }] }));
    },
    [fire],
  );

  const adoptBlock = useCallback(
    (untilMs: number, minutes: number) => {
      fire((current) => {
        const mine = current.block.until ? new Date(current.block.until).getTime() : 0;
        if (!untilMs || untilMs <= mine) return null;
        const block = { minutes, until: new Date(untilMs).toISOString() };
        return { next: { ...current, block }, ops: [{ kind: "block", block }] };
      });
    },
    [fire],
  );
```

beginBlock: N분 잠금 시작 (끝 시각 계산해서 저장).

adoptBlock: Java 서비스 쪽 잠금이 **더 늦게 끝나면** 화면 쪽 잠금도 그 시각으로 맞춘다. 더 짧으면 무시.

```
  const mergeNative = useCallback(
    async (batch: NativeBatch) => {
      const ids = [
        ...(batch.sessions ?? []).map((row) => row.id),
        ...(batch.interventions ?? []).map((row) => row.id),
        ...(batch.lockAttempts ?? []).map((row) => row.id),
      ].filter(Boolean);
      await mutate((current) => {
        const added = convertNativeBatch(current, batch);
        // "도움이 됐나요?" 답을 해당 개입 기록에 붙인다 (같은 묶음에 온 기록 포함).
        const answers = new Map(added.feedback.map((f) => [f.refId, f.feedback]));
        const withFeedback = (rows: AppData["interventions"]) =>
          rows.map((row) =>
            answers.has(row.interventionId) ? { ...row, feedback: answers.get(row.interventionId) ?? null } : row,
          );
        const merged: AppData = {
          ...current,
          usageLogs: [...current.usageLogs, ...added.usageLogs],
          interventions: withFeedback([...current.interventions, ...added.interventions]),
          lockAttempts: [...current.lockAttempts, ...added.lockAttempts],
        };
```

mergeNative — 서비스 기록 합치기 (1). Java 대기열에서 받은 묶음(batch)을 저장한다.

1. **345~349줄** 받은 기록 id들을 모아 둔다 (끝나면 돌려줘서 ack 하게).
2. **351줄** logic.ts의 convertNativeBatch로 앱 데이터 모양으로 바꾼다 (이미 있는 것은 뺌).
3. **353~357줄** "도움이 됐나요?" 답을 해당 개입 기록에 붙인다.
4. **358~363줄** 시청, 개입, 잠금 기록을 기존 목록 뒤에 붙인다.

```
        // 시트에서 푼 카드의 본 횟수·정답 수를 올린다.
        const cardHits = new Map<string, { shown: number; correct: number }>();
        for (const row of added.interventions) {
          if (!row.cardId || row.cardCorrect === null) continue;
          const hit = cardHits.get(row.cardId) ?? { shown: 0, correct: 0 };
          hit.shown += 1;
          if (row.cardCorrect) hit.correct += 1;
          cardHits.set(row.cardId, hit);
```

mergeNative (2) 카드.

- **365~372줄** 시트에서 푼 카드마다 본 횟수, 맞힌 횟수를 센다.
- **373~382줄** 해당 카드의 숫자를 올리고, 바뀐 카드를 따로 모은다(저장용).
- **383~388줄** "이상한 문제" 신고된 카드는 목록에서 지운다.

```
        }
        const touchedCards: StudyCard[] = [];
        if (cardHits.size > 0) {
          merged.cards = current.cards.map((card) => {
            const hit = cardHits.get(card.id);
            if (!hit) return card;
            const next = { ...card, shown: card.shown + hit.shown, correct: card.correct + hit.c
orrect };
            touchedCards.push(next);
            return next;
          });
        }
        // "이상한 문제"로 신고된 카드는 지운다.
        const reported = new Set(added.reports);
        const removedCards = (merged.cards ?? current.cards).filter((card) => reported.has(card.
id));
        if (removedCards.length > 0) {
          merged.cards = (merged.cards ?? current.cards).filter((card) => !reported.has(card.i
d));
        }
```

```
        const settled = settleReentry(merged);
        const ops: Op[] = [
          ...removedCards.map((card) => ({ kind: "deleteCard" as const, id: card.id })),
          ...touchedCards.filter((row) => !reported.has(row.id)).map((row) => ({ kind: "card" as
const, row })),
          ...added.usageLogs.map((row) => ({ kind: "session" as const, row })),
          ...added.lockAttempts.map((row) => ({ kind: "lock" as const, row })),
          // 재진입 판정이 붙은 개입은 최신 내용으로 덮어쓴다.
          ...settled.data.interventions
            .filter(
              (row) =>
                added.interventions.some((a) => a.interventionId === row.interventionId) ||
                answers.has(row.interventionId) ||
                settled.changed.some((c) => c.interventionId === row.interventionId),
            )
            .map((row) => ({ kind: "intervention" as const, row })),
        ];
        if (ops.length === 0) return null;
        return { next: settled.data, ops };
      });
      return ids;
    },
    [mutate],
  );
```

mergeNative (3) 저장 작업. 재진입 판정(30분 안에 다시 봤는지)을 채운 뒤, 저장소 작업 목록을 만든다: 카드 삭제, 바뀐 카드, 시청, 잠금, 그리고 **새로 왔거나 답이 붙었거나 판정이 바뀐** 개입 기록. 할 일이 없으면 null. 저장이 끝난 뒤 id 목록을 돌려준다 → guard-bridge가 ack 한다(저장이 끝나기 전에 Java가 지우지 않게).

```
  const addCards = useCallback(
    (cards: StudyCard[]) => {
      if (cards.length === 0) return;
      fire((current) => ({
        next: { ...current, cards: [...current.cards, ...cards] },
```

addCards: 카드 여러 장을 추가 저장. **deleteCard**: 카드 한 장 삭제.

markReportSeen: 오늘 리포트를 봤다고 표시 (이미 표시했으면 아무것도 안 함).

```
      ops: cards.map((row) => ({ kind: "card" as const, row })),
    }));
  },
  [fire],
);

const deleteCard = useCallback(
  (id: string) => {
    fire((current) => ({
      next: { ...current, cards: current.cards.filter((card) => card.id !== id) },
      ops: [{ kind: "deleteCard", id }],
    }));
  },
  [fire],
);

const markReportSeen = useCallback(() => {
  const today = todayKey();
  fire((current) => {
    if (current.lastNotifiedDate === today) return null;
    return {
      next: { ...current, lastNotifiedDate: today },
      ops: [{ kind: "lastNotifiedDate", date: today }],
    };
  });
}, [fire]);
```

```
// 브라우저에서만: 앱이 열려 있을 때 리포트 시각이 지나면 Notification API로 알린다.
const notifyInBrowser = useCallback((current: AppData) => {
  if (!current.settings.notifyEnabled) return "알림이 꺼져 있습니다.";
  if (typeof Notification === "undefined") return "이 브라우저는 알림을 지원하지 않습니다.";
  if (Notification.permission !== "granted") return "브라우저 알림 권한이 없습니다.";
  const stats = statsForDate(current, todayKey());
  const body = reportLines(stats).slice(0, 4).join(" · ");
  new Notification("오늘의 Shorts 사용 리포트", { body, lang: "ko" });
  return null;
}, []);
```

notifyInBrowser: 브라우저에서 리포트 알림을 띄운다. 꺼짐, 미지원, 권한 없음이면 이유 문구를 돌려준다. 아니면 오늘 통계 앞 4줄을 " · "로 이어 알림 본문으로.

```
useEffect(() => {
  if (!ready || isNativeAndroid()) return;
  const check = () => {
    const current = dataRef.current;
    if (!current || !current.settings.notifyEnabled) return;
    if (!isReportDue(current.settings.notificationTime)) return;
    const today = todayKey();
    if (current.lastNotifiedDate === today) return;
    if (typeof Notification === "undefined" || Notification.permission !== "granted") return;
    if (notifyInBrowser(current)) return;
    fire((latest) => ({
      next: { ...latest, lastNotifiedDate: today },
      ops: [{ kind: "lastNotifiedDate", date: today }],
```

브라우저 리포트 알림 확인 (폰 아님). 15초마다: 알림 켜짐, 리포트 시각 지남, 오늘 아직 안 보냄, 권한 있음이면 알림을 띄우고 "오늘 보냄"을 저장. 화면이 사라지면 타이머를 멈춘다.

```
      }));
    };
    check();
    const id = window.setInterval(check, 15000);
    return () => window.clearInterval(id);
  }, [fire, notifyInBrowser, ready]);
```

```
  // 30분이 지난 개입의 재진입 여부를 채운다 (앱이 열려 있는 동안 1분마다).
  useEffect(() => {
    if (!ready) return;
    const settle = () =>
      fire((current) => {
        const settled = settleReentry(current);
        if (settled.changed.length === 0) return null;
        return {
          next: settled.data,
          ops: settled.changed.map((row) => ({ kind: "intervention" as const, row })),
        };
      });
    settle();
    const id = window.setInterval(settle, 60_000);
    return () => window.clearInterval(id);
  }, [fire, ready]);
```

재진입 판정 채우기. 앱이 열려 있는 동안 1분마다, 30분이 지난 개입에 "다시 봤는지"를 채우고 바뀐 것만 저장한다. AI 점수 계산에 쓰인다.

```
  const requestNotificationPermission = useCallback(async () => {
    const result = await requestNotifyPermission();
    setPermission(result);
    if (result === "unsupported") return "이 기기에서는 알림을 쓸 수 없습니다.";
    if (result === "denied") {
      return isNativeAndroid()
        ? "알림이 꺼져 있습니다. 휴대폰 설정 → 애플리케이션 → ShortsAI → 알림에서 허용해 주세요."
        : "알림이 차단되어 있습니다. 브라우저 사이트 설정에서 허용해 주세요.";
    }
    if (result !== "granted") return "알림 권한을 허용하지 않았습니다.";
    return null;
  }, []);

  const previewNotification = useCallback(async () => {
    const current = dataRef.current;
    if (!current) return "기록을 아직 불러오지 못했습니다.";
    if (isNativeAndroid()) return previewReport();
    return notifyInBrowser(current);
  }, [notifyInBrowser]);
```

requestNotificationPermission: 알림 권한을 요청하고, 거절이면 어디서 허용하는지 안내 문구를 돌려준다 (폰/브라우저 다르게).

previewNotification: 알림 미리 보기. 폰이면 notify.ts, 브라우저면 notifyInBrowser.

```
513   const getData = useCallback(() => dataRef.current, []);
514
515   const value = useMemo<StoreContextValue>(
516     () => ({
517       ready,
518       corrupt,
519       saveError,
520       data,
521       storageKind,
522       permission,
523       notifyIssue,
524       getData,
525       reload,
526       reset,
527       dismissSaveError,
528       saveSettings,
529       recordIntervention,
530       recordLockAttempt,
531       startWatching,
532       flushWatch,
533       advanceClip,
534       stopWatching,
535       beginBlock,
536       adoptBlock,
537       mergeNative,
538       markReportSeen,
539       addCards,
540       deleteCard,
541       cancelDetach,
542       scheduleDetach,
543       requestNotificationPermission,
544       previewNotification,
545     }),
546     [
547       ready,
548       corrupt,
549       saveError,
550       data,
551       storageKind,
552       permission,
553       notifyIssue,
554       getData,
555       reload,
556       reset,
557       dismissSaveError,
558       saveSettings,
559       recordIntervention,
560       recordLockAttempt,
561       startWatching,
562       flushWatch,
```

getData: 최신 데이터를 꺼내는 함수 (타이머 안에서 쓰기 좋음).

515~576줄 위의 값과 기능을 하나로 묶는다. `useMemo`로 묶어서, 아래 목록 중 하나가 바뀔 때만 새 묶음을 만든다 (그래야 쓸데없이 모든 화면이 다시 그려지지 않는다).

578줄 공유 상자에 담아 안쪽 화면들(children)에 내려 준다.

```
      advanceClip,
      stopWatching,
      beginBlock,
      adoptBlock,
      mergeNative,
      markReportSeen,
      addCards,
      deleteCard,
      cancelDetach,
      scheduleDetach,
      requestNotificationPermission,
      previewNotification,
    ],
  );

  return <StoreContext.Provider value={value}>{children}</StoreContext.Provider>;
}
```

```
export function useStore() {
  const value = useContext(StoreContext);
  if (!value) throw new Error("StoreProvider가 필요합니다.");
  return value;
}
```

useStore: 화면 부품에서 창고를 꺼내는 함수. StoreProvider 바깥에서 부르면 오류를 낸다 (실수를 바로 알 수 있게).

guard-bridge.tsx — 웹과 Java를 잇는 다리 `components/guard-bridge.tsx (107줄)`

화면에는 아무것도 그리지 않지만(`return null`), 앱이 열려 있는 동안 **1.5초마다** Java 접근성 서비스와 데이터를 주고받는 핵심 부품. "식당 비유"의 **주문 전달 담당**이다.

```
"use client";

import { useEffect, useRef } from "react";
import { usePathname, useRouter } from "next/navigation";
import { useStore } from "@/components/store";
import { HanbakjaGuard, isGuardTarget, isNativeAndroid } from "@/lib/guard";
import { todayKey, webSecondsOn } from "@/lib/logic";
import { riskHours, hourLabel } from "@/lib/ai";
import { onNotificationAction, scheduleRiskNotice } from "@/lib/notify";
import { buildPolicy, cardsForService } from "@/lib/policy";

/**
 * 안드로이드에서 1.5초마다 접근성 서비스와 맞춘다.
 * 1. 정책표·공부 카드·웹이 잰 오늘 초를 넘기고, 서비스가 쌓은 세션·개입·잠금 시도를 받는다.
 * 2. SQLite에 저장한 뒤에만 ack 해서, 저장 전에 앱이 꺼져도 기록이 남는다.
 * 3. 서비스가 넘긴 개입(Level 3, Instagram)이 있으면 개입 화면을 연다.
 */
```

3~10줄 가져오기: React, 화면 이동(router), 창고, 플러그인, 계산, AI 위험 시간, 알림, 정책표.

12~17줄 설명: ① 정책표·공부 카드·웹이 잰 초를 보내고 서비스 기록을 받는다, ② SQLite에 저장한 **뒤에만** ack(받았다는 확인) → 저장 전에 앱이 꺼져도 Java 쪽에 기록이 남아 다음에 다시 받는다, ③ 서비스가 넘긴 개입이 있으면 개입 화면을 연다.

```
export function GuardBridge() {
  const router = useRouter();
```

19~21줄 화면 이동 도구, 지금 주소, 창고에서 필요한 것 꺼내기.

```
const pathname = usePathname();
const { ready, data, getData, mergeNative, adoptBlock, beginBlock } = useStore();
const pathRef = useRef(pathname);
const busy = useRef(false);

useEffect(() => {
  pathRef.current = pathname;
}, [pathname]);
```

22~27줄 지금 주소를 기억 칸에 계속 넣어 둔다 (1.5초 타이머 안에서 최신 주소를 보려고). **23줄 busy**: 이전 맞추기가 아직 진행 중이면 겹쳐 실행하지 않으려는 표시.

```
useEffect(
  () =>
    onNotificationAction({
      open: (route) => router.push(route),
      lock: (minutes) => {
        beginBlock(minutes);
        void HanbakjaGuard.startBlock({ minutes }).catch(() => undefined);
        router.push("/");
      },
    }),
  [beginBlock, router],
);
```

알림 버튼 처리 연결. 알림을 누르면 해당 화면으로 이동. 위험 시간 알림의 [1시간 잠그기]면 화면 쪽 잠금 + Java 쪽 잠금(startBlock)을 함께 걸고 홈으로 간다.

```
// AI: 위험 시간 알림은 하루에 한 번, 그리고 AI 설정이 바뀔 때 다시 예약한다.
const aiOn = data?.settings.aiEnabled ?? false;
const today = todayKey();
useEffect(() => {
  if (!ready || !isNativeAndroid()) return;
  const current = getData();
  const risk = current && aiOn ? riskHours(current) : null;
  void scheduleRiskNotice(risk ? risk.hours[0] : null, risk ? hourLabel(risk.hours[0]) : "");
}, [aiOn, getData, ready, today]);
```

AI 위험 시간 알림 예약. AI 켜짐 여부나 날짜가 바뀔 때(= 하루 한 번) 다시 계산한다. AI가 켜져 있으면 riskHours로 가장 위험한 시간 하나를 골라 그 10분 전 알림을 예약, 꺼져 있으면 null을 넘겨 예약을 지운다.

```
useEffect(() => {
  if (!ready || !isNativeAndroid()) return;
  let stopped = false;

  async function sync() {
    if (busy.current) return;
    const data = getData();
    if (!data) return;
    busy.current = true;
    try {
      const status = await HanbakjaGuard.getStatus();
      if (stopped) return;
      if (
        status.blockerEnabled &&
        isGuardTarget(status.pendingTarget) &&
        !pathRef.current.startsWith("/watch")
      ) {
        router.push(`/watch?guard=${status.pendingTarget}`);
      }
```

sync — 1.5초마다 하는 일 (1) 상태 확인.

1. 이미 진행 중이거나 데이터가 없으면 건너뛴다.
2. **62줄** getStatus로 Java 상태를 받는다. `stopped`는 화면이 사라졌는지 (사라졌으면 그만).
3. **64~70줄** 개입 스위치가 켜져 있고, 서비스가 넘긴 개입이 있고, 지금 개입 화면이 아니면 → `/watch?guard=youtube`로 이동해 앱 화면 개입을 연다.
4. **71~73줄** Java 쪽 잠금이 아직 안 끝났으면 화면 쪽 잠금도 맞춘다.

```
        if (status.blockedUntil > Date.now()) {
          adoptBlock(status.blockedUntil, status.blockMinutes);
        }
```

```
        const today = todayKey();
        const batch = await HanbakjaGuard.sync({
          level1Threshold: data.settings.level1Threshold,
          level3Threshold: data.settings.level3Threshold,
          webDate: today,
          webSeconds: webSecondsOn(data, today),
          policy: buildPolicy(data.settings, data),
          cards: cardsForService(data),
        });
        if (stopped) return;
        const ids = await mergeNative(batch);
        if (ids.length > 0) await HanbakjaGuard.ack({ ids });
      } catch {
        // 저장에 실패하면 ack 하지 않는다. 다음 주기에 다시 받는다.
      } finally {
        busy.current = false;
      }
    }
```

sync (2) 기록 주고받기.

1. **75~82줄** Java에 보냄: 단계 기준 분, 오늘 날짜, 웹 연습 화면에서 잰 오늘 초, **정책표**(buildPolicy: AI 표, 문구, 공부 모드 등), **공부 카드**(최대 60장). 돌려받음: 쌓인 시청·개입·잠금 기록 묶음.
2. **84줄** 창고에 합쳐 저장(mergeNative). 저장이 끝나야 다음 줄로.
3. **85줄** 저장된 id를 ack → Java가 대기열에서 지운다.
4. **86~87줄** 중간에 실패하면 ack 하지 않는다 → 다음 1.5초에 같은 기록을 다시 받는다 (기록이 사라지지 않음).
5. **88~90줄** 무슨 일이 있어도 진행 중 표시는 끈다.

```
    void sync();
    const id = window.setInterval(() => void sync(), 1500);
    const onVisible = () => {
      if (document.visibilityState === "visible") void sync();
    };
    document.addEventListener("visibilitychange", onVisible);
    return () => {
      stopped = true;
      window.clearInterval(id);
      document.removeEventListener("visibilitychange", onVisible);
    };
  }, [adoptBlock, getData, mergeNative, ready, router]);

  return null;
}
```

93줄 바로 한 번 실행. **94줄** 1.5초마다 반복. **95~98줄** 앱이 다시 화면에 보일 때도 즉시 실행 (다른 앱에서 돌아왔을 때 빨리 반영).

99~103줄 정리: 화면이 사라지면 멈춤 표시, 타이머와 듣기를 끈다. **106줄** 화면에는 아무것도 그리지 않는다.

use-now.ts — 지금 시각 훅 `components/use-now.ts (12줄)`

```
"use client";

import { useEffect, useState } from "react";

export function useNow(intervalMs = 1000) {
  const [now, setNow] = useState(() => Date.now());
  useEffect(() => {
    const id = window.setInterval(() => setNow(Date.now()), intervalMs);
```

useNow: 1초마다(기본) 현재 시각을 새로 넣어서, 이걸 쓰는 화면이 **자동으로 다시 그려지게** 한다. 잠금 남은 시간 "12:34"처럼 계속 줄어드는 숫자에 쓴다. 화면이 사라지면 타이머를 끈다. (`use`로 시작하는 함수를 "훅"이라 부르며, React 기능을 묶어 재사용하는 방법이다.)

```
    return () => window.clearInterval(id);
  }, [intervalMs]);
  return now;
}
```

pwa-register.tsx — 브라우저 설치용 등록 `components/pwa-register.tsx (22줄)`

```tsx
"use client";

import { useEffect } from "react";

type NativeWindow = Window & {
  Capacitor?: { isNativePlatform?: () => boolean };
};

export function PwaRegister() {
  useEffect(() => {
    const native = (window as NativeWindow).Capacitor?.isNativePlatform?.();
    if (native || !("serviceWorker" in navigator)) return;
    const register = () => {
      void navigator.serviceWorker.register("/sw.js").catch(() => {
        // 설치 안내는 설정 화면에 있고, 등록 실패가 앱 사용을 막지는 않습니다.
      });
    };
    if (document.readyState === "complete") register();
    else window.addEventListener("load", register, { once: true });
  }, []);
  return null;
}
```

PwaRegister: **브라우저에서만** 서비스 워커(`/sw.js`)를 등록한다. 서비스 워커는 앱 파일을 저장해 두어 인터넷이 없어도 열리게 하고, 홈 화면에 "앱처럼 설치"할 수 있게 해 준다(PWA).

- **5~7줄** Capacitor가 있는 창의 모양 (폰 앱인지 확인용).
- **11~12줄** 폰 앱 안이거나 서비스 워커를 지원하지 않으면 아무것도 안 함 (폰 앱은 파일이 이미 APK 안에 있음).
- **13~19줄** 페이지가 다 열린 뒤 등록한다 (첫 화면을 느리게 하지 않으려고). 실패해도 앱 사용에는 지장 없으니 무시.

app-shell.tsx — 앱 틀 (메뉴, 로딩, 오류) `components/app-shell.tsx (179줄)`

모든 화면을 감싸는 **틀**. 로딩 중 화면, 기록 손상 화면, 저장 오류 띠, 그리고 메뉴(폰: 아래 탭 4개, 넓은 화면: 왼쪽 메뉴)를 그린다.

```tsx
"use client";

import Link from "next/link";
import { usePathname } from "next/navigation";
import { ChartColumn, House, NotebookText, Settings } from "lucide-react";
import { useStore } from "@/components/store";
import { isReportDue, todayKey } from "@/lib/logic";
import { cn } from "@/lib/utils";

const NAV = [
  { href: "/", label: "홈", icon: House },
  { href: "/stats", label: "통계", icon: ChartColumn },
```

3~8줄 가져오기: 링크, 지금 주소, 아이콘(lucide-react 무료 아이콘 모음), 창고, 리포트 시각 계산, 모양 합치기.

10~15줄 NAV: 메뉴 4개 — 홈(/), 통계, 리포트, 설정. 주소, 이름, 아이콘.

```
  { href: "/report", label: "리포트", icon: NotebookText },
  { href: "/settings", label: "설정", icon: Settings },
];
```

```
function Mark({ className }: { className?: string }) {
  return (
    <img
      src="/icons/icon-192.png"
      alt=""
      className={cn("h-8 w-8 rounded-lg", className)}
    />
  );
}
```

Mark: 앱 아이콘 그림 (32×32, 둥근 모서리). `alt=""`는 화면 읽기 도구가 읽지 않게 하는 장식 표시.

```
function LoadingScreen() {
  return (
    <div
      className="flex min-h-dvh items-center justify-center px-6"
      role="status"
      aria-live="polite"
    >
      <div className="w-full max-w-sm space-y-4">
        <div className="h-8 w-28 animate-pulse rounded-lg bg-muted" />
        <div className="h-24 animate-pulse rounded-2xl bg-muted" />
        <div className="h-12 animate-pulse rounded-2xl bg-muted" />
        <p className="text-sm text-muted-foreground">기록을 불러오는 중입니다.</p>
      </div>
    </div>
  );
}
```

LoadingScreen — 불러오는 중 화면. 회색 막대 3개가 깜빡이는(`animate-pulse`) "자리 표시" 모양과 "기록을 불러오는 중입니다." `role="status"`는 화면 읽기 도구에 상태 알림임을 알려 준다.

```
function CorruptScreen() {
  const { reload, reset } = useStore();
  return (
    <div className="flex min-h-dvh items-center justify-center px-6">
      <div className="w-full max-w-md space-y-4 rounded-3xl bg-card p-6 ring-1 ring-foreground/10">
        <p className="text-sm font-medium text-destructive">기록을 읽지 못했습니다</p>
        <h1 className="text-2xl font-semibold tracking-tight">
          이 브라우저에 저장된 데이터가 손상되었습니다.
        </h1>
        <p className="text-sm leading-6 text-muted-foreground">
          사용 시간과 개입 기록을 열 수 없어 통계를 표시하지 않습니다. 다시
          읽거나, 초기화한 뒤 새로 시작할 수 있습니다.
        </p>
        <div className="flex flex-col gap-2 sm:flex-row">
          <button
            type="button"
            onClick={reload}
            className="inline-flex h-11 items-center justify-center rounded-xl bg-primary px-4 text-sm font-medium text-primary-foreground"
```

CorruptScreen — 기록 손상 화면. 저장된 데이터를 읽지 못했을 때 통계를 보여 주지 않고, 두 버튼을 준다: [다시 읽기](reload), [초기화하고 시작](reset). 잘못된 숫자를 보여 주는 것보다 솔직히 알리는 쪽을 택했다.

```
          >
            다시 읽기
          </button>
          <button
            type="button"
            onClick={reset}
            className="inline-flex h-11 items-center justify-center rounded-xl border border-bor
der bg-background px-4 text-sm font-medium"
          >
            초기화하고 시작
          </button>
        </div>
      </div>
    </div>
  );
}
```

```
export function AppShell({ children }: { children: React.ReactNode }) {
  const pathname = usePathname();
  const { ready, corrupt, saveError, dismissSaveError, data } = useStore();
  const immersive = pathname.startsWith("/watch");
  const reportDue = data
    ? isReportDue(data.settings.notificationTime) &&
      data.lastNotifiedDate !== todayKey()
    : false;

  if (!ready) return <LoadingScreen />;
  if (corrupt) return <CorruptScreen />;

  if (immersive) {
    return <div className="min-h-dvh">{children}</div>;
  }
```

AppShell 시작.

- **81줄 immersive**: 개입·시청 화면(/watch)은 메뉴 없이 전체 화면으로.
- **82~85줄 reportDue**: 리포트 시각이 지났고 오늘 아직 안 봤으면 → 메뉴의 리포트에 점 표시.
- **87~92줄** 순서대로: 불러오는 중 → 로딩 화면, 손상 → 손상 화면, /watch → 틀 없이 내용만.

```
  return (
    <div className="min-h-dvh md:grid md:grid-cols-[240px_minmax(0,1fr)]">
      <aside className="hidden border-r border-border/80 md:flex md:flex-col md:px-4 md:py-6">
        <Link href="/" className="flex items-center gap-2 px-2">
          <Mark />
          <span className="text-lg font-semibold tracking-tight">Focus on</span>
        </Link>
        <p className="mt-2 px-2 text-xs leading-5 text-muted-foreground">
          Shorts · Reels 사용 조절
        </p>
        <nav className="mt-8 flex flex-col gap-1" aria-label="주요">
          {NAV.map((item) => {
            const active = pathname === item.href;
            const Icon = item.icon;
            return (
              <Link
                key={item.href}
                href={item.href}
```

넓은 화면용 왼쪽 메뉴 (`md:` = 태블릿 이상 너비에서만, 폰에서는 `hidden`).

- 앱 아이콘 + "Focus on", 설명 "Shorts · Reels 사용 조절".
- **105~127줄** 메뉴 4개를 반복해서 그린다. 지금 화면이면 진하게(`aria-current="page"`). 리포트 메뉴에는 볼 리포트가 있을 때 작은 점.

```
                aria-current={active ? "page" : undefined}
                className={cn(
                  "flex h-11 items-center gap-2 rounded-xl px-3 text-sm",
                  active
                    ? "bg-secondary font-medium text-foreground"
                    : "text-muted-foreground hover:bg-secondary/70 hover:text-foreground",
                )}
              >
                <Icon className="size-4" />
                {item.label}
                {item.href === "/report" && reportDue ? (
                  <span className="ml-auto size-2 rounded-full bg-primary" />
                ) : null}
              </Link>
            );
          })}
        </nav>
      </aside>
```

```
      <div className="flex min-h-dvh flex-col">
        {saveError ? (
          <div
            className="flex items-start justify-between gap-3 border-b border-destructive/20 bg-destructive/10 px-4 py-3 text-sm text-destructive"
            role="alert"
          >
            <p>{saveError}</p>
            <button type="button" className="shrink-0 underline" onClick={dismissSaveError}>
              닫기
            </button>
          </div>
        ) : null}
        <div className="mx-auto flex w-full max-w-5xl flex-1 flex-col px-4 pt-5 pb-28 md:px-8 md:pt-8 md:pb-10">
          <div className="mb-4 flex items-center gap-2 md:hidden">
            <Mark />
            <span className="text-lg font-semibold tracking-tight">Focus on</span>
          </div>
          {children}
        </div>
      </div>
```

본문 영역.

- **131~141줄** 저장 오류가 있으면 빨간 띠로 문구와 [닫기]. `role="alert"` 는 화면 읽기 도구가 바로 읽게 하는 표시.
- **142~148줄** 가운데 정렬된 내용 칸. 폰에서는 위에 아이콘 + 이름을 보이고, 아래 탭에 가리지 않게 아래 여백(pb-28)을 크게 둔다. `{children}` 자리에 각 화면(홈, 통계 …)이 들어간다.

```
      <nav
        className="fixed inset-x-0 bottom-0 z-20 border-t border-border/80 bg-background/95 backdrop-blur md:hidden"
        style={{ paddingBottom: "max(0.4rem, env(safe-area-inset-bottom))" }}
        aria-label="주요"
      >
        <ul className="grid grid-cols-4">
          {NAV.map((item) => {
            const active = pathname === item.href;
```

폰용 아래 탭 메뉴 (`md:hidden` = 넓은 화면에선 숨김). 화면 맨 아래에 고정 (`fixed bottom-0`), 반투명 배경. **152줄** 아이폰 홈 막대 같은 아래 안전 영역만큼 여백을 더 준다. 4칸 격자로 아이콘 + 이름.

```
            const Icon = item.icon;
            return (
              <li key={item.href}>
                <Link
                  href={item.href}
                  aria-current={active ? "page" : undefined}
                  className={cn(
                    "flex h-14 flex-col items-center justify-center gap-1 text-[11px]",
                    active ? "font-medium text-foreground" : "text-muted-foreground",
                  )}
                >
                  <Icon className="size-5" />
                  {item.label}
                </Link>
              </li>
            );
          })}
        </ul>
      </nav>
    </div>
  );
}
```

home-view.tsx — 홈 화면 `components/home-view.tsx (246줄)`

앱을 열면 처음 보이는 화면. 오늘 사용량 막대, 남은 시간, 시간대별 막대, 오늘의 개입 숫자, 리포트 바로 가기, 최근 개입 목록을 보여 준다.

```
"use client";

import { useRouter } from "next/navigation";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { GoalSuggestionCard } from "@/components/goal-suggestion";
import { GuardPanel } from "@/components/guard-panel";
import { useStore } from "@/components/store";
import { useNow } from "@/components/use-now";
import {
  BAND_LABEL,
  blockRemainingSeconds,
  formatClock,
  formatTime,
  FRAMINGS,
  goalRatio,
  hourlyUsage,
  insightFacts,
  insightLine,
  isBlocked,
```

3~10줄 가져오기: 화면 이동, 작은 부품들(배지, 버튼, 카드), 주간 제안 카드, 개입 스위치 패널, 창고, 1초 시계.

11~31줄 logic.ts에서 쓸 함수와 글자표(단계 이름, 결과 이름 등).

```
  isReportDue,
  LEVEL_SUMMARY,
  levelForUsage,
  nextLevelGap,
  OUTCOME_LABEL,
  reasonLabel,
  statsForDate,
  todayKey,
  usageSecondsOn,
} from "@/lib/logic";
```

```
export function HomeView() {
  const router = useRouter();
  const { data } = useStore();
  const now = useNow();
  if (!data) return null;

  const today = todayKey(new Date(now));
  const seconds = usageSecondsOn(data, today);
  const level = levelForUsage(seconds, data.settings);
  const stats = statsForDate(data, today);
  const gap = nextLevelGap(seconds, data.settings);
  const blocked = isBlocked(data.block, now);
  const remaining = blockRemainingSeconds(data.block, now);
  const reportReady = isReportDue(data.settings.notificationTime, new Date(now));
  const recent = data.interventions
    .filter((log) => log.date === today)
    .slice()
    .reverse()
    .slice(0, 3);
```

HomeView 계산 (1). `useNow()` 덕분에 1초마다 다시 계산된다.

- 오늘 날짜, 오늘 본 초, 지금 단계(약함/중간/강함), 오늘 통계, 다음 단계까지 남은 시간.
- 잠금 중인지, 남은 초, 리포트 시각이 지났는지.
- **47~51줄** 오늘 개입 기록 중 **최근 3개** (뒤집어서 최신부터). `slice()`로 복사한 뒤 뒤집어 원래 목록을 건드리지 않는다.

```
  const goal = data.settings.level3Threshold;
  const ratio = goalRatio(seconds, data.settings);
  const percent = Math.round(ratio * 100);
  const budgetLeft = goal * 60 - seconds;
  // 막대 끝은 목표의 135%: 넘친 만큼도 보이게 한다. 50%·100% 지점에 강도 경계.
  const scaleMax = 1.35;
  const marker = Math.min(100, (ratio / scaleMax) * 100);
  const l1Width = (0.5 / scaleMax) * 100;
  const l2Width = (0.5 / scaleMax) * 100;
  const l3Width = 100 - l1Width - l2Width;
  const hours = hourlyUsage(data, today);
  const peakHour = Math.max(...hours, 1);
  // 시간 체감 문구: 하루 동안은 같은 종류를 쓰고, 숫자가 없는 종류는 건너뛴다.
  const facts = insightFacts(data, new Date(now));
  const framing =
    FRAMINGS.map((_, i) => insightLine(FRAMINGS[(new Date(now).getDate() + i) % FRAMINGS.length], facts)).find(
      (line): line is string => line !== null,
    ) ?? null;
```

HomeView 계산 (2) 막대.

- 목표(분), 목표 대비 비율, 퍼센트, 남은 초(음수면 넘음).
- **56~61줄** 막대 끝을 목표의 **135%**로 둔다 (넘친 만큼도 보이게). 막대를 세 색으로: 0~50% 약함, 50~100% 중간, 100% 넘으면 강함. 동그란 표시는 지금 위치.
- **62~63줄** 시간대별(0~23시) 시청 초와 가장 큰 값 (막대 높이 기준).
- **64~69줄** 시간 체감 문구: 날짜에 따라 문구 종류를 돌리되, 숫자가 없어서 만들 수 없는 종류는 건너뛰고 첫 번째로 만들어지는 문장을 고른다.

```
  function openShorts() {
    router.push("/watch");
  }

  return (
    <div className="flex flex-col gap-6">
      <header className="flex items-end justify-between gap-3">
        <div>
          <p className="text-sm text-muted-foreground">YouTube Shorts · Instagram Reels</p>
          <h1 className="mt-1 text-3xl font-semibold tracking-tight">Focus on</h1>
        </div>
        <Badge variant={level === 3 ? "destructive" : "secondary"}>
          {BAND_LABEL[level]}
        </Badge>
      </header>

      <GoalSuggestionCard />

      <GuardPanel />
```

openShorts: [쇼츠 열기]를 누르면 연습 화면(/watch)으로.

75~85줄 머리: "YouTube Shorts · Instagram Reels", 큰 제목 "Focus on", 오른쪽에 지금 단계 배지(강함이면 빨강).

87~89줄 AI 주간 목표 제안 카드, 개입 스위치·접근성 안내 패널.

```
      <div className="grid gap-6 lg:grid-cols-[minmax(0,1.15fr)_minmax(260px,0.85fr)]">
        <Card className="gap-5">
          <CardHeader>
            <CardTitle className="text-sm font-medium text-muted-foreground">
              오늘 목표의 {percent}% 사용
            </CardTitle>
            <p className="text-4xl font-semibold tracking-tight tabular-nums">
              {budgetLeft > 0 ? `${formatClock(budgetLeft)} 남음` : `${formatClock(-budgetLeft)} 넘음`}
            </p>
            <p className="text-sm text-muted-foreground">
              오늘 {formatClock(seconds)} / 목표 {goal}분 · {LEVEL_SUMMARY[level]}
            </p>
          </CardHeader>
```

오늘 사용 카드 머리. 넓은 화면에선 두 칸으로 나눈다. "오늘 목표의 N% 사용", 큰 글씨 "12:34 남음"(넘었으면 "넘음"), "오늘 12:34 / 목표 30분 · 단계 설명". `tabular-nums`는 숫자 폭을 같게 해 숫자가 바뀔 때 흔들리지 않게 한다.

```
          <CardContent className="space-y-3">
            <div
              className="relative h-3 overflow-hidden rounded-full bg-muted"
              role="img"
              aria-label={`오늘 목표의 ${percent}% 사용. 목표의 절반부터 중간, 목표를 넘으면 강함입니다.`}
            >
              <div className="absolute inset-0 flex">
                <div className="h-full bg-level-1/80" style={{ width: `${l1Width}%` }} />
                <div className="h-full bg-level-2/85" style={{ width: `${l2Width}%` }} />
                <div className="h-full bg-level-3/85" style={{ width: `${l3Width}%` }} />
              </div>
              <div
                aria-background bg-foreground shadow"
                style={{ left: `clamp(0px, calc(${marker}% - 8px), calc(100% - 16px))` }}
```

사용량 막대. 세 색 구간을 깔고(**110~114줄**), 그 위에 지금 위치 동그라미(**115~118줄**, `clamp`로 막대 밖으로 나가지 않게). 아래에 눈금 "0분 / 절반 15분 / 목표 30분". `aria-label`은 화면 읽기 도구용 설명.

```
            />
          </div>
          <div className="flex justify-between text-xs text-muted-foreground tabular-nums">
            <span>0분</span>
            <span>절반 {Math.round(goal / 2)}분</span>
            <span>목표 {goal}분</span>
          </div>
```

```
          <p className="text-sm">
            {gap
              ? `${gap.label} ${formatClock(gap.remainSeconds)} 남았어요.`
              : "오늘 목표를 넘었어요. 다음에 열면 오늘을 마무리할지 고르게 돼요."}
          </p>
          {framing ? <p className="text-sm text-muted-foreground">{framing}</p> : null}
```

다음 단계까지 남은 시간 문장 (예: "중간까지 3:20 남았어요."). 목표를 넘었으면 "오늘 목표를 넘었어요 …". 그 아래 시간 체감 문구(있으면).

```
          <div aria-label="오늘 시간대별 시청" role="img">
            <div className="flex h-12 items-end gap-[2px]">
              {hours.map((value, hour) => (
                <span
                  key={hour}
                  title={`${hour}시 ${formatClock(value)}`}
                  className={value > 0 ? "flex-1 rounded-sm bg-primary/80" : "flex-1 rounded-sm bg-muted"}
                  style={{ height: `${Math.max(6, (value / peakHour) * 100)}%` }}
                />
              ))}
            </div>
            <div className="mt-1 flex justify-between text-[10px] text-muted-foreground tabular-nums">
              <span>0시</span>
              <span>6시</span>
              <span>12시</span>
              <span>18시</span>
              <span>24시</span>
            </div>
          </div>
```

시간대별 막대 24개. 각 시의 시청을 가장 많이 본 시 대비 높이로 그린다 (최소 6%, 0이면 회색). 손가락을 올리면(`title`) "21시 5:00". 아래 눈금 0·6·12·18·24시.

```
          {blocked ? (
            <div className="rounded-2xl bg-destructive/10 px-4 py-3 text-sm">
              <p className="font-medium text-destructive">쇼츠 열기가 차단되어 있습니다</p>
              <p className="mt-1 tabular-nums text-foreground">
                남은 시간 {formatClock(remaining)}
              </p>
            </div>
          ) : null}
          <Button
            type="button"
            className="h-12 w-full text-base"
            onClick={openShorts}
          >
            {blocked ? "차단 상태 보기" : "쇼츠 열기"}
```

- **150~157줄** 잠금 중이면 빨간 상자 "쇼츠 열기가 차단되어 있습니다 / 남은 시간 12:34".
- **158~164줄** 큰 버튼: 잠금 중이면 [차단 상태 보기], 아니면 [쇼츠 열기].
- **165~168줄** 안내: 이 버튼은 연습이고, 안드로이드에서 접근성을 켜면 실제 쇼츠에서 같은 개입이 열린다.

```
            </Button>
            <p className="text-xs leading-5 text-muted-foreground">
              이 버튼은 연습입니다. 안드로이드에서 접근성을 켜면 실제 YouTube
              Shorts와 Instagram Reels 화면에서 같은 개입이 열립니다.
            </p>
```

```
          </CardContent>
        </Card>

        <div className="grid gap-3 sm:grid-cols-2 lg:grid-cols-1">
          <Card size="sm">
            <CardHeader>
              <CardTitle>오늘의 개입</CardTitle>
            </CardHeader>
            <CardContent className="grid grid-cols-3 gap-2 text-center">
              <Stat label="스스로 멈춤" value={`${stats.selfStopCount}번`} />
              <Stat label="쉰 시간" value={stats.restSeconds >= 60 ? formatClock(stats.restSeconds) : "0분"} />
              <Stat label="잠금 연속" value={`${stats.lockStreak}일`} />
            </CardContent>
          </Card>
          <Card size="sm">
            <CardHeader>
              <CardTitle>일일 리포트</CardTitle>
            </CardHeader>
            <CardContent className="space-y-2 text-sm">
              <p>
                {reportReady
                  ? "오늘의 리포트를 볼 수 있습니다."
                  : `${data.settings.notificationTime}에 하루를 정리합니다.`}
              </p>
              <Button variant="outline" className="h-10" onClick={() => router.push("/report")}>
                리포트 열기
              </Button>
            </CardContent>
          </Card>
        </div>
      </div>
```

오른쪽 카드 2개.

- **오늘의 개입**: 스스로 멈춤 횟수, 쉰 시간, 잠금 연속 일수 (작은 Stat 부품 3개).
- **일일 리포트**: 리포트 시각이 지났으면 "볼 수 있습니다", 아니면 "21:00에 하루를 정리합니다." + [리포트 열기].

```
      <section className="space-y-3">
        <h2 className="text-lg font-semibold">최근 개입</h2>
        {recent.length === 0 ? (
          <Card>
            <CardContent className="py-8 text-sm leading-6 text-muted-foreground">
              아직 오늘 쇼츠를 열지 않았습니다. 열면 이유와 레벨, 그다음 선택이
              여기에 남습니다.
            </CardContent>
          </Card>
        ) : (
          <ul className="space-y-2">
            {recent.map((log) => (
```

최근 개입 목록. 오늘 기록이 없으면 안내 카드. 있으면 최대 3줄: "Level 2 · 심심해서", 시각, 결과(예: "3분만 보기") · 고른 대체 행동 · "수행함". `<time dateTime>`은 기계가 읽을 수 있는 시각 표시.

```
              <li
                key={log.interventionId}
                className="rounded-2xl bg-card px-4 py-3 ring-1 ring-foreground/10"
              >
                <div className="flex items-baseline justify-between gap-3">
                  <p className="font-medium">
                    Level {log.level} · {reasonLabel(log)}
                  </p>
                  <time className="text-xs text-muted-foreground tabular-nums" dateTime={log.tim
estamp}>
                    {formatTime(log.timestamp)}
                  </time>
                </div>
                <p className="mt-1 text-sm text-muted-foreground">
                  {OUTCOME_LABEL[log.outcome]}
                  {log.alternativeAction ? ` · ${log.alternativeAction}` : ""}
                  {log.alternativeCompleted ? " · 수행함" : ""}
                </p>
              </li>
            ))}
          </ul>
        )}
      </section>
    </div>
  );
}
```

```
function Stat({ label, value }: { label: string; value: string }) {
  return (
    <div>
      <p className="text-xs text-muted-foreground">{label}</p>
      <p className="mt-1 text-lg font-semibold tabular-nums">{value}</p>
    </div>
  );
}
```

Stat: 작은 이름 + 큰 숫자 한 칸을 그리는 작은 부품.

goal-suggestion.tsx — AI 주간 목표 제안 카드 `components/goal-suggestion.tsx (60줄)`

```
"use client";

import { Button } from "@/components/ui/button";
import { useStore } from "@/components/store";
import { goalSuggestion } from "@/lib/ai";
import { settingsFromGoal } from "@/lib/storage";

/**
 * AI 주간 목표 제안 (홈). 지난주 목표를 지킨 날이 5일 이상이면 2~5분 줄이자고 제안하고,
 * 최종 목표에 닿았으면 유지 단계라고 알린다. 한 주에 한 번, 사용자가 답하면 사라진다.
 * 문구는 사용자 확인 완료 (docs/HANDOFF.md).
```

설명(8~12줄): 지난주 목표를 5일 이상 지켰으면 2~5분 줄이자고 제안, 최종 목표에 닿았으면 유지 단계라고 알린다. 한 주에 한 번, 답하면 사라진다.

14~18줄 보여 주지 않는 경우: 데이터가 없음, AI 꺼짐, 제안 없음(ai.ts의 goalSuggestion이 null), 이번 주에 이미 답함.

```
 */
export function GoalSuggestionCard() {
  const { data, saveSettings } = useStore();
  if (!data || !data.settings.aiEnabled) return null;
  const suggestion = goalSuggestion(data);
  if (!suggestion || data.settings.goalWeekAnswered === suggestion.week) return null;
  const settings = data.settings;

  function answer(accept: boolean) {
    if (!suggestion) return;
    const next =
      accept && suggestion.reduce
        ? settingsFromGoal(suggestion.suggested, { ...settings, goalWeekAnswered: suggestion.week })
        : { ...settings, goalWeekAnswered: suggestion.week };
    saveSettings(next);
  }

  return (
    <section className="space-y-3 rounded-3xl bg-secondary px-5 py-4">
      <p className="text-xs font-medium text-muted-foreground">AI 이번 주 제안</p>
      {suggestion.reduce ? (
        <>
          <p className="text-base leading-7">
            지난주 {suggestion.keptDays}일 동안 목표 {suggestion.current}분을 지켰어요. 이번 주는{" "}
            <strong>{suggestion.suggested}분</strong>으로 줄여 볼까요?
          </p>
          <div className="flex flex-col gap-2 sm:flex-row">
            <Button type="button" className="h-11" onClick={() => answer(true)}>
              {suggestion.suggested}분으로 할게요
            </Button>
            <Button type="button" variant="outline" className="h-11" onClick={() => answer(false)}>
              이번 주는 그대로
            </Button>
          </div>
        </>
      ) : (
        <>
          <p className="text-base leading-7">
            최종 목표 {suggestion.current}분에 닿았어요. 이제는 지키는 단계예요. 지난주{" "}
            {suggestion.keptDays}일을 지켰어요.
          </p>
          <Button type="button" className="h-11" onClick={() => answer(false)}>
            좋아요
          </Button>
        </>
      )}
    </section>
```

answer — 답하기. [줄일게요]면 settingsFromGoal로 새 목표에 맞춰 단계 기준(절반 등)까지 다시 계산해서 저장. 아니면 "이번 주 답함"만 저장한다. 어느 쪽이든 이번 주에는 다시 뜨지 않는다.

화면. 작은 제목 "AI 이번 주 제안".

- **줄이기 제안**: "지난주 6일 동안 목표 30분을 지켰어요. 이번 주는 **25분**으로 줄여 볼까요?" + [25분으로 할게요] [이번 주는 그대로].
- **유지**: "최종 목표 10분에 닿았어요. 이제는 지키는 단계예요 …" + [좋아요].

`<>...</>`는 감싸는 태그 없이 여러 개를 묶는 표시. `{" "}`는 줄바꿈 사이 띄어쓰기 한 칸.

```
  );
}
```

guard-panel.tsx — 개입 스위치와 접근성 안내 `components/guard-panel.tsx (223줄)`

홈 화면의 "쇼츠·릴스 개입" 스위치, 끄기 확인, 접근성 켜는 방법 안내를 담당한다.

```tsx
"use client";

import { useEffect, useState } from "react";
import { Button } from "@/components/ui/button";
import { Label } from "@/components/ui/label";
import { Switch } from "@/components/ui/switch";
import { useStore } from "@/components/store";
import { HanbakjaGuard, isNativeAndroid } from "@/lib/guard";
import { blockRemainingSeconds, formatClock, isBlocked, todayKey, usageSecondsOn } from "@/lib/logic";

const OFF_WAIT_SEC = 3;

/** 끄기 전에 한 번 더: 오늘 본 시간을 보여 주고 스스로 정한 약속을 떠올리게 한다. */
const OFF_LINES = [
  (minutes: number) => `오늘 쇼츠를 벌써 ${minutes}분 봤어요. 지금 끄면 아무도 멈춰주지 않아요.`,
  (minutes: number) =>
    `끄는 건 쉬워요. 다시 켜는 건 생각보다 어려워요. 오늘 ${minutes}분, 정말 괜찮으세요?`,
];
```

11줄 끄기 버튼이 눌리기까지 3초 기다림.

13~18줄 OFF_LINES: 끄기 전에 보여 주는 문장 2개 (번갈아). 오늘 본 분을 넣어 "정말 괜찮은지" 한 번 더 생각하게 한다. 끄는 것을 **막지는 않는다** (강제하지 않기).

```tsx
export function GuardPanel() {
  const { data } = useStore();
  const [native, setNative] = useState(false);
  const [enabled, setEnabled] = useState<boolean | null>(null);
  const [blocker, setBlocker] = useState<boolean | null>(null);
  const [confirmOff, setConfirmOff] = useState(false);
  const [offWait, setOffWait] = useState(0);
  const [offLine, setOffLine] = useState(0);

  useEffect(() => {
    if (!confirmOff || offWait <= 0) return;
    const id = window.setTimeout(() => setOffWait((value) => value - 1), 1000);
    return () => window.clearTimeout(id);
  }, [confirmOff, offWait]);
```

기억하는 값: 폰 앱인지, 접근성 켜짐, 스위치 상태(null = 아직 확인 중), 끄기 확인창 열림, 남은 기다림 초, 고른 문장.

29~33줄 확인창이 열려 있으면 1초마다 기다림을 1씩 줄인다 (3, 2, 1, 0).

```tsx
  function setSwitch(next: boolean) {
    setBlocker(next);
    void HanbakjaGuard.setBlocker({ enabled: next }).catch(() => {
      setBlocker(!next);
    });
  }
```

setSwitch: 화면 스위치를 먼저 바꾸고 Java에 알린다. 실패하면 원래대로 되돌린다.

```
42  useEffect(() => {
43    const onAndroid = isNativeAndroid();
44    setNative(onAndroid);
45    let stopped = false;
46
47    async function read() {
48      try {
49        const status = await HanbakjaGuard.getStatus();
50        if (stopped) return;
51        setBlocker(status.blockerEnabled);
52        if (onAndroid) setEnabled(status.enabled);
53      } catch {
54        if (stopped) return;
55        setBlocker(true);
56        if (onAndroid) setEnabled(false);
57      }
58    }
59
60    void read();
61    const id = window.setInterval(() => void read(), 2000);
62    const onVisible = () => {
63      if (document.visibilityState === "visible") void read();
64    };
65    document.addEventListener("visibilitychange", onVisible);
66    return () => {
67      stopped = true;
68      window.clearInterval(id);
69      document.removeEventListener("visibilitychange", onVisible);
70    };
71  }, []);
72
73  const blockerOn = blocker !== false;
```

Java 상태 읽기. 처음과 2초마다, 그리고 앱이 다시 보일 때 getStatus로 스위치와 접근성 상태를 읽는다 (사용자가 설정 앱에서 접근성을 켜고 돌아오면 바로 반영). 실패하면 스위치는 켜짐, 접근성은 꺼짐으로 본다.

73줄 아직 모를 때(null)는 켜짐으로 보여 준다.

```
75  return (
76    <section className="space-y-3 rounded-3xl bg-card px-4 py-4 ring-1 ring-foreground/10">
77      <div className="flex items-start justify-between gap-4">
78        <div className="space-y-1">
79          <Label htmlFor="blocker-enabled" className="text-base font-medium">
80            쇼츠.릴스 개입
81          </Label>
82          <p className="text-sm leading-6 text-muted-foreground">
83            {blocker === null
84              ? "개입 스위치를 확인하고 있습니다."
85              : blockerOn
86                ? "켜져 있습니다. 쇼츠가 열리면 영상을 멈추고 그 위에 질문을 띄웁니다."
87                : "꺼져 있습니다. 질문과 차단은 멈추고, 쇼츠 사용 시간만 기록합니다."}
88          </p>
89        </div>
90        <Switch
91          id="blocker-enabled"
```

스위치 줄. 이름 "쇼츠·릴스 개입"과 상태 설명 (확인 중 / 켜짐: 영상을 멈추고 질문을 띄움 / 꺼짐: 시간만 기록).

95~104줄 스위치를 켜면 바로 켠다. **끄려고 하면** 바로 끄지 않고, 문장을 바꾸고 3초 기다림을 걸고 확인창을 연다.

```
          checked={blockerOn}
          disabled={blocker === null}
          aria-label="쇼츠·릴스 개입"
          onCheckedChange={(checked) => {
            if (checked) {
              setConfirmOff(false);
              setSwitch(true);
              return;
            }
            setOffLine((value) => (value + 1) % OFF_LINES.length);
            setOffWait(OFF_WAIT_SEC);
            setConfirmOff(true);
          }}
        />
      </div>
```

```
      {confirmOff && data ? (
        <OffConfirm
          minutes={Math.round(usageSecondsOn(data, todayKey()) / 60)}
          line={offLine}
          wait={offWait}
          blockedMinutes={
            isBlocked(data.block) ? Math.ceil(blockRemainingSeconds(data.block) / 60) : 0
          }
          onKeep={() => setConfirmOff(false)}
          onOff={() => {
            setConfirmOff(false);
            setSwitch(false);
          }}
        />
      ) : null}
```

끄기 확인창(OffConfirm)을 연다: 오늘 본 분, 문장 번호, 남은 기다림, 지금 걸린 잠금의 남은 분. [계속 켜둘게요]는 창 닫기, [그래도 끌래요]는 끄기.

```
      <h2 className="text-base font-medium">실제 쇼츠·릴스 개입</h2>
      <p className="text-sm leading-6 text-muted-foreground">
        안드로이드에서 접근성 서비스를 직접 켜면, YouTube Shorts 플레이어가 열릴 때
        영상을 멈추고 그 위에 질문이 올라옵니다. 오늘 목표의 절반까지는 이유와 볼 시간만
        정하고, 절반을 넘으면 숨 고르기·문제 하나 같은 멈추는 활동을 거칩니다. 정한 시간이
        되면 다시 멈춥니다. 목표를 넘으면 오늘을 마무리할지, 잠시 잠글지 고릅니다. 잠금
        중에 쇼츠를 열면 남은 시간이 보이는 잠금 화면이 뜹니다. 일반 영상, 홈 피드,
        검색에서는 개입하지 않습니다. Instagram Reels는 지금처럼 이 앱 화면으로 개입합니다.
      </p>
      <p className="text-sm leading-6 text-muted-foreground">
        기록은 이 기기에만 남습니다. 화면의 글, 메시지, 비밀번호, 계정은 읽거나
        저장하거나 보내지 않습니다. 쇼츠를 보다 다른 앱으로 넘어갔는지는 앱 이름만으로
        판단하고, 쇼츠를 작은 창(PIP)으로 계속 보는지는 화면에 떠 있는 YouTube 작은 창만
        확인합니다. 다른 앱의 이름은 기록하지 않습니다. 사용 시간과 차단 시간은 항상 실제
        시간입니다.
      </p>
```

설명 문단: 접근성을 켜면 무엇을 하는지(단계별 개입, 잠금 화면, 일반 영상·홈·검색에는 개입 안 함), 그리고 **개인정보**: 기록은 기기에만, 화면 글·메시지·비밀번호·계정은 읽지 않음, 다른 앱 이름은 기록하지 않음.

```
      <ol className="list-decimal space-y-1 pl-5 text-sm leading-6 text-muted-foreground">
        <li>아래 버튼을 누르거나 휴대폰의 설정 앱을 엽니다.</li>
```

접근성 켜는 순서 3단계와 안드로이드 13 이상의 "제한된 설정 허용" 안내.

```
        <li>
          <strong className="font-medium text-foreground">설정 → 접근성</strong>으로
          이동합니다. ShortsAI가 바로 없으면{" "}
          <strong className="font-medium text-foreground">설치된 앱</strong> 또는{" "}
          <strong className="font-medium text-foreground">다운로드한 앱</strong>을
          엽니다.
        </li>
        <li>ShortsAI를 켜고, 확인 창에서 허용을 누릅니다.</li>
      </ol>
      <p className="text-sm leading-6 text-muted-foreground">
        안드로이드 13 이상에서는 그 전에{" "}
        <strong className="font-medium text-foreground">
          설정 → 앱 → ShortsAI → 오른쪽 위 ⋮ → 제한된 설정 허용
        </strong>
        이 필요할 수 있습니다.
      </p>
```

```
      {native ? (
        <p className="text-sm font-medium">
          {enabled === null
            ? "접근성 상태를 확인하고 있습니다."
            : enabled
              ? "이 기기에서 접근성 서비스가 켜져 있습니다."
              : "이 기기에서 접근성 서비스가 꺼져 있습니다."}
        </p>
      ) : (
        <p className="text-sm leading-6 text-muted-foreground">
          이 브라우저와 아이폰에서는 다른 앱의 Shorts·Reels를 막을 수 없습니다.
          접근성 설정 버튼은 설치한 안드로이드 앱 안에 있습니다. 스위치 선택은 이
          브라우저에 남습니다.
        </p>
      )}
      {native ? (
        <Button
          type="button"
          className="h-11"
          onClick={() => {
            void HanbakjaGuard.openSettings();
          }}
        >
          접근성 설정 열기
        </Button>
      ) : null}
    </section>
  );
}
```

폰 앱이면 접근성 상태 문장과 [접근성 설정 열기] 버튼(Java가 설정 화면을 연다). 브라우저면 "이 브라우저와 아이폰에서는 다른 앱을 막을 수 없습니다" 안내.

```
function OffConfirm({
  minutes,
  line,
  wait,
```

OffConfirm — 끄기 확인창. "정말 끌까요?" + 문장 + (잠금 중이면) "지금 걸린 차단은 남은 시간 동안 계속돼요." 큰 버튼 [계속 켜둘게요], 작은 글씨 버튼 [그

```
  blockedMinutes,
  onKeep,
  onOff,
}: {
  minutes: number;
  line: number;
  wait: number;
  blockedMinutes: number;
  onKeep: () => void;
  onOff: () => void;
}) {
  return (
    <div className="space-y-3 rounded-2xl bg-secondary px-4 py-4" role="alertdialog" aria-label
="개입 끄기 확인">
      <p className="text-lg font-semibold">정말 끌까요?</p>
      <p className="text-sm leading-6">{OFF_LINES[line](minutes)}</p>
      {blockedMinutes > 0 ? (
        <p className="text-sm leading-6 text-muted-foreground">
          지금 걸린 차단은 남은 {formatClock(blockedMinutes * 60)} 동안 계속돼요.
        </p>
      ) : null}
      <Button type="button" className="h-12 w-full text-base" onClick={onKeep}>
        계속 켜둘게요
      </Button>
      <button
        type="button"
        disabled={wait > 0}
        onClick={onOff}
        className="h-9 w-full text-sm text-muted-foreground underline-offset-4 enabled:hover:und
erline disabled:opacity-60"
      >
        {wait > 0 ? `그래도 끌래요 (${wait})` : "그래도 끌래요"}
      </button>
    </div>
  );
}
```

래도 끌래요 (3)] — 3초 동안은 눌리지 않는다(`disabled`). 큰 버튼이 "계속 켜기"인 것은 의도된 설계다.

intervention-flow.tsx — 앱 화면 개입 단계 `components/intervention-flow.tsx (552줄)`

앱 안에서 보여 주는 **개입 화면** (브라우저 연습 화면과 Instagram Reels, 그리고 서비스가 넘긴 Level 3). 이유 고르기 → 단계별 활동 → 결과를 기록한다. 실제 YouTube 쇼츠 위 개입은 Java의 GuardOverlay가 그리고, 이 파일은 **앱 화면판**이다.

단계(step) 흐름: reason(이유) → Level 1: wait(5초) → ready(볼지 고르기) / Level 2: alternative(대체행동 고르기) → doing(하는 중) → reenter(다시 볼지) / Level 3: warning(경고) → confirm-block(차단 확인).

```tsx
"use client";

import { useEffect, useRef, useState } from "react";
import {
  BookOpen,
  Droplets,
  Eye,
  Footprints,
  PersonStanding,
  Wind,
} from "lucide-react";
import { Button } from "@/components/ui/button";
import { Progress, ProgressLabel } from "@/components/ui/progress";
import { Textarea } from "@/components/ui/textarea";
import {
  ALTERNATIVES,
  BLOCK_MINUTES,
  formatClock,
  LEVEL_SUMMARY,
  minutesUntilDayEnd,
  recommendAlternative,
  REASONS,
  type Alternative,
} from "@/lib/logic";
import type { InterventionInput, Level } from "@/lib/types";
import { cn } from "@/lib/utils";

const ICONS = {
  "물 마시기": Droplets,
  "간단한 스트레칭": PersonStanding,
  "5분 독서": BookOpen,
  "잠시 걷기": Footprints,
  "눈 쉬기": Eye,
  "호흡하기": Wind,
};
```

3~26줄 가져오기: React, 아이콘 6개, 버튼·진행 막대·글 상자, logic.ts의 대체행동 목록, 차단 시간 목록, 이유 목록, 대체행동 추천 등.

28~35줄 ICONS: 대체행동 이름 → 아이콘 (물 → 물방울, 스트레칭 → 사람, 독서 → 책 …).

```tsx
type Step =
  | "reason"
  | "wait"
  | "ready"
  | "alternative"
  | "doing"
  | "reenter"
  | "warning"
  | "confirm-block";
```

Step: 화면 단계 8가지 이름 (위 표 참고). 이름 중 하나만 들어갈 수 있게 정해 두면 오타를 TypeScript가 잡아 준다.

```tsx
export function InterventionFlow({
  level,
  usageSeconds,
  onFinish,
  sourceLabel = null,
  dayEndHour = 6,
}: {
  level: Level;
  usageSeconds: number;
  onFinish: (input: InterventionInput) => void;
  sourceLabel?: string | null;
  dayEndHour?: number;
}) {
```

InterventionFlow가 받는 값: 단계(level), 오늘 사용 초, 끝났을 때 부를 함수(onFinish), 대상 이름(예: "Instagram Reels", 없으면 연습), 하루 끝 시각(기본 6시).

```tsx
  const [step, setStep] = useState<Step>("reason");
  const [reason, setReason] = useState<string | null>(null);
  const [note, setNote] = useState("");
  const [alternative, setAlternative] = useState<Alternative | null>(null);
  const [completed, setCompleted] = useState(false);
  const [left, setLeft] = useState(5);
  const [actionLeft, setActionLeft] = useState(0);
  const [blockMinutes, setBlockMinutes] = useState<number | null>(null);
  const headingRef = useRef<HTMLHeadingElement>(null);
  const locked = useRef(false);

  const pauseDone = step === "wait" && left <= 0;
  const actionDone = completed || (step === "doing" && actionLeft <= 0);
  const shownStep =
    pauseDone ? "ready" : step === "doing" && actionLeft <= 0 ? "reenter" : step;
```

기억하는 값: 지금 단계, 고른 이유, 기타 메모, 고른 대체행동, 완료 여부, 5초 남은 초, 대체행동 남은 초, 고른 차단 분.

- **68줄 headingRef**: 단계가 바뀔 때 제목으로 초점을 옮기려는 칸 (화면 읽기 도구 사용자용).
- **69줄 locked**: 결과를 **한 번만** 기록하게 하는 잠금 (버튼 두 번 눌림 방지).
- **71~74줄** 시간이 다 되면 보여 줄 단계를 자동으로 바꾼다: 5초 끝 → ready, 대체행동 시간 끝 → reenter.

```tsx
  useEffect(() => {
    headingRef.current?.focus();
  }, [shownStep]);

  useEffect(() => {
    if (step !== "wait" || left <= 0) return;
    const id = window.setTimeout(() => setLeft((value) => value - 1), 1000);
    return () => window.clearTimeout(id);
  }, [step, left]);

  useEffect(() => {
    if (step !== "doing" || actionLeft <= 0) return;
    const id = window.setTimeout(() => setActionLeft((value) => value - 1), 1000);
    return () => window.clearTimeout(id);
  }, [step, actionLeft]);
```

- **76~78줄** 보여 주는 단계가 바뀌면 제목에 초점.
- **80~84줄** wait 단계에서 1초마다 5→0.
- **86~90줄** doing 단계에서 1초마다 대체행동 시간 줄이기.

```tsx
  function finish(
    partial: Omit<
      InterventionInput,
      | "level"
      | "reason"
```

finish — 결과 기록. 이미 기록했거나 이유가 없으면 무시. 받은 일부 값(결과, 차단 등)에 단계, 이유, 메모, 사용 초를 합쳐 onFinish로 넘긴다. `Omit<A, "x">`는 "A에서 x 칸을 뺀 모양", `&`는 두 모양을 합친 것.

```
      | "reasonNote"
      | "usageSeconds"
      | "alternativeAction"
      | "alternativeCompleted"
    > & {
      alternativeAction?: string | null;
      alternativeCompleted?: boolean;
    },
  ) {
    if (locked.current || !reason) return;
    locked.current = true;
    onFinish({
      level,
      reason,
      reasonNote: note.trim(),
      alternativeAction: partial.alternativeAction ?? null,
      alternativeCompleted: partial.alternativeCompleted ?? false,
      reEntered: partial.reEntered,
      blocked: partial.blocked,
      blockDuration: partial.blockDuration,
      usageSeconds,
      outcome: partial.outcome,
    });
  }
```

```
  function exitNow() {
    if (locked.current) return;
    locked.current = true;
    onFinish({
      level,
      reason: reason ?? "선택하지 않음",
      reasonNote: note.trim(),
      alternativeAction: alternative?.label ?? null,
      alternativeCompleted: false,
      reEntered: false,
      blocked: false,
      blockDuration: 0,
      usageSeconds,
      outcome: "exit",
    });
  }
```

exitNow — 바로 나가기. 이유를 안 골랐어도 나갈 수 있다 ("선택하지 않음"으로 기록). 결과는 exit.

```
  const levelTone =
    level === 1 ? "text-level-1" : level === 2 ? "text-level-2" : "text-level-3";

  return (
    <div className="mx-auto flex min-h-dvh w-full max-w-lg flex-col px-5 py-6 md:justify-center md:px-0">
      <p className={cn("text-sm font-medium", levelTone)}>{LEVEL_SUMMARY[level]}</p>
      <p className="mt-1 text-sm text-muted-foreground">
        오늘 누적 {formatClock(usageSeconds)}
```

139~140줄 단계별 글자 색. **143~148줄** 화면 맨 위: 단계 설명(색), "오늘 누적 12:34 · Instagram Reels".

```
        {sourceLabel ? ` · ${sourceLabel}` : ""}
      </p>
```

```
      {step === "reason" ? (
        <section className="mt-6 flex flex-1 flex-col">
          <h1
            ref={headingRef}
            tabIndex={-1}
            className="text-3xl font-semibold tracking-tight outline-none"
          >
            {sourceLabel?.includes("Reels")
              ? "왜 릴스를 실행하셨나요?"
              : "왜 쇼츠를 실행하셨나요?"}
          </h1>
          <p className="mt-2 text-sm leading-6 text-muted-foreground">
            이유를 고르기 전에는 피드가 열리지 않습니다. 나가기는 선택 없이도
            됩니다.
            {sourceLabel
              ? " 나가기, 강제 종료, 시간 차단을 고르면 그 화면을 닫고 ShortsAI로 돌아갑니다."
              : ""}
          </p>
```

reason 단계 (1): 제목 "왜 쇼츠를 실행하셨나요?"(릴스면 "릴스를"). `tabIndex={-1}`은 코드로 초점을 줄 수 있게 하는 표시. 설명: 이유를 고르기 전에는 피드가 열리지 않고, 나가기는 언제나 된다.

```
          <div className="mt-5 space-y-2" role="listbox" aria-label="실행 이유">
            {REASONS.map((item) => {
              const selected = reason === item.id;
              return (
                <button
                  key={item.id}
                  type="button"
                  role="option"
                  aria-selected={selected}
                  onClick={() => setReason(item.id)}
                  className={cn(
                    "w-full rounded-2xl border px-4 py-3 text-left",
                    selected
                      ? "border-primary bg-primary/10"
                      : "border-border bg-card hover:bg-secondary",
                  )}
                >
                  <span className="block text-sm font-medium">{item.id}</span>
                  <span className="mt-0.5 block text-xs text-muted-foreground">
                    {item.hint}
                  </span>
                </button>
              );
            })}
          </div>
          {reason === "기타" ? (
            <Textarea
              className="mt-3 min-h-20"
              value={note}
```

reason 단계 (2) 이유 목록. REASONS(심심해서, 습관적으로, 스트레스 …)를 버튼으로 나열. 고른 것은 테두리 강조(`aria-selected`). "기타"를 고르면 80자까지 적을 수 있는 칸이 나온다.

```
            onChange={(event) => setNote(event.target.value)}
            placeholder="이유를 짧게 적어도 됩니다."
            maxLength={80}
            aria-label="기타 이유"
          />
        ) : null}
```

```
        <div className="mt-5 flex flex-col gap-2">
          <Button
            type="button"
            className="h-12 text-base"
            disabled={!reason}
            onClick={() => {
              if (!reason) return;
              if (level === 1) {
                setLeft(5);
                setStep("wait");
              } else if (level === 2) {
                setAlternative(recommendAlternative(reason));
                setStep("alternative");
              } else {
                setStep("warning");
              }
            }}
          >
            {level === 1 ? "5초 쉬러 가기" : level === 2 ? "대체행동 보기" : "경고 보기"}
          </Button>
          <Button type="button" variant="ghost" className="h-11" onClick={exitNow}>
            나가기
          </Button>
        </div>
      </section>
    ) : null}
```

reason 단계 (3) 다음 버튼. 이유를 골라야 눌린다. 단계에 따라:

- Level 1 → 5초 쉬기(wait), 버튼 이름 "5초 쉬러 가기".
- Level 2 → 이유에 맞는 대체행동을 추천해서 alternative, "대체행동 보기".
- Level 3 → warning, "경고 보기".

그 아래 [나가기].

```
    {step === "wait" || step === "ready" || pauseDone ? (
      <section className="mt-8 flex flex-1 flex-col">
        <h1
          ref={headingRef}
          tabIndex={-1}
          className="text-3xl font-semibold tracking-tight outline-none"
        >
          {shownStep === "wait" ? "한 박자만 쉬어 보세요." : "이제 볼 수 있습니다."}
        </h1>
        <p className="mt-2 text-sm text-muted-foreground">고른 이유 · {reason}</p>
        <p
          className="my-10 text-center text-7xl font-semibold tabular-nums"
          aria-live="polite"
        >
          {shownStep === "wait" ? left : 0}
        </p>
        <p className="text-center text-sm text-muted-foreground">
```

wait / ready 단계 (Level 1). "한 박자만 쉬어 보세요." 큰 숫자 5,4,3,2,1 (`aria-live`로 숫자 변화를 읽어 줌). 시간이 끝나면 "이제 볼 수 있습니다." "그래도 볼지, 여기서 끝낼지 고르세요."

```
          {shownStep === "wait"
            ? "5초가 지나기 전에는 피드가 열리지 않습니다."
            : "그래도 볼지, 여기서 끝낼지 고르세요."}
        </p>
        <div className="mt-auto flex flex-col gap-2 pt-8">
```

```
          {shownStep === "ready" ? (
            <Button
              type="button"
              className="h-12 text-base"
              onClick={() =>
                finish({
                  reEntered: true,
                  blocked: false,
                  blockDuration: 0,
                  outcome: "watch",
                })
              }
            >
              쇼츠 보기
            </Button>
          ) : null}
          <Button type="button" variant="outline" className="h-11" onClick={exitNow}>
            나가기
          </Button>
          <button
            type="button"
            className="h-10 text-sm text-muted-foreground underline-offset-4 hover:underline"
            onClick={() => setStep("reason")}
          >
            이유 다시 고르기
          </button>
        </div>
      </section>
    ) : null}
```

버튼: (시간이 끝난 뒤에만) [쇼츠 보기] → 결과 watch 기록. [나가기]. [이유 다시 고르기]는 처음 단계로.

```
    {step === "alternative" && alternative ? (
      <section className="mt-6 flex flex-1 flex-col">
        <h1
          ref={headingRef}
          tabIndex={-1}
          className="text-3xl font-semibold tracking-tight outline-none"
        >
          보기 전에 하나만 해볼까요.
        </h1>
        <p className="mt-2 text-sm text-muted-foreground">
          {reason}에 맞춰 {alternative.label}을 추천합니다. 다른 행동을 골라도 됩니다.
        </p>
        <div className="mt-5 grid grid-cols-2 gap-2">
          {ALTERNATIVES.map((item) => {
            const Icon = ICONS[item.label as keyof typeof ICONS];
```

alternative 단계 (Level 2) (1). "보기 전에 하나만 해볼까요." "심심해서에 맞춰 잠시 걷기를 추천합니다. 다른 행동을 골라도 됩니다." 대체행동 6개를 2칸 격자 버튼으로 (아이콘, 이름, 안내 초). 누르면 그것으로 바뀐다.

```
              const selected = alternative.id === item.id;
              return (
                <button
                  key={item.id}
                  type="button"
                  onClick={() => setAlternative(item)}
                  className={cn(
                    "rounded-2xl border px-3 py-3 text-left",
                    selected
                      ? "border-primary bg-primary/10"
                      : "border-border bg-card",
                  )}
                  aria-pressed={selected}
                >
                  <Icon className="size-4" />
                  <span className="mt-2 block text-sm font-medium">{item.label}</span>
                  <span className="mt-1 block text-xs text-muted-foreground">
                    안내 {item.seconds}초
                  </span>
                </button>
              );
            })}
          </div>
```

```
          <div className="mt-5 flex flex-col gap-2">
            <Button
              type="button"
              className="h-12"
              onClick={() => {
                setCompleted(false);
                setActionLeft(alternative.seconds);
                setStep("doing");
              }}
            >
              지금 해볼게요
            </Button>
            <Button
              type="button"
              variant="outline"
              className="h-11"
              onClick={() => {
                setCompleted(true);
                setStep("reenter");
              }}
            >
              이미 했어요
            </Button>
            <Button
              type="button"
              variant="ghost"
              className="h-11"
```

alternative 단계 (2) 버튼.

- [지금 해볼게요] → 안내 시간을 걸고 doing으로.
- [이미 했어요] → 완료로 보고 reenter로.
- [건너뛸게요] → 미완료로 reenter로.
- [나가기].

```
              onClick={() => {
                setCompleted(false);
                setStep("reenter");
              }}
            >
              건너뛸게요
            </Button>
            <Button type="button" variant="ghost" className="h-11" onClick={exitNow}>
              나가기
            </Button>
          </div>
        </section>
      ) : null}
```

```
      {step === "doing" && alternative && !actionDone ? (
        <section className="mt-8 flex flex-1 flex-col">
          <h1
            ref={headingRef}
            tabIndex={-1}
            className="text-3xl font-semibold tracking-tight outline-none"
          >
            {alternative.label}
          </h1>
          <p className="mt-3 text-sm leading-6 text-muted-foreground">{alternative.guide}</p>
          <p className="my-8 text-center text-6xl font-semibold tabular-nums" aria-live="polit
e">
            {actionLeft}
          </p>
          <Progress
            value={
              alternative.seconds === 0
                ? 100
                : ((alternative.seconds - actionLeft) / alternative.seconds) * 100
            }
            aria-label="대체행동 진행"
          >
            <ProgressLabel>안내 시간</ProgressLabel>
          </Progress>
          <div className="mt-auto flex flex-col gap-2 pt-8">
            <Button
              type="button"
              className="h-12"
              onClick={() => {
                setCompleted(true);
                setStep("reenter");
              }}
            >
              완료했어요
            </Button>
            <Button type="button" variant="outline" className="h-11" onClick={exitNow}>
              나가기
```

doing 단계. 대체행동 이름, 안내 문장(예: "물 한 컵을 천천히 마셔요"), 큰 숫자 카운트다운, 진행 막대(지난 시간 ÷ 전체 × 100%). [완료했어요]를 누르면 바로 reenter, 시간이 끝나도 자동으로 reenter.

```
            </Button>
          </div>
        </section>
      ) : null}

      {shownStep === "reenter" ? (
        <section className="mt-8 flex flex-1 flex-col">
          <h1
            ref={headingRef}
            tabIndex={-1}
            className="text-3xl font-semibold tracking-tight outline-none"
          >
            {actionDone ? "대체행동을 마쳤습니다." : "대체행동을 건너뛰었습니다."}
          </h1>
          <p className="mt-3 text-sm leading-6 text-muted-foreground">
            {alternative?.label ?? "대체행동"} 이후에도 쇼츠를 볼지 골라 주세요. 이
            선택이 통계의 재실행 여부로 남습니다.
          </p>
          <div className="mt-auto flex flex-col gap-2 pt-8">
            <Button
              type="button"
              className="h-12 text-base"
              onClick={() =>
                finish({
                  alternativeAction: alternative?.label ?? null,
                  alternativeCompleted: actionDone,
                  reEntered: true,
                  blocked: false,
                  blockDuration: 0,
                  outcome: "watch",
                })
              }
            >
              볼래요
            </Button>
            <Button
              type="button"
              variant="outline"
              className="h-11"
              onClick={() =>
                finish({
                  alternativeAction: alternative?.label ?? null,
                  alternativeCompleted: actionDone,
                  reEntered: false,
                  blocked: false,
                  blockDuration: 0,
                  outcome: "exit",
                })
              }
            >
              오늘은 여기까지
```

reenter 단계. "대체행동을 마쳤습니다."(또는 건너뛰었습니다) + "이후에도 쇼츠를 볼지 골라 주세요. 이 선택이 통계의 재실행 여부로 남습니다."

- [볼래요] → 결과 watch, 다시 들어감(reEntered) 참.
- [오늘은 여기까지] → 결과 exit.

둘 다 고른 대체행동과 완료 여부를 함께 기록한다.

```
            </Button>
          </div>
        </section>
      ) : null}
```

```
      {step === "warning" ? (
        <section className="mt-6 flex flex-1 flex-col rounded-3xl bg-[oklch(0.22_0.02_45)] px-5
py-6 text-[oklch(0.96_0.012_85)]">
          <h1
            ref={headingRef}
            tabIndex={-1}
            className="text-3xl font-semibold tracking-tight outline-none"
          >
            오늘은 여기까지 해볼까요?
          </h1>
          <p className="mt-3 text-sm leading-6 text-[oklch(0.8_0.02_80)]">
            오늘 벌써 {formatClock(usageSeconds)} 봤어요. 한 개만 더 보려다 30분이 지나기
            쉬워요. 지금 멈추거나, 잠시 쇼츠를 잠가 둘 수 있어요.
          </p>
```

warning 단계 (Level 3). 어두운 배경(`oklch(...)`는 색을 밝기·채도·색상으로 적는 방식). "오늘은 여기까지 해볼까요?" "오늘 벌써 45:00 봤어요. 한 개만 더 보려다 30분이 지나기 쉬워요 …"

```
          <div className="mt-6 flex flex-col gap-2">
            <Button
              type="button"
              className="h-12 bg-background text-foreground hover:bg-background/90"
              onClick={() =>
                finish({
                  alternativeAction: null,
                  reEntered: false,
                  blocked: true,
                  blockDuration: minutesUntilDayEnd(dayEndHour),
                  outcome: "day-end",
                })
              }
            >
              오늘은 끝 · 내일 {dayEndHour}시까지
            </Button>
            <div className="grid grid-cols-2 gap-2">
              {BLOCK_MINUTES.map((minutes) => (
                <Button
                  key={minutes}
                  type="button"
                  variant="outline"
                  className="h-11 border-white/20 bg-transparent text-[oklch(0.96_0.012_85)] hov
er:bg-white/10"
                  onClick={() => {
                    setBlockMinutes(minutes);
                    setStep("confirm-block");
                  }}
                >
                  {minutes === 60 ? "1시간" : `${minutes}분`} 차단
                </Button>
```

warning 버튼.

- [오늘은 끝 · 내일 6시까지] → 내일 하루 끝 시각까지 차단(day-end).
- BLOCK_MINUTES(예: 10·30·60분)마다 [N분 차단] → 바로 막지 않고 확인 단계로.
- [지금은 나가기] → 차단 없이 나가기.

```
            ))}
          </div>
          <Button
            type="button"
            variant="ghost"
            className="h-11 text-[oklch(0.9_0.01_85)] hover:bg-white/10"
            onClick={exitNow}
          >
            지금은 나가기
          </Button>
        </div>
      </section>
    ) : null}
```

```
    {step === "confirm-block" && blockMinutes ? (
      <section className="mt-8 flex flex-1 flex-col">
        <h1
          ref={headingRef}
          tabIndex={-1}
          className="text-3xl font-semibold tracking-tight outline-none"
        >
          {blockMinutes === 60 ? "1시간" : `${blockMinutes}분`} 동안 막을까요?
        </h1>
        <p className="mt-3 text-sm leading-6 text-muted-foreground">
          이 시간이 끝나기 전에는 쇼츠 피드가 열리지 않습니다. 차단 시간은 실제
          분입니다.
        </p>
        <div className="mt-auto flex flex-col gap-2 pt-8">
          <Button
            type="button"
            className="h-12"
            onClick={() =>
              finish({
                alternativeAction: null,
                reEntered: false,
                blocked: true,
                blockDuration: blockMinutes,
                outcome: "timed-block",
              })
            }
          >
            {blockMinutes === 60 ? "1시간" : `${blockMinutes}분`} 동안 막기
          </Button>
          <Button
            type="button"
            variant="outline"
            className="h-11"
            onClick={() => setStep("warning")}
          >
            다시 고르기
          </Button>
```

confirm-block 단계. "30분 동안 막을까요?" "이 시간이 끝나기 전에는 쇼츠 피드가 열리지 않습니다. 차단 시간은 실제 분입니다." [30분 동안 막기] → timed-block 기록, [다시 고르기] → warning으로. 차단은 **사용자가 직접 확인**해야 걸린다 (강제하지 않기).

```
          </div>
        </section>
      ) : null}
    </div>
  );
}
```

watch-view.tsx — 개입 화면과 연습 피드 (/watch) `components/watch-view.tsx (315줄)`

/watch 주소의 화면. ① 잠금 중이면 잠금 안내, ② 아니면 개입 단계(InterventionFlow), ③ 연습에서 [쇼츠 보기]를 고르면 가짜 피드(Player)를 보여 준다. 서비스가 넘긴 개입(Level 3, Instagram)도 여기서 처리하고 결과를 Java에 돌려준다.

```
"use client";

import { useEffect, useRef, useState } from "react";
import { useRouter } from "next/navigation";
import { Button } from "@/components/ui/button";
import { InterventionFlow } from "@/components/intervention-flow";
import { useStore } from "@/components/store";
import { useNow } from "@/components/use-now";
import { CLIPS } from "@/lib/clips";
import {
  guardSourceLabel,
  HanbakjaGuard,
  isGuardTarget,
  isNativeAndroid,
  type GuardTarget,
} from "@/lib/guard";
import {
  blockRemainingSeconds,
  formatClock,
  isBlocked,
  levelForUsage,
  todayKey,
  usageSecondsOn,
} from "@/lib/logic";
import type { InterventionInput, Level } from "@/lib/types";

/** 서비스가 감지 순간 정한 Level과 누적 초. 웹 기록보다 앞서 있을 수 있어 이 값을 쓴다. */
type NativeEntry = { target: GuardTarget; level: Level | null; usageSeconds: number };
```

3~25줄 가져오기: React, 화면 이동, 버튼, 개입 단계 부품, 창고, 1초 시계, 연습 문구, 플러그인, 계산 함수.

27~28줄 NativeEntry: 서비스가 감지 순간 정한 대상, 단계, 누적 초. 웹 기록은 1.5초마다 받아서 조금 늦을 수 있으므로 서비스 값이 더 정확하다.

```
export function WatchView() {
  const router = useRouter();
  const store = useStore();
  const now = useNow();
  const [phase, setPhase] = useState<"gate" | "play">("gate");
  const [entryUsage, setEntryUsage] = useState<number | null>(null);
  const [entry, setEntry] = useState<NativeEntry | null>(null);
```

WatchView 기억하는 값: phase(gate = 개입 중, play = 연습 피드 보는 중), 들어올 때의 사용 초, 서비스 개입 정보. **37줄** 잠금 중 열기를 한 번만 기록하는 표시, **38줄** 서비스 정보의 최신 칸.

```
const recordedBlock = useRef(false);
const entryRef = useRef<NativeEntry | null>(null);
```

```
async function resolveEntry(): Promise<NativeEntry | null> {
  if (entryRef.current) return entryRef.current;
  if (!isNativeAndroid()) return null;
  try {
    const status = await HanbakjaGuard.getStatus();
    if (!isGuardTarget(status.pendingTarget)) return null;
    const level =
      status.pendingLevel === 1 || status.pendingLevel === 2 || status.pendingLevel === 3
        ? (status.pendingLevel as Level)
        : null;
    const next = {
      target: status.pendingTarget,
      level,
      usageSeconds: status.pendingUsageSeconds,
    };
    entryRef.current = next;
    setEntry(next);
    return next;
  } catch {
    return null;
  }
}
```

resolveEntry — 서비스가 넘긴 개입 읽기. 이미 읽었으면 그대로. 폰이 아니면 없음. getStatus로 대상(youtube/instagram)과 단계(1~3만 인정), 누적 초를 읽어 저장한다. 실패하면 없음.

```
useEffect(() => {
  // 개입 화면이 열릴 때 서비스가 넘긴 대상을 한 번 읽는다 (플러그인 응답 뒤 상태를 바꾼다).
  // eslint-disable-next-line react-hooks/set-state-in-effect
  void resolveEntry();
}, []);

if (store.data && entryUsage === null) {
  setEntryUsage(usageSecondsOn(store.data, todayKey()));
}

const blocked = store.data ? isBlocked(store.data.block, now) : false;
const guardTarget = entry?.target ?? null;
```

- **63~67줄** 화면이 열릴 때 한 번 읽는다.
- **69~71줄** 들어올 때의 오늘 사용 초를 **한 번만** 정해 둔다 (그 뒤 시간이 늘어도 단계가 흔들리지 않게).
- **73~74줄** 잠금 중인지, 서비스 대상.

```
// 연습 피드(브라우저)에서 차단 중에 연 경우. 실제 쇼츠는 서비스가 잠금 화면으로 처리한다.
useEffect(() => {
  if (!store.data || !blocked || phase === "play" || recordedBlock.current) return;
  if (isNativeAndroid() && entryRef.current) return;
  recordedBlock.current = true;
  store.recordLockAttempt("practice", blockRemainingSeconds(store.data.block));
}, [blocked, phase, store]);
```

연습 화면에서 잠금 중에 연 경우 잠금 시도를 한 번 기록한다. 실제 쇼츠는 Java 서비스가 잠금 화면과 기록을 처리하므로 여기서는 빼고 센다.

```
const leftRealScreen = useRef(false);
useEffect(() => {
  if (!blocked || phase === "play" || leftRealScreen.current || !isNativeAndroid()) return;
  let cancelled = false;
  void resolveEntry().then(async (found) => {
```

서비스가 넘긴 개입인데 이미 잠금 중이면 Java에 "leave(나감)"로 결과를 알려, 서비스가 대기 중인 개입을 끝내게 한다 (한 번만). `cancelled`는 화면이 사라진 뒤 결과가 와도 무시하는 표시.

```
      if (cancelled || leftRealScreen.current || !found) return;
      leftRealScreen.current = true;
      await HanbakjaGuard.finish({ outcome: "leave", target: found.target }).catch(() => undefin
ed);
    });
    return () => {
      cancelled = true;
    };
  }, [blocked, phase]);
```

```
  if (!store.data || entryUsage === null) {
    return (
      <div className="flex min-h-dvh items-center justify-center px-6" role="status">
        <p className="text-sm text-muted-foreground">개입 화면을 준비하고 있습니다.</p>
      </div>
    );
  }
```

데이터를 아직 못 읽었으면 "개입 화면을 준비하고 있습니다."

```
  if (blocked && phase !== "play") {
    const remaining = blockRemainingSeconds(store.data.block, now);
    return (
      <div className="mx-auto flex min-h-dvh w-full max-w-lg flex-col justify-center px-5">
        <p className="text-sm font-medium text-destructive">쇼츠 쉬는 중</p>
        <h1 className="mt-2 text-3xl font-semibold tracking-tight">
          {guardTarget === "instagram"
            ? "지금은 릴스를 열 수 없습니다."
            : "지금은 쇼츠를 열 수 없습니다."}
        </h1>
        <p className="mt-3 text-sm leading-6 text-muted-foreground">
          선택한 차단이 끝나기 전에는 피드가 시작되지 않습니다.
        </p>
        <p className="my-8 text-5xl font-semibold tabular-nums">{formatClock(remaining)}</p>
        <Button type="button" className="h-12" onClick={() => router.push("/")}>
          홈으로
        </Button>
      </div>
    );
  }
```

잠금 안내 화면. "쇼츠 쉬는 중", "지금은 쇼츠를 열 수 없습니다."(릴스면 릴스), 설명, 큰 남은 시간 "24:13" (1초마다 줄어듦), [홈으로].

```
  if (phase === "play") {
    return <Player />;
  }

  const usage = entry?.level ? entry.usageSeconds : entryUsage;
  const level: Level = entry?.level ?? levelForUsage(entryUsage, store.data.settings);
```

연습 피드를 보는 중이면 Player를 그린다.

131~132줄 쓸 사용 초와 단계: 서비스가 단계를 정해 넘겼으면 그 값을, 아니면 웹 기록으로 계산.

```
function finish(input: InterventionInput) {
  const id = store.recordIntervention({
    ...input,
    target: entryRef.current?.target ?? (isNativeAndroid() ? null : "practice"),
  });
  const blocking = input.outcome === "timed-block" || input.outcome === "day-end";
  if (blocking) {
    store.beginBlock(input.blockDuration);
  }
  if (input.outcome === "watch" && !entryRef.current) {
    store.startWatching(input.level, id);
  }
```

finish — 개입 결과 처리 (1).

1. 개입 기록을 저장 (대상: 서비스 대상, 아니면 연습 "practice").
2. 차단(시간 차단, 오늘은 끝)이면 화면 쪽 잠금 시작.
3. 연습에서 [보기]면 연습 시청을 시작.

```
  void (async () => {
    const found = await resolveEntry();
    if (found) {
      const outcome =
        input.outcome === "watch" ? "watch" : blocking ? "block" : "leave";
      try {
        await HanbakjaGuard.finish({
          outcome,
          target: found.target,
          blockMinutes: blocking ? input.blockDuration : undefined,
          dayEnd: input.outcome === "day-end",
        });
      } catch {
        // Still leave the intervention screen if the bridge is unavailable.
      }
      router.push("/");
      return;
    }
    if (blocking && isNativeAndroid()) {
      await HanbakjaGuard.startBlock({ minutes: input.blockDuration }).catch(() => undefined);
    }
    if (input.outcome === "watch") {
      setPhase("play");
      return;
    }
    router.push("/");
  })();
}
```

finish (2) Java에 알리기.

- **서비스가 넘긴 개입이면**: 결과를 watch/block/leave로 바꿔 finish로 보낸다 (차단 분, 오늘은 끝 여부 포함). Java가 YouTube로 돌려보내거나 닫는다. 실패해도 홈으로 이동.
- **연습인데 폰 앱이면**: 차단을 Java에도 건다(startBlock) — 실제 쇼츠도 같이 잠기게.
- [보기]면 연습 피드로, 그 밖에는 홈으로.

`void (async () => {...})()`는 기다리는 작업을 바로 실행하되 결과를 기다리지 않는 표현.

```
const sourceLabel = guardTarget ? guardSourceLabel(guardTarget) : null;

return (
  <InterventionFlow
    level={level}
    usageSeconds={usage}
    onFinish={finish}
    sourceLabel={sourceLabel}
    dayEndHour={store.data.settings.dayEndHour}
```

대상 이름("Instagram Reels" 등)과 함께 개입 단계 부품을 그린다. 하루 끝 시각은 설정 값.

```
    />
  );
}
```

```
function Player() {
  const router = useRouter();
  const store = useStore();
  const [index, setIndex] = useState(0);
  const [clipMs, setClipMs] = useState(0);
  const [paused, setPaused] = useState(false);
  const clipMsRef = useRef(0);
  const storeRef = useRef(store);

  useEffect(() => {
    storeRef.current = store;
  });
```

Player — 연습 피드. 지금 영상 번호, 영상 안 경과(ms), 일시정지 여부.
194~199줄 타이머 안에서 최신 창고를 쓰려고 매번 기억 칸에 넣는다.

```
  useEffect(() => {
    storeRef.current.cancelDetach();
    const id = window.setInterval(() => {
      const visible = document.visibilityState === "visible";
      setPaused(!visible);
      storeRef.current.flushWatch(visible);
    }, 500);
    const onVisible = () => {
      const visible = document.visibilityState === "visible";
      setPaused(!visible);
      storeRef.current.flushWatch(visible);
    };
    document.addEventListener("visibilitychange", onVisible);
    return () => {
      window.clearInterval(id);
      document.removeEventListener("visibilitychange", onVisible);
      storeRef.current.scheduleDetach();
    };
  }, []);
```

시청 시간 쌓기. 들어오면 예약된 "닫기"를 취소. 0.5초마다(그리고 탭 전환 때) 탭이 보이는지 확인해 **보일 때만** 시간을 더한다(flushWatch). 화면을 떠나면 0.4초 뒤 마감 예약(scheduleDetach).

```
  useEffect(() => {
    const id = window.setInterval(() => {
      if (document.visibilityState !== "visible") return;
      clipMsRef.current += 250;
      if (clipMsRef.current >= 8000) {
        clipMsRef.current = 0;
        setClipMs(0);
        setIndex((value) => (value + 1) % CLIPS.length);
        storeRef.current.advanceClip();
        return;
      }
      setClipMs(clipMsRef.current);
    }, 250);
```

영상 넘기기. 0.25초마다 경과를 늘리고, **8초**가 되면 다음 연습 문구로(끝이면 처음으로) 넘기고 영상 수 +1. 탭이 안 보이면 멈춘다.

```
    return () => window.clearInterval(id);
  }, []);
```

```
  const data = store.data;
  const session = data?.activeSession;
  const today = todayKey();
  const total = data ? usageSecondsOn(data, today) : 0;
  const liveLevel = data ? levelForUsage(total, data.settings) : 1;
  const clip = CLIPS[index] ?? CLIPS[0];

  function leave() {
    store.stopWatching();
    router.push("/");
  }

  function nextClip() {
    clipMsRef.current = 0;
    setClipMs(0);
    setIndex((value) => (value + 1) % CLIPS.length);
    store.advanceClip();
  }
```

오늘 합계, 지금 단계, 지금 문구를 계산. **leave**: [시청 종료] → 마감 후 홈. **nextClip**: [다음 쇼츠] → 바로 다음 문구, 영상 수 +1.

```
  return (
    <div className="min-h-dvh bg-[#14110e] text-[#f6f1e7] md:grid md:place-items-center">
      <div className="mx-auto flex min-h-dvh w-full max-w-md flex-col md:min-h-[820px] md:overfl
ow-hidden md:rounded-[28px] md:ring-1 md:ring-white/10">
        <header className="flex items-start justify-between gap-3 px-4 pt-4">
          <div>
            <p className="text-xs text-white/60">시뮬레이션 피드 · 실제 YouTube 아님</p>
            <p className="mt-1 text-sm tabular-nums">
              이번 시청 {formatClock(session?.duration ?? 0)}
            </p>
            <p className="text-xs text-white/60 tabular-nums">오늘 {formatClock(total)}</p>
          </div>
          <Button
            type="button"
            variant="outline"
            className="h-10 border-white/20 bg-transparent text-[#f6f1e7] hover:bg-white/10"
            onClick={leave}
          >
            시청 종료
          </Button>
        </header>
        {session && liveLevel !== session.levelAtStart ? (
          <p className="mx-4 mt-3 rounded-xl bg-white/10 px-3 py-2 text-xs leading-5">
            사용량이 Level {liveLevel} 구간에 들어왔습니다. 이번 영상은 이어서 볼 수
            있고, 다음에 열면 Level {liveLevel} 개입이 시작됩니다.
          </p>
        ) : null}
```

피드 화면 (1). 어두운 배경. 위에 "시뮬레이션 피드 · 실제 YouTube 아님", 이번 시청 시간, 오늘 합계, [시청 종료]. 보는 중에 다음 단계로 넘어가면 "사용량이 Level 2 구간에 들어왔습니다. 이번 영상은 이어서 볼 수 있고, 다음에 열면 Level 2 개입이 시작됩니다."

```
        <div className="flex flex-1 items-center px-4 py-6">
          <article
            className="relative flex aspect-[9/16] w-full max-h-[68dvh] flex-col justify-end overflow-hidden rounded-3xl p-5"
            style={{ backgroundImage: `linear-gradient(160deg, ${clip.from}, ${clip.to})` }}
          >
            <div className="absolute inset-x-0 top-0 h-1 bg-white/15">
              <div
                className="h-full bg-white"
                style={{ width: `${(clipMs / 8000) * 100}%` }}
              />
            </div>
            <p className="text-xs text-white/70">쇼츠 {index + 1}</p>
            <h1 className="mt-2 text-2xl font-semibold">{clip.title}</h1>
            <p className="mt-2 text-sm leading-6 text-white/80">{clip.caption}</p>
          </article>
        </div>
        {paused ? (
          <p className="px-4 pb-2 text-center text-xs text-white/70" role="status">
            이 탭을 보고 있을 때만 사용 시간이 쌓입니다. 지금은 일시정지입니다.
          </p>
        ) : null}
        <div className="px-4 pb-6">
          <Button
            type="button"
            className="h-12 w-full bg-[#f6f1e7] text-[#14110e] hover:bg-white"
            onClick={nextClip}
          >
            다음 쇼츠
          </Button>
        </div>
      </div>
    </div>
  );
}
```

피드 화면 (2). 세로 9:16 카드에 그라데이션 배경, 위에 8초 진행 막대, "쇼츠 3", 제목, 설명 문장. 탭이 안 보이면 "이 탭을 보고 있을 때만 사용 시간이 쌓입니다." 아래 [다음 쇼츠].

cards-panel.tsx — 내 공부 카드 (설정 화면) `components/cards-panel.tsx (332줄)`

설정 화면 안의 "내 공부 카드" 칸. 기능 설명, 공부 모드 스위치, 직접 만들기, ChatGPT·Gemini 결과 붙여넣기, 방금 가져온 문제 확인, 카드 목록·삭제를 담당한다. 카드는 폰 안에만 저장되고 서버는 없다.

```
"use client";

import { useState } from "react";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
```

3~12줄 가져오기: 입력 부품들, 창고, policy.ts의 **질문 문장(CARD_PROMPT)**과 **붙여넣기 검사(parseCardImport)**.

14줄 빈 보기 4칸. **16줄** 권장 카드 수 30장 (사용자 요청: 오래 쓰려면 10장보다 많이).

```
import { Switch } from "@/components/ui/switch";
import { Textarea } from "@/components/ui/textarea";
import { useStore } from "@/components/store";
import { CARD_PROMPT, parseCardImport } from "@/lib/policy";
import type { StudyCard } from "@/lib/types";
import { cn } from "@/lib/utils";

const EMPTY_OPTIONS = ["", "", "", ""];
/** 공부 모드에서 이보다 적으면 같은 문제가 자주 나올 수 있다고 알려 준다. */
const RECOMMENDED_CARDS = 30;
```

```
/** 가져오기 결과 알림. 문구는 사용자 확인 완료 (2026-10-13). */
function importMessage(result: ReturnType<typeof parseCardImport>): string {
  if (result.error) return result.error;
  let line = `${result.cards.length}개를 가져왔어요.`;
  if (result.skipped > 0) line += ` 형식이 이상한 ${result.skipped}개는 건너뛰었어요.`;
  if (result.duplicates > 0) line += ` 이미 있는 문제 ${result.duplicates}개는 넣지 않았어요.`;
  if (result.suspectIds.length > 0) {
    line += ` 해설이 정답과 맞지 않아 보이는 문제 ${result.suspectIds.length}개에 표시했어요. 아래에서 확인해 주세요.`;
  }
  return line;
}
```

importMessage — 가져오기 결과 문장. 오류면 오류 문장. 아니면 "12개를 가져왔어요." 뒤에 필요한 것만 붙인다: 형식이 이상해서 건너뛴 수, 이미 있어서 안 넣은 수, 해설이 정답과 안 맞아 보여 표시한 수. `ReturnType<typeof f>`는 "f가 돌려주는 값의 모양".

```
/**
 * 내 공부 카드. 쇼츠 개입에서 문제 1개로 나온다 (공부 모드면 매번).
 * 직접 입력하거나, ChatGPT·Gemini 무료 채팅에서 만든 결과를 붙여넣는다 (서버 없음).
 */
export function CardsPanel() {
  const { data, addCards, deleteCard, saveSettings } = useStore();
  const [question, setQuestion] = useState("");
  const [options, setOptions] = useState<string[]>(EMPTY_OPTIONS);
  const [answer, setAnswer] = useState(0);
  const [explanation, setExplanation] = useState("");
  const [pasted, setPasted] = useState("");
  const [message, setMessage] = useState<string | null>(null);
  const [confirmDelete, setConfirmDelete] = useState<string | null>(null);
  const [showList, setShowList] = useState(false);
  /** 방금 가져온 문제 (확인 목록). 의심 문제 id는 따로. */
  const [review, setReview] = useState<{ ids: string[]; suspect: string[] } | null>(null);

  if (!data) return null;
  const settings = data.settings;
  const cards = data.cards;
  const reviewCards: StudyCard[] = review
    ? review.ids.map((id) => cards.find((card) => card.id === id)).filter((card): card is StudyCard => !!card)
    : [];
```

CardsPanel 기억하는 값: 직접 만들기의 문제·보기 4개·정답 번호·해설, 붙여넣은 글, 알림 문장, 지우기 확인 중인 카드, 목록 펼침, 방금 가져온 문제 목록(의심 표시 id 포함).

50~52줄 방금 가져온 id들로 실제 카드를 찾는다 (그새 지운 것은 빠짐). `card is StudyCard`는 "이 필터를 지나면 빈 값이 없다"고 TypeScript에 알려 주는 표시.

```
  function addManual() {
    const filled = options.map((o) => o.trim());
```

addManual — 직접 만든 카드 추가. 문제가 비었거나 보기 중 빈 칸이 있으면 안내만 하고 멈춘다. 아니면 새 id, 앞뒤 공백을 지운 글, 정답 번호, 만든 시각,

```
    if (!question.trim()) {
      setMessage("문제를 적어 주세요.");
      return;
    }
    if (filled.some((o) => !o)) {
      setMessage("보기 4개를 모두 적어 주세요.");
      return;
    }
    addCards([
      {
        id: crypto.randomUUID(),
        question: question.trim(),
        options: filled,
        answer,
        createdAt: new Date().toISOString(),
        source: "manual",
        shown: 0,
        correct: 0,
        explanation: explanation.trim(),
      },
    ]);
    setQuestion("");
    setOptions(EMPTY_OPTIONS);
    setAnswer(0);
    setExplanation("");
    setMessage("카드를 추가했어요.");
  }
```

출처 "manual", 횟수 0으로 카드를 저장하고 입력 칸을 비운다.

```
  function importPasted() {
    const result = parseCardImport(pasted, undefined, undefined, cards);
    if (result.cards.length > 0) {
      addCards(result.cards);
      setPasted("");
      setReview({ ids: result.cards.map((card) => card.id), suspect: result.suspectIds });
    }
    setMessage(importMessage(result));
  }
```

importPasted — 붙여넣은 문제 가져오기. parseCardImport가 글을 검사한다(JSON 형식, 보기 4개, 정답 번호, "모두 정답" 같은 보기 거르기, 중복, 해설-정답 불일치 의심). 통과한 카드만 저장하고, **방금 가져온 문제** 확인 목록을 연다.

```
  async function copyPrompt() {
    try {
      await navigator.clipboard.writeText(CARD_PROMPT);
      setMessage("질문 문장을 복사했어요. ChatGPT나 Gemini에 붙여넣고, 공부 자료를 붙이거나 PDF 파일을 첨부해 주세요.");
    } catch {
      setMessage("복사하지 못했어요. 아래 문장을 길게 눌러 직접 복사해 주세요.");
    }
  }
```

copyPrompt — 질문 문장 복사. 클립보드에 CARD_PROMPT(ChatGPT·Gemini에 붙일 "이 자료로 4지선다 문제를 JSON으로 만들어 줘" 문장)를 넣는다. 실패하면 직접 복사하라고 안내.

```
  return (
    <section className="space-y-4 rounded-3xl bg-card p-5 ring-1 ring-foreground/10">
      <div className="space-y-3">
```

자세한 설명 (사용자 확인 완료 문구). 네 덩어리:

```
        <h2 className="text-base font-medium">내 공부 카드</h2>
        <div className="space-y-3 text-sm leading-6 text-muted-foreground">
          <div>
            <p className="font-medium text-foreground">쇼츠 대신 문제 하나</p>
            <p>
              쇼츠를 열려는 순간, 내 공부 자료로 만든 문제를 하나 풀고 넘어가게 해 주는 기능이에요. 카드는
              이 폰 안에만 저장돼요.
            </p>
          </div>
          <div>
            <p className="font-medium text-foreground">어떻게 나오나요?</p>
            <ul className="list-disc space-y-1 pl-5">
              <li>평소에는 다른 활동(숨 고르기, 호흡 등)과 번갈아 가끔 나와요.</li>
              <li>공부 모드를 켜면 쇼츠를 열 때마다 문제가 나와요.</li>
              <li>
                틀린 문제는 더 자주, 맞힌 문제는 점점 드물게 나와요. 같은 문제는 몇 시간 안에 다시 나오지
                않아요.
              </li>
            </ul>
          </div>
          <div>
            <p className="font-medium text-foreground">어떤 자료가 좋나요?</p>
            <p>
              시험 범위 강의 자료, 교재 PDF, 단어장처럼 외우거나 이해해야 하는 내용이 좋아요. 범위를 나눠 여러
              번 만들어 30장 이상 쌓아 두면 좋아요.
            </p>
          </div>
          <div>
            <p className="font-medium text-foreground">문제가 이상하면?</p>
            <p>
              문제 아래 &quot;이 문제 이상해요&quot;를 누르면 빠져요. 아래 목록에서 직접 지울 수도 있어요.
            </p>
          </div>
        </div>
        <p className="text-sm">지금 {cards.length}장 있어요.</p>
      </div>
```

- 쇼츠 대신 문제 하나: 쇼츠를 열려는 순간 문제를 하나 풀고 넘어간다, 카드는 폰 안에만.
- 어떻게 나오나요: 평소엔 다른 활동과 번갈아, 공부 모드면 매번, 틀린 문제는 더 자주·맞힌 문제는 드물게, 같은 문제는 몇 시간 안에 다시 안 나옴.
- 어떤 자료가 좋나요: 강의 자료, 교재 PDF, 단어장. 범위를 나눠 30장 이상.
- 문제가 이상하면: "이 문제 이상해요"를 누르면 빠짐.

그리고 "지금 N장 있어요." `"`는 큰따옴표 글자.

```
      <div className="flex items-start justify-between gap-4 rounded-2xl bg-background px-4 py-3 ring-1 ring-foreground/10">
        <div className="space-y-1">
          <Label htmlFor="study-mode" className="text-base font-medium">
            공부 모드
          </Label>
          <p className="text-sm leading-6 text-muted-foreground">
            켜면 쇼츠를 열 때 숨 고르기나 다른 활동 대신 내 공부 카드 문제가 나와요. 카드가 없으면 평소대로
            해요.
```

공부 모드 스위치. 설명: 켜면 다른 활동 대신 공부 카드 문제가 나오고, 카드가 없으면 평소대로. 공부 모드가 켜져 있는데 카드가 30장보다 적으면 "같은 문제가 자주 나올 수 있어요" 안내. 스위치를 바꾸면 설정에 저장 → 다음 sync 때 정책표에 실려 Java로 간다.

```
          </p>
          {settings.studyMode && cards.length < RECOMMENDED_CARDS ? (
            <p className="text-sm leading-6" role="status">
              카드가 {cards.length}장이라 같은 문제가 자주 나올 수 있어요. 오래 쓰려면 {RECOMMENDED_CARDS}장 이상
              만들어 두면 좋아요.
            </p>
          ) : null}
        </div>
        <Switch
          id="study-mode"
          checked={settings.studyMode}
          aria-label="공부 모드"
          onCheckedChange={(checked) => saveSettings({ ...settings, studyMode: Boolean(checked)})}
        />
      </div>
```

```
      <div className="space-y-2">
        <Label htmlFor="card-question">직접 만들기</Label>
        <Textarea
          id="card-question"
          value={question}
          maxLength={300}
          placeholder="문제"
          onChange={(event) => setQuestion(event.target.value)}
        />
        <div className="space-y-2" role="radiogroup" aria-label="정답 고르기">
          {options.map((option, index) => (
            <div key={index} className="flex items-center gap-2">
              <button
                type="button"
                role="radio"
                aria-checked={answer === index}
                aria-label={`${index + 1}번을 정답으로`}
                onClick={() => setAnswer(index)}
                className={cn(
                  "flex size-9 shrink-0 items-center justify-center rounded-full border text-sm",
                  answer === index ? "border-primary bg-primary text-primary-foreground" : "border-border",
                )}
              >
                {index + 1}
              </button>
              <Input
                value={option}
                maxLength={120}
                placeholder={`보기 ${index + 1}`}
                onChange={(event) =>
                  setOptions(options.map((o, i) => (i === index ? event.target.value : o)))
```

직접 만들기. 문제 칸(300자), 보기 4줄 — 각 줄 왼쪽 동그라미 버튼을 눌러 정답을 고른다(`role="radio"`로 화면 읽기 도구에도 "하나만 고르는 버튼"임을 알림). 보기 120자, 해설 500자(선택, 문제를 푼 뒤 보여 줌). [카드 추가].

197줄 보기 하나를 바꿀 때, 바꾼 칸만 새 글로 하고 나머지는 그대로 둔 새 목록을 만든다.

```
                }
              />
            </div>
          ))}
        </div>
        <p className="text-xs text-muted-foreground">동그라미를 눌러 정답을 고르세요.</p>
        <Textarea
          value={explanation}
          maxLength={500}
          placeholder="해설 (선택) · 문제를 푼 뒤에 보여 줘요"
          onChange={(event) => setExplanation(event.target.value)}
        />
        <Button type="button" variant="outline" className="h-11 w-full" onClick={addManual}>
          카드 추가
        </Button>
      </div>
```

ChatGPT·Gemini로 한 번에 만들기 (무료). 순서 3단계 안내, "자료가 길면 범위를 나눠서" 팁, [질문 문장 복사], 붙여넣기 칸(예시 JSON이 흐리게 보임), [붙여넣은 문제 가져오기](비어 있으면 안 눌림). 그 아래 결과 알림 문장.

```
      <div className="space-y-2">
        <Label htmlFor="card-import">ChatGPT·Gemini로 한 번에 만들기 (무료)</Label>
        <ol className="list-decimal space-y-1 pl-5 text-sm leading-6 text-muted-foreground">
          <li>아래 버튼으로 질문 문장을 복사해요.</li>
          <li>ChatGPT나 Gemini 무료 채팅에 붙여넣고, 공부 자료를 붙이거나 PDF 파일을 첨부해요.</li>
          <li>나온 결과를 통째로 복사해서 아래 칸에 붙여넣어요.</li>
        </ol>
        <p className="text-sm leading-6 text-muted-foreground">
          자료가 길면 &amp;quot;3장만&amp;quot;, &amp;quot;1~10쪽만&amp;quot;처럼 범위를 나눠서 여러 번 만들면 문제가 고르게 나와요.
        </p>
        <Button type="button" variant="outline" className="h-10" onClick={() => void copyPrompt()}>
          질문 문장 복사
        </Button>
        <Textarea
          id="card-import"
          className="min-h-28 font-mono text-xs"
          value={pasted}
          placeholder='[{"question":"...","options":["...","...","...","..."],"answer":1}]'
          onChange={(event) => setPasted(event.target.value)}
        />
        <Button
          type="button"
          className="h-11 w-full"
          disabled={!pasted.trim()}
          onClick={importPasted}
        >
          붙여넣은 문제 가져오기
        </Button>
      </div>

      {message ? (
        <p className="text-sm" role="status">
```

```
          {message}
        </p>
      ) : null}
```

```
      {review && reviewCards.length > 0 ? (
        <div className="space-y-2">
          <p className="text-sm font-medium">방금 가져온 문제</p>
          <ul className="space-y-2">
            {reviewCards.map((card) => (
              <li key={card.id} className="rounded-2xl bg-background px-4 py-3 text-sm ring-1 ri
ng-foreground/10">
                {review.suspect.includes(card.id) ? (
                  <p className="mb-1 text-xs font-medium text-destructive">해설이 정답과 달라 보여
요</p>
                ) : null}
                <p className="font-medium">{card.question}</p>
                <p className="mt-1 text-xs text-muted-foreground">정답: {card.options[card.answe
r]}</p>
                {card.explanation ? (
                  <p className="mt-1 text-xs text-muted-foreground">해설: {card.explanation}</p>
                ) : null}
                <Button
                  type="button"
                  variant="outline"
                  className="mt-2 h-9"
                  onClick={() => deleteCard(card.id)}
                >
                  지우기
                </Button>
              </li>
            ))}
          </ul>
          <Button type="button" variant="ghost" className="h-10" onClick={() => setReview(null)}
>
            확인했어요
          </Button>
        </div>
      ) : null}
```

방금 가져온 문제 확인. 각 문제: (의심이면) 빨간 "해설이 정답과 달라 보여요", 문제, 정답, 해설, [지우기]. [확인했어요]로 목록 닫기. AI가 만든 문제를 사용자가 **한 번 훑어보게** 하는 안전장치.

```
      {cards.length > 0 ? (
        <div className="space-y-2">
          <Button type="button" variant="ghost" className="h-10" onClick={() => setShowList(!sho
wList)}>
            {showList ? "카드 목록 닫기" : `카드 목록 보기 (${cards.length})`}
          </Button>
          {showList ? (
            <ul className="space-y-2">
              {cards.map((card) => (
                <li key={card.id} className="rounded-2xl bg-background px-4 py-3 text-sm ring-1
ring-foreground/10">
                  <p className="font-medium">{card.question}</p>
```

카드 목록. [카드 목록 보기 (N)]로 펼치기/닫기. 각 카드: 문제, 정답 · 나온 횟수 · 맞힘, 해설. [이 카드 지우기] → 그 자리에서 [지우기] [취소] 확인 (실수로 지우지 않게 두 번 누르기).

```
                  <p className="mt-1 text-xs text-muted-foreground">
                    정답: {card.options[card.answer]} · 나온 횟수 {card.shown} · 맞힘 {card.correct}
                  </p>
                  {card.explanation ? (
                    <p className="mt-1 text-xs text-muted-foreground">해설: {card.explanation}</p>
                  ) : null}
                  {confirmDelete === card.id ? (
                    <div className="mt-2 flex gap-2">
                      <Button
                        type="button"
                        variant="destructive"
                        className="h-9"
                        onClick={() => {
                          deleteCard(card.id);
                          setConfirmDelete(null);
                        }}
                      >
                        지우기
                      </Button>
                      <Button type="button" variant="outline" className="h-9" onClick={() => setConfirmDelete(null)}>
                        취소
                      </Button>
                    </div>
                  ) : (
                    <button
                      type="button"
                      className="mt-2 text-xs text-muted-foreground underline"
                      onClick={() => setConfirmDelete(card.id)}
                    >
                      이 카드 지우기
                    </button>
                  )}
                </li>
              ))}
            </ul>
          ) : null}
        </div>
      ) : null}
    </section>
  );
}
```

settings-view.tsx — 설정 화면 `components/settings-view.tsx (367줄)`

목표, 하루 끝 시각, 일어나는 시각, 리포트 알림을 고치는 입력 양식과, 개입 스위치·AI·공부 카드 패널, 알림 권한, 기록 초기화, 설치 안내, 앱 정보를 모은 화면.

```tsx
"use client";

import { useState } from "react";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Separator } from "@/components/ui/separator";
import { Switch } from "@/components/ui/switch";
import { AiPanel } from "@/components/ai-panel";
import { CardsPanel } from "@/components/cards-panel";
import { GuardPanel } from "@/components/guard-panel";
import { useStore } from "@/components/store";
import { APP_VERSION, RELEASES_URL } from "@/lib/app-info";
import { isNativeAndroid } from "@/lib/guard";
import { normalizeClock, validateSettings } from "@/lib/logic";
import { settingsFromGoal } from "@/lib/storage";
import type { Settings } from "@/lib/types";

type Draft = {
  goal: string;
  finalGoal: string;
  dayEndHour: string;
  wakeTime: string;
  notificationTime: string;
  notifyEnabled: boolean;
};

function toDraft(settings: Settings): Draft {
  return {
    goal: String(settings.level3Threshold),
    finalGoal: String(settings.finalGoal),
    dayEndHour: String(settings.dayEndHour),
    wakeTime: settings.wakeTime,
    notificationTime: settings.notificationTime,
    notifyEnabled: settings.notifyEnabled,
  };
}
```

3~17줄 가져오기: 입력 부품, AI 패널, 카드 패널, 스위치 패널, 창고, 앱 버전, 폰 확인, 시각 정리·설정 검사, 목표로 설정 만들기.

19~26줄 Draft: 입력 중인 값 (아직 저장 전). 입력 칸에는 글자가 들어가므로 숫자도 글자(string)로 둔다.

28~37줄 toDraft: 저장된 설정 → 입력 칸 값으로.

```tsx
export function SettingsView() {
  const {
    data,
    permission,
    notifyIssue,
    storageKind,
    saveSettings,
```

기억하는 값: 입력 중인 값(없으면 저장된 설정을 그대로 보여 줌), 오류 문장, 저장됨 표시, 권한 안내, 알림 결과 문장, 초기화 확인 중.

update: 입력 칸 하나를 바꾸면 초안에 반영하고, "저장됨"과 오류 표시를 지운다. `Partial<Draft>`는 "Draft의 일부 칸만".

```
    reset,
    requestNotificationPermission,
    previewNotification,
  } = useStore();
  const native = isNativeAndroid();
  const [draft, setDraft] = useState<Draft | null>(null);
  const [error, setError] = useState<string | null>(null);
  const [saved, setSaved] = useState(false);
  const [permissionMessage, setPermissionMessage] = useState<string | null>(null);
  const [notice, setNotice] = useState<string | null>(null);
  const [confirmReset, setConfirmReset] = useState(false);

  if (!data) return null;
  const form = draft ?? toDraft(data.settings);

  function update(partial: Partial<Draft>) {
    setDraft({ ...form, ...partial });
    setSaved(false);
    setError(null);
  }
```

```
  function save() {
    if (!data) return;
    const goal = Number(form.goal);
    const finalGoal = Number(form.finalGoal);
    const dayEndHour = Number(form.dayEndHour);
    const notificationTime = normalizeClock(form.notificationTime);
    const wakeTime = normalizeClock(form.wakeTime);
    const message = validateSettings({
      goal,
      finalGoal,
      dayEndHour,
      notificationTime: notificationTime ?? form.notificationTime,
      wakeTime: wakeTime ?? form.wakeTime,
    });
    if (message || !notificationTime || !wakeTime) {
      setError(message ?? "알림 시각을 다시 확인해 주세요.");
      setSaved(false);
      return;
    }
    saveSettings(
      settingsFromGoal(goal, {
        notificationTime,
        notifyEnabled: form.notifyEnabled,
        finalGoal,
        dayEndHour,
        wakeTime,
        aiEnabled: data.settings.aiEnabled,
        strictMode: data.settings.strictMode,
        banditV2: data.settings.banditV2,
        studyMode: data.settings.studyMode,
```

save — 저장.

1. 글자를 숫자로 바꾸고, 시각은 "9:5" → "09:05"처럼 정리(normalizeClock, 이상하면 null).
2. validateSettings로 범위 검사 (목표 1~240분, 최종 목표 ≤ 목표, 하루 끝 0~12시 등). 문제가 있으면 오류 문장을 보이고 멈춘다.
3. **86~99줄** settingsFromGoal로 목표에 맞춰 단계 기준(절반 = 중간 시작)까지 계산해 저장. 이 화면에 없는 설정(AI, 엄격 모드, 학습 개선, 공부 모드, 주간 답)은 **기존 값을 그대로** 넘긴다.
4. "설정을 저장했습니다." 표시.

```
        goalWeekAnswered: data.settings.goalWeekAnswered,
      }),
    );
    setError(null);
    setSaved(true);
  }
```

```
  return (
    <div className="mx-auto flex w-full max-w-xl flex-col gap-6">
      <header>
        <p className="text-sm text-muted-foreground">목표, 알림, 공부 카드</p>
        <h1 className="mt-1 text-3xl font-semibold tracking-tight">설정</h1>
      </header>

      <GuardPanel />

      <form
        className="space-y-5 rounded-3xl bg-card p-5 ring-1 ring-foreground/10"
        onSubmit={(event) => {
          event.preventDefault();
          save();
        }}
      >
```

머리 "목표, 알림, 공부 카드 / 설정", 개입 스위치 패널(홈과 같은 부품).
113~119줄 입력 양식: 저장 버튼(또는 키보드 엔터)을 누르면 페이지를 새로 고치지 않고(`preventDefault`) save를 부른다.

```
        <div className="space-y-2">
          <Label htmlFor="goal">하루 목표 (분)</Label>
          <Input
            id="goal"
            inputMode="numeric"
            value={form.goal}
            onChange={(event) => update({ goal: event.target.value })}
            aria-invalid={Boolean(error)}
          />
          <p className="text-xs leading-5 text-muted-foreground">
            오늘 쇼츠에 쓸 시간입니다. 목표의 절반까지는 가볍게, 절반을 넘으면 멈추는 활동을
            하나 거치고, 목표를 넘으면 오늘을 마무리할지 고릅니다. 기본값은 20분입니다.
          </p>
        </div>
        <div className="space-y-2">
          <Label htmlFor="final-goal">최종 목표 (분)</Label>
          <Input
            id="final-goal"
            inputMode="numeric"
            value={form.finalGoal}
            onChange={(event) => update({ finalGoal: event.target.value })}
            aria-invalid={Boolean(error)}
          />
          <p className="text-xs leading-5 text-muted-foreground">
            언젠가 줄이고 싶은 하루 시간입니다. 지금은 기록만 하고, 나중에 하루 목표를 이쪽으로
            조금씩 줄이자고 제안할 때 씁니다.
```

- **하루 목표 (분)**: 숫자 키보드(`inputMode="numeric"`). 설명: 절반까지 가볍게, 절반 넘으면 멈추는 활동, 목표 넘으면 오늘 마무리 선택. 기본 20분.
- **최종 목표 (분)**: 언젠가 줄이고 싶은 시간. AI 주간 제안이 여기까지 조금씩 줄이자고 할 때 쓴다.

`aria-invalid`는 오류가 있을 때 화면 읽기 도구에 "잘못된 입력"이라 알린다.

```
          </p>
        </div>
```

```
        <div className="space-y-2">
          <Label htmlFor="day-end">&amp;quot;오늘은 끝&amp;quot;이 풀리는 시각 (시)</Label>
          <Input
            id="day-end"
            inputMode="numeric"
            value={form.dayEndHour}
            onChange={(event) => update({ dayEndHour: event.target.value })}
            aria-invalid={Boolean(error)}
          />
          <p className="text-xs leading-5 text-muted-foreground">
            목표를 넘었을 때 &amp;quot;오늘은 끝&amp;quot;을 고르면 다음 날 이 시각까지 쇼츠가 잠깁니다.
            0~12 사이로 적습니다. 기본값은 6시입니다.
          </p>
        </div>
        <div className="space-y-2">
          <Label htmlFor="wake-time">일어나는 시각</Label>
          <Input
            id="wake-time"
            type="time"
            value={form.wakeTime}
            onChange={(event) => update({ wakeTime: event.target.value })}
          />
          <p className="text-xs leading-5 text-muted-foreground">
            밤에 쇼츠를 열면 &amp;quot;지금 자면 몇 시간 잘 수 있어요&amp;quot;를 알려 줄 때 씁니다. 휴대폰에
            12시간 안에 울리는 알람이 있으면 알람 시각을 먼저 씁니다.
          </p>
        </div>
```

- **"오늘은 끝"이 풀리는 시각**: 다음 날 이 시각까지 잠김. 0~12시, 기본 6시.
- **일어나는 시각** (`type="time"` = 시각 고르기 칸): 밤에 "지금 자면 몇 시간 잘 수 있어요"에 쓴다. 폰에 12시간 안 알람이 있으면 알람 시각이 먼저.

```
        <div className="space-y-2">
          <Label htmlFor="notify-time">일일 리포트 시각</Label>
          <Input
            id="notify-time"
            type="time"
            value={form.notificationTime}
            onChange={(event) => update({ notificationTime: event.target.value })}
          />
          <p className="text-xs leading-5 text-muted-foreground">
            {native
              ? "이 기기 시각 기준입니다. 앱이 꺼져 있어도 매일 이 시각에 알림이 오고, 누르면 오늘 리포트가 열립니다."
              : "이 기기 시각 기준입니다. 앱이 열려 있을 때 브라우저 알림을 보내고, 리포트 화면에도 같은 내용을 표시합니다."}
          </p>
        </div>
        <div className="flex items-center justify-between gap-3">
          <div>
            <Label htmlFor="notify-enabled">{native ? "리포트 알림" : "브라우저 알림"}</Label>
            <p className="mt-1 text-xs text-muted-foreground">
```

- **일일 리포트 시각**: 폰이면 앱이 꺼져 있어도 매일 알림, 브라우저면 열려 있을 때만.
- **리포트 알림 스위치**: 끄면 예약 알림만 끄고 리포트 화면은 유지.
- 오류 문장(빨강), "설정을 저장했습니다.", [저장] 버튼.

```
              끄면 예약 알림을 보내지 않습니다. 리포트 화면은 유지됩니다.
            </p>
          </div>
          <Switch
            id="notify-enabled"
            checked={form.notifyEnabled}
            onCheckedChange={(checked) => update({ notifyEnabled: checked })}
          />
        </div>
        {error ? (
          <p className="text-sm text-destructive" role="alert">
            {error}
          </p>
        ) : null}
        {saved ? (
          <p className="text-sm text-muted-foreground" role="status">
            설정을 저장했습니다.
          </p>
        ) : null}
        <Button type="submit" className="h-11 w-full">
          저장
        </Button>
      </form>
```

```
      <AiPanel />

      <CardsPanel />
```

AI 개인화 패널, 공부 카드 패널.

```
      <section className="space-y-3 rounded-3xl bg-card p-5 ring-1 ring-foreground/10">
        <h2 className="text-base font-medium">알림 권한</h2>
        <p className="text-sm leading-6 text-muted-foreground">
          {permission === "granted"
            ? native
              ? "알림을 보낼 수 있습니다. 리포트 알림과 시간 차단 중 남은 시간 알림이 옵니다."
              : "이 브라우저에서 알림을 보낼 수 있습니다."
            : permission === "denied"
              ? native
                ? "알림이 꺼져 있습니다. 휴대폰 설정 → 애플리케이션 → ShortsAI → 알림에서 허용해 주세요."
                : "알림이 차단되어 있습니다. 주소창의 사이트 설정에서 허용해 주세요."
              : permission === "unsupported"
                ? "이 기기에서는 알림을 쓸 수 없습니다."
                : "아직 권한을 묻지 않았습니다."}
        </p>
        {notifyIssue ? (
          <p className="text-sm text-destructive" role="alert">
            {notifyIssue}
          </p>
        ) : null}
        {permissionMessage ? (
          <p className="text-sm text-destructive" role="alert">
```

알림 권한 칸. 권한 상태별 문장 (허용 / 거부: 어디서 허용하는지 / 지원 안 함 / 아직 안 물음). 그 아래 알림 예약 문제, 권한 안내(빨강), 결과 문장.

```
            {permissionMessage}
          </p>
        ) : null}
        {notice ? (
          <p className="text-sm" role="status">
            {notice}
          </p>
        ) : null}
```

```
        <div className="flex flex-col gap-2 sm:flex-row">
          <Button
            type="button"
            variant="outline"
            className="h-11"
            onClick={() => {
              void requestNotificationPermission().then((message) => {
                setPermissionMessage(message);
                setNotice(message ? null : "알림을 허용했습니다.");
              });
            }}
          >
            알림 허용 요청
          </Button>
          <Button
            type="button"
            variant="outline"
            className="h-11"
            onClick={() => {
              void previewNotification().then((message) => {
                setPermissionMessage(message);
                setNotice(message ? null : "알림을 보냈습니다.");
              });
            }}
          >
            알림 미리보기
          </Button>
        </div>
      </section>
```

[알림 허용 요청]: 권한을 묻고 결과에 따라 안내 또는 "알림을 허용했습니다."
[알림 미리보기]: 1.5초 뒤 테스트 알림.

```
      <Separator />

      <section className="space-y-3">
        <h2 className="text-base font-medium">기록 초기화</h2>
        <p className="text-sm leading-6 text-muted-foreground">
          {storageKind === "sqlite" ? "이 기기에" : "이 브라우저에"} 저장된 사용 시간,
          개입, 잠금 시도, 차단을 지웁니다. 설정은 기본값으로 돌아갑니다. 공부 카드는
          남기고 푼 횟수만 지웁니다.
        </p>
        {confirmReset ? (
          <div className="flex flex-col gap-2 sm:flex-row">
            <Button
```

기록 초기화. 무엇이 지워지는지 설명 (사용 시간, 개입, 잠금 시도, 차단, 설정은 기본값, **공부 카드는 남기고** 푼 횟수만). [기록 모두 지우기] → [정말 지우기] [취소] 두 번 확인. 지우면 입력 초안도 비운다.

```
              type="button"
              variant="destructive"
              className="h-11"
              onClick={() => {
                void reset();
                setDraft(null);
                setConfirmReset(false);
                setSaved(false);
              }}
            >
              정말 지우기
            </Button>
            <Button
              type="button"
              variant="outline"
              className="h-11"
              onClick={() => setConfirmReset(false)}
            >
              취소
            </Button>
          </div>
        ) : (
          <Button
            type="button"
            variant="outline"
            className="h-11"
            onClick={() => setConfirmReset(true)}
          >
            기록 모두 지우기
          </Button>
        )}
      </section>
```

```
      <section className="space-y-2 text-sm leading-6 text-muted-foreground">
        <h2 className="text-base font-medium text-foreground">폰에 설치</h2>
        <p>
          아이폰은 Safari에서 이 페이지를 연 뒤 공유 버튼을 누르고{" "}
          <strong className="font-medium text-foreground">홈 화면에 추가</strong>를
          고르세요. 한 번 열린 뒤에는 네트워크가 없어도 홈 화면 아이콘으로 열립니다.
        </p>
        <p>
          안드로이드는{" "}
          {RELEASES_URL ? (
            <a className="underline" href={RELEASES_URL}>
              ShortsAI APK
            </a>
          ) : (
            "GitHub Releases의 ShortsAI APK"
          )}
          를 받아 설치하세요. 출처를 알 수 없는 앱 설치를 허용해야 합니다. 설치 후
          설정 → 접근성에서 ShortsAI를 켜야 실제 쇼츠와 릴스 플레이어에 개입합니다.
```

폰에 설치 안내. 아이폰: Safari → 공유 → 홈 화면에 추가 (PWA). 안드로이드: GitHub Releases의 APK (RELEASES_URL이 있으면 링크), 출처를 알 수 없는 앱 허용, 접근성 켜기.

```
        </p>
      </section>
```

```
      <section className="space-y-1 text-sm leading-6 text-muted-foreground">
        <h2 className="text-base font-medium text-foreground">앱 정보</h2>
        <p>ShortsAI {APP_VERSION ? `버전 ${APP_VERSION}` : ""}</p>
        <p>
          기록 저장소: {storageKind === "sqlite" ? "기기 데이터베이스(SQLite)" : "브라우저 저장소"}
        </p>
      </section>

      <section className="space-y-2 text-sm leading-6 text-muted-foreground">
        <h2 className="text-base font-medium text-foreground">이 앱이 하지 않는 일</h2>
        <p>
          브라우저에서는 다른 앱을 막지 않습니다. 안드로이드 설치본은 접근성을 켠
          뒤에 YouTube Shorts 플레이어와 Instagram Reels 플레이어만 구분합니다.
          쇼츠 플레이어가 열리면 영상을 멈추고 그 위에 질문을 띄웁니다. Level 3에서는
          YouTube를 닫고 ShortsAI로 돌아옵니다. 화면의 글이나 다른 앱의 내용은 읽거나
          저장하지 않고, 기록은 이 기기 밖으로 보내지 않습니다.
        </p>
      </section>
    </div>
  );
}
```

앱 정보: 버전, 저장소 종류(SQLite / 브라우저).

이 앱이 하지 않는 일: 브라우저에서는 다른 앱을 막지 않음, 안드로이드는 쇼츠·릴스 플레이어만 구분, 화면 글이나 다른 앱 내용은 읽지 않고 기록은 기기 밖으로 보내지 않음.

ai-panel.tsx — AI 개인화 패널 (설정 화면) `components/ai-panel.tsx (132줄)`

AI 켜기/끄기, 엄격 모드, 그리고 **AI가 지금까지 배운 것**을 그대로 보여 주는 칸. "왜 이렇게 개입하는지" 사용자가 볼 수 있게 하는 설명 가능성 장치다.

```
"use client";

import { Label } from "@/components/ui/label";
import { Switch } from "@/components/ui/switch";
import { useStore } from "@/components/store";
import {
  banditTables,
  hourLabel,
  MIN_DAYS_FOR_RISK,
  recordedDays,
  riskHours,
  type TimeSlot,
} from "@/lib/ai";

const SLOT_LABEL: Record<TimeSlot, string> = {
  morning: "아침",
  day: "낮",
  evening: "저녁",
  night: "밤",
```

3~13줄 가져오기: 스위치, 창고, ai.ts의 밴딧 표·위험 시간·기록 일수.

15~20줄 시간대 이름 (morning → 아침 …). **22~29줄** 방식 이름 (pause → 5초 숨 고르기, framing → 시간 체감 문구, breath → 호흡, alternative → 대체행동, reflection → 성찰 질문, card → 공부 카드). `Record<K, V>`는 K → V 표.

```
};

const METHOD_LABEL: Record<string, string> = {
  pause: "5초 숨 고르기",
  framing: "시간 체감 문구",
  breath: "호흡 따라 하기",
  alternative: "대체행동",
  reflection: "성찰 질문",
  card: "공부 카드",
};
```

```
/**
 * 시간대별로 가장 잘 먹힌 방식 (기록 3번 이상인 것만).
 * 그 칸에서 직접 쌓인 기록만 본다 (다른 칸에서 빌려 온 값은 섞지 않는다).
 */
function bestBySlot(counts: Record<string, Record<string, [number, number]>>) {
  const bySlot = new Map<string, { arm: string; mean: number; n: number }>();
  for (const [context, arms] of Object.entries(counts)) {
    const slot = context.split(":")[1] ?? "";
    for (const [arm, [s, f]] of Object.entries(arms)) {
      const n = s + f;
      if (n < 3) continue;
      const mean = (s + 1) / (n + 2);
      const prev = bySlot.get(slot);
      if (!prev || mean > prev.mean) bySlot.set(slot, { arm, mean, n });
    }
  }
  return bySlot;
}
```

bestBySlot — 시간대별 가장 잘 먹힌 방식. 칸별 자기 기록(다른 칸에서 빌려 온 값은 섞지 않음)을 보고, 칸 이름 "2:night:weekday"의 둘째 조각(시간대)으로 묶는다. 기록이 **3번 이상**인 방식만, 성공률 (성공 + 1) ÷ (전체 + 2)가 가장 높은 것을 고른다. `Object.entries`는 표를 [이름, 값] 쌍 목록으로 바꾼다.

```
/**
 * 설정의 AI 패널. 켜기/끄기(AI 전후 비교용), 엄격 모드, 그리고 AI가 지금까지 배운 것을
 * 그대로 보여 준다 (설명 가능성: 왜 이렇게 개입하는지 사용자가 볼 수 있게).
 */
export function AiPanel() {
  const { data, saveSettings } = useStore();
  if (!data) return null;
  const settings = data.settings;
  const days = recordedDays(data);
  const risk = settings.aiEnabled ? riskHours(data) : null;
  const tables = settings.aiEnabled ? banditTables(data) : null;
  const best = tables ? bestBySlot(tables.ownMethods) : new Map();
  const feedbackCount = data.interventions.filter((log) => log.feedback !== null).length;
```

AiPanel 계산. 지난 2주 기록 일수, (AI 켜짐이면) 위험 시간, 밴딧 표, 시간대별 최고 방식, "도움이 됐나요?" 답 수.

```
  return (
    <section className="space-y-4 rounded-3xl bg-card p-5 ring-1 ring-foreground/10">
      <div className="flex items-start justify-between gap-4">
        <div className="space-y-1">
          <Label htmlFor="ai-enabled" className="text-base font-medium">
            AI 개인화
          </Label>
```

AI 개인화 스위치. 설명: 내 기록을 보고 잘 맞는 방식과 문구를 고르고, 평소 오래 보는 시간엔 조금 더 짧게. 계산은 폰 안에서, 기록은 밖으로 보내지 않음. 끄면 정해진 규칙대로 번갈아. (켜고 끄기는 AI 전후 비교에도 쓴다.)

```
          <p className="text-sm leading-6 text-muted-foreground">
            내 기록을 보고, 나에게 잘 맞는 개입 방식과 문구를 고르고, 평소 오래 보는 시간에는
            조금 더 짧게 잡습니다. 계산은 모두 이 폰 안에서 하고, 기록은 밖으로 보내지 않습니다.
            끄면 정해진 규칙대로 번갈아 보여 줍니다.
          </p>
        </div>
        <Switch
          id="ai-enabled"
          checked={settings.aiEnabled}
          aria-label="AI 개인화"
          onCheckedChange={(checked) => saveSettings({ ...settings, aiEnabled: Boolean(checked)
})}
        />
      </div>

      <div className="flex items-start justify-between gap-4">
        <div className="space-y-1">
          <Label htmlFor="strict-mode" className="text-base font-medium">
            엄격 모드
          </Label>
          <p className="text-sm leading-6 text-muted-foreground">
            켜면 약속 시간을 늘리는 &quot;조금 더 볼래요&quot;와 목표를 넘긴 뒤 &quot;3분만 더&quot;
가
            사라집니다. 스스로 정한 약속을 더 단단하게 지키고 싶을 때 켜세요.
          </p>
        </div>
        <Switch
          id="strict-mode"
          checked={settings.strictMode}
          aria-label="엄격 모드"
          onCheckedChange={(checked) => saveSettings({ ...settings, strictMode: Boolean(checked)
})}
        />
      </div>

      {settings.aiEnabled ? (
        <div className="space-y-2 rounded-2xl bg-background px-4 py-3 text-sm leading-6 ring-1 r
ing-foreground/10">
          <p className="font-medium">AI가 지금까지 배운 것</p>
          <p className="text-muted-foreground">
            지난 2주 중 기록이 있는 날: {days}일
            {days < MIN_DAYS_FOR_RISK
              ? ` (평소 오래 보는 시간은 ${MIN_DAYS_FOR_RISK}일 이상 쌓이면 찾기 시작해요)`
              : ""}
          </p>
          <p>
            평소 오래 보는 시간:{" "}
            {risk ? risk.hours.map(hourLabel).join(", ") : "아직 찾지 못했어요"}
          </p>
          {(["morning", "day", "evening", "night"] as TimeSlot[]).map((slot) => {
            const top = best.get(slot);
```

엄격 모드 스위치. 켜면 "조금 더 볼래요"(약속 연장)와 목표 넘긴 뒤 "3분만 더"가 사라진다.

AI가 지금까지 배운 것 (AI 켜짐일 때만).

- 지난 2주 중 기록 있는 날 (5일 미만이면 "5일 이상 쌓이면 찾기 시작해요").
- 평소 오래 보는 시간: "밤 11시, 새벽 1시" 또는 "아직 찾지 못했어요".
- 시간대 4개마다: "밤: 호흡 따라 하기이(가) 잘 맞았어요 (멈춘 비율 72%)" 또는 "아직 배우는 중이에요".
- "처음 몇 주는 여러 방식을 골고루 보여 주며 배웁니다." + 도움 답 수.

```
            return (
              <p key={slot} className="text-muted-foreground">
                {SLOT_LABEL[slot]}: {top ? `${METHOD_LABEL[top.arm] ?? top.arm}이(가) 잘 맞았어요 (멈춘 비율 ${Math.round(top.mean * 100)}%)` : "아직 배우는 중이에요"}
              </p>
            );
          })}
          <p className="text-muted-foreground">
            처음 몇 주는 여러 방식을 골고루 보여 주며 배웁니다. 남겨 주신 &amp;quot;도움 됐어요&amp;quot; 답: {feedbackCount}개
          </p>
        </div>
      ) : null}
    </section>
  );
}
```

report-view.tsx — 일일 리포트 화면 `components/report-view.tsx (117줄)`

```
"use client";

import { useEffect, useState } from "react";
import { Button } from "@/components/ui/button";
import { useStore } from "@/components/store";
import { useNow } from "@/components/use-now";
import { isNativeAndroid } from "@/lib/guard";
import {
  formatKoreanDate,
  isReportDue,
  reportLines,
  statsForDate,
  todayKey,
} from "@/lib/logic";
```

가져오기: React, 버튼, 창고, 시계, 폰 확인, 날짜 글자·리포트 시각·리포트 문장·통계 함수.

```
export function ReportView() {
  const { data, permission, previewNotification, markReportSeen } = useStore();
  const now = useNow(5000);
  const [notice, setNotice] = useState<string | null>(null);
  const native = isNativeAndroid();
  const due = data ? isReportDue(data.settings.notificationTime, new Date(now)) : false;

  // 리포트 시각이 지난 뒤 이 화면을 열면 오늘 리포트를 본 것으로 표시한다.
  useEffect(() => {
    if (due) markReportSeen();
  }, [due, markReportSeen]);
```

ReportView 시작. 5초마다 시각을 갱신(리포트 시각을 넘기는 순간 제목이 바뀌게). `due` = 리포트 시각이 지났는지. **23~26줄** 시각이 지난 뒤 이 화면을 열면 "오늘 리포트 봄"으로 표시 (메뉴의 점이 사라짐).

```
  if (!data) return null;

  const today = todayKey(new Date(now));
  const stats = statsForDate(data, today);
  const empty =
    stats.usageSeconds === 0 &&
    stats.interventionCount === 0 &&
    stats.clipCount === 0;
  const lines = reportLines(stats);
```

오늘 통계를 내고, 사용·개입·영상이 모두 0이면 "빈 날". reportLines가 통계를 문장 목록으로 만든다 (예: "오늘 쇼츠 23분", "스스로 멈춤 3번" …).

```
  return (
    <div className="mx-auto flex w-full max-w-xl flex-col gap-6">
      <header>
        <p className="text-sm text-muted-foreground">{formatKoreanDate(today)}</p>
        <h1 className="mt-1 text-3xl font-semibold tracking-tight">
          {due ? "오늘의 Shorts 사용 리포트" : "오늘의 중간 집계"}
        </h1>
      </header>
```

머리: 오늘 날짜("10월 13일 화요일"), 제목 — 시각 지남: "오늘의 Shorts 사용 리포트", 전: "오늘의 중간 집계".

```
      {empty ? (
        <div className="rounded-3xl bg-card px-5 py-8 ring-1 ring-foreground/10">
          <p className="text-lg font-medium">오늘 기록된 사용이 없습니다.</p>
          <p className="mt-2 text-sm leading-6 text-muted-foreground">
            {due
              ? "리포트 시각은 지났지만, 쇼츠를 열지 않아 집계가 비어 있습니다."
              : `${data.settings.notificationTime}에 리포트가 도착합니다. 그 전에 쇼츠를 열면 사용 시간과 개입이 여기 모입니다.`}
          </p>
        </div>
      ) : (
        <article className="rounded-3xl bg-card px-5 py-6 ring-1 ring-foreground/10">
          <p className="text-sm text-muted-foreground">
            {due
              ? "하루를 이렇게 정리했습니다."
              : `${data.settings.notificationTime}까지는 중간 집계입니다.`}
          </p>
          <ul className="mt-4 space-y-2">
            {lines.map((line) => (
              <li key={line} className="text-lg leading-8">
                {line}
              </li>
            ))}
          </ul>
          {stats.forceQuitCount > 0 ? (
            <p className="mt-4 text-sm text-muted-foreground">
              강제 종료 {stats.forceQuitCount}회
            </p>
          ) : null}
        </article>
      )}
```

본문. 빈 날이면 "오늘 기록된 사용이 없습니다." + 상황 설명. 아니면 "하루를 이렇게 정리했습니다."(또는 "21:00까지는 중간 집계입니다.") 아래에 문장 목록을 큰 글씨로, 강제 종료가 있었으면 횟수.

```
      <div className="space-y-2 text-sm leading-6 text-muted-foreground">
        {!data.settings.notifyEnabled ? (
          <p>리포트 알림이 설정에서 꺼져 있습니다. 이 화면의 리포트는 그대로 볼 수 있습니다.</p>
        ) : native && permission === "granted" ? (
          <p>
            매일 {data.settings.notificationTime}에 앱이 꺼져 있어도 알림이 옵니다. 누르면 이
            화면이 열립니다.
          </p>
        ) : native ? (
          <p>알림 권한이 없어 리포트 알림이 오지 않습니다. 설정 화면에서 알림을 허용해 주세요.</p>
        ) : permission === "granted" ? (
          <p>
            알림 권한이 있습니다. 앱이 열려 있고 시각이 {data.settings.notificationTime}을
            지나면 같은 내용을 브라우저 알림으로도 보냅니다.
          </p>
        ) : permission === "denied" ? (
          <p>
            브라우저가 알림을 막았습니다. 사이트 설정에서 허용하거나, 이 화면에서
            리포트를 확인하세요.
          </p>
        ) : permission === "unsupported" ? (
          <p>이 브라우저는 알림을 지원하지 않습니다. 리포트는 이 화면에서 확인합니다.</p>
        ) : (
          <p>알림을 받으려면 설정에서 브라우저 알림을 허용하세요.</p>
        )}
        {notice ? <p role="status">{notice}</p> : null}
        <Button
          type="button"
          variant="outline"
          className="h-10"
          onClick={() => {
            void previewNotification().then((message) => setNotice(message ?? "알림을 보냈습니
다."));
          }}
        >
          알림 미리보기
        </Button>
      </div>
    </div>
  );
}
```

알림 상태 안내 — 경우별 한 문장: 알림 꺼짐 / 폰 + 권한 있음(앱이 꺼져 있어도 매일 알림) / 폰 + 권한 없음 / 브라우저 + 권한 있음(열려 있을 때만) / 차단됨 / 미지원 / 아직 안 물음. 그 아래 [알림 미리보기]와 결과 문장. 조건 ? A : 조건 ? B : C 처럼 이어 쓴 것은 "if / else if / else"와 같다.

stats-view.tsx — 통계 화면 `components/stats-view.tsx (278줄)`

최근 7일 막대 그래프와, 고른 날의 숫자 14가지, 개입 기록 상세, 시청 기록, 잠금 중 시도를 보여 준다.

```tsx
"use client";

import { useState } from "react";
import { Badge } from "@/components/ui/badge";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { useStore } from "@/components/store";
import { useNow } from "@/components/use-now";
import {
  BAND_LABEL,
  formatClock,
  formatKoreanDate,
  formatShortDate,
  formatTime,
  lastDates,
  OUTCOME_LABEL,
  reasonLabel,
  secondsOnDate,
  statsForDate,
  TARGET_LABEL,
  todayKey,
  usageSecondsOn,
} from "@/lib/logic";
import { cn } from "@/lib/utils";
```

가져오기: React, 배지, 카드, 창고, 시계, logic.ts의 글자표·날짜 함수·통계 함수, 모양 합치기.

```tsx
export function StatsView() {
  const { data } = useStore();
  const now = useNow(5000);
  const today = todayKey(new Date(now));
  const [selected, setSelected] = useState(today);
  if (!data) return null;

  const days = lastDates(7, new Date(now));
  const active = days.includes(selected) ? selected : today;
  const series = days.map((date) => ({
    date,
    seconds: usageSecondsOn(data, date),
  }));
  const max = Math.max(...series.map((item) => item.seconds), 1);
  const stats = statsForDate(data, active);
  const interventions = data.interventions
    .filter((log) => log.date === active)
    .slice()
    .reverse();
  // 자정을 넘긴 세션은 걸친 두 날짜에 모두 보인다. 시간은 그 날짜 몫만 표시한다.
  const sessions = data.usageLogs
```

StatsView 계산.

- 5초마다 시각 갱신, 고른 날(처음엔 오늘). 고른 날이 7일 안에 없으면 오늘.
- **34~38줄** 최근 7일 각각의 시청 초와 가장 큰 값 (막대 높이 기준).
- **39~43줄** 고른 날의 통계와 개입 기록(최신부터).
- **44~48줄** 시청 기록: **자정을 넘긴 시청**은 걸친 두 날 모두에 보이고, 시간은 그 날 몫만 센다(secondsOnDate).
- 잠금 중 시도, 세 가지가 모두 없으면 "빈 날".

```
  .map((log) => ({ log, seconds: secondsOnDate(log, active) }))
  .filter((item) => item.seconds > 0 || item.log.date === active)
  .reverse();
const locks = data.lockAttempts.filter((item) => item.date === active).reverse();
const empty = interventions.length === 0 && sessions.length === 0 && locks.length === 0;
```

```
return (
  <div className="flex flex-col gap-6">
    <header>
      <p className="text-sm text-muted-foreground">일별 사용과 개입</p>
      <h1 className="mt-1 text-3xl font-semibold tracking-tight">통계</h1>
    </header>

    <section aria-label="최근 7일 사용 시간">
      <div className="grid grid-cols-7 gap-2">
        {series.map((item) => {
          const height = Math.max(8, Math.round((item.seconds / max) * 96));
          const on = item.date === active;
          return (
            <button
              key={item.date}
              type="button"
              onClick={() => setSelected(item.date)}
              aria-pressed={on}
              className={cn(
                "flex flex-col items-center gap-2 rounded-2xl px-1 py-2",
                on ? "bg-secondary" : "hover:bg-secondary/60",
              )}
            >
              <span className="flex h-28 items-end">
                <span
                  className={cn(
                    "w-3 rounded-full",
                    item.seconds > 0 ? "bg-primary" : "bg-muted",
                  )}
                  style={{ height }}
                />
              </span>
              <span className="text-[11px] text-muted-foreground">
                {formatShortDate(item.date, today)}
              </span>
            </button>
          );
        })}
      </div>
    </section>
```

머리와 7일 막대. "일별 사용과 개입 / 통계". 7칸 격자에 날마다 버튼 하나: 막대 높이 = 그날 초 ÷ 최대 × 96px (최소 8px), 아래에 "오늘", "어제", "10/9" 같은 짧은 날짜. 누르면 그 날을 고른다(`aria-pressed`로 눌린 상태 표시).

```
      <div>
        <h2 className="text-lg font-semibold">{formatKoreanDate(active)}</h2>
        {empty ? (
          <Card className="mt-3">
            <CardContent className="py-8 text-sm leading-6 text-muted-foreground">
              이 날의 기록이 없습니다. 쇼츠를 열면 사용 시간, 실행 이유, 레벨,
              대체행동, 재실행 여부가 여기에 쌓입니다.
            </CardContent>
          </Card>
        ) : (
          <div className="mt-3 grid gap-3 sm:grid-cols-2 lg:grid-cols-4">
            <Metric label="스스로 멈춘 횟수" value={`${stats.selfStopCount}번`} />
            <Metric label="사용 시간" value={formatClock(stats.usageSeconds)} />
            <Metric label="실행" value={`${stats.clipCount}회`} />
            <Metric label="개입" value={`${stats.interventionCount}회`} />
            <Metric label="많은 이유" value={stats.topReason ?? "없음"} />
            <Metric label="약속 지킴" value={`${stats.keptCount}번`} />
            <Metric label="약속 연장" value={`${stats.extendCount}번`} />
            <Metric label="가벼움 / 중간 / 강함" value={`${stats.levelCounts[1]} / ${stats.levelC
ounts[2]} / ${stats.levelCounts[3]}`} />
            <Metric label="오늘은 끝" value={stats.dayEndCount > 0 ? "스스로 마무리" : "없음"} />
            <Metric label="목표 초과" value={`${stats.overdraftCount}번`} />
            <Metric label="시간 잠금" value={`${stats.blockCount}회`} />
            <Metric label="잠금 중 시도" value={`${stats.lockAttemptCount}회`} />
            <Metric label="쉰 시간" value={formatClock(stats.restSeconds)} />
            <Metric label="푼 문제" value={`${stats.cardsSolved}개 (정답 ${stats.cardsCorrect})`}
/>
          </div>
        )}
      </div>
```

고른 날 숫자. 날짜 제목, 빈 날이면 안내. 아니면 작은 카드(Metric) 14개: 스스로 멈춘 횟수, 사용 시간, 실행, 개입, 많은 이유, 약속 지킴, 약속 연장, 단계별 횟수, 오늘은 끝, 목표 초과, 시간 잠금, 잠금 중 시도, 쉰 시간, 푼 문제(정답 수).

```
      <section className="space-y-3">
        <h2 className="text-lg font-semibold">개입 기록</h2>
        {interventions.length === 0 ? (
          <p className="text-sm text-muted-foreground">이 날의 개입이 없습니다.</p>
        ) : (
          <ul className="space-y-3">
            {interventions.map((log) => (
              <li key={log.interventionId}>
                <Card size="sm">
                  <CardHeader>
                    <CardTitle className="flex items-center justify-between gap-2">
                      <span>
                        {formatTime(log.timestamp)} · {BAND_LABEL[log.level]}
                      </span>
                      <Badge variant="outline">{OUTCOME_LABEL[log.outcome]}</Badge>
                    </CardTitle>
                  </CardHeader>
                  <CardContent>
                    <dl className="grid grid-cols-2 gap-x-3 gap-y-2 text-sm">
```

개입 기록 (1). 각 개입마다 카드: 제목 "21:34 · 중간" + 결과 배지("3분만 보기" 등). `<dl>`(이름-값 목록) 안에 실행 이유, 방식.

```
                    <Field label="실행 이유" value={reasonLabel(log)} />
                    <Field label="방식" value={METHOD_LABEL[log.interventionType] ?? log.interv
entionType} />
```

```
                    {log.commitMinutes > 0 ? (
                      <Field label="약속 시간" value={`${log.commitMinutes}분`} />
                    ) : null}
                    {log.reflection ? <Field label="성찰" value={log.reflection} /> : null}
                    {log.cardCorrect !== null ? (
                      <Field label="문제" value={log.cardCorrect ? "맞힘" : "틀림"} />
                    ) : null}
                    <Field label="당시 누적" value={formatClock(log.usageSeconds)} />
                    <Field label="대체행동" value={log.alternativeAction ?? "없음"} />
                    <Field
                      label="수행 여부"
                      value={log.alternativeCompleted ? "수행함" : "하지 않음"}
                    />
                    <Field label="재실행" value={log.reEntered ? "다시 봄" : "보지 않음"} />
                    <Field label="차단 여부" value={log.blocked ? "차단함" : "차단 안 함"} />
```

개입 기록 (2) 있을 때만 보이는 칸: 약속 시간, 성찰 답, 문제 맞힘/틀림. 그리고 당시 누적, 대체행동, 수행 여부, 재실행, 차단 여부.

```
                    <Field
                      label="30분 내 재진입"
                      value={
                        log.outcome === "watch"
                          ? "해당 없음"
                          : log.reenteredWithin30 === null
                            ? "확인 중"
                            : log.reenteredWithin30
                              ? "다시 열었음"
                              : "열지 않음"
                      }
                    />
                    <Field
                      label="차단 시간"
                      value={
                        log.blockDuration > 0
                          ? log.blockDuration === 60
                            ? "1시간"
                            : `${log.blockDuration}분`
                          : "없음"
                      }
                    />
                  </dl>
                </CardContent>
              </Card>
            </li>
          ))}
        </ul>
      )}
    </section>
```

개입 기록 (3): 30분 내 재진입 — 보기를 골랐으면 "해당 없음", 아직 30분 안 지났으면 "확인 중", 그 뒤 "다시 열었음/열지 않음". 차단 시간 — "1시간", "30분", "없음".

```
      <section className="space-y-3">
        <h2 className="text-lg font-semibold">사용 기록</h2>
        {sessions.length === 0 ? (
          <p className="text-sm text-muted-foreground">이 날의 시청 기록이 없습니다.</p>
        ) : (
          <ul className="space-y-2">
            {sessions.map(({ log, seconds }) => (
              <li
                key={log.usageId}
                className="flex items-center justify-between gap-3 rounded-2xl bg-card px-4 py-3
text-sm ring-1 ring-foreground/10"
              >
                <span>
                  <span className="block tabular-nums">
                    {formatTime(log.startTime)} – {formatTime(log.endTime)}
                  </span>
                  <span className="text-xs text-muted-foreground">{TARGET_LABEL[log.target]}</sp
an>
                </span>
                <span className="text-muted-foreground tabular-nums">
                  {formatClock(seconds)}
                  {Math.round(seconds) < Math.round(log.duration) ? " (자정 넘김)" : ""}
                </span>
              </li>
            ))}
          </ul>
        )}
      </section>
```

사용 기록. 시청 한 번마다 "21:30 – 21:42", 대상(YouTube Shorts, 연습 …), 그날 몫 시간. 그날 몫이 전체보다 짧으면 "(자정 넘김)".

```
      {locks.length > 0 ? (
        <section className="space-y-3">
          <h2 className="text-lg font-semibold">잠금 중 시도</h2>
          <LockList items={locks} />
        </section>
      ) : null}
    </div>
  );
}
```

잠금 중 시도가 있으면 목록으로.

```
const METHOD_LABEL: Record<string, string> = {
  pause: "5초 숨 고르기",
  framing: "시간 바꿔 말하기",
  breath: "호흡 따라 하기",
  alternative: "대체행동",
  reflection: "성찰 질문",
  card: "공부 카드",
  warning: "경고",
  commit_end: "약속 시간 끝",
  strong: "마무리 선택",
  locked: "잠금",
```

METHOD_LABEL: 개입 방식 이름 (통계용으로 AI 패널보다 많다): 경고, 약속 시간 끝, 마무리 선택, 잠금, "나가기 후 작은 창으로 계속 봄"(PIP) 등.

```
  pip: "나가기 후 작은 창으로 계속 봄",
};
```

```
function LockList({ items }: { items: { id: string; createdAt: string; remainingSec: number }[]
}) {
  return (
    <ul className="space-y-2">
      {items.map((item) => (
        <li
          key={item.id}
          className="flex items-center justify-between gap-3 rounded-2xl bg-card px-4 py-3 text-
sm ring-1 ring-foreground/10"
        >
          <span className="tabular-nums">{formatTime(item.createdAt)}</span>
          <span className="text-muted-foreground tabular-nums">
            남은 잠금 {formatClock(item.remainingSec)}
          </span>
        </li>
      ))}
    </ul>
  );
}
```

LockList: 잠금 중 시도 한 줄 = 시각 + "남은 잠금 12:30".

```
function Metric({ label, value }: { label: string; value: string }) {
  return (
    <Card size="sm">
      <CardHeader>
        <CardTitle className="text-xs font-medium text-muted-foreground">{label}</CardTitle>
      </CardHeader>
      <CardContent>
        <p className="text-xl font-semibold tracking-tight">{value}</p>
      </CardContent>
    </Card>
  );
}

function Field({ label, value }: { label: string; value: string }) {
  return (
    <div>
      <dt className="text-xs text-muted-foreground">{label}</dt>
      <dd className="mt-0.5">{value}</dd>
    </div>
  );
}
```

Metric: 작은 카드 하나 (이름 + 큰 값). **Field**: 이름(`dt`) + 값(`dd`) 한 칸.

ui/badge.tsx — 배지 (작은 이름표) `components/ui/badge.tsx (51줄)`

components/ui/ 폴더란? 버튼, 카드, 입력 칸 같은 **기본 부품**. 직접 처음부터 쓴 것이 아니라 **shadcn/ui**라는 무료 부품 모음에서 명령어로 복사해 온 코드다 (components.json에 설정). 동작(키보드, 접근성)은 **Base UI** 라이브러리가 맡고, 모양은 Tailwind 모양 이름으로 입힌다. 긴 `className` 줄은 "기본, 마우스 올림, 초점, 비활성, 어두운 모드일 때 모양"을 한 줄에 적은 것이라 한 단어씩 읽을 필요는 없다.

```tsx
import { mergeProps } from "@base-ui/react/merge-props"
import { useRender } from "@base-ui/react/use-render"
import { cva, type VariantProps } from "class-variance-authority"
import { cn } from "cn"

const badgeVariants = cva(
  "group/badge inline-flex h-5 w-fit shrink-0 items-center justify-center gap-1 overflow-hidden rounded-4xl border border-transparent px-2 py-0.5 text-xs font-medium whitespace-nowrap transition-all focus-visible:border-ring focus-visible:ring-[3px] focus-visible:ring-ring/50 has-data-[icon=inline-end]:pr-1.5 has-data-[icon=inline-start]:pl-1.5 aria-invalid:border-destructive aria-invalid:ring-destructive/20 dark:aria-invalid:ring-destructive/40 [&>svg]:pointer-events-none [&>svg]:size-3!",
  {
    variants: {
      variant: {
        default: "bg-primary text-primary-foreground [a]:hover:bg-primary/80",
        secondary:
          "bg-secondary text-secondary-foreground [a]:hover:bg-secondary/80",
        destructive:
          "bg-destructive/10 text-destructive focus-visible:ring-destructive/20 dark:bg-destructive/20 dark:focus-visible:ring-destructive/40 [a]:hover:bg-destructive/20",
        outline:
          "border-border text-foreground [a]:hover:bg-muted [a]:hover:text-muted-foreground",
        ghost:
          "hover:bg-muted hover:text-muted-foreground dark:hover:bg-muted/50",
        link: "text-primary underline-offset-4 hover:underline",
      },
    },
    defaultVariants: {
      variant: "default",
    },
  }
)
```

1~4줄 가져오기: 값 합치기, "어떤 태그로 그릴지" 도구, **cva**(종류별 모양 표를 만드는 도구), 모양 합치기.

6~27줄 badgeVariants: 공통 모양(높이 20px, 둥근 알약, 작은 글씨) + 종류별 모양: default(주 색), secondary(옅은 색), destructive(빨강), outline(테두리만), ghost, link. 기본은 default. 홈 화면 단계 배지(강함이면 destructive)에 쓰인다.

```tsx
function Badge({
  className,
  variant = "default",
  render,
  ...props
}: useRender.ComponentProps<"span"> & VariantProps<typeof badgeVariants>) {
  return useRender({
    defaultTagName: "span",
    props: mergeProps<"span">(
```

Badge: 종류에 맞는 모양 이름을 만들어 `<span>`으로 그린다. `render`를 주면 다른 태그(예: 링크)로도 그릴 수 있다. `...props`는 "나머지 받은 값 전부"를 그대로 넘기는 표시.

```
      {
        className: cn(badgeVariants({ variant }), className),
      },
      props
    ),
    render,
    state: {
      slot: "badge",
      variant,
    },
  })
}

export { Badge, badgeVariants }
```

ui/button.tsx — 버튼 `components/ui/button.tsx (57줄)`

```
import { Button as ButtonPrimitive } from "@base-ui/react/button"
import { cva, type VariantProps } from "class-variance-authority"
import { cn } from "cn"

const buttonVariants = cva(
  "group/button inline-flex shrink-0 items-center justify-center rounded-lg border border-transparent bg-clip-padding text-sm font-medium whitespace-nowrap transition-all outline-none select-none focus-visible:border-ring focus-visible:ring-3 focus-visible:ring-ring/50 active:not-aria-[haspopup]:translate-y-px disabled:pointer-events-none disabled:opacity-50 aria-invalid:border-destructive aria-invalid:ring-3 aria-invalid:ring-destructive/20 dark:aria-invalid:border-destructive/50 dark:aria-invalid:ring-destructive/40 [&_svg]:pointer-events-none [&_svg]:shrink-0 [&_svg:not([class*='size-'])]:size-4",
  {
    variants: {
      variant: {
        default: "bg-primary text-primary-foreground hover:bg-primary/80",
        outline:
          "border-border bg-background hover:bg-muted hover:text-foreground aria-expanded:bg-muted aria-expanded:text-foreground dark:border-input dark:bg-input/30 dark:hover:bg-input/50",
        secondary:
          "bg-secondary text-secondary-foreground hover:bg-[color-mix(in_oklch,var(--secondary),var(--foreground)_5%)] aria-expanded:bg-secondary aria-expanded:text-secondary-foreground",
        ghost:
          "hover:bg-muted hover:text-foreground aria-expanded:bg-muted aria-expanded:text-foreground dark:hover:bg-muted/50",
        destructive:
          "bg-destructive/10 text-destructive hover:bg-destructive/20 focus-visible:border-destructive/40 focus-visible:ring-destructive/20 dark:bg-destructive/20 dark:hover:bg-destructive/30 dark:focus-visible:ring-destructive/40",
        link: "text-primary underline-offset-4 hover:underline",
      },
      size: {
        default:
```

buttonVariants: 공통 모양(가운데 정렬, 둥근 모서리, 눌리면 1px 내려감, 비활성이면 반투명) + 종류(default 주 색, outline 테두리, secondary, ghost 배경 없음, destructive 빨강, link 밑줄) + 크기(default 32px, xs, sm, lg, 아이콘용 정사각형들).

```
          "h-8 gap-1.5 px-2.5 has-data-[icon=inline-end]:pr-2 has-data-[icon=inline-start]:pl-2",
        xs: "h-6 gap-1 rounded-[min(var(--radius-md),10px)] px-2 text-xs in-data-[slot=button-group]:rounded-lg has-data-[icon=inline-end]:pr-1.5 has-data-[icon=inline-start]:pl-1.5 [&_svg:not([class*='size-'])]:size-3",
        sm: "h-7 gap-1 rounded-[min(var(--radius-md),12px)] px-2.5 text-[0.8rem] in-data-[slot=button-group]:rounded-lg has-data-[icon=inline-end]:pr-1.5 has-data-[icon=inline-start]:pl-1.5 [&_svg:not([class*='size-'])]:size-3.5",
        lg: "h-9 gap-1.5 px-2.5 has-data-[icon=inline-end]:pr-2 has-data-[icon=inline-start]:pl-2",
        icon: "size-8",
        "icon-xs":
          "size-6 rounded-[min(var(--radius-md),10px)] in-data-[slot=button-group]:rounded-lg [&_svg:not([class*='size-'])]:size-3",
        "icon-sm":
          "size-7 rounded-[min(var(--radius-md),12px)] in-data-[slot=button-group]:rounded-lg",
        "icon-lg": "size-9",
      },
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  }
)

function Button({
  className,
  variant = "default",
  size = "default",
  ...props
}: ButtonPrimitive.Props & VariantProps<typeof buttonVariants>) {
  return (
    <ButtonPrimitive
      data-slot="button"
      className={cn(buttonVariants({ variant, size, className }))}
      {...props}
    />
  )
}

export { Button, buttonVariants }
```

Button: Base UI의 버튼(키보드·접근성 처리됨)에 종류·크기 모양을 입혀 그린다. 앱 곳곳의 `<Button variant="outline" className="h-11">`은 여기서 모양을 받고, `h-11`처럼 넘긴 값이 기본 높이를 덮어쓴다.

ui/card.tsx — 카드 (둥근 상자) `components/ui/card.tsx (102줄)`

```
import * as React from "react"
import { cn } from "cn"

function Card({
```

Card: 둥근 모서리, 카드 배경, 옅은 테두리의 상자. `size="sm"`이면 안쪽 여백이 작아진다 (`data-size` 표시를 보고 CSS가 간격을 바꿈).

```
  className,
  size = "default",
  ...props
}: React.ComponentProps<"div"> & { size?: "default" | "sm" }) {
  return (
    <div
      data-slot="card"
      data-size={size}
      className={cn(
        "group/card flex flex-col gap-(--card-spacing) overflow-hidden rounded-xl bg-card py-(--card-spacing) text-sm text-card-foreground ring-1 ring-foreground/10 [--card-spacing:--spacing(4)] has-data-[slot=card-footer]:pb-0 has-[>img:first-child]:pt-0 data-[size=sm]:[--card-spacing:--spacing(3)] data-[size=sm]:has-data-[slot=card-footer]:pb-0 *:[img:first-child]:rounded-t-xl *:[img:last-child]:rounded-b-xl",
        className
      )}
      {...props}
    />
  )
}
```

```
function CardHeader({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="card-header"
      className={cn(
        "group/card-header @container/card-header grid auto-rows-min items-start gap-1 rounded-t-xl px-(--card-spacing) has-data-[slot=card-action]:grid-cols-[1fr_auto] has-data-[slot=card-description]:grid-rows-[auto_auto] [.border-b]:pb-(--card-spacing)",
        className
      )}
      {...props}
    />
  )
}

function CardTitle({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="card-title"
      className={cn(
        "font-heading text-base leading-snug font-medium group-data-[size=sm]/card:text-sm",
        className
      )}
      {...props}
    />
  )
}

function CardDescription({ className, ...props }: React.ComponentProps<"div">) {
  return (
```

CardHeader(머리 칸, 격자), **CardTitle**(제목 글씨, 작은 카드면 더 작게), **CardDescription**(흐린 설명 글씨).

```
    <div
      data-slot="card-description"
      className={cn("text-sm text-muted-foreground", className)}
      {...props}
    />
  )
}
```

```
function CardAction({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="card-action"
      className={cn(
        "col-start-2 row-span-2 row-start-1 self-start justify-self-end",
        className
      )}
      {...props}
    />
  )
}

function CardContent({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="card-content"
      className={cn("px-(--card-spacing)", className)}
      {...props}
    />
  )
}

function CardFooter({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="card-footer"
      className={cn(
        "flex items-center rounded-b-xl border-t bg-muted/50 p-(--card-spacing)",
        className
      )}
      {...props}
    />
  )
}

export {
  Card,
  CardHeader,
  CardFooter,
  CardTitle,
  CardAction,
  CardDescription,
```

CardAction(머리 오른쪽 버튼 자리), **CardContent**(본문, 좌우 여백), **CardFooter**(아래 띠, 위 테두리). 마지막에 7개를 내보낸다. 이 앱은 Card, CardHeader, CardTitle, CardContent를 쓴다.

```
  CardContent,
}
```

ui/input.tsx — 한 줄 입력 칸 `components/ui/input.tsx (19줄)`

```tsx
import * as React from "react"
import { Input as InputPrimitive } from "@base-ui/react/input"
import { cn } from "cn"

function Input({ className, type, ...props }: React.ComponentProps<"input">) {
  return (
    <InputPrimitive
      type={type}
      data-slot="input"
      className={cn(
        "h-8 w-full min-w-0 rounded-lg border border-input bg-transparent px-2.5 py-1 text-base transition-colors outline-none file:inline-flex file:h-6 file:border-0 file:bg-transparent file:text-sm file:font-medium file:text-foreground placeholder:text-muted-foreground focus-visible:border-ring focus-visible:ring-3 focus-visible:ring-ring/50 disabled:pointer-events-none disabled:cursor-not-allowed disabled:bg-input/50 disabled:opacity-50 aria-invalid:border-destructive aria-invalid:ring-3 aria-invalid:ring-destructive/20 md:text-sm dark:bg-input/30 dark:disabled:bg-input/80 dark:aria-invalid:border-destructive/50 dark:aria-invalid:ring-destructive/40",
        className
      )}
      {...props}
    />
  )
}

export { Input }
```

Input: Base UI의 입력 칸에 모양을 입힌다: 높이 32px, 테두리, 흐린 안내 글씨, 초점이면 테두리 강조, 비활성이면 반투명, `aria-invalid`(잘못된 입력)이면 빨간 테두리. 폰에서는 글씨 16px(`text-base`) — 이보다 작으면 iPhone이 입력할 때 화면을 확대해 버리기 때문.

ui/label.tsx — 입력 칸 이름표 `components/ui/label.tsx (19줄)`

```tsx
"use client"

import * as React from "react"
import { cn } from "cn"

function Label({ className, ...props }: React.ComponentProps<"label">) {
  return (
    <label
      data-slot="label"
      className={cn(
        "flex items-center gap-2 text-sm leading-none font-medium select-none group-data-[disabled=true]:pointer-events-none group-data-[disabled=true]:opacity-50 peer-disabled:cursor-not-allowed peer-disabled:opacity-50",
        className
```

Label: `<label>`에 모양을 입힌다. `htmlFor="goal"`로 입력 칸과 연결하면, 이름을 눌러도 칸에 초점이 가고 화면 읽기 도구가 칸 이름을 읽어 준다. 연결된 칸이 비활성이면 반투명.

```
      )}
      {...props}
    />
  )
}

export { Label }
```

ui/progress.tsx — 진행 막대 `components/ui/progress.tsx (82줄)`

```tsx
"use client"

import { Progress as ProgressPrimitive } from "@base-ui/react/progress"
import { cn } from "cn"

function Progress({
  className,
  children,
  value,
  ...props
}: ProgressPrimitive.Root.Props) {
  return (
    <ProgressPrimitive.Root
      value={value}
      data-slot="progress"
      className={cn("flex flex-wrap gap-3", className)}
      {...props}
    >
      {children}
      <ProgressTrack>
        <ProgressIndicator />
      </ProgressTrack>
    </ProgressPrimitive.Root>
  )
}
```

Progress: 값(0~100)을 받아 바깥 틀 안에 이름(children)과 막대를 그린다. 대체 행동 "안내 시간" 막대에 쓰인다.

```tsx
function ProgressTrack({ className, ...props }: ProgressPrimitive.Track.Props) {
  return (
    <ProgressPrimitive.Track
      className={cn(
        "relative flex h-1 w-full items-center overflow-x-hidden rounded-full bg-muted",
        className
      )}
      data-slot="progress-track"
      {...props}
    />
  )
}

```

ProgressTrack: 회색 바탕 줄 (높이 4px, 둥근 끝). **ProgressIndicator**: 채워지는 부분 (주 색, 부드럽게 늘어남). 채움 너비는 Base UI가 값으로 계산한다.

```tsx
function ProgressIndicator({
  className,
  ...props
}: ProgressPrimitive.Indicator.Props) {
  return (
    <ProgressPrimitive.Indicator
      data-slot="progress-indicator"
      className={cn("h-full bg-primary transition-all", className)}
      {...props}
    />
  )
}

function ProgressLabel({ className, ...props }: ProgressPrimitive.Label.Props) {
  return (
    <ProgressPrimitive.Label
      className={cn("text-sm font-medium", className)}
      data-slot="progress-label"
      {...props}
    />
  )
}

function ProgressValue({ className, ...props }: ProgressPrimitive.Value.Props) {
  return (
    <ProgressPrimitive.Value
      className={cn(
        "ml-auto text-sm text-muted-foreground tabular-nums",
        className
      )}
      data-slot="progress-value"
      {...props}
    />
  )
}

export {
  Progress,
  ProgressTrack,
  ProgressIndicator,
  ProgressLabel,
  ProgressValue,
}
```

ProgressLabel: 막대 이름 글씨. **ProgressValue**: 오른쪽 숫자 (이 앱에선 안 씀). 다섯 개를 내보낸다.

ui/separator.tsx — 구분선 `components/ui/separator.tsx (24줄)`

```tsx
"use client"

import { Separator as SeparatorPrimitive } from "@base-ui/react/separator"
```

Separator: 1px 회색 선. 기본 가로, `orientation="vertical"`이면 세로. 설정 화면에서 알림 권한과 기록 초기화 사이에 쓰인다. 화면 읽기 도구에는 "구분

```tsx
import { cn } from "cn"

function Separator({
  className,
  orientation = "horizontal",
  ...props
}: SeparatorPrimitive.Props) {
  return (
    <SeparatorPrimitive
      data-slot="separator"
      orientation={orientation}
      className={cn(
        "shrink-0 bg-border data-horizontal:h-px data-horizontal:w-full data-vertical:w-px data-vertical:self-stretch",
        className
      )}
      {...props}
    />
  )
}

export { Separator }
```

선"으로 알려진다.

ui/switch.tsx — 켜기/끄기 스위치 components/ui/switch.tsx (31줄)

```tsx
"use client"

import { Switch as SwitchPrimitive } from "@base-ui/react/switch"
import { cn } from "cn"

function Switch({
  className,
  size = "default",
  ...props
}: SwitchPrimitive.Root.Props & {
  size?: "sm" | "default"
}) {
  return (
    <SwitchPrimitive.Root
      data-slot="switch"
      data-size={size}
      className={cn(
        "peer group/switch relative inline-flex shrink-0 items-center rounded-full border border-transparent transition-all outline-none group-has-[:focus-visible]/field-label:border-transparent group-has-[:focus-visible]/field-label:ring-0 after:absolute after:-inset-x-3 after:-inset-y-2 focus-visible:border-ring focus-visible:ring-3 focus-visible:ring-ring/50 aria-invalid:border-destructive aria-invalid:ring-3 aria-invalid:ring-destructive/20 data-[size=default]:h-[18.4px] data-[size=default]:w-[32px] data-[size=sm]:h-[14px] data-[size=sm]:w-[24px] dark:aria-invalid:border-destructive/50 dark:aria-invalid:ring-destructive/40 data-checked:bg-primary data-unchecked:bg-input dark:data-unchecked:bg-input/80
```

Switch: Base UI 스위치(Root = 바깥 알약, Thumb = 움직이는 동그라미)에 모양을 입힌다. 켜짐(`data-checked`)이면 주 색 바탕에 동그라미가 오른쪽으로, 꺼짐이면 회색에 왼쪽. **18줄** `after:-inset-x-3 -inset-y-2`로 눌리는 영역을 보이는 것보다 넓혀 손가락으로 누르기 쉽게 한다. 개입, AI, 엄격 모드, 공부 모드, 알림 스위치에 쓰인다.

```
data-disabled:cursor-not-allowed data-disabled:opacity-50",
        className
      )}
      {...props}
    >
      <SwitchPrimitive.Thumb
        data-slot="switch-thumb"
        className="pointer-events-none block rounded-full bg-background ring-0 transition-transform group-data-[size=default]/switch:size-4 group-data-[size=sm]/switch:size-3 group-data-[size=default]/switch:data-checked:translate-x-[calc(100%-2px)] group-data-[size=sm]/switch:data-checked:translate-x-[calc(100%-2px)] dark:data-checked:bg-primary-foreground group-data-[size=default]/switch:data-unchecked:translate-x-0 group-data-[size=sm]/switch:data-unchecked:translate-x-0 dark:data-unchecked:bg-foreground"
      />
    </SwitchPrimitive.Root>
  )
}

export { Switch }
```

ui/textarea.tsx — 여러 줄 입력 칸 `components/ui/textarea.tsx (17줄)`

```tsx
import * as React from "react"
import { cn } from "cn"

function Textarea({ className, ...props }: React.ComponentProps<"textarea">) {
  return (
    <textarea
      data-slot="textarea"
      className={cn(
        "flex field-sizing-content min-h-16 w-full rounded-lg border border-input bg-transparent px-2.5 py-2 text-base transition-colors outline-none placeholder:text-muted-foreground focus-visible:border-ring focus-visible:ring-3 focus-visible:ring-ring/50 disabled:cursor-not-allowed disabled:bg-input/50 disabled:opacity-50 aria-invalid:border-destructive aria-invalid:ring-3 aria-invalid:ring-destructive/20 md:text-sm dark:bg-input/30 dark:disabled:bg-input/80 dark:aria-invalid:border-destructive/50 dark:aria-invalid:ring-destructive/40",
        className
      )}
      {...props}
    />
  )
}

export { Textarea }
```

Textarea: 여러 줄 입력 칸. `field-sizing-content`로 글이 늘어나면 칸도 자동으로 커진다 (최소 64px). 나머지 모양은 Input과 같다. 카드 문제·해설·붙여넣기, "기타 이유"에 쓰인다.

6부. 화면 주소와 전체 틀 (app/, Next.js)

`app/` 폴더는 Next.js의 **"폴더 = 주소"** 규칙을 따른다. `app/page.tsx`는 `/`(홈), `app/stats/page.tsx`는 `/stats`. 각 page는 화면 부품 하나만 부르는 얇은 파일이고, 실제 내용은 components/에 있다. 빌드할 때 Next.js가 주소마다 HTML 파일을 미리 만들어(정적 내보내기, `out/` 폴더) Capacitor가 APK 안에 넣는다.

layout.tsx — 모든 화면의 바깥 틀 `app/layout.tsx (61줄)`

```tsx
import type { Metadata, Viewport } from "next";
import "@fontsource/noto-sans-kr/korean-400.css";
import "@fontsource/noto-sans-kr/korean-500.css";
import "@fontsource/noto-sans-kr/korean-700.css";
import "@fontsource/noto-sans-kr/latin-400.css";
import "@fontsource/noto-sans-kr/latin-500.css";
import "@fontsource/noto-sans-kr/latin-700.css";
import { AppShell } from "@/components/app-shell";
import { GuardBridge } from "@/components/guard-bridge";
import { PwaRegister } from "@/components/pwa-register";
import { StoreProvider } from "@/components/store";
import "./globals.css";
```

1줄 페이지 정보 모양. **2~7줄** 한글 글꼴 Noto Sans KR(보통·중간·굵게, 한글·영문)을 **앱 안에 포함**해서 불러온다 (인터넷 없이도 같은 글꼴). **8~11줄** 앱 틀, 다리, PWA 등록, 창고. **12줄** 전체 CSS.

```tsx
export const metadata: Metadata = {
  title: {
    default: "Focus on",
    template: "%s · Focus on",
  },
  description:
    "YouTube Shorts를 열기 전에 이유를 묻고, 오늘 사용량에 따라 단계적으로 개입하는 사용 조절 앱.",
  applicationName: "Focus on",
  manifest: "/manifest.webmanifest",
  appleWebApp: {
    capable: true,
    title: "Focus on",
    statusBarStyle: "default",
  },
  icons: {
    icon: [
      { url: "/icons/icon-192.png", sizes: "192x192", type: "image/png" },
      { url: "/icons/icon-512.png", sizes: "512x512", type: "image/png" },
    ],
    apple: [{ url: "/icons/apple-touch-icon.png", sizes: "180x180", type: "image/png" }],
  },
};
```

metadata — 페이지 정보: 탭 제목 "Focus on" (각 화면은 "통계 · Focus on" 형태), 설명, 앱 이름, PWA 설명서(manifest) 위치, 아이폰 홈 화면 앱 설정, 아이콘 (192, 512, 아이폰 180).

```tsx
export const viewport: Viewport = {
  width: "device-width",
  initialScale: 1,
```

viewport: 폰 화면 너비에 맞추고 확대 1배, 브라우저 위 띠 색 #f4efe4 (앱 배경 베이지).

```
  themeColor: "#f4efe4",
};
```

```
export default function RootLayout({
  children,
}: Readonly<{ children: React.ReactNode }>) {
  return (
    <html lang="ko" className="h-full antialiased">
      <head>
        <meta name="apple-mobile-web-app-capable" content="yes" />
      </head>
      <body className="min-h-full">
        <noscript>Focus on은 자바스크립트가 필요합니다.</noscript>
        <StoreProvider>
          <PwaRegister />
          <GuardBridge />
          <AppShell>{children}</AppShell>
        </StoreProvider>
      </body>
    </html>
  );
}
```

RootLayout — 감싸는 순서가 중요하다.

1. `<html lang="ko">`: 한국어 페이지 (화면 읽기 도구 발음).
2. `noscript`: 자바스크립트가 꺼져 있으면 안내.
3. **StoreProvider**가 가장 바깥 → 안쪽 모든 부품이 기록 창고를 쓸 수 있다.
4. 그 안에 PwaRegister, **GuardBridge**(화면 없이 1.5초마다 Java와 맞춤), AppShell(메뉴 틀) 안에 각 화면(children).

그래서 어느 화면에 있든 다리는 계속 돌아간다.

page.tsx — 홈 주소 (/) `app/page.tsx (10줄)`

```
import type { Metadata } from "next";
import { HomeView } from "@/components/home-view";

export const metadata: Metadata = {
  title: "홈",
};

export default function HomePage() {
  return <HomeView />;
}
```

탭 제목 "홈", 내용은 HomeView. `export default`는 "이 파일의 대표"라는 표시로, Next.js가 이 함수를 그 주소의 화면으로 쓴다. 아래 4개 page 파일도 모양이 똑같다.

stats/page.tsx — /stats `app/stats/page.tsx (10줄)`

```
import type { Metadata } from "next";
import { StatsView } from "@/components/stats-view";

export const metadata: Metadata = {
  title: "통계",
};

export default function StatsPage() {
```

제목 "통계", 내용 StatsView.

```
  return <StatsView />;
}
```

report/page.tsx — /report `app/report/page.tsx (10줄)`

```tsx
import type { Metadata } from "next";
import { ReportView } from "@/components/report-view";

export const metadata: Metadata = {
  title: "리포트",
};

export default function ReportPage() {
  return <ReportView />;
}
```

제목 "리포트", 내용 ReportView. 리포트 알림을 누르면 이 주소가 열린다.

settings/page.tsx — /settings `app/settings/page.tsx (10줄)`

```tsx
import type { Metadata } from "next";
import { SettingsView } from "@/components/settings-view";

export const metadata: Metadata = {
  title: "설정",
};

export default function SettingsPage() {
  return <SettingsView />;
}
```

제목 "설정", 내용 SettingsView.

watch/page.tsx — /watch `app/watch/page.tsx (10줄)`

```tsx
import type { Metadata } from "next";
import { WatchView } from "@/components/watch-view";

export const metadata: Metadata = {
  title: "개입",
};

export default function WatchPage() {
  return <WatchView />;
}
```

제목 "개입", 내용 WatchView. 서비스가 넘긴 개입이면 `/watch?guard=youtube` 처럼 열린다.

error.tsx — 화면 오류 대비 `app/error.tsx (26줄)`

```tsx
"use client";

export default function Error({
  reset,
}: {
  error: Error & { digest?: string };
  reset: () => void;
}) {
  return (
    <div className="flex min-h-[60vh] items-center justify-center">
      <div className="max-w-md space-y-4">
        <h1 className="text-2xl font-semibold tracking-tight">화면을 표시하지 못했습니다.</h1>
        <p className="text-sm leading-6 text-muted-foreground">
          잠시 문제가 생겼습니다. 다시 시도해도 기록이 지워지지는 않습니다.
        </p>
        <button
          type="button"
          onClick={reset}
          className="inline-flex h-11 items-center rounded-xl bg-primary px-4 text-sm font-mediu
m text-primary-foreground"
        >
          다시 시도
        </button>
      </div>
    </div>
  );
}
```

화면을 그리다 예상 못 한 오류가 나면 Next.js가 앱 전체를 하얗게 만드는 대신 이 화면을 보여 준다: "화면을 표시하지 못했습니다. 다시 시도해도 기록이 지워지지는 않습니다." + [다시 시도](reset = 그 화면만 다시 그리기). `error` 값은 받지만 화면에는 내부 오류 내용을 보이지 않는다.

globals.css — 전체 색과 모양 `app/globals.css (136줄)`

CSS는 화면 모양(색, 글꼴, 둥글기)을 정하는 언어다. 이 파일은 **색 이름표**를 정해 두고, 부품들은 `bg-primary`처럼 이름으로만 색을 쓴다. 색을 바꾸려면 여기만 고치면 된다.

```css
@import "tailwindcss";
@import "tw-animate-css";
@import "shadcn/tailwind.css";

@custom-variant dark (&:is(.dark *));
```

1~3줄 Tailwind(모양 이름 도구), 움직임 효과 모음, shadcn 기본 CSS를 불러온다. **5줄** `.dark` 안에서는 어두운 모드 모양을 쓰겠다는 규칙.

```css
@theme inline {
  --color-background: var(--background);
  --color-foreground: var(--foreground);
  --font-sans: "Noto Sans KR", "WenQuanYi Micro Hei", ui-sans-serif, sans-serif;
  --font-mono: var(--font-geist-mono);
  --color-level-1: oklch(0.46 0.07 155);
  --color-level-2: oklch(0.58 0.12 70);
```

@theme — Tailwind에 이름 알려 주기. `--color-primary`를 정하면 `bg-primary`, `text-primary` 같은 이름이 생긴다.

- **10, 15줄** 글꼴: Noto Sans KR 먼저.

```
  --color-level-3: oklch(0.5 0.15 32);
  --font-heading: "Noto Sans KR", "WenQuanYi Micro Hei", ui-sans-serif, sans-serif;
  --color-sidebar-ring: var(--sidebar-ring);
  --color-sidebar-border: var(--sidebar-border);
  --color-sidebar-accent-foreground: var(--sidebar-accent-foreground);
  --color-sidebar-accent: var(--sidebar-accent);
  --color-sidebar-primary-foreground: var(--sidebar-primary-foreground);
  --color-sidebar-primary: var(--sidebar-primary);
  --color-sidebar-foreground: var(--sidebar-foreground);
  --color-sidebar: var(--sidebar);
  --color-chart-5: var(--chart-5);
  --color-chart-4: var(--chart-4);
  --color-chart-3: var(--chart-3);
  --color-chart-2: var(--chart-2);
  --color-chart-1: var(--chart-1);
  --color-ring: var(--ring);
  --color-input: var(--input);
  --color-border: var(--border);
  --color-destructive: var(--destructive);
  --color-accent-foreground: var(--accent-foreground);
  --color-accent: var(--accent);
  --color-muted-foreground: var(--muted-foreground);
  --color-muted: var(--muted);
  --color-secondary-foreground: var(--secondary-foreground);
  --color-secondary: var(--secondary);
  --color-primary-foreground: var(--primary-foreground);
  --color-primary: var(--primary);
  --color-popover-foreground: var(--popover-foreground);
  --color-popover: var(--popover);
  --color-card-foreground: var(--card-foreground);
  --color-card: var(--card);
  --radius-sm: calc(var(--radius) * 0.6);
  --radius-md: calc(var(--radius) * 0.8);
  --radius-lg: var(--radius);
  --radius-xl: calc(var(--radius) * 1.4);
  --radius-2xl: calc(var(--radius) * 1.8);
  --radius-3xl: calc(var(--radius) * 2.2);
  --radius-4xl: calc(var(--radius) * 2.6);
}
```

- **12~14줄 단계 색**(이 앱에서 직접 정함): 약함 초록, 중간 주황, 강함 붉은색. 홈 막대와 개입 화면 글씨에 쓰인다.
- 나머지는 아래 :root 값을 가리키는 연결(색, 차트, 사이드바)과 둥글기 단계(기본 0.625rem의 0.6배 ~ 2.6배).

```
:root {
  --background: oklch(0.968 0.014 85);
  --foreground: oklch(0.25 0.02 55);
  --card: oklch(0.992 0.006 90);
  --card-foreground: oklch(0.25 0.02 55);
  --popover: oklch(0.992 0.006 90);
  --popover-foreground: oklch(0.25 0.02 55);
  --primary: oklch(0.48 0.13 35);
  --primary-foreground: oklch(0.98 0.01 85);
  --secondary: oklch(0.94 0.018 85);
  --secondary-foreground: oklch(0.28 0.02 55);
```

:root — 밝은 모드 실제 색. `oklch(밝기 채도 색상)` 형식. 배경은 따뜻한 베이지(0.968, 색상 85), 글씨는 짙은 갈색, **주 색(primary)은 벽돌색**(색상 35), 빨강(destructive), 테두리, 초점 테두리. 차트·사이드바 색은 부품 모음 기본값 그대로(이 앱에서 거의 안 씀).

```
  --muted: oklch(0.935 0.016 85);
  --muted-foreground: oklch(0.48 0.02 55);
  --accent: oklch(0.93 0.03 150);
  --accent-foreground: oklch(0.32 0.04 155);
  --destructive: oklch(0.52 0.17 28);
  --border: oklch(0.88 0.02 80);
  --input: oklch(0.88 0.02 80);
  --ring: oklch(0.55 0.1 35);
  --chart-1: oklch(0.87 0 0);
  --chart-2: oklch(0.556 0 0);
  --chart-3: oklch(0.439 0 0);
  --chart-4: oklch(0.371 0 0);
  --chart-5: oklch(0.269 0 0);
  --radius: 0.625rem;
  --sidebar: oklch(0.985 0 0);
  --sidebar-foreground: oklch(0.145 0 0);
  --sidebar-primary: oklch(0.205 0 0);
  --sidebar-primary-foreground: oklch(0.985 0 0);
  --sidebar-accent: oklch(0.97 0 0);
  --sidebar-accent-foreground: oklch(0.205 0 0);
  --sidebar-border: oklch(0.922 0 0);
  --sidebar-ring: oklch(0.708 0 0);
}
```

```
.dark {
  --background: oklch(0.145 0 0);
  --foreground: oklch(0.985 0 0);
  --card: oklch(0.205 0 0);
  --card-foreground: oklch(0.985 0 0);
  --popover: oklch(0.205 0 0);
  --popover-foreground: oklch(0.985 0 0);
  --primary: oklch(0.922 0 0);
  --primary-foreground: oklch(0.205 0 0);
  --secondary: oklch(0.269 0 0);
  --secondary-foreground: oklch(0.985 0 0);
  --muted: oklch(0.269 0 0);
  --muted-foreground: oklch(0.708 0 0);
  --accent: oklch(0.269 0 0);
  --accent-foreground: oklch(0.985 0 0);
  --destructive: oklch(0.704 0.191 22.216);
  --border: oklch(1 0 0 / 10%);
  --input: oklch(1 0 0 / 15%);
  --ring: oklch(0.556 0 0);
  --chart-1: oklch(0.87 0 0);
  --chart-2: oklch(0.556 0 0);
  --chart-3: oklch(0.439 0 0);
  --chart-4: oklch(0.371 0 0);
  --chart-5: oklch(0.269 0 0);
  --sidebar: oklch(0.205 0 0);
  --sidebar-foreground: oklch(0.985 0 0);
  --sidebar-primary: oklch(0.488 0.243 264.376);
```

.dark — 어두운 모드 색. 부품 모음 기본값 (회색 계열). 이 앱은 지금 어두운 모드를 켜지 않으므로 실제로는 쓰이지 않는다.

```
  --sidebar-primary-foreground: oklch(0.985 0 0);
  --sidebar-accent: oklch(0.269 0 0);
  --sidebar-accent-foreground: oklch(0.985 0 0);
  --sidebar-border: oklch(1 0 0 / 10%);
  --sidebar-ring: oklch(0.556 0 0);
}
```

```
@layer base {
  * {
    @apply border-border outline-ring/50;
  }
  body {
    @apply bg-background text-foreground;
  }
  html {
    @apply font-sans;
  }
  ::selection {
    background: oklch(0.86 0.05 40);
  }
}
```

@layer base — 기본 모양. 모든 요소의 테두리 색과 초점 테두리, body 배경·글씨 색, html 글꼴, 글자를 드래그해 선택했을 때 색(옅은 벽돌색).

7부. 빌드 도구 스크립트 (scripts/)

앱 자체가 아니라 **앱을 만들 때 쓰는 도구**. `npm run apk:debug` 같은 명령이 이 파일들을 실행한다 (package.json의 scripts 칸). `.mjs`는 Node.js(컴퓨터에서 도는 JavaScript)용 파일, `.py`는 Python 파일이다.

build-apk.mjs — APK 한 번에 만들기 `scripts/build-apk.mjs (58줄)`

`npm run apk:debug` 하나로 웹 빌드 → Capacitor 복사 → Gradle 빌드 → release 폴더 복사 → 지문(SHA-256) 출력까지 한다.

```
// npm run apk        → 서명된 release APK (android/keystore.properties 필요)
// npm run apk:debug  → 디버그 APK (테스트용, 로그 태그 ShortsAIGuard 출력)
// 웹 빌드 → cap sync → gradle → release/shortsai-v<버전>[-debug].apk
import { execSync } from "node:child_process";
import { createHash } from "node:crypto";
import { copyFileSync, existsSync, mkdirSync, readFileSync, statSync } from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";

const root = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");
const mode = process.argv[2] === "debug" ? "debug" : "release";
const { version } = JSON.parse(readFileSync(path.join(root, "package.json"), "utf8"));
const android = path.join(root, "android");
```

1~3줄 사용법 설명. **4~8줄** Node 기본 도구: 명령 실행, 지문 계산, 파일 읽기·복사, 경로.

- **10줄** 프로젝트 폴더 (이 파일의 한 단계 위).
- **11줄** `node build-apk.mjs debug`면 디버그, 아니면 release.
- **12줄** package.json에서 버전(예: 0.9.2)을 읽는다 → 파일 이름에 쓰임.

```
if (mode === "release" && !existsSync(path.join(android, "keystore.properties"))) {
  console.error(`
android/keystore.properties 가 없어 서명된 APK를 만들 수 없습니다.

1. 키스토어를 직접 만듭니다 (비밀번호는 본인만 알게 정하세요):
   keytool -genkeypair -v -keystore android/shortsai-release.jks -alias shortsai -keyalg RSA -keysize 2048 -validity 10000

2. android/keystore.properties 를 만들고 아래처럼 적습니다 (git에 올라가지 않습니다):
   storeFile=shortsai-release.jks
   storePassword=<키스토어 비밀번호>
   keyAlias=shortsai
   keyPassword=<키 비밀번호>

3. .jks 파일과 비밀번호를 따로 백업하세요. 잃어버리면 같은 앱으로 업데이트 설치가 안 됩니다.

테스트용 디버그 APK는 npm run apk:debug 로 만들 수 있습니다.
`);
  process.exit(1);
}
```

release인데 서명 설정이 없으면 멈춤. android/keystore.properties가 없으면 만드는 방법을 안내한다: ① keytool로 키스토어(.jks) 직접 만들기, ② keystore.properties에 파일 이름·비밀번호 적기(git에 안 올라감), ③ .jks와 비밀번호 백업 — 잃으면 같은 앱으로 업데이트가 안 된다. 테스트는 apk:debug로.

```
function run(command, cwd = root) {
  console.log(`\n> ${command}`);
  execSync(command, { cwd, stdio: "inherit" });
}

run("npm run build");
run("npx cap sync android");
const gradlew = path.join(android, process.platform === "win32" ? "gradlew.bat" : "gradlew");
run(`"${gradlew}" ${mode === "release" ? "assembleRelease" : "assembleDebug"} --console=plain`, android);
```

run: 명령을 화면에 보여 주고 실행 (출력도 그대로 보임, 실패하면 멈춤).

1. `npm run build`: Next.js 웹 빌드 → out/ 폴더.
2. `npx cap sync android`: out/을 안드로이드 프로젝트 안으로 복사, 플러그인 연결.
3. Gradle(윈도우는 gradlew.bat)로 `assembleDebug` 또는 `assembleRelease` → APK 생성.

```
const built = path.join(android, "app", "build", "outputs", "apk", mode, `app-${mode}.apk`);
if (!existsSync(built)) {
  console.error(`APK를 찾지 못했습니다: ${built}`);
  process.exit(1);
}
const outDir = path.join(root, "release");
mkdirSync(outDir, { recursive: true });
const name = `shortsai-v${version}${mode === "debug" ? "-debug" : ""}.apk`;
const dest = path.join(outDir, name);
copyFileSync(built, dest);
const sha = createHash("sha256").update(readFileSync(dest)).digest("hex").toUpperCase();
console.log(`\n완료: release/${name}`);
console.log(`크기: ${statSync(dest).size.toLocaleString("en-US")} bytes`);
console.log(`SHA-256: ${sha}`);
```

결과 정리. Gradle이 만든 app-debug.apk를 찾아(없으면 오류), `release/shortsai-v0.9.2-debug.apk` 이름으로 복사하고, 크기와 SHA-256 지문을 출력한다. 지문은 "올린 파일이 내가 만든 파일과 같은지" 확인하는 데 쓴다.

make-icons.py — 아이콘 크기별로 만들기 `scripts/make-icons.py (60줄)`

```
"""Scale assets/launcher-icon.png into the PWA and Android launcher sizes.

Does not redraw the artwork. Every output is that exact image, resized.
"""

from pathlib import Path

from PIL import Image

ROOT = Path(__file__).resolve().parents[1]
SOURCE = ROOT / "assets" / "launcher-icon.png"
PUBLIC = ROOT / "public" / "icons"
RES = ROOT / "android" / "app" / "src" / "main" / "res"

LAUNCHER = {
    "mdpi": 48,
    "hdpi": 72,
    "xhdpi": 96,
    "xxhdpi": 144,
    "xxxhdpi": 192,
}
```

1~4줄 설명: assets/launcher-icon.png 한 장을 PWA와 안드로이드에 필요한 크기로 **줄이기만** 한다 (그림을 새로 그리지 않음).

8줄 Pillow(PIL) 이미지 도구. **10~13줄** 원본, 웹 아이콘 폴더, 안드로이드 리소스 폴더 경로.

15~28줄 안드로이드 화면 밀도별 크기: 일반 아이콘 48~192px, 적응형 아이콘 앞면 108~432px.

```
FOREGROUND = {
    "mdpi": 108,
    "hdpi": 162,
    "xhdpi": 216,
    "xxhdpi": 324,
    "xxxhdpi": 432,
}
```

```
def resized(size: int) -> Image.Image:
    image = Image.open(SOURCE).convert("RGB")
    if image.size == (size, size):
        return image
    return image.resize((size, size), Image.Resampling.LANCZOS)


def write(image: Image.Image, path: Path) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    image.save(path, "PNG")

```

resized: 원본을 열어 정사각형 size로 줄인다 (LANCZOS = 선명하게 줄이는 방법). 이미 그 크기면 그대로. **write**: 폴더를 만들고 PNG로 저장.

```
PUBLIC.mkdir(parents=True, exist_ok=True)
write(resized(192), PUBLIC / "icon-192.png")
write(resized(512), PUBLIC / "icon-512.png")
write(resized(180), PUBLIC / "apple-touch-icon.png")
write(resized(512), PUBLIC / "icon-maskable-512.png")
write(resized(512), ROOT / "app" / "icon.png")

for density, size in LAUNCHER.items():
    folder = RES / f"mipmap-{density}"
    image = resized(size)
    write(image, folder / "ic_launcher.png")
    write(image, folder / "ic_launcher_round.png")

for density, size in FOREGROUND.items():
    folder = RES / f"mipmap-{density}"
    write(resized(size), folder / "ic_launcher_foreground.png")

print(SOURCE)
```

실행 부분. 웹용 192, 512, 아이폰 180, 마스커블 512, 브라우저 탭 아이콘(app/icon.png). 그리고 밀도별 폴더(mipmap-hdpi 등)에 ic_launcher, ic_launcher_round, ic_launcher_foreground. 끝에 원본 경로 출력.

serve-static.mjs — 미리 보기용 작은 서버 `scripts/serve-static.mjs (70줄)`

빌드된 out/ 폴더를 컴퓨터에서 브라우저로 열어 볼 수 있게 하는 아주 작은 웹 서버 (개발 중 확인용, 배포용 아님). `.claude/launch.json`이 이걸로 미리 보기를 띄운다.

```
1  import { createReadStream, statSync } from "node:fs";
2  import { createServer } from "node:http";
3  import path from "node:path";
4
5  const root = path.resolve(process.argv[2] ?? "out");
6  const port = Number(process.env.PORT ?? 47231);
7  const host = process.env.HOST ?? "0.0.0.0";
8
9  const types = {
10   ".html": "text/html; charset=utf-8",
11   ".js": "text/javascript; charset=utf-8",
12   ".css": "text/css; charset=utf-8",
13   ".json": "application/json; charset=utf-8",
14   ".webmanifest": "application/manifest+json; charset=utf-8",
15   ".png": "image/png",
16   ".svg": "image/svg+xml",
17   ".ico": "image/x-icon",
18   ".woff": "font/woff",
19   ".woff2": "font/woff2",
20   ".txt": "text/plain; charset=utf-8",
21   ".apk": "application/vnd.android.package-archive",
22   ".map": "application/json; charset=utf-8",
23 };
```

5~7줄 보여 줄 폴더(기본 out), 포트 47231, 주소 0.0.0.0(같은 와이파이의 폰에서도 접속 가능).

9~23줄 파일 확장자 → 브라우저에 알려 줄 파일 종류 (html, js, css, png, 글꼴, apk …).

```
25 function fileFor(urlPath) {
26   const decoded = decodeURIComponent(urlPath.split("?")[0]);
27   const rel = path.normalize(decoded).replace(/^(\.\.[/\\])+/, "").replace(/^[/\\]+/, "");
28   const full = path.resolve(root, rel);
29   if (full !== root && !full.startsWith(root + path.sep)) return null;
30   try {
31     const stat = statSync(full);
32     if (stat.isDirectory()) {
33       const index = path.join(full, "index.html");
34       statSync(index);
35       return index;
36     }
37     return full;
38   } catch {
39     if (!path.extname(full)) {
40       const html = `${full}.html`;
41       try {
42         statSync(html);
43         return html;
44       } catch {
45         return null;
```

fileFor — 주소를 파일로 바꾸기.

1. ? 뒤를 떼고, 한글 등 인코딩을 풀고, **"../"로 폴더 밖을 보려는 시도를 막는다** (26~29줄, 보안).
2. 폴더면 그 안의 index.html.
3. 파일이 없고 확장자도 없으면 "주소.html"을 찾는다 (/stats → stats.html).
4. 못 찾으면 null.

```
      }
    }
    return null;
  }
}
```

```
const server = createServer((req, res) => {
  const file = fileFor(req.url ?? "/");
  if (!file) {
    res.writeHead(404, { "Content-Type": "text/plain; charset=utf-8" });
    res.end("찾을 수 없습니다.");
    return;
  }
  const ext = path.extname(file);
  res.writeHead(200, {
    "Content-Type": types[ext] ?? "application/octet-stream",
    "Cache-Control": ext === ".html" || ext === ".webmanifest" ? "no-cache" : "public, max-age=3600",
    "Service-Worker-Allowed": "/",
  });
  createReadStream(file).pipe(res);
});

server.listen(port, host, () => {
  console.log(`한박자 static ${host}:${port} -> ${root}`);
});
```

서버. 파일이 없으면 404 "찾을 수 없습니다." 있으면 종류를 붙이고, HTML은 매번 새로(no-cache), 나머지는 1시간 저장 허용. 서비스 워커가 전체 주소를 맡을 수 있게 허용 헤더. 파일을 흘려보낸다. 시작하면 주소를 출력 (예전 이름 "한박자"가 남아 있음).

write-sw.mjs — 오프라인용 서비스 워커 만들기 `scripts/write-sw.mjs (84줄)`

빌드가 끝난 뒤 out/ 폴더의 모든 파일 목록을 넣어 `sw.js`를 **자동으로 써 준다**. 브라우저(PWA)에서 한 번 열면 인터넷 없이도 열리게 하는 장치다. (폰 APK는 파일이 이미 안에 있어서 쓰지 않음.)

```
import { readdir, writeFile } from "node:fs/promises";
import path from "node:path";

const outDir = path.resolve("out");

async function walk(dir) {
  const entries = await readdir(dir, { withFileTypes: true });
  const files = [];
  for (const entry of entries) {
    const full = path.join(dir, entry.name);
    if (entry.isDirectory()) files.push(...(await walk(full)));
    else files.push(full);
  }
  return files;
}
```

walk: 폴더를 끝까지 들어가며 모든 파일 경로를 모은다 (자기 자신을 다시 부르는 "재귀").

17~28줄 파일마다 주소를 만든다 (sw.js, apk, map 파일은 제외). index.html은 폴더 주소(/, /stats/)로도 등록. `Set`은 중복 없는 목록.

```
16
17 const files = await walk(outDir);
18 const urls = new Set();
19
20 for (const file of files) {
21   const rel = path.relative(outDir, file).split(path.sep).join("/");
22   if (rel === "sw.js" || rel.endsWith(".apk") || rel.endsWith(".map")) continue;
23   if (rel.endsWith("/index.html") || rel === "index.html") {
24     const dir = rel.slice(0, -"index.html".length);
25     urls.add(dir === "" ? "/" : `/${dir}`);
26   }
27   urls.add(`/${rel}`);
28 }
```

```
30 const version = new Date().toISOString().replace(/[:.]/g, "-");
31 const body = `/* 한박자 오프라인 캐시. 빌드 때 생성됩니다. */
32 const CACHE = "hanbakja-${version}";
33 const PRECACHE = ${JSON.stringify([...urls].sort(), null, 2)};
```

30줄 빌드 시각으로 버전 이름을 만든다 → 새로 빌드하면 저장소 이름이 바뀌어 옛 파일이 정리된다. **31~33줄**부터 sw.js에 들어갈 글이 시작: 저장소 이름, 미리 저장할 주소 목록.

```
35 self.addEventListener("install", (event) => {
36   event.waitUntil(
37     caches.open(CACHE).then((cache) => cache.addAll(PRECACHE)).then(() => self.skipWaiting()),
38   );
39 });
40
41 self.addEventListener("activate", (event) => {
42   event.waitUntil(
43     caches
44       .keys()
45       .then((keys) => Promise.all(keys.filter((key) => key !== CACHE).map((key) => caches.delete
(key))))
46       .then(() => self.clients.claim()),
47   );
48 });
```

(sw.js 내용) install: 설치될 때 목록의 파일을 모두 저장하고 바로 활성화. **activate**: 이름이 다른 옛 저장소를 지우고 열린 화면을 바로 맡는다.

```
50 self.addEventListener("fetch", (event) => {
51   const request = event.request;
52   if (request.method !== "GET") return;
53   const url = new URL(request.url);
54   if (url.origin !== self.location.origin) return;
55   if (url.pathname.endsWith(".apk")) return;
56
57   event.respondWith(
58     (async () => {
59       const cache = await caches.open(CACHE);
60       const cached = await cache.match(request);
61       try {
62         const fresh = await fetch(request);
63         if (fresh.ok) await cache.put(request, fresh.clone());
64         return fresh;
65       } catch {
```

(sw.js 내용) fetch — 파일 요청 가로채기. GET이고 같은 사이트이고 apk가 아닌 요청만. **먼저 인터넷에서 새로 받고**(성공하면 저장소도 갱신), 인터넷이 안 되면 저장된 것 → 폴더 주소 → index.html → 홈 순서로 찾는다. 다 없으면 "오프라인 상태이고 이 화면은 아직 저장되지 않았습니다."

```
        if (cached) return cached;
        const pathName = url.pathname.endsWith("/") ? url.pathname : url.pathname + "/";
        return (
          (await cache.match(pathName)) ||
          (await cache.match(pathName + "index.html")) ||
          (await cache.match("/")) ||
          new Response("오프라인 상태이고 이 화면은 아직 저장되지 않았습니다.", {
            status: 503,
            headers: { "Content-Type": "text/plain; charset=utf-8" },
          })
        );
      }
    })(),
  );
});
`;
```

```
await writeFile(path.join(outDir, "sw.js"), body);
console.log(`service worker precache ${urls.size} urls`);
```

out/sw.js로 파일을 쓰고 저장한 주소 수를 출력한다.

8부. 프로젝트 설정 파일 (맨 위 폴더)

프로젝트 맨 위 폴더의 설정 파일들. 코드는 아니지만 "어떤 도구를 어떤 버전으로, 어떻게 빌드할지"를 정한다. `.json`은 이름-값 표 형식의 글 파일이다.

package.json — 프로젝트 이름표와 부품 목록 `package.json (43줄)`

```
{
  "name": "hanbakja",
  "version": "0.9.2",
  "private": true,
  "scripts": {
    "dev": "next dev --hostname 0.0.0.0 --port 47231",
    "build": "next build && node scripts/write-sw.mjs",
    "icons": "python3 scripts/make-icons.py",
    "serve:static": "node scripts/serve-static.mjs",
    "start": "next start",
    "lint": "eslint",
    "test": "node --test lib/*.test.ts",
    "apk": "node scripts/build-apk.mjs release",
    "apk:debug": "node scripts/build-apk.mjs debug"
  },
```

2~4줄 이름(옛 이름 hanbakja 그대로), **버전 0.9.2** (설정 화면 표시, APK versionName, 파일 이름이 모두 여기서 나온다 → 버전 올릴 때 이것만 고치면 됨), 공개 패키지 아님.

5~15줄 scripts — 명령 단축키 (`npm run 이름`):

- **dev**: 개발 서버 (고치면 바로 반영). **build**: 웹 빌드 + sw.js 만들기.
- **icons**: 아이콘 만들기. **serve:static**: 빌드 결과 미리 보기. **start**: Next 서버 (이 앱은 안 씀).
- **lint**: 코드 검사. **test**: lib의 테스트 실행 (Node 기본 테스트 도구).
- **apk** / **apk:debug**: APK 만들기.

```
  "dependencies": {
    "@base-ui/react": "^1.8.0",
    "@capacitor-community/sqlite": "^8.1.1",
    "@capacitor/android": "^8.5.2",
    "@capacitor/core": "^8.5.2",
    "@capacitor/local-notifications": "^8.3.1",
    "@fontsource/noto-sans-kr": "^5.3.0",
    "class-variance-authority": "^0.7.1",
    "cn": "^0.4.0",
    "lucide-react": "^1.48.0",
    "next": "16.3.6",
    "react": "19.2.8",
    "react-dom": "19.2.8",
    "shadcn": "^4.21.0",
    "tw-animate-css": "^1.4.0"
  },
```

dependencies — 앱에 들어가는 부품 (^는 "이 버전 이상의 호환 버전"):

- @base-ui/react: 버튼·스위치 등의 동작. @capacitor/*: 웹을 안드로이드 앱으로, 로컬 알림. @capacitor-community/sqlite: 폰 SQLite.
- @fontsource/noto-sans-kr: 한글 글꼴. class-variance-authority, cn, shadcn, tw-animate-css: 부품 모양 도구.
- lucide-react: 아이콘. next, react, react-dom: 화면 프레임워크 (버전 고정).

```
  "devDependencies": {
    "@capacitor/cli": "^8.5.2",
    "@tailwindcss/postcss": "^4",
    "@types/node": "^20",
    "@types/react": "^19",
    "@types/react-dom": "^19",
    "eslint": "^9",
    "eslint-config-next": "16.3.6",
```

devDependencies — 만들 때만 쓰는 도구 (앱에는 안 들어감): Capacitor 명령어, Tailwind, TypeScript와 타입 정보, ESLint.

```
    "tailwindcss": "^4",
    "typescript": "^5"
  }
}
```

capacitor.config.ts — Capacitor 설정 `capacitor.config.ts (12줄)`

```ts
import type { CapacitorConfig } from "@capacitor/cli";

const config: CapacitorConfig = {
  appId: "app.focuson.shorts",
  appName: "ShortsAI",
  webDir: "out",
  android: {
    allowMixedContent: false,
  },
};

export default config;
```

appId `app.focuson.shorts`: 안드로이드 패키지 이름 (폰이 앱을 구별하는 고유 이름, 바꾸면 다른 앱이 됨). **appName** ShortsAI. **webDir** out: 빌드된 웹 파일 위치 (cap sync가 여기서 복사). **allowMixedContent: false**: 보안 연결이 아닌 http 내용 섞기 금지.

next.config.ts — Next.js 설정 `next.config.ts (16줄)`

```ts
import { readFileSync } from "node:fs";
import type { NextConfig } from "next";

const { version } = JSON.parse(readFileSync("./package.json", "utf8")) as { version: string };

const nextConfig: NextConfig = {
  agentRules: false,
  allowedDevOrigins: ["127.0.0.1", "localhost"],
  output: "export",
  trailingSlash: true,
  images: { unoptimized: true },
  // 설정 화면에 보이는 버전. APK의 versionName도 같은 package.json 값을 쓴다.
  env: { NEXT_PUBLIC_APP_VERSION: version },
};

export default nextConfig;
```

4줄 package.json의 버전을 읽는다.

- **output: "export"**: 서버 없이 HTML 파일로 내보내기 (APK 안에 넣으려면 필수).
- **trailingSlash**: 주소 끝에 / (stats/index.html 형태로 만들어 파일로 열기 쉽게).
- **images unoptimized**: 서버가 없으니 이미지 자동 최적화 끄기.
- **allowedDevOrigins**: 개발 서버 접속 허용 주소. **agentRules**: Next.js의 AI 도우미 규칙 파일 생성 끄기.
- **13줄 env**: 버전을 `NEXT_PUBLIC_APP_VERSION`으로 화면 코드에 넣는다 → app-info.ts가 읽음.

tsconfig.json — TypeScript 설정 `tsconfig.json (35줄)`

```
{
  "compilerOptions": {
    "target": "ES2017",
    "lib": ["dom", "dom.iterable", "esnext"],
```

- **strict: true**: 가장 엄격한 검사 (빈 값 실수 등을 잡음).
- **noEmit**: 검사만 하고 파일은 Next.js가 만든다. **jsx: react-jsx**: 화면 태그 문법 사용.

```
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "allowImportingTsExtensions": true,
    "jsx": "react-jsx",
    "incremental": true,
    "plugins": [
      {
        "name": "next"
      }
    ],
    "paths": {
      "@/*": ["./*"]
    }
  },
  "include": [
    "next-env.d.ts",
    "**/*.ts",
    "**/*.tsx",
    ".next/types/**/*.ts",
    ".next/dev/types/**/*.ts",
    "**/*.mts"
  ],
  "exclude": ["node_modules"]
}
```

- **allowImportingTsExtensions**: 테스트에서 `./logic.ts`처럼 확장자를 붙여 가져오기 허용 (Node 테스트용).
- **22~24줄 paths**: `@/lib/guard`의 **@ = 프로젝트 맨 위 폴더**. 그래서 어디서든 같은 방식으로 가져온다.
- **include / exclude**: 검사할 파일(모든 .ts, .tsx)과 뺄 폴더(node_modules).

eslint.config.mjs — 코드 검사 규칙 `eslint.config.mjs (18줄)`

```
import { defineConfig, globalIgnores } from "eslint/config";
import nextVitals from "eslint-config-next/core-web-vitals";
import nextTs from "eslint-config-next/typescript";

const eslintConfig = defineConfig([
  ...nextVitals,
  ...nextTs,
  // Override default ignores of eslint-config-next.
  globalIgnores([
    // Default ignores of eslint-config-next:
    ".next/**",
    "out/**",
    "build/**",
    "next-env.d.ts",
  ]),
```

ESLint는 "실수하기 쉬운 코드"를 찾아 주는 도구. Next.js 권장 규칙(성능, 접근성)과 TypeScript 규칙을 쓰고, 빌드 결과 폴더(.next, out, build)는 검사하지 않는다. `npm run lint`로 실행.

```
]);

export default eslintConfig;
```

postcss.config.mjs — CSS 처리 설정 `postcss.config.mjs (7줄)`

```
const config = {
  plugins: {
    "@tailwindcss/postcss": {},
  },
};

export default config;
```

CSS를 빌드할 때 Tailwind 처리기를 쓴다. 이 덕분에 globals.css의 `@import "tailwindcss"`와 화면의 `className="p-4"` 같은 이름이 실제 CSS로 바뀐다.

components.json — shadcn 부품 설정 `components.json (25줄)`

```
{
  "$schema": "https://ui.shadcn.com/schema.json",
  "style": "base-nova",
  "rsc": true,
  "tsx": true,
  "tailwind": {
    "config": "",
    "css": "app/globals.css",
    "baseColor": "neutral",
    "cssVariables": true,
    "prefix": ""
  },
  "iconLibrary": "lucide",
  "rtl": false,
  "aliases": {
    "components": "@/components",
    "utils": "@/lib/utils",
    "ui": "@/components/ui",
    "lib": "@/lib",
    "hooks": "@/hooks"
  },
  "menuColor": "default",
  "menuAccent": "subtle",
  "registries": {}
}
```

`npx shadcn add button` 같은 명령으로 부품을 복사해 올 때의 설정: 스타일(base-nova), TSX 사용, CSS 파일 위치(app/globals.css), 색 변수 사용, 아이콘 lucide, 부품을 넣을 폴더(`@/components/ui`) 등. 앱 실행에는 영향이 없다.

manifest.webmanifest — 웹 앱 설명서 (PWA) `public/manifest.webmanifest (35줄)`

```
{
  "id": "/",
  "name": "Focus on",
  "short_name": "Focus on",
  "description": "쇼츠를 열기 전에 이유를 묻고, 오늘 사용량에 따라 단계적으로 개입합니다.",
  "lang": "ko",
  "dir": "ltr",
  "start_url": "/",
  "scope": "/",
  "display": "standalone",
  "display_override": ["standalone", "browser"],
  "orientation": "portrait",
  "background_color": "#f4efe4",
  "theme_color": "#f4efe4",
  "icons": [
    {
      "src": "/icons/icon-192.png",
      "sizes": "192x192",
      "type": "image/png",
      "purpose": "any"
    },
    {
      "src": "/icons/icon-512.png",
      "sizes": "512x512",
      "type": "image/png",
      "purpose": "any"
    },
    {
      "src": "/icons/icon-maskable-512.png",
      "sizes": "512x512",
      "type": "image/png",
      "purpose": "maskable"
    }
  ]
}
```

브라우저에서 "홈 화면에 추가"할 때 쓰는 설명서. 이름 "Focus on", 한국어, 시작 주소 /, **standalone**(주소창 없이 앱처럼), 세로 고정, 배경·띠 색 베이지, 아이콘 3개 (**maskable** = 안드로이드가 원·둥근 사각형으로 잘라 써도 되는 아이콘). `public/` 폴더의 파일은 주소 그대로 제공된다 (/manifest.webmanifest).

.gitignore — git에 올리지 않을 파일 `.gitignore (55줄)`

```
# See https://help.github.com/articles/ignoring-files/ for more about ignoring files.

# dependencies
/node_modules
/.pnp
.pnp.*
.yarn/*
```

여기 적힌 파일·폴더는 git이 무시한다 (GitHub에도 안 올라감). `#`은 설명 줄, `!`는 "이건 예외로 올림".

- node_modules (npm install로 다시 받을 수 있는 부품, 매우 큼), 테스트 결과, .next·out (빌드 결과).

```
!.yarn/patches
!.yarn/plugins
!.yarn/releases
!.yarn/versions

# testing
/coverage

# next.js
/.next/
/out/

```

```
# android build output
android/.gradle/
android/app/build/
android/build/
android/local.properties
android/capacitor-cordova-android-plugins/
*.keystore
*.jks
!android/app/debug.keystore
android/keystore.properties

# built apks
release/*.apk
```

안드로이드 쪽 (중요).

- Gradle 빌드 결과, local.properties(내 컴퓨터의 SDK 경로).
- ***.keystore, *.jks, keystore.properties**: 서명 키와 비밀번호 — **절대 올리면 안 되는 파일**.
- **28줄** `!android/app/debug.keystore`는 처음 가져온 프로젝트에 있던 예외 줄이다. 지금 그 파일은 저장소에 없고, 디버그 키는 각 PC의 `~/.android/debug.keystore`를 쓴다. 디버그 키도 GitHub에 올리지 않는다는 팀 규칙은 그대로다.
- release/*.apk: 만든 APK (GitHub Releases에 따로 올림).

```
# production
/build

# misc
.DS_Store
*.pem

# debug
npm-debug.log*
yarn-debug.log*
yarn-error.log*
.pnpm-debug.log*

# env files (can opt-in for committing if needed)
.env*

# vercel
.vercel

# typescript
*.tsbuildinfo
next-env.d.ts
```

그 밖에: 빌드 폴더, 맥 시스템 파일, 인증서(*.pem), 로그, **.env 파일**(API 키 같은 비밀 값을 두는 곳 — 이 앱엔 없음), Vercel 설정, TypeScript 캐시.

.claude/launch.json — 미리 보기 실행 설정 `.claude/launch.json (11줄)`

```
{
  "version": "0.0.1",
  "configurations": [
    {
      "name": "static",
      "runtimeExecutable": "node",
      "runtimeArgs": ["scripts/serve-static.mjs"],
      "port": 47231
    }
  ]
}
```

Claude Code 데스크톱의 미리 보기 창이 쓰는 설정. "static"이라는 이름으로 `node scripts/serve-static.mjs`를 실행하고 47231 포트를 연다 → 빌드된 웹 화면을 브라우저 창에서 확인. 앱에는 들어가지 않는다.

9부. 테스트 코드

테스트는 "이 함수에 이 값을 넣으면 이 결과가 나와야 한다"를 적어 둔 코드다. 코드를 고친 뒤 테스트를 돌리면, 예전 기능이 망가졌는지 **사람이 폰을 만지지 않고도** 바로 알 수 있다. 지금 웹 54개, Java 27개가 모두 통과한다.

테스트 읽는 법:

- Java(JUnit): `@Test`가 붙은 함수 하나 = 테스트 하나. `assertEquals(기대값, 실제값)` 같으면 통과, `assertTrue(x)` x가 참이어야 통과, `assertNull(x)` 비어 있어야 통과. 소수는 `assertEquals(0.8, x, 0.01)`처럼 허용 오차를 준다.
- 실행: Java는 `android` 폴더에서 `gradlew test`, 웹은 `npm test`.
- 무작위가 들어간 테스트는 `new Random(7)`처럼 **씨앗 값**을 고정해서 매번 같은 결과가 나오게 한다.

GuardBanditTest.java — AI 고르기 테스트 `android/app/src/test/java/app/focuson/shorts/GuardBanditTest.java (125줄)`

```java
package app.focuson.shorts;

import static org.junit.Assert.assertEquals;
import static org.junit.Assert.assertNull;
import static org.junit.Assert.assertTrue;

import java.util.Arrays;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Random;
import org.junit.Test;

public class GuardBanditTest {
    @Test
    public void betaDrawsAverageToTheMean() {
        Random random = new Random(7);
        double sum = 0;
        int n = 20000;
        for (int i = 0; i < n; i++) sum += GuardBandit.beta(8, 2, random);
        assertEquals(0.8, sum / n, 0.01);
        double small = 0;
        for (int i = 0; i < n; i++) small += GuardBandit.beta(0.5, 0.5, random);
        assertEquals(0.5, small / n, 0.02);
    }
```

3~12줄 검사 도구와 표·목록·무작위 가져오기.

betaDrawsAverageToTheMean: 베타 분포 뽑기를 2만 번 하면 평균이 이론값에 가까운가. Beta(8, 2)는 평균 0.8(±0.01), Beta(0.5, 0.5)는 0.5(±0.02). 직접 만든 뽑기 함수가 맞게 동작하는지 확인.

```
    @Test
    public void picksTheStrongerArmMostOfTheTime() {
        Map<String, double[]> posteriors = new HashMap<>();
        posteriors.put("breath", new double[] {30, 5});   // stops ~86%
        posteriors.put("card", new double[] {5, 30});     // stops ~14%
        List<String> arms = Arrays.asList("breath", "card");
        Random random = new Random(42);
        int breath = 0;
        for (int i = 0; i < 1000; i++) {
            if ("breath".equals(GuardBandit.pick(arms, posteriors, random))) breath++;
        }
        assertTrue("breath picked " + breath, breath > 950);
    }
```

picksTheStrongerArmMostOfTheTime: 호흡은 [30, 5](멈춘 비율 약 86%), 카드는 [5, 30](약 14%)일 때 1000번 중 950번 넘게 호흡을 고르는가. → 기록이 분명하면 잘 맞는 쪽을 고른다.

```
    @Test
    public void unseenArmsStillGetTried() {
        Map<String, double[]> posteriors = new HashMap<>();
        posteriors.put("pause", new double[] {6, 6}); // middling, some data
        List<String> arms = Arrays.asList("pause", "reflection"); // reflection: no record → Beta(1, 1)
        Random random = new Random(3);
        int reflection = 0;
        for (int i = 0; i < 1000; i++) {
            if ("reflection".equals(GuardBandit.pick(arms, posteriors, random))) reflection++;
        }
        assertTrue("reflection picked " + reflection, reflection > 300 && reflection < 700);
    }
```

unseenArmsStillGetTried: 기록이 있는 pause [6, 6]과 기록이 **전혀 없는** reflection(→ [1, 1])이 있을 때, reflection도 300~700번 고르는가. → 새 방식도 계속 시도된다(탐색).

```
    @Test
    public void handlesEmptyAndMissingTables() {
        assertNull(GuardBandit.pick(Arrays.<String>asList(), null, new Random(1)));
        assertEquals("only", GuardBandit.pick(Arrays.asList("only"), null, new Random(1)));
        assertEquals(0.5, GuardBandit.mean(null), 0.0);
    }
```

handlesEmptyAndMissingTables: 선택지가 없으면 null, 하나면 그것, 표가 없을 때 평균은 0.5. 빈 값으로 앱이 죽지 않는지.

```
    @Test
    public void recentlyShownArmsAreLessLikelyForAWhile() {
        Map<String, double[]> posteriors = new HashMap<>();
        posteriors.put("breath", new double[] {6, 6});
        posteriors.put("card", new double[] {6, 6});
        List<String> arms = Arrays.asList("breath", "card");
        long now = 100L * 3_600_000L;
        Map<String, Long> shown = new HashMap<>();
        shown.put("breath", now - 60_000L); // a minute ago
        Random random = new Random(11);
        int breath = 0;
        for (int i = 0; i < 2000; i++) {
            if ("breath".equals(GuardBandit.pick(arms, posteriors, random, shown, now, 0.3, 6))) breath++;
        }
        assertTrue("breath picked " + breath, breath < 800);
```

recentlyShownArmsAreLessLikelyForAWhile — 질림 반영. 점수가 같은 두 방식 중 호흡을 1분 전에 보여 줬으면, 2000번 중 800번 미만으로 고른다 (반반이면 1000).

- **78~82줄** freshness(새로움 점수): 세기 0이면 1(효과 없음), 본 적 없으면 1, 방금 봤으면 0.7, 6시간 지나면 0.85, 72시간 지나면 거의 1.

```
76
77          // Off (strength 0) or long ago: back to an even split.
78          assertEquals(1.0, GuardBandit.freshness(now - 60_000L, now, 0, 6), 0.0);
79          assertEquals(1.0, GuardBandit.freshness(0L, now, 0.3, 6), 0.0);
80          assertEquals(0.7, GuardBandit.freshness(now, now, 0.3, 6), 1e-9);
81          assertEquals(0.85, GuardBandit.freshness(now - 6 * 3_600_000L, now, 0.3, 6), 1e-9);
82          assertTrue(GuardBandit.freshness(now - 72 * 3_600_000L, now, 0.3, 6) > 0.999);
83      }
```

```
85      @Test
86      public void looksUpWeekdayCellsThenFallsBackToOldKeys() {
87          GuardPolicy.Ai ai = new GuardPolicy.Ai();
88          Map<String, double[]> weekend = new HashMap<>();
89          weekend.put("breath", new double[] {9, 1});
90          Map<String, double[]> old = new HashMap<>();
91          old.put("card", new double[] {2, 2});
92          ai.methods.put("2:night:weekend", weekend);
93          ai.methods.put("2:day", old);
94          java.util.Calendar saturdayNight = java.util.Calendar.getInstance();
95          saturdayNight.set(2026, java.util.Calendar.OCTOBER, 10, 23, 0); // Saturday
96          java.util.Calendar mondayNoon = java.util.Calendar.getInstance();
97          mondayNoon.set(2026, java.util.Calendar.OCTOBER, 12, 13, 0); // Monday
98          assertEquals("weekend", GuardPolicy.dayType(saturdayNight));
99          assertEquals("weekday", GuardPolicy.dayType(mondayNoon));
100         assertEquals(weekend, ai.methodsAt(2, saturdayNight));
101         assertEquals(old, ai.methodsAt(2, mondayNoon));
102         assertNull(ai.methodsAt(1, mondayNoon));
103     }
```

looksUpWeekdayCellsThenFallsBackToOldKeys — 평일/주말 칸 찾기. 표에 "2:night:weekend"와 옛 형식 "2:day"만 있을 때:

- 2026-10-10(토) 23시 → 주말, "2:night:weekend" 칸을 찾는다.
- 2026-10-12(월) 13시 → 평일, "2:day:weekday"가 없으니 옛 "2:day"로 돌아간다 (예전 버전 정책표와 호환).
- 단계 1은 칸이 없으니 null.

```
105     @Test
106     public void studyCardsComeBackSoonerWhenWrong() {
107         long hour = 3_600_000L;
108         long last = 1_000L * hour;
109         assertEquals(0L, GuardState.cardDueAt(0L, 0));
110         assertEquals(last + hour, GuardState.cardDueAt(last, 0));
111         assertEquals(last + 24 * hour, GuardState.cardDueAt(last, 1));
112         assertEquals(last + 72 * hour, GuardState.cardDueAt(last, 2));
113         assertEquals(last + 168 * hour, GuardState.cardDueAt(last, 3));
114         assertEquals(last + 168 * hour, GuardState.cardDueAt(last, 9));
115     }
```

studyCardsComeBackSoonerWhenWrong — 카드 다시 나오는 시각. 한 번도 안 풀었으면 0(바로 가능), 틀렸으면(연속 0) 1시간 뒤, 연속 1번 맞힘 24시간, 2번 72시간, 3번 이상 168시간(7일), 9번이어도 7일.

```
117     @Test
118     public void timeSlotsMatchTheWebSide() {
119         assertEquals("morning", GuardPolicy.timeSlot(5));
120         assertEquals("day", GuardPolicy.timeSlot(11));
121         assertEquals("evening", GuardPolicy.timeSlot(21));
122         assertEquals("night", GuardPolicy.timeSlot(22));
123         assertEquals("night", GuardPolicy.timeSlot(3));
124     }
125 }
```

timeSlotsMatchTheWebSide: 시간대 나누기가 웹(ai.ts)과 같은지 — 5시 아침, 11시 낮, 21시 저녁, 22시·3시 밤. Java와 웹 규칙이 어긋나면 AI 표의 칸을 못 찾기 때문.

GuardCopyTest.java — 화면 문구 테스트 `android/app/src/test/java/app/focuson/shorts/GuardCopyTest.java (75줄)`

```java
package app.focuson.shorts;

import static org.junit.Assert.assertEquals;
import static org.junit.Assert.assertNull;
import static org.junit.Assert.assertTrue;

import org.junit.Test;

public class GuardCopyTest {
    private static GuardCopy.Facts facts(double usageMin, double goalMin) {
        GuardCopy.Facts f = new GuardCopy.Facts();
        f.usage = usageMin * 60;
        f.goal = goalMin * 60;
        return f;
    }

    @Test
    public void minutesReadNaturally() {
        assertEquals("23분", GuardCopy.minutes(23 * 60 + 10));
        assertEquals("2시간 40분", GuardCopy.minutes(160 * 60));
        assertEquals("1시간", GuardCopy.minutes(3600));
        assertEquals("1분", GuardCopy.minutes(5));
    }
```

facts: 테스트용 사실 묶음 만들기 (사용 분, 목표 분 → 초).

minutesReadNaturally: 시간 글자가 자연스러운가 — 23분 10초 → "23분", 160분 → "2시간 40분", 3600초 → "1시간", 5초 → "1분"(0분이라 하지 않음).

```java
    @Test
    public void timeLinesNeedTheirNumbers() {
        GuardCopy.Facts f = facts(23, 30);
        assertEquals("오늘 쇼츠에 쓴 23분, 다시 돌아오지 않아요.", GuardCopy.insight("guilt", f));
        assertEquals("지금 멈추면 오늘 목표 안에서 끝낼 수 있어요. (7분 남음)", GuardCopy.insight("motivate", f));
        assertEquals("정한 30분 중 23분을 썼어요.", GuardCopy.insight("goal", f));
        // Unknown numbers: skipped.
        assertNull(GuardCopy.insight("yesterday", f));
        assertNull(GuardCopy.insight("average", f));
        assertNull(GuardCopy.insight("weekly", f));
        assertNull(GuardCopy.insight("sleep", f));
        // Less than a minute watched: nothing to say yet.
        assertNull(GuardCopy.insight("guilt", facts(0.5, 30)));

        f.yesterdayByNow = 12 * 60;
        f.avg7 = 15 * 60;
        f.weekProjection = 160 * 60;
        assertEquals("어제 이 시간엔 12분이었는데, 오늘은 벌써 23분이에요.", GuardCopy.insight("yesterday", f));
        assertEquals("이번 주 평균보다 8분 더 보고 있어요.", GuardCopy.insight("average", f));
        assertEquals("이 속도면 이번 주에 2시간 40분을 쇼츠에 써요.", GuardCopy.insight("weekly", f));

```

timeLinesNeedTheirNumbers — 시간 체감 문구. 23분 사용, 목표 30분일 때:

- guilt "오늘 쇼츠에 쓴 23분, 다시 돌아오지 않아요." / motivate "...(7분 남음)" / goal "정한 30분 중 23분을 썼어요."
- **숫자가 없는 문구는 null** (어제, 평균, 주간, 수면). 1분도 안 봤으면 아직 말하지 않음.
- **39~49줄** 숫자를 채우면: 어제 같은 시각 비교, 주 평균보다 8분 더, 이 속도면 이번 주 2시간 40분, 수면 "지금 자면 6시간 20분 ... (7:10 알람 기준)".

```
        f.sleepSeconds = (6 * 60 + 20) * 60;
        f.sleepBasis = "7:10 알람 기준";
        assertEquals("지금 자면 6시간 20분 잘 수 있어요. 10분 더 보면 6시간 10분이에요. (7:10 알람 기준)",
            GuardCopy.insight("sleep", f));
    }

    @Test
    public void overdraftLinesEscalate() {
        GuardCopy.Facts f = facts(26, 20);
        assertEquals("오늘 1번째 3분이에요", GuardCopy.overdraftTitle(1, 3));
        assertTrue(GuardCopy.overdraft(1, 3, f).startsWith("오늘 목표를 넘어서 보는 첫 3분"));
        assertEquals("벌써 두 번째 3분이에요. 이번 3분까지 목표보다 9분 더 보게 돼요.", GuardCopy.overdraft(2, 3, f));
        assertEquals("오늘 목표보다 6분 넘게 봤어요. 4번째 3분이에요.", GuardCopy.overdraft(4, 3, f));
        f.yesterdayByNow = 12 * 60;
        assertEquals("오늘 목표보다 6분 넘게 봤어요. 어제는 이 시간까지 12분이었어요.", GuardCopy.overdraft(5, 3, f));
    }

    @Test
    public void praiseAndLockLines() {
        GuardCopy.Facts f = facts(10, 20);
        f.keptToday = 2;
        assertEquals("정한 만큼만 봤어요. 오늘 2번째예요.", GuardCopy.praise("kept", 1, f, 0));
        assertEquals("좋은 선택이에요. 30분 뒤에 다시 만나요.", GuardCopy.praise("lock", 1, f, 30));
        assertEquals("1시간, 폰 말고 다른 걸 해 볼 시간이에요.", GuardCopy.praise("lock", 2, f, 60));
        assertEquals("30분 잘 쉬었어요. 그동안 뭐 했어요?", GuardCopy.afterLockQuestion(30));
        assertEquals("12분 남았어요. 지금까지 오늘 45분 쉬었어요.", GuardCopy.locked(12 * 60_000L, 45 * 60));
        assertEquals("12분 남았어요.", GuardCopy.locked(12 * 60_000L, 0));
        assertEquals("약속한 3분 중 2분 남았어요.", GuardCopy.resumeToast(3, 110));
    }
}
```

overdraftLinesEscalate — 목표 넘긴 뒤 "3분만 더" 문구가 점점 진해지는지. 1번째: "오늘 1번째 3분이에요" + "첫 3분…", 2번째: "벌써 두 번째 3분 … 9분 더", 4번째: "6분 넘게 … 4번째", 어제 기록이 있으면 어제와 비교.

praiseAndLockLines: 약속 지킴 칭찬 "오늘 2번째예요", 잠금 선택 칭찬 (30분, 1시간), 잠금 끝난 뒤 질문, 잠금 화면 "12분 남았어요. 지금까지 오늘 45분 쉬었어요."(쉰 시간 0이면 뒷말 없음), 다시 보기 알림 "약속한 3분 중 2분 남았어요."

GuardPolicyTest.java — 강도·약속 규칙 테스트 `android/app/src/test/java/app/focuson/shorts/GuardPolicyTest.java (54줄)`

```
package app.focuson.shorts;

import static org.junit.Assert.assertEquals;

import java.util.Arrays;
import java.util.Calendar;
import java.util.Collections;
import java.util.TimeZone;
import org.junit.Test;
```

12~13줄 서울 시간대, 목표 20분 기본 정책.

bandsFollowShareOfGoal: 9.9분까지 약함, 10분(절반)부터 중간, 20분(목표)부터 강함. 13분 사용이면 남은 7분, 25분이면 0(음수 아님).

```
10
11 public class GuardPolicyTest {
12     private static final TimeZone SEOUL = TimeZone.getTimeZone("Asia/Seoul");
13     private final GuardPolicy policy = GuardPolicy.defaults(20);
14
15     @Test
16     public void bandsFollowShareOfGoal() {
17         assertEquals(GuardPolicy.BAND_LIGHT, policy.band(0));
18         assertEquals(GuardPolicy.BAND_LIGHT, policy.band(9.9 * 60));
19         assertEquals(GuardPolicy.BAND_MEDIUM, policy.band(10 * 60));
20         assertEquals(GuardPolicy.BAND_MEDIUM, policy.band(19.9 * 60));
21         assertEquals(GuardPolicy.BAND_STRONG, policy.band(20 * 60));
22         assertEquals(7 * 60, policy.remainingSeconds(13 * 60), 0.001);
23         assertEquals(0, policy.remainingSeconds(25 * 60), 0.001);
24     }
```

```
26     @Test
27     public void commitmentChoicesFitBudgetAndLadder() {
28         assertEquals(Arrays.asList(3, 5, 10), policy.commitChoices(15 * 60, 0));
29         assertEquals(Arrays.asList(3, 5), policy.commitChoices(7 * 60, 0));
30         // Extension ladder: second ask caps at 10, third at 5, then 3.
31         assertEquals(Arrays.asList(3, 5, 10), policy.commitChoices(15 * 60, 1));
32         assertEquals(Arrays.asList(3, 5), policy.commitChoices(15 * 60, 2));
33         assertEquals(Collections.singletonList(3), policy.commitChoices(15 * 60, 3));
34         assertEquals(Collections.singletonList(3), policy.commitChoices(15 * 60, 9));
35         // Less than the smallest option left: offer what is left, rounded up.
36         assertEquals(Collections.singletonList(2), policy.commitChoices(100, 0));
37         assertEquals(Collections.emptyList(), policy.commitChoices(10, 0));
38     }
```

commitmentChoicesFitBudgetAndLadder — 약속 시간 고르기 버튼.

- 남은 15분 → [3, 5, 10], 남은 7분 → [3, 5] (남은 시간보다 긴 건 빼기).
- **연장 사다리**: 연장 횟수가 늘수록 최대가 줄어든다 — 2번째 10, 3번째 5, 그 뒤 3분만.
- 100초 남음 → [2](올림), 10초 남음 → 없음.

```
40     @Test
41     public void dayEndIsNextMorning() {
42         Calendar cal = Calendar.getInstance(SEOUL);
43         cal.clear();
44         cal.set(2026, Calendar.OCTOBER, 6, 23, 30, 0);
45         long night = cal.getTimeInMillis();
46         cal.set(2026, Calendar.OCTOBER, 7, 6, 0, 0);
47         assertEquals(cal.getTimeInMillis(), policy.nextDayEnd(night, SEOUL));
48
49         cal.set(2026, Calendar.OCTOBER, 7, 2, 0, 0);
50         long lateNight = cal.getTimeInMillis();
51         cal.set(2026, Calendar.OCTOBER, 7, 6, 0, 0);
52         assertEquals(cal.getTimeInMillis(), policy.nextDayEnd(lateNight, SEOUL));
53     }
54 }
```

dayEndIsNextMorning: "오늘은 끝"을 23:30에 고르면 다음 날 6:00까지, 새벽 2:00에 고르면 **같은 날** 6:00까지 (내일 6시가 아니라).

GuardScreensTest.java — 쇼츠 화면 구별 테스트 `android/app/src/test/java/app/focuson/shorts/GuardScreensTest.java (150줄)`

오탐(일반 영상을 쇼츠로 착각)과 미탐을 막는 가장 중요한 테스트. 화면 글이 아니라 **viewId(화면 부품 이름표)와 클래스 이름**만으로 판단하는지 확인한다.

```
package app.focuson.shorts;

import static org.junit.Assert.assertEquals;
import static org.junit.Assert.assertFalse;
import static org.junit.Assert.assertNull;
import static org.junit.Assert.assertTrue;

import java.util.Arrays;
import java.util.Collections;
import java.util.List;
import org.junit.Test;

public class GuardScreensTest {
    @Test
    public void detectsYoutubeShortsPlayerOnly() {
        assertEquals(
            "youtube",
            GuardScreens.classify(
                "com.google.android.youtube",
                Collections.singletonList("com.google.android.youtube:id/reel_player_page_contai
ner")
            )
        );
        assertNull(
            GuardScreens.classify(
                "com.google.android.youtube",
                Arrays.asList(
                    "com.google.android.youtube:id/watch_player",
                    "com.google.android.youtube:id/reel_recycler",
                    "com.google.android.youtube:id/shorts_shelf"
                )
            )
        );
        assertNull(
            GuardScreens.classify(
                "com.google.android.youtube",
                Collections.singletonList("com.google.android.youtube:id/watch_player")
            )
        );
        assertNull(
            GuardScreens.classify(
                "com.google.android.youtube",
                Collections.singletonList("com.google.android.youtube:id/shorts_shelf")
            )
```

detectsYoutubeShortsPlayerOnly: 쇼츠 플레이어(reel_player_page_container)만 있으면 "youtube". 일반 영상 플레이어(watch_player)가 함께 있거나, 홈 피드의 쇼츠 선반(shorts_shelf)만 있으면 null (개입 안 함).

```
        );
    }
```

```
    private static final String YT = "com.google.android.youtube:id/";

    @Test
    public void youtubeShortsNeedsExactShortsIdWithoutLongform() {
        assertTrue(GuardScreens.isYoutubeShorts(Collections.singletonList(YT + "reel_recycle
r")));
        assertTrue(GuardScreens.isYoutubeShorts(Arrays.asList(
            YT + "reel_recycler", YT + "reel_player_page_container", YT + "reel_time_bar")));
        // Long-form to Shorts animation: both visible, not Shorts yet.
        assertFalse(GuardScreens.isYoutubeShorts(Arrays.asList(YT + "reel_recycler", YT + "watch
_player")));
        assertFalse(GuardScreens.isYoutubeShorts(Arrays.asList(
            YT + "reel_player_page_container", YT + "metapanel_overlay_recycler_view")));
        assertFalse(GuardScreens.isYoutubeShorts(Collections.<String>emptyList()));
        assertFalse(GuardScreens.isYoutubeShorts(null));
    }
```

youtubeShortsNeedsExactShortsIdWithoutLongform: reel_recycler 하나나 쇼츠 id 여러 개 → 쇼츠. 일반 영상 → 쇼츠로 넘어가는 애니메이션 중(둘 다 보임)이나 메타 패널이 함께 있으면 아직 쇼츠 아님. 빈 목록·null도 아님.

```
    @Test
    public void youtubeIgnoresUnreliableAndSimilarIds() {
        assertFalse(GuardScreens.isYoutubeShorts(Collections.singletonList(YT + "reel_time_ba
r")));
        assertFalse(GuardScreens.isYoutubeShorts(Arrays.asList(
            YT + "reel_feedback_play", YT + "reel_feedback_pause")));
        // Old substring markers no longer decide.
        assertFalse(GuardScreens.isYoutubeShorts(Collections.singletonList(YT + "reel_player_pag
e")));
        assertFalse(GuardScreens.isYoutubeShorts(Collections.singletonList(YT + "reel_watch_frag
ment")));
        assertFalse(GuardScreens.isYoutubeShorts(Collections.singletonList(YT + "reel_recycler_
x")));
        assertFalse(GuardScreens.isYoutubeShorts(Collections.singletonList("reel_recycler")));
        assertFalse(GuardScreens.isYoutubeShorts(
            Collections.singletonList("com.instagram.android:id/reel_recycler")));
    }
```

youtubeIgnoresUnreliableAndSimilarIds: 진행 막대(reel_time_bar)만, 재생/정지 표시만, 비슷하지만 다른 이름(reel_player_page, reel_recycler_x), 앞부분 없는 이름, Instagram의 같은 이름 → 모두 쇼츠 아님. **이름이 정확히 같아야** 인정.

```
    @Test
    public void detectsSelectedShortsAndReelsTabsOnly() {
        assertEquals("youtube", GuardScreens.selectedTabKind("Shorts", true));
        assertEquals("youtube", GuardScreens.selectedTabKind("쇼츠", true));
        assertEquals("instagram", GuardScreens.selectedTabKind("Reels", true));
        assertEquals("instagram", GuardScreens.selectedTabKind("릴스 탭", true));
        assertNull(GuardScreens.selectedTabKind("Shorts", false));
        assertNull(GuardScreens.selectedTabKind("shortcuts", true));
        assertNull(GuardScreens.selectedTabKind("오늘 본 릴스 캡션이 길어서 저장하면 안 되는 문장",
true));
        assertNull(
            GuardScreens.classify(
                "com.google.android.youtube",
```

detectsSelectedShortsAndReelsTabsOnly: 아래 탭 이름이 "Shorts/쇼츠"이고 선택됨 → youtube, "Reels/릴스 탭" → instagram. 선택 안 됨, "shortcuts", 긴 캡션 문장은 null. 탭 신호 하나만으로는 classify가 개입하지 않는다 (플레이어 id가 있어야 함).

```
                new GuardScreens.Hit(Collections.emptyList(), true, false, false)
            )
        );
        assertNull(
            GuardScreens.classify(
                "com.instagram.android",
                new GuardScreens.Hit(Collections.emptyList(), false, true, false)
            )
        );
        assertNull(
            GuardScreens.classify(
                "com.google.android.youtube",
                new GuardScreens.Hit(Collections.emptyList(), false, true, false)
            )
        );
    }
```

```
    @Test
    public void detectsClipsClassButNotStoryClass() {
        assertTrue(GuardScreens.viewClassIsReels("com.instagram.clips.ClipsViewer"));
        assertFalse(GuardScreens.viewClassIsReels("com.instagram.reels.ReelViewer"));
        assertNull(
            GuardScreens.classify(
                "com.instagram.android",
                new GuardScreens.Hit(Collections.emptyList(), false, false, true)
            )
        );
        assertNull(
            GuardScreens.classify(
                "com.instagram.android",
                Collections.singletonList("com.instagram.android:id/clips_video_container")
            )
        );
    }
```

detectsClipsClassButNotStoryClass: 클래스 이름 ClipsViewer는 릴스, ReelViewer(스토리)는 아님. 클래스 신호나 비슷한 id(clips_video_container)만으로는 개입하지 않음.

```
    @Test
    public void detectsInstagramReelsNotStories() {
        assertEquals(
            "instagram",
            GuardScreens.classify(
                "com.instagram.android",
                Collections.singletonList("com.instagram.android:id/clips_viewer_container")
            )
        );
        assertNull(
            GuardScreens.classify(
                "com.instagram.android",
                Collections.singletonList("com.instagram.android:id/reel_viewer_root")
            )
        );
    }
```

detectsInstagramReelsNotStories: clips_viewer_container → instagram, 스토리(reel_viewer_root) → 아님. **ignoresOtherPackages**: Chrome 앱에서 같은 id가 보여도 무시, 목록이 null이어도 안전.

```
139
140     @Test
141     public void ignoresOtherPackages() {
142         assertNull(
143             GuardScreens.classify(
144                 "com.android.chrome",
145                 Collections.singletonList("com.google.android.youtube:id/reel_player_page_contai
ner")
146             )
147         );
148         assertNull(GuardScreens.classify("com.google.android.youtube", (List<String>) null));
149     }
150 }
```

GuardTimeTest.java — 날짜·시간 나누기 테스트 `android/app/src/test/java/app/focuson/shorts/GuardTimeTest.java (53줄)`

```
1  package app.focuson.shorts;
2
3  import static org.junit.Assert.assertEquals;
4  import static org.junit.Assert.assertTrue;
5
6  import java.util.Calendar;
7  import java.util.Map;
8  import java.util.TimeZone;
9  import org.junit.Test;
10
11 public class GuardTimeTest {
12     private static final TimeZone SEOUL = TimeZone.getTimeZone("Asia/Seoul");
13
14     private static long at(int year, int month, int day, int hour, int minute, int second) {
15         Calendar cal = Calendar.getInstance(SEOUL);
16         cal.clear();
17         cal.set(year, month - 1, day, hour, minute, second);
18         return cal.getTimeInMillis();
19     }
20
21     @Test
22     public void dayKeyUsesLocalDate() {
23         assertEquals("2026-09-30", GuardTime.dayKey(at(2026, 9, 30, 23, 59, 59), SEOUL));
24         assertEquals("2026-10-01", GuardTime.dayKey(at(2026, 10, 1, 0, 0, 0), SEOUL));
25     }
```

at: 서울 시간으로 년-월-일 시:분:초 → 밀리초. (Java Calendar는 월이 0부터라 `month - 1`.)

dayKeyUsesLocalDate: 9/30 23:59:59는 "2026-09-30", 10/1 0:00은 "2026-10-01".

```
27     @Test
28     public void splitsAcrossMidnight() {
29         Map<String, Double> parts = GuardTime.splitByDay(
30             at(2026, 9, 30, 23, 58, 0), at(2026, 10, 1, 0, 3, 0), SEOUL);
31         assertEquals(2, parts.size());
32         assertEquals(120.0, parts.get("2026-09-30"), 0.001);
33         assertEquals(180.0, parts.get("2026-10-01"), 0.001);
```

splitsAcrossMidnight: 23:58~0:03 시청은 전날 120초 + 다음 날 180초로 나뉜다.

splitsWithinOneDayAndEmpty: 같은 날 45초는 45초. 시작 = 끝이거나 끝이 시작보다 앞이면 빈 결과.

```
    }

    @Test
    public void splitsWithinOneDayAndEmpty() {
        Map<String, Double> parts = GuardTime.splitByDay(
            at(2026, 10, 1, 10, 0, 0), at(2026, 10, 1, 10, 0, 45), SEOUL);
        assertEquals(45.0, parts.get("2026-10-01"), 0.001);
        assertTrue(GuardTime.splitByDay(5000, 5000, SEOUL).isEmpty());
        assertTrue(GuardTime.splitByDay(5000, 1000, SEOUL).isEmpty());
    }
```

```
    @Test
    public void levelBandsMatchWeb() {
        assertEquals(1, GuardTime.levelFor(0, 10, 20));
        assertEquals(1, GuardTime.levelFor(9.9 * 60, 10, 20));
        assertEquals(2, GuardTime.levelFor(10 * 60, 10, 20));
        assertEquals(2, GuardTime.levelFor(19.9 * 60, 10, 20));
        assertEquals(3, GuardTime.levelFor(20 * 60, 10, 20));
    }
}
```

levelBandsMatchWeb: 단계 계산이 웹과 같은지 (기준 10분/20분: 9.9분 1단계, 10분 2단계, 20분 3단계).

ExampleUnitTest.java — 기본 예제 `android/app/src/test/java/com/getcapacitor/myapp/ExampleUnitTest.java (18줄)`

```
package com.getcapacitor.myapp;

import static org.junit.Assert.*;

import org.junit.Test;

/**
 * Example local unit test, which will execute on the development machine (host).
 *
 * @see <a href="http://d.android.com/tools/testing">Testing documentation</a>
 */
public class ExampleUnitTest {

    @Test
    public void addition_isCorrect() throws Exception {
        assertEquals(4, 2 + 2);
    }
}
```

Capacitor 프로젝트를 만들 때 자동으로 생긴 예제 (2 + 2 = 4). 테스트 도구가 동작하는지만 보여 준다. 앱 기능과 무관해 지워도 된다.

ExampleInstrumentedTest.java — 기기 테스트 예제 `android/app/src/androidTest/java/com/getcapacitor/myapp/ExampleInstrumentedTest.java (26줄)`

```
package com.getcapacitor.myapp;

import static org.junit.Assert.*;
```

실제 폰·에뮬레이터에서 도는 테스트의 자동 생성 예제. 앱 패키지 이름이 "com.getcapacitor.app"인지 확인하는데, 이 앱은 `app.focuson.shorts`로 바꿨

```
4
5  import android.content.Context;
6  import androidx.test.ext.junit.runners.AndroidJUnit4;
7  import androidx.test.platform.app.InstrumentationRegistry;
8  import org.junit.Test;
9  import org.junit.runner.RunWith;
10
11 /**
12  * Instrumented test, which will execute on an Android device.
13  *
14  * @see <a href="http://d.android.com/tools/testing">Testing documentation</a>
15  */
16 @RunWith(AndroidJUnit4.class)
17 public class ExampleInstrumentedTest {
18
19     @Test
20     public void useAppContext() throws Exception {
21         // Context of the app under test.
22         Context appContext = InstrumentationRegistry.getInstrumentation().getTargetContext();
23
24         assertEquals("com.getcapacitor.app", appContext.getPackageName());
25     }
26 }
```

으므로 **기기에서 돌리면 실패한다**. 평소 쓰는 `gradlew test`(컴퓨터에서 도는 테스트)에는 포함되지 않아 문제가 드러나지 않았다. 지우거나 패키지 이름을 고치면 된다.

ai.test.ts — AI 계산 테스트 (웹) `lib/ai.test.ts (382줄)`

웹 테스트 읽는 법 (Node 기본 테스트): `describe("묶음 이름", () => {...})` 안에 `it("해야 하는 일", () => {...})`가 테스트 하나. `assert.equal(실제, 기대)` 같아야 통과, `assert.deepEqual` 안쪽까지 같아야 통과, `assert.ok(x)` 참이어야 통과. Java와 달리 **실제값이 먼저**다.

```
1  import assert from "node:assert/strict";
2  import { describe, it } from "node:test";
3  import {
4    BANDIT_SHARE_K,
5    dayType,
6    gradedReward,
7    similarity,
8    watchedSecondsBetween,
9    banditTables,
10   goalSuggestion,
11   hourLabel,
12   reward,
13   riskHours,
14   timeSlot,
15   weekKey,
16 } from "./ai.ts";
17 import { convertNativeBatch } from "./logic.ts";
18 import { buildPolicy } from "./policy.ts";
```

1~2줄 Node 기본 검사·테스트 도구. **3~20줄** 시험할 함수들 (ai.ts, logic.ts, policy.ts, storage.ts).

22~25줄 도우미: `at(10, 5, 23)` = 2026년 10월 5일 23시, `key(10, 5)` = "2026-10-05".

```ts
import { defaultData, readIntervention } from "./storage.ts";
import type { AppData, InterventionLog, InterventionType, UsageLog } from "./types.ts";

const at = (month: number, day: number, hour: number, minute = 0) =>
  new Date(2026, month - 1, day, hour, minute);
const key = (month: number, day: number) =>
  `2026-${String(month).padStart(2, "0")}-${String(day).padStart(2, "0")}`;
```

```ts
function session(id: string, start: Date, minutes: number): UsageLog {
  return {
    usageId: id,
    date: key(start.getMonth() + 1, start.getDate()),
    startTime: start.toISOString(),
    endTime: new Date(start.getTime() + minutes * 60_000).toISOString(),
    duration: minutes * 60,
    clipCount: 1,
    target: "youtube",
    source: "native",
  };
}

function row(partial: Partial<InterventionLog> & { interventionId: string; timestamp: string }): InterventionLog {
  const log = readIntervention({
    date: partial.timestamp.slice(0, 10),
    level: 2,
    reason: "습관적으로",
    reEntered: false,
    blocked: false,
    outcome: "exit",
    stage: "entry",
    ...partial,
  });
  assert.ok(log);
  return { ...log, ...partial } as InterventionLog;
}
```

session: 테스트용 시청 기록 한 줄 (시작 시각, 분 → 끝 시각, 초).

row: 테스트용 개입 기록 한 줄. 기본값(2단계, 습관적으로, 나감, 진입)에 바꾸고 싶은 칸만 덮어쓴다. storage.ts의 readIntervention을 거쳐서 실제 저장소에서 읽은 것과 같은 모양이 되게 한다.

```ts
describe("ai basics", () => {
  it("maps hours to slots and labels", () => {
    assert.equal(timeSlot(6), "morning");
    assert.equal(timeSlot(13), "day");
    assert.equal(timeSlot(20), "evening");
    assert.equal(timeSlot(23), "night");
    assert.equal(timeSlot(2), "night");
    assert.equal(hourLabel(23), "밤 11시");
    assert.equal(hourLabel(21), "밤 9시");
    assert.equal(hourLabel(19), "저녁 7시");
    assert.equal(hourLabel(14), "오후 2시");
    assert.equal(hourLabel(9), "오전 9시");
    assert.equal(hourLabel(0), "밤 12시");
  });
```

ai basics.

- 시를 시간대로: 6시 아침, 13시 낮, 20시 저녁, 23시·2시 밤. 시 이름: "밤 11시", "저녁 7시", "오후 2시", "오전 9시", 0시는 "밤 12시".
- 예전 점수(reward): 나가고 다시 안 옴 1, 지키고 30분 안에 다시 옴 0.5, 계속 봄 0, "도움 안 됐어요"면 절반.

```
  it("scores stopping, re-entry and feedback", () => {
    const base = row({ interventionId: "r", timestamp: at(10, 1, 22).toISOString() });
    assert.equal(reward({ ...base, outcome: "exit", reenteredWithin30: false }), 1);
    assert.equal(reward({ ...base, outcome: "kept", reenteredWithin30: true }), 0.5);
    assert.equal(reward({ ...base, outcome: "watch" }), 0);
    assert.equal(reward({ ...base, outcome: "exit", reenteredWithin30: false, feedback: "not"
}), 0.5);
  });
});
```

```
describe("bandit tables", () => {
  it("with learning v2 off, matches v0.9.0: Beta per band and slot, skipping unsettled rows", ()
=> {
    const now = at(10, 6, 12);
    const data: AppData = {
      ...defaultData(),
      interventions: [
        row({ interventionId: "a", timestamp: at(10, 5, 23).toISOString(), hour: 23, band: 2, in
terventionType: "breath", outcome: "exit", reenteredWithin30: false, framing: "sleep" }),
        row({ interventionId: "b", timestamp: at(10, 5, 23, 30).toISOString(), hour: 23, band:
2, interventionType: "breath", outcome: "watch" }),
        row({ interventionId: "c", timestamp: at(10, 5, 23, 40).toISOString(), hour: 23, band:
2, interventionType: "card", outcome: "kept", reenteredWithin30: true }),
        // not settled yet (re-entry unknown): skipped
        row({ interventionId: "d", timestamp: at(10, 6, 11, 50).toISOString(), hour: 11, band:
1, interventionType: "pause", outcome: "exit", reenteredWithin30: null }),
        // older than 30 days: skipped
        row({ interventionId: "e", timestamp: at(8, 1, 23).toISOString(), hour: 23, band: 2, int
erventionType: "card", outcome: "exit", reenteredWithin30: false }),
      ],
    };
    const { methods, framings, samples } = banditTables(data, now, { v2: false });
    assert.equal(samples, 3);
    assert.deepEqual(methods["2:night"]?.breath, [2, 2]);
    assert.deepEqual(methods["2:night"]?.card, [1.5, 1.5]);
    assert.equal(methods["1:day"], undefined);
    assert.deepEqual(framings.night?.sleep, [2, 1]);
  });
});
```

bandit tables — 학습 개선을 끄면 v0.9.0과 같다. 기록 5개 중 재진입 판정이 안 끝난 것(d)과 30일 넘은 것(e)은 빼서 3개. 2단계·밤의 호흡은 성공 1·실패 1 → [2, 2], 카드는 0.5점 → [1.5, 1.5]. 문구 sleep은 [2, 1].

```ts
/**
 * 밴딧 테스트용 기록. watched = 개입 뒤 바로 이어서 본 분 (0이면 바로 나감).
 * 기록끼리 30분 창이 겹치지 않게 시각을 띄워서 만든다.
 */
type Entry = { when: Date; band: 1 | 2; arm: InterventionType; watched: number; framing?: string };

function scenario(entries: Entry[], settings: Partial<AppData["settings"]> = {}): AppData {
  const base = defaultData();
  return {
    ...base,
    settings: { ...base.settings, ...settings },
    interventions: entries.map((e, i) =>
      row({
        interventionId: `i${i}`,
        timestamp: e.when.toISOString(),
        hour: e.when.getHours(),
        band: e.band,
        interventionType: e.arm,
        outcome: e.watched > 0 ? "watch" : "exit",
        reenteredWithin30: e.watched > 0 ? null : false,
        framing: e.framing ?? null,
      }),
    ),
    usageLogs: entries.filter((e) => e.watched > 0).map((e, i) => session(`u${i}`, e.when, e.watched)),
  };
}

const mean = ([a, b]: [number, number]) => a / (a + b);
const close = (actual: number, expected: number) => assert.ok(Math.abs(actual - expected) < 1e-6, `${actual} ≈ ${expected}`);
/** 2026년 10월 평일 / 주말 날짜 */
const WEEKDAYS = [1, 2, 5, 6, 7, 8, 9, 12, 13, 14, 15, 16, 19, 20];
const WEEKENDS = [3, 4, 10, 11, 17, 18];
/** 최근 가중치를 사실상 끈다 (계산만 보고 싶을 때) */
const FLAT = { v2: true, halfLifeDays: 1e9 } as const;
```

학습 개선 테스트용 도구.

- **Entry / scenario**: "언제, 몇 단계, 어떤 방식, 개입 뒤 몇 분 봤는지"만 적으면 개입 기록과 시청 기록을 함께 만들어 준다 (0분이면 바로 나감).
- **mean**: [α, β]의 평균 = α ÷ (α + β). **close**: 소수 비교 (아주 작은 차이 허용).
- **WEEKDAYS / WEEKENDS**: 2026년 10월의 평일·주말 날짜.
- **FLAT**: 반감기를 엄청 길게 해서 최근 가중치를 사실상 끈 설정 (계산만 보려고).

```ts
describe("bandit learning v2", () => {
  const now = at(10, 21, 12); // 수요일

  it("scores by what was watched in the 30 minutes after", () => {
    const t = at(10, 20, 20);
    const log = row({ interventionId: "g", timestamp: t.toISOString() });
    close(gradedReward(log, []), 1); // 바로 나감
    close(gradedReward(log, [session("a", t, 3)]), 0.8); // 3분 약속 지킴
    close(gradedReward(log, [session("a", t, 20)]), 0); // 15분 넘게 봄
    close(gradedReward(log, [session("a", at(10, 20, 20, 20), 6)]), 0.6); // 20분 뒤 다시 와서 6분
    close(gradedReward(log, [session("a", at(10, 20, 20, 25), 10)]), 1 - 5 / 15); // 창 안 5분만
```

세밀한 점수(gradedReward) — 개입 뒤 30분 동안 더 본 시간:

- 바로 나감 1, 3분 지킴 0.8, 20분 봄 0(15분 넘으면 0), 20분 뒤 다시 와서 6분 0.6.
- 25분 뒤 시작한 10분 시청은 **30분 창 안의 5분만** 센다 → 1 − 5/15. 창 밖은 1.
- "도움 안 됐어요" 0.5, 연습 화면 기록은 세지 않음. 1분 구간에 겹친 시청 60초.

셈

```
    close(gradedReward(log, [session("a", at(10, 20, 21, 0), 10)]), 1); // 30분 창 밖
    close(gradedReward({ ...log, feedback: "not" }, []), 0.5);
    close(
      gradedReward(log, [{ ...session("p", t, 10), target: "practice" }]),
      1,
    ); // 연습 기록은 세지 않음
    close(watchedSecondsBetween([session("a", t, 3)], t.getTime(), t.getTime() + 60_000), 60);
  });
```

```
  it("waits 30 minutes before scoring a record", () => {
    const data = scenario([{ when: at(10, 21, 11, 45), band: 2, arm: "breath", watched: 0 }]);
    assert.equal(banditTables(data, now, FLAT).samples, 0);
    assert.equal(banditTables(data, at(10, 21, 12, 16), FLAT).samples, 1);
  });

  it("1. with no records at all, every arm is Beta(1, 1) like before", () => {
    const { methods, framings } = banditTables(scenario([]), now, { v2: true });
    assert.deepEqual(methods, {});
    assert.deepEqual(framings, {});
  });
```

30분이 지나야 점수를 매긴다: 11:45 기록은 12:00에는 0개, 12:16에는 1개.

1. 기록이 없으면 표도 비어 있다 → Java는 모든 방식을 Beta(1, 1)로 본다 (예전과 같음).

```
  it("2. an empty cell starts from what worked in other cells, capped at K", () => {
    const entries: Entry[] = [];
    for (const d of WEEKDAYS.slice(0, 10)) {
      entries.push({ when: at(10, d, 17), band: 2, arm: "breath", watched: 0 }); // 잘 됨
      entries.push({ when: at(10, d, 19), band: 2, arm: "card", watched: 20 }); // 안 됨
    }
    const { methods } = banditTables(scenario(entries), now, FLAT);
    const morning = methods["2:morning:weekday"];
    assert.ok(morning, "empty cell gets a starting value");
    assert.ok(mean(morning.breath) > 0.7);
    assert.ok(mean(morning.card) < 0.3);
    const [a, b] = morning.breath;
    close(a + b - 2, BANDIT_SHARE_K);
  });
```

2. 빈 칸은 다른 칸 경험으로 시작, 최대 K(4)만큼. 평일 저녁(17시, 19시)에 호흡은 매번 성공, 카드는 매번 실패. 기록 없는 평일 아침 칸도 호흡 평균 0.7 초과, 카드 0.3 미만으로 시작하고, 빌려 온 양(α + β − 2)은 정확히 4.

```
  it("3. a cell with plenty of its own records follows its own records", () => {
    const entries: Entry[] = [];
    // 다른 칸(평일 밤): 호흡이 매번 성공
    for (const d of WEEKDAYS) entries.push({ when: at(10, d, 23), band: 2, arm: "breath", watche
d: 0 });
    // 이 칸(평일 저녁): 호흡은 20번 중 18번 실패, 대체행동은 20번 중 18번 성공
    for (let i = 0; i < 40; i += 1) {
      const when = at(10, WEEKDAYS[i % 14], 17 + Math.floor(i / 14));
      const breath = i < 20;
      const ok = breath ? i < 2 : i < 38;
      entries.push({ when, band: 2, arm: breath ? "breath" : "alternative", watched: ok ? 0 : 20
});

    }
    const { methods } = banditTables(scenario(entries), now, FLAT);
```

3. 자기 기록이 충분한 칸은 자기 기록을 따른다. 다른 칸(평일 밤)에서 호흡이 매번 성공해도, 평일 저녁 칸에서 호흡이 20번 중 18번 실패했으면 그 칸의 호흡 평균은 0.3 미만이고 대체행동이 더 높다.

```
    const evening = methods["2:evening:weekday"];
    assert.ok(mean(evening.breath) < 0.3);
    assert.ok(mean(evening.alternative) > mean(evening.breath));
  });
```

```
  it("4. the cell's own records are not counted twice", () => {
    // 기록이 한 칸에만 있으면 빌려 올 다른 칸이 없으므로 자기 기록만 쓴 것과 같다.
    const entries: Entry[] = [
      { when: at(10, 19, 23), band: 2, arm: "breath", watched: 0 },
      { when: at(10, 20, 1), band: 2, arm: "breath", watched: 20 },
      { when: at(10, 20, 23), band: 2, arm: "breath", watched: 0 },
    ];
    const { methods, ownMethods } = banditTables(scenario(entries), now, FLAT);
    const [s, f] = ownMethods["2:night:weekday"].breath;
    close(s, 2);
    close(f, 1);
    const [a, b] = methods["2:night:weekday"].breath;
    close(a, 3);
    close(b, 2);
  });
```

4. 자기 칸 기록을 두 번 세지 않는다. 기록이 한 칸에만 있으면(성공 2, 실패 1) 빌려 올 곳이 없으니 결과는 [1 + 2, 1 + 1] = [3, 2].

```
  it("5. with learning v2 off (setting), tables equal the v0.9.0 calculation", () => {
    const data: AppData = {
      ...defaultData(),
      settings: { ...defaultData().settings, banditV2: false },
      interventions: [
        row({ interventionId: "y1", timestamp: at(10, 19, 23).toISOString(), hour: 23, band: 2, interventionType: "breath", outcome: "exit", reenteredWithin30: false, framing: "sleep" }),
        row({ interventionId: "y2", timestamp: at(10, 10, 8).toISOString(), hour: 8, band: 1, interventionType: "pause", outcome: "watch", framing: "goal" }),
        row({ interventionId: "y3", timestamp: at(10, 5, 14).toISOString(), hour: 14, band: 2, interventionType: "card", outcome: "kept", reenteredWithin30: false }),
      ],
    };
    const { methods, framings, v2 } = banditTables(data, now);
    assert.equal(v2, false);
    assert.deepEqual(methods, {
      "2:night": { breath: [2, 1] },
      "1:morning": { pause: [1, 2] },
      "2:day": { card: [2, 1] },
    });
    assert.deepEqual(framings, { night: { sleep: [2, 1] }, morning: { goal: [1, 2] } });
  });

  it("6. records outside the 30-day window are not borrowed either", () => {
    const data = scenario([{ when: at(9, 1, 23), band: 2, arm: "breath", watched: 0 }]);
    const { methods, ownMethods } = banditTables(data, now, { v2: true });
    assert.deepEqual(methods, {});
    assert.deepEqual(ownMethods, {});
  });
```

5. 설정에서 학습 개선을 끄면 표가 v0.9.0 계산과 똑같다 (칸 이름에 평일/주말 없음, 공유 없음).

6. 30일 밖 기록은 빌려 오지도 않는다 → 표가 비어 있음.

```
  it("an empty weekend cell leans on the same time slot on weekdays most", () => {
    const entries: Entry[] = [];
    for (const d of WEEKDAYS.slice(0, 8)) {
      entries.push({ when: at(10, d, 23), band: 2, arm: "breath", watched: 0 }); // 평일 밤: 잘 됨
      entries.push({ when: at(10, d, 8), band: 2, arm: "breath", watched: 20 }); // 평일 아침: 안 됨
    }
    const { methods } = banditTables(scenario(entries), now, FLAT);
    // 주말 밤은 평일 밤(0.8)을 평일 아침(0.4)보다 더 믿는다.
    assert.ok(mean(methods["2:night:weekend"].breath) > 0.5);
    assert.ok(mean(methods["2:morning:weekend"].breath) < 0.5);
    // 주말 기록이 쌓이면 주말 기록을 따른다.
    const weekendBad = [...entries];
    for (const d of WEEKENDS) {
      for (const h of [22, 23, 0]) {
        const day = h === 0 ? d + 1 : d;
        if (h === 0 && d % 7 === 4) continue; // 일요일 다음 0시는 평일이라 뺀다
        weekendBad.push({ when: at(10, day, h), band: 2, arm: "breath", watched: 20 });
      }
    }
    const after = banditTables(scenario(weekendBad), now, FLAT).methods;
    assert.ok(mean(after["2:night:weekend"].breath) < 0.3);
  });
```

빈 주말 칸은 같은 시간대의 평일을 가장 많이 참고한다. 평일 밤은 호흡 성공, 평일 아침은 실패 → 주말 밤 칸은 0.5 초과(평일 밤 비슷함 0.8), 주말 아침 칸은 0.5 미만. 그 뒤 주말 밤에 실패 기록이 쌓이면 주말 밤 칸은 0.3 미만 → **주말만의 패턴**을 따른다. (일요일 다음 0시는 월요일이라 뺀다.)

```
  it("similarity: weekday↔weekend is closest, then time slot, then band", () => {
    assert.equal(similarity("2:night:weekday", "2:night:weekend"), 0.8);
    assert.equal(similarity("2:night:weekday", "2:morning:weekday"), 0.5);
    assert.equal(similarity("2:night:weekday", "1:night:weekday"), 0.5);
    assert.equal(similarity("2:night:weekday", "1:morning:weekend"), 0.5 * 0.5 * 0.8);
    assert.equal(similarity("night:weekday", "night:weekend"), 0.8);
    assert.equal(dayType(at(10, 10, 12)), "weekend");
    assert.equal(dayType(at(10, 12, 12)), "weekday");
  });
```

비슷한 정도: 평일↔주말만 다름 0.8, 시간대 다름 0.5, 강도 다름 0.5, 셋 다 다르면 곱해서 0.2. 문구 칸에도 같은 규칙. 10/10(토) 주말, 10/12(월) 평일.

```
  it("weights recent records more (half every 7 days)", () => {
    const entries: Entry[] = [
      { when: at(10, 21, 11), band: 2, arm: "breath", watched: 0 }, // 1시간 전
      { when: at(10, 14, 12), band: 2, arm: "breath", watched: 0 }, // 7일 전
    ];
    const { ownMethods } = banditTables(scenario(entries), now, { v2: true });
    const [s, f] = ownMethods["2:day:weekday"].breath;
    close(s, Math.pow(0.5, 1 / 24 / 7) + 0.5);
    assert.equal(f, 0);
  });
```

최근 가중치 (7일마다 절반). 1시간 전 성공은 거의 1, 7일 전 성공은 0.5만큼 센다.

```
  it("follows a changed habit faster than equal weights", () => {
    const entries: Entry[] = [];
    // 3주쯤 전: 호흡이 잘 됐다. 최근: 호흡이 안 된다.
    for (const d of [1, 2, 5, 6, 7, 8, 9]) entries.push({ when: at(10, d, 23), band: 2, arm: "breath", watched: 0 });
```

바뀐 습관을 더 빨리 따라간다. 3주쯤 전엔 호흡이 7번 성공, 최근 이틀엔 4번 실패. 최근 가중치를 쓰면 평균 0.5 미만(요즘 안 통함을 반영), 같은 무게면 0.5 초과(옛 기록에 끌려감).

```
    for (const d of [19, 20]) {
      entries.push({ when: at(10, d, 22), band: 2, arm: "breath", watched: 20 });
      entries.push({ when: at(10, d, 23), band: 2, arm: "breath", watched: 20 });
    }
    const data = scenario(entries);
    const weighted = banditTables(data, now, { v2: true }).methods["2:night:weekday"].breath;
    const flat = banditTables(data, now, FLAT).methods["2:night:weekday"].breath;
    assert.ok(mean(weighted) < 0.5);
    assert.ok(mean(flat) > 0.5);
  });
});
```

```
describe("risk hours", () => {
  it("needs five recorded days, then finds the long-watch hours", () => {
    const now = at(10, 10, 12);
    const few = { ...defaultData(), usageLogs: [1, 2, 3].map((d) => session(`s${d}`, at(10, d, 23), 20)) };
    assert.equal(riskHours(few, now), null);

    const logs: UsageLog[] = [];
    for (let d = 1; d <= 7; d += 1) {
      logs.push(session(`n${d}`, at(10, d, 23), 25)); // 밤 11시에 25분
      logs.push(session(`m${d}`, at(10, d, 8), 2)); // 아침 8시에 2분
    }
    const data = { ...defaultData(), usageLogs: logs };
    const risk = riskHours(data, now);
    assert.ok(risk);
    assert.deepEqual(risk.hours, [23]);
    assert.ok(risk.avgSeconds > 20 * 60);
  });
});
```

위험 시간. 기록 3일이면 없음(null). 7일 동안 밤 11시에 25분, 아침 8시에 2분 봤으면 위험 시간은 [23]만, 평균 20분 초과.

```
describe("weekly goal suggestion", () => {
  const monday = at(10, 12, 9); // 2026-10-12 월요일
  const lastWeek = [5, 6, 7, 8, 9, 10, 11];

  it("suggests a smaller goal after five kept days, down to the final goal", () => {
    const data = {
      ...defaultData(),
      usageLogs: lastWeek.map((d) => session(`w${d}`, at(10, d, 20), 15)),
    };
    const s = goalSuggestion(data, monday);
    assert.ok(s);
    assert.equal(s.week, weekKey(monday));
    assert.equal(s.keptDays, 7);
    assert.equal(s.current, 20);
    assert.equal(s.suggested, 17);
    assert.equal(s.reduce, true);

    const atFloor = { ...data, settings: { ...data.settings, level3Threshold: 10, finalGoal: 10 } };
```

주간 목표 제안.

- 지난주 7일 모두 15분(목표 20분 안) → 7일 지킴, 20 → **17분** 제안(목표 15~29분이면 3분 줄임).
- 이미 최종 목표 10분이고 지켰으면 → "유지"(reduce false).
- 7일 중 3일만 지킨 주 → 제안 없음 (못 지킨 주엔 더 조이지 않음).

```
    const kept = { ...atFloor, usageLogs: lastWeek.map((d) => session(`f${d}`, at(10, d, 20),
8)) };
    const hold = goalSuggestion(kept, monday);
    assert.equal(hold?.reduce, false);
  });

  it("does not tighten after a missed week", () => {
    const data = {
      ...defaultData(),
      usageLogs: lastWeek.map((d, i) => session(`x${d}`, at(10, d, 20), i < 3 ? 15 : 40)),
    };
    assert.equal(goalSuggestion(data, monday), null);
  });
});

describe("policy and feedback", () => {
  it("switches the policy between rules and AI", () => {
    const data = defaultData();
    const on = buildPolicy(data.settings, data);
    assert.equal(on.version, "ai-2");
    assert.equal(buildPolicy({ ...data.settings, banditV2: false }, data).version, "ai-1");
    assert.deepEqual(on.ai.habituation, { strength: 0.3, recoveryHours: 6 });
    assert.equal(buildPolicy({ ...data.settings, banditV2: false }, data).ai.habituation.strengt
h, 0);
    assert.equal(on.ai.enabled, true);
    const off = buildPolicy({ ...data.settings, aiEnabled: false }, data);
    assert.equal(off.version, "rule-2");
    assert.equal(off.ai.enabled, false);
    assert.equal(buildPolicy({ ...data.settings, strictMode: true }, data).ai.strict, true);
  });

  it("turns feedback rows into answers for the intervention", () => {
    const result = convertNativeBatch(defaultData(), {
      sessions: [],
      lockAttempts: [],
      interventions: [
        { id: "f1", createdAt: at(10, 6, 22).getTime(), target: "youtube", level: 2, reason: "선
택하지 않음", interventionType: "feedback", alternativeAction: null, outcome: "feedback", usageSeconds:
0, stage: "feedback", refId: "row-1", feedback: "helpful" },
      ],
    });
    assert.equal(result.interventions.length, 0);
    assert.deepEqual(result.feedback, [{ refId: "row-1", feedback: "helpful" }]);
  });
});
```

정책표 버전. AI + 학습 개선 → "ai-2", 학습 개선 끔 → "ai-1"(질림 반영 0), AI 끔 → "rule-2". 질림 설정 0.3 / 6시간, 엄격 모드가 정책표에 실리는지.

"도움이 됐나요?" 답 변환. Java가 보낸 feedback 줄은 새 개입 기록으로 넣지 않고, 어느 개입(refId)의 답인지만 꺼낸다.

logic.test.ts — 기록·통계·카드 테스트 (웹) `lib/logic.test.ts (689줄)`

```
  1 import assert from "node:assert/strict";
  2 import { describe, it } from "node:test";
  3 import {
  4   applyFlush,
  5   blockUntilFromMinutes,
  6   buildIntervention,
  7   convertNativeBatch,
  8   finalizeSession,
  9   hourlyUsage,
 10   insightLine,
 11   lockStreak,
 12   isBlocked,
 13   isReportDue,
 14   levelForUsage,
 15   minutesUntilDayEnd,
 16   restSecondsOn,
 17   usageStats,
 18   reportLines,
 19   secondsOnDate,
 20   settleReentry,
 21   statsForDate,
 22   usageSecondsOn,
 23   validateSettings,
 24   webSecondsOn,
 25 } from "./logic.ts";
 26 import { buildPolicy, CARD_PROMPT, cardsForService, explanationLooksWrong, parseCardImport } fro
m "./policy.ts";
 27 import {
 28   defaultData,
 29   defaultSettings,
 30   normalizeData,
 31   readIntervention,
 32   readSettings,
 33   settingsFromGoal,
 34 } from "./storage.ts";
 35 import type { AppData, InterventionLog, UsageLog } from "./types.ts";
 36
 37 const settings = defaultSettings();
```

1~35줄 시험할 함수들: logic.ts(단계, 시청 갱신, 잠금, 리포트, 통계, 자정 나누기, Java 기록 변환, 재진입), policy.ts(정책표, 카드 가져오기), storage.ts(기본값, 읽기 검사). **37줄** 기본 설정 (목표 20분).

```
describe("level", () => {
  it("follows the 10 and 20 minute bands", () => {
    assert.equal(levelForUsage(0, settings), 1);
    assert.equal(levelForUsage(9.9 * 60, settings), 1);
    assert.equal(levelForUsage(10 * 60, settings), 2);
    assert.equal(levelForUsage(19.9 * 60, settings), 2);
    assert.equal(levelForUsage(20 * 60, settings), 3);
  });
});
```

level: 9.9분까지 1단계, 10분부터 2단계, 20분부터 3단계 (Java GuardTimeTest와 같은 값 → 두 쪽이 일치).

```
describe("settings", () => {
  it("rejects a final goal above the daily goal and a bad day-end hour", () => {
    assert.ok(validateSettings({ goal: 20, finalGoal: 30, dayEndHour: 6, notificationTime: "21:00" }));
    assert.ok(validateSettings({ goal: 20, finalGoal: 10, dayEndHour: 13, notificationTime: "21:00" }));
    assert.ok(validateSettings({ goal: 0, finalGoal: 0, dayEndHour: 6, notificationTime: "21:00" }));
  });

  it("accepts a clock with seconds", () => {
    assert.equal(
      validateSettings({ goal: 20, finalGoal: 10, dayEndHour: 6, notificationTime: "09:30:00" }),
      null,
    );
  });
```

settings 검사: 최종 목표가 하루 목표보다 크거나, 하루 끝 13시, 목표 0분이면 오류 문장이 나와야 한다(`assert.ok` = 문장이 있음). "09:30:00"처럼 초가 붙은 시각은 통과(null).

```
  it("derives the medium band start from the goal", () => {
    const settings = settingsFromGoal(30, {
      notificationTime: "21:00",
      notifyEnabled: true,
      finalGoal: 15,
      dayEndHour: 6,
      wakeTime: "07:00",
      aiEnabled: true,
      strictMode: false,
      banditV2: true,
      studyMode: false,
      goalWeekAnswered: "",
    });
    assert.equal(settings.level1Threshold, 15);
    assert.equal(settings.level3Threshold, 30);
    assert.equal(levelForUsage(14 * 60, settings), 1);
    assert.equal(levelForUsage(15 * 60, settings), 2);
    assert.equal(levelForUsage(30 * 60, settings), 3);
  });

  it("reads old saved settings with defaults for the new fields", () => {
    const settings = readSettings({ level1Threshold: 10, level2Threshold: 20, level3Threshold: 20, notificationTime: "21:00" });
```

목표로 단계 기준 만들기: 목표 30분 → 중간 시작 15분, 강함 30분. 14분 1단계, 15분 2단계, 30분 3단계.

옛 설정 읽기: 예전 버전에 없던 칸(최종 목표, 하루 끝)은 기본값(10분, 6시)으로 채운다 → 업데이트해도 앱이 깨지지 않음.

```
    assert.equal(settings.level3Threshold, 20);
    assert.equal(settings.finalGoal, 10);
    assert.equal(settings.dayEndHour, 6);
  });
});
```

```
const local = (d: number, h: number, m = 0, s = 0) =>
  new Date(2026, 8, d, h, m, s).toISOString();

function practiceLog(partial: Partial<UsageLog> & Pick<UsageLog, "usageId">): UsageLog {
  return {
    date: "2026-09-27",
    startTime: local(27, 10),
    endTime: local(27, 10),
    duration: 0,
    clipCount: 1,
    target: "practice",
    source: "web",
    ...partial,
  };
}

function intervention(partial: Partial<InterventionLog>): InterventionLog {
  const row = readIntervention({
    interventionId: "x",
    timestamp: local(27, 10),
    date: "2026-09-27",
    level: 1,
    reason: "습관적으로",
    reasonNote: "",
    alternativeAction: null,
    alternativeCompleted: false,
    reEntered: false,
    blocked: false,
    blockDuration: 0,
    usageSeconds: 0,
    outcome: "exit",
    ...partial,
  });
  assert.ok(row);
  return row;
}
```

도우미: `local(27, 10)` = 2026-09-27 10:00 (JavaScript Date는 월이 0부터라 8 = 9월). **practiceLog**: 테스트용 시청 한 줄. **intervention**: 테스트용 개입 한 줄 (기본값 + 바꿀 칸).

```
describe("usage flush", () => {
  it("adds real seconds only while visible", () => {
    const start = 1_000;
    const data: AppData = {
      ...defaultData(),
      activeSession: {
        usageId: "u1",
        date: "2026-09-27",
```

usage flush — 화면이 보일 때만 시간이 쌓이는가. 0.5초 동안 숨겨져 있으면 0초, 그 뒤 1초 보이면 1초. 마감(finalizeSession)하면 열린 시청이 닫히고 기록에 1초가 남는다.

```
        startTime: new Date(start).toISOString(),
        duration: 0,
        clipCount: 1,
        interventionId: "i1",
        levelAtStart: 1,
        lastTickAt: start,
      },
      usageLogs: [
        practiceLog({
          usageId: "u1",
          startTime: new Date(start).toISOString(),
          endTime: new Date(start).toISOString(),
        }),
      ],
    };
    const hidden = applyFlush(data, false, start + 500);
    assert.equal(hidden.activeSession?.duration, 0);
    const visible = applyFlush(hidden, true, start + 1500);
    assert.equal(visible.activeSession?.duration, 1);
    const done = finalizeSession(visible, start + 1500);
    assert.equal(done.activeSession, null);
    assert.equal(done.usageLogs[0]?.duration, 1);
  });
});
```

```
describe("block and report", () => {
  it("keeps a block for real minutes", () => {
    const now = Date.parse("2026-09-27T12:00:00");
    const until = blockUntilFromMinutes(10, now);
    assert.equal(Date.parse(until) - now, 600_000);
    assert.equal(isBlocked({ until, minutes: 10 }, now + 599_000), true);
    assert.equal(isBlocked({ until, minutes: 10 }, now + 601_000), false);
  });

  it("treats the report as due at the configured minute", () => {
    const morning = new Date(2026, 8, 27, 8, 0, 0);
    const evening = new Date(2026, 8, 27, 21, 0, 0);
    assert.equal(isReportDue("21:00", morning), false);
    assert.equal(isReportDue("21:00", evening), true);
  });
});
```

잠금: 10분 잠금은 정확히 600,000ms 뒤에 끝난다. 599초엔 잠김, 601초엔 풀림. **리포트 시각**: 21:00 설정이면 아침 8시엔 아직, 21시엔 도착.

```
describe("stats", () => {
  it("counts clips, reasons, reentry, timed blocks and lock attempts", () => {
    const data = defaultData();
    data.usageLogs.push(
      practiceLog({
        usageId: "u",
        startTime: local(27, 10),
        endTime: local(27, 10, 2),
        duration: 120,
```

통계 계산. 하루에 시청 1번(2분, 영상 4개), 개입 3번(대체행동 후 봄, 그냥 봄, 10분 잠금), 잠금 중 시도 1번을 넣고:

- 사용 120초, 영상 4, 개입 3, 많은 이유 "습관적으로", 재진입 1, 잠금 1, 잠금 중 시도 1.
- 리포트 첫 줄이 "스스로 멈춘 횟수 : 1번"으로 시작하고 "잠금 중 시도 : 1회"가 들어 있다 (`assert.match` = 글이 패턴에 맞는지).

```
        clipCount: 4,
      }),
    );
    data.interventions.push(
      intervention({
        interventionId: "a",
        level: 2,
        alternativeAction: "호흡하기",
        alternativeCompleted: true,
        reEntered: true,
        usageSeconds: 60,
        outcome: "watch",
      }),
      intervention({ interventionId: "a2", reEntered: true, outcome: "watch" }),
      intervention({
        interventionId: "b",
        level: 3,
        reason: "심심해서",
        blocked: true,
        blockDuration: 10,
        outcome: "timed-block",
      }),
    );
    data.lockAttempts.push({
      id: "l1",
      createdAt: local(27, 10, 6),
      date: "2026-09-27",
      remainingSec: 300,
      target: "youtube",
    });
    const stats = statsForDate(data, "2026-09-27");
    assert.equal(stats.usageSeconds, 120);
    assert.equal(stats.clipCount, 4);
    assert.equal(stats.interventionCount, 3);
    assert.equal(stats.topReason, "습관적으로");
    assert.equal(stats.reentryCount, 1);
    assert.equal(stats.blockCount, 1);
    assert.equal(stats.lockAttemptCount, 1);
    const report = reportLines(stats).join("\n");
    assert.match(report, /^스스로 멈춘 횟수 : 1번/);
    assert.match(report, /잠금 중 시도 : 1회/);
  });
});
```

```
describe("midnight split", () => {
  it("splits a session that crosses midnight by the time on each date", () => {
    const log = practiceLog({
      usageId: "night",
      date: "2026-09-27",
      startTime: local(27, 23, 58),
      endTime: local(28, 0, 3),
```

자정 나누기: 23:58~0:03(300초) 시청은 27일 120초, 28일 180초, 29일 0초. 하루 합계도 그날 몫만.

```
      duration: 300,
      target: "youtube",
      source: "native",
    });
    assert.equal(Math.round(secondsOnDate(log, "2026-09-27")), 120);
    assert.equal(Math.round(secondsOnDate(log, "2026-09-28")), 180);
    assert.equal(secondsOnDate(log, "2026-09-29"), 0);
    const data = { ...defaultData(), usageLogs: [log] };
    assert.equal(Math.round(usageSecondsOn(data, "2026-09-28")), 180);
  });
```

```
  it("counts only web seconds for the service baseline", () => {
    const data = {
      ...defaultData(),
      usageLogs: [
        practiceLog({ usageId: "p", startTime: local(27, 9), endTime: local(27, 9, 1), duration: 60 }),
        practiceLog({
          usageId: "n",
          startTime: local(27, 9, 5),
          endTime: local(27, 9, 7),
          duration: 120,
          target: "youtube",
          source: "native",
        }),
      ],
    };
    assert.equal(Math.round(webSecondsOn(data, "2026-09-27")), 60);
    assert.equal(Math.round(usageSecondsOn(data, "2026-09-27")), 180);
  });
});
```

Java에 보낼 "웹이 잰 초"는 연습 화면 기록만 (60초). 하루 전체는 Java 기록까지 180초. (Java가 이미 아는 시간을 두 번 세지 않게.)

```
describe("native batch", () => {
  it("converts service rows and skips ones already stored", () => {
    const startedAt = new Date(2026, 8, 27, 10, 0, 0).getTime();
    const batch = {
      sessions: [
        { id: "s1", target: "youtube", startedAt, endedAt: startedAt + 90_000, durationSec: 90 },
        { id: "bad", target: "youtube", startedAt, endedAt: startedAt, durationSec: 0 },
      ],
      interventions: [
        {
          id: "i1",
          createdAt: startedAt,
          target: "youtube",
          level: 2,
          reason: "심심해서",
          interventionType: "alternative",
          alternativeAction: "잠시 걷기",
          outcome: "exit",
```

Java 기록 묶음 변환.

- 정상 시청은 앱 기록으로 (출처 native, 날짜 9/27), 0초 시청("bad")은 버린다.
- 개입은 시(10시), 방식, 결과를 채우고, 잠금 남은 초 120.4 → 120.
- **301~308줄** 같은 묶음을 다시 받으면(ack 전에 다시 온 경우) **아무것도 추가하지 않는다** → 중복 저장 방지.

```
          usageSeconds: 700,
        },
      ],
      lockAttempts: [{ id: "k1", createdAt: startedAt, remainingSec: 120.4, target: "youtube"
}],
    };
    const first = convertNativeBatch(defaultData(), batch);
    assert.equal(first.usageLogs.length, 1);
    assert.equal(first.usageLogs[0]?.source, "native");
    assert.equal(first.usageLogs[0]?.date, "2026-09-27");
    assert.equal(first.interventions[0]?.hour, 10);
    assert.equal(first.interventions[0]?.interventionType, "alternative");
    assert.equal(first.interventions[0]?.outcome, "exit");
    assert.equal(first.lockAttempts[0]?.remainingSec, 120);

    const stored = {
      ...defaultData(),
      usageLogs: first.usageLogs,
      interventions: first.interventions,
      lockAttempts: first.lockAttempts,
    };
    const again = convertNativeBatch(stored, batch);
    assert.equal(again.usageLogs.length + again.interventions.length + again.lockAttempts.lengt
h, 0);
  });
});
```

```
describe("reentry within 30 minutes", () => {
  it("fills the flag only after 30 minutes and skips watch choices", () => {
    const left = intervention({ interventionId: "left", timestamp: local(27, 10), outcome: "exi
t" });
    const watched = intervention({ interventionId: "w", timestamp: local(27, 10), outcome: "watc
h" });
    const back = practiceLog({
      usageId: "back",
      startTime: local(27, 10, 20),
      endTime: local(27, 10, 21),
      duration: 60,
      target: "youtube",
      source: "native",
    });
    const data = { ...defaultData(), interventions: [left, watched], usageLogs: [back] };

    const early = settleReentry(data, new Date(2026, 8, 27, 10, 10).getTime());
    assert.equal(early.changed.length, 0);

    const later = settleReentry(data, new Date(2026, 8, 27, 10, 31).getTime());
    assert.equal(later.changed.length, 1);
    assert.equal(later.data.interventions[0]?.reenteredWithin30, true);
    assert.equal(later.data.interventions[1]?.reenteredWithin30, null);

```

30분 내 재진입 판정. 10:00에 나간 개입과 10:20 다시 본 시청:

- 10:10엔 아직 판정 안 함 (30분 안 지남).
- 10:31엔 "다시 열었음"(true). [보기]를 고른 개입은 판정 대상 아님(null).
- 다시 본 기록이 없으면 11:00에 "열지 않음"(false).

```
    const quiet = settleReentry(
      { ...data, usageLogs: [] },
      new Date(2026, 8, 27, 11, 0).getTime(),
    );
    assert.equal(quiet.data.interventions[0]?.reenteredWithin30, false);
  });
});
```

```
describe("intervention record", () => {
  it("adds hour, weekday and type", () => {
    const row = buildIntervention(
      {
        level: 2,
        reason: "심심해서",
        reasonNote: "",
        alternativeAction: null,
        alternativeCompleted: false,
        reEntered: false,
        blocked: false,
        blockDuration: 0,
        usageSeconds: 0,
        outcome: "exit",
      },
      "id1",
      new Date(2026, 8, 27, 22, 15),
    );
    assert.equal(row.hour, 22);
    assert.equal(row.weekday, 0);
    assert.equal(row.interventionType, "alternative");
    assert.equal(row.reenteredWithin30, null);
  });
});
```

개입 기록 만들기: 9/27(일) 22:15 → 시 22, 요일 0(일요일), 2단계는 방식 "alternative", 재진입은 아직 null.

```
describe("storage", () => {
  it("rejects a broken document", () => {
    assert.equal(normalizeData({ usageLogs: "nope" }), null);
  });

  it("keeps good rows when one row is broken and fills new fields", () => {
    const data = normalizeData({
      usageLogs: [
        { usageId: "old", date: "2026-09-27", startTime: local(27, 9), endTime: local(27, 9, 1),
duration: 60, clipCount: 2 },
        { usageId: 3 },
      ],
      interventions: [
        {
          interventionId: "i",
          timestamp: local(27, 9),
          date: "2026-09-27",
          level: 1,
```

저장소 읽기 검사. 형식이 아예 망가진 문서는 null(→ 손상 화면). 일부 줄만 망가졌으면 **좋은 줄은 살리고** 나쁜 줄(usageId가 숫자)만 버린다. 옛 기록에 없던 칸은 기본값으로 (대상 practice, 결과 watch, 방식 pause, 잠금 시도 빈 목록).

```
          reason: "기타",
          reEntered: true,
          blocked: false,
        },
      ],
    });
    assert.ok(data);
    assert.equal(data.usageLogs.length, 1);
    assert.equal(data.usageLogs[0]?.target, "practice");
    assert.equal(data.interventions[0]?.outcome, "watch");
    assert.equal(data.interventions[0]?.interventionType, "pause");
    assert.deepEqual(data.lockAttempts, []);
  });
});
```

```
describe("v0.8.0 helpers", () => {
  it("builds the approved time lines and skips unknown numbers", () => {
    const facts = {
      usage: 23 * 60,
      goal: 30 * 60,
      weekProjection: -1,
      avg7: -1,
      yesterdayByNow: -1,
      sleepSeconds: -1,
      sleepBasis: "",
    };
    assert.equal(insightLine("guilt", facts), "오늘 쇼츠에 쓴 23분, 다시 돌아오지 않아요.");
    assert.equal(insightLine("motivate", facts), "지금 멈추면 오늘 목표 안에서 끝낼 수 있어요. (7분 남음)");
    assert.equal(insightLine("yesterday", facts), null);
    assert.equal(insightLine("sleep", facts), null);
    assert.equal(
      insightLine("yesterday", { ...facts, yesterdayByNow: 12 * 60 }),
      "어제 이 시간엔 12분이었는데, 오늘은 벌써 23분이에요.",
    );
    assert.equal(
      insightLine("average", { ...facts, avg7: 15 * 60 }),
      "이번 주 평균보다 8분 더 보고 있어요.",
    );
    assert.equal(insightLine("guilt", { ...facts, usage: 30 }), null);
  });
```

시간 체감 문구 (웹판): Java GuardCopyTest와 같은 문장이 나오는지. 모르는 숫자(-1)면 null, 30초만 봤으면 null.

```
  it("computes stats for the lines from past days", () => {
    const now = new Date(2026, 8, 30, 21, 0); // 수요일
    const data = {
      ...defaultData(),
      usageLogs: [
        practiceLog({ usageId: "y", date: "2026-09-29", startTime: new Date(2026, 8, 29, 20, 0).toISOString(), endTime: new Date(2026, 8, 29, 20, 10).toISOString(), duration: 600 }),
        practiceLog({ usageId: "m", date: "2026-09-28", startTime: new Date(2026, 8, 28, 9, 0).toISOString(), endTime: new Date(2026, 8, 28, 9, 20).toISOString(), duration: 1200 }),
```

지난 기록 통계 (9/30 수 21시 기준): 오늘 전 기록 일수 2일(월, 화), 지난주 합계 1800초, 어제 19시까지 0초·20시까지 600초, 7일 평균 900초 — 기록이 시작된 날부터 나눈다 (처음 쓰는 사람의 평균이 낮게 나오지 않게).

```
      ],
    };
    const stats = usageStats(data, now);
    assert.equal(stats.daysBeforeToday, 2); // 월, 화
    assert.equal(Math.round(stats.pastWeekSeconds), 1800);
    assert.equal(Math.round(stats.yesterdayCumulative?.[19] ?? -1), 0);
    assert.equal(Math.round(stats.yesterdayCumulative?.[20] ?? -1), 600);
    assert.equal(Math.round(stats.avg7Seconds), 900); // 기록이 시작된 9/28부터 2일 평균
  });
```

```
  it("splits usage into hours", () => {
    const data = {
      ...defaultData(),
      usageLogs: [
        practiceLog({ usageId: "h", startTime: local(27, 9, 50), endTime: local(27, 10, 10), dur
ation: 1200 }),
      ],
    };
    const hours = hourlyUsage(data, "2026-09-27");
    assert.equal(Math.round(hours[9]), 600);
    assert.equal(Math.round(hours[10]), 600);
    assert.equal(hours[11], 0);
  });

  it("counts minutes until the next day-end hour", () => {
    assert.equal(minutesUntilDayEnd(6, new Date(2026, 8, 27, 23, 0)), 7 * 60);
    assert.equal(minutesUntilDayEnd(6, new Date(2026, 8, 28, 5, 30)), 30);
    assert.equal(minutesUntilDayEnd(6, new Date(2026, 8, 28, 6, 0)), 24 * 60);
  });
```

시간대별 나누기: 9:50~10:10 시청 → 9시 600초, 10시 600초, 11시 0.

하루 끝까지 분: 23:00 → 다음 6시까지 420분, 5:30 → 30분, 정확히 6:00 → 24시간.

```
  it("maps sheet results from the service", () => {
    const at = new Date(2026, 8, 27, 22, 0).getTime();
    const base = {
      createdAt: at,
      target: "youtube",
      level: 2,
      reason: "습관적으로",
      interventionType: "breath",
      alternativeAction: null,
      usageSeconds: 700,
    };
    const { interventions } = convertNativeBatch(defaultData(), {
      sessions: [],
      lockAttempts: [],
      interventions: [
        { ...base, id: "a", outcome: "kept", stage: "commit_end", interventionType: "commit_end"
},
        { ...base, id: "b", outcome: "day_end", blockMinutes: 480, band: 3, interventionType: "s
trong" },
        { ...base, id: "c", outcome: "lock", blockMinutes: 10, band: 3 },
        { ...base, id: "d", outcome: "watch", verification: "verified", commitMinutes: 5, framin
```

시트 결과 변환 (Java 개입 시트의 여러 결과):

- kept(약속 지킴) → kept, commit_end 방식. day_end → day-end, 480분, 강함. lock → timed-block.
- watch → 확인 결과(verified), 약속 5분, 대체행동 수행함. extend + 카드 정답.
- 통계: 스스로 멈춤 3번(지킴, 오늘은 끝, 시간 잠금), 지킴 1, 오늘은 끝 1, 연장 1, 푼 문제 1. 리포트 첫 줄에 "3번".

```
g: "book" },
        { ...base, id: "e", outcome: "extend", cardId: "k1", cardCorrect: true },
      ],
    });
    const by = (id: string) => interventions.find((row) => row.interventionId === id)!;
    assert.equal(by("a").outcome, "kept");
    assert.equal(by("a").interventionType, "commit_end");
    assert.equal(by("b").outcome, "day-end");
    assert.equal(by("b").blockDuration, 480);
    assert.equal(by("b").band, 3);
    assert.equal(by("c").outcome, "timed-block");
    assert.equal(by("d").interventionType, "breath");
    assert.equal(by("d").verification, "verified");
    assert.equal(by("d").commitMinutes, 5);
    assert.equal(by("d").alternativeCompleted, true);
    assert.equal(by("e").cardCorrect, true);

    const stats = statsForDate({ ...defaultData(), interventions }, "2026-09-27");
    assert.equal(stats.selfStopCount, 3); // kept, day-end, timed-block
    assert.equal(stats.keptCount, 1);
    assert.equal(stats.dayEndCount, 1);
    assert.equal(stats.extendCount, 1);
    assert.equal(stats.cardsSolved, 1);
    assert.match(reportLines(stats)[0] ?? "", /스스로 멈춘 횟수 : 3번/);
  });
```

```
  it("does not count an in-stay commitment sheet as re-entry", () => {
    const left = intervention({ interventionId: "left", timestamp: local(27, 10), outcome: "kept" });
    const sameStay = intervention({
      interventionId: "cs",
      timestamp: local(27, 10, 5),
      outcome: "kept",
      stage: "commit_end",
    } as Partial<InterventionLog>);
    const extended = intervention({ interventionId: "x", timestamp: local(27, 10), outcome: "extend" });
    const data = { ...defaultData(), interventions: [left, sameStay, extended] };
    const settled = settleReentry(data, new Date(2026, 8, 27, 11, 0).getTime());
    assert.equal(settled.data.interventions[0]?.reenteredWithin30, false);
    assert.equal(settled.data.interventions[2]?.reenteredWithin30, null);
  });
});
```

보는 중 약속 시트는 재진입이 아니다. 같은 시청 안에서 5분 뒤 뜬 "약속 끝" 시트(commit_end)를 "다시 열었다"로 세지 않는다 → 첫 개입은 false. 연장(extend)은 판정 대상 아님.

```
describe("policy and cards", () => {
  it("builds the rule policy from settings", () => {
    const policy = buildPolicy({ ...defaultSettings(), level3Threshold: 30, dayEndHour: 7 });
    assert.equal(policy.goalMinutes, 30);
    assert.equal(policy.dayEndHour, 7);
    assert.deepEqual(policy.commitOptions, [3, 5, 10]);
    assert.ok(policy.methods.medium.includes("card"));
  });

  it("imports pasted cards and skips broken ones", () => {
    let n = 0;
    const text = `여기 문제예요:
\`\`\`json
[{"question":"1+1은?","options":["1","2","3","4"],"answer":2},
 {"question":"","options":["a","b"],"answer":1},
 {"question":"수도는?","options":["서울","부산"],"answer":3}]
\`\`\``;
    const result = parseCardImport(text, new Date(2026, 8, 27), () => `c${++n}`);
    assert.equal(result.cards.length, 1);
    assert.equal(result.skipped, 2);
    assert.equal(result.cards[0]?.answer, 1);
    assert.equal(result.cards[0]?.source, "import");
    assert.ok(parseCardImport("그냥 글").error);
  });
```

규칙 정책표: 목표 30, 하루 끝 7시, 약속 버튼 [3, 5, 10], 중간 단계 방식에 카드 포함.

카드 붙여넣기: 설명 글과 ```json 감싸기가 있어도 읽는다. 문제가 빈 것, 정답 번호가 보기 수보다 큰 것은 건너뛰고(2개) 1개만. 정답 "2"(사람 기준 1부터) → 내부 1(0부터). JSON이 아니면 오류.

```
  it("sends the least-shown cards to the service", () => {
    const card = (id: string, shown: number) => ({
      id,
      question: id,
      options: ["a", "b"],
      answer: 0,
      createdAt: "2026-09-27T00:00:00.000Z",
      source: "manual" as const,
      shown,
      correct: 0,
      explanation: "",
    });
    const data = { ...defaultData(), cards: [card("seen", 5), card("new", 0)] };
    assert.deepEqual(cardsForService(data, 1).map((c) => c.id), ["new"]);
  });
});
```

Java에 보낼 카드는 덜 본 것부터 (5번 본 카드보다 0번 본 카드 먼저).

```
describe("v0.9.2 study cards", () => {
  const studyCard = (id: string, question: string) => ({
    id,
    question,
    options: ["a", "b"],
    answer: 0,
    createdAt: "2026-10-13T00:00:00.000Z",
    source: "import" as const,
    shown: 0,
```

v0.9.2 공부 카드 — 이상한 보기 거르기. 5개 중 정상 1개만 통과: 보기가 사실상 같음("물가 상승" / "물가 상승!"), "보기1·보기2" 같은 빈 내용, "…"만 있는 보기, 너무 짧은 문제는 건너뛴다.

```
    correct: 0,
    explanation: "",
  });

  it("skips cards with empty, junk or repeated options", () => {
    let n = 0;
    const text = `[
  {"question":"인플레이션이란?","options":["물가 상승","물가 하락","환율","금리"],"answer":1},
  {"question":"보기가 겹치는 문제","options":["물가 상승","물가  상승!","환율","금리"],"answer":1},
  {"question":"내용 없는 보기 문제","options":["보기1","보기2","보기3","보기4"],"answer":1},
  {"question":"점만 있는 보기 문제","options":["...","물가","환율","금리"],"answer":2},
  {"question":"짧","options":["a","b"],"answer":1}
]`;
    const result = parseCardImport(text, new Date(2026, 9, 13), () => `c${++n}`);
    assert.equal(result.cards.length, 1);
    assert.equal(result.skipped, 4);
  });
```

```
  it("does not import the same question twice", () => {
    let n = 0;
    const text = `[
  {"question":"인플레이션이란?","options":["물가 상승","물가 하락"],"answer":1},
  {"question":"인플레이션이란 ?","options":["물가 상승","물가 하락"],"answer":1},
  {"question":"기회비용이란?","options":["포기한 것의 가치","낸 돈"],"answer":1}
]`;
    const existing = [studyCard("old", "기회비용이란?")];
    const result = parseCardImport(text, new Date(2026, 9, 13), () => `c${++n}`, existing);
    assert.equal(result.cards.length, 1);
    assert.equal(result.duplicates, 2);
  });
```

중복 막기: 띄어쓰기만 다른 같은 질문, 이미 저장된 질문은 넣지 않는다 → 3개 중 1개만, 중복 2.

```
  it("marks cards whose explanation points at another option", () => {
    let n = 0;
    const text = `[
  {"question":"서유럽 밀 가격은 몇 배?","options":["2배","2.6배","3.3배","4배"],"answer":4,"explanation":"자료에 따르면 2.6배 올랐어요."},
  {"question":"영국 밀 가격은 몇 배?","options":["2배","2.6배","3.3배","4배"],"answer":2,"explanation":"영국은 2.6배, 서유럽은 4배였어요."},
  {"question":"인플레이션이란?","options":["물가 상승","물가 하락"],"answer":1,"explanation":"물가가 전반적으로 계속 오르는 상태예요."}
]`;
    const result = parseCardImport(text, new Date(2026, 9, 13), () => `c${++n}`);
    assert.equal(result.cards.length, 3);
    assert.deepEqual(result.suspectIds, ["c1"]);
    assert.equal(explanationLooksWrong({ options: ["A", "B"], answer: 0, explanation: "" }), false);
  });
```

해설-정답 불일치 의심: 정답이 "4배"인데 해설이 "2.6배 올랐어요"만 말하면 의심 표시(c1). 해설이 정답과 다른 보기를 **둘 다** 말하면(비교 설명) 의심 안 함. 해설이 없으면 의심 안 함.

```
  it("passes study mode to the service and keeps the usual activities as fallback", () => {
    const base = defaultData();
    const withCards = { ...base, cards: [studyCard("k", "문제 하나")] };
```

공부 모드가 정책표에 실리고, 평소 활동(호흡 등)도 남아 있어 카드가 없을 때 대신 쓴다. 꺼져 있으면 false.

```
    const on = buildPolicy({ ...base.settings, studyMode: true }, withCards);
    assert.equal(on.studyMode, true);
    assert.ok(on.methods.medium.includes("breath"));
    assert.equal(buildPolicy(base.settings, withCards).studyMode, false);
  });

  it("asks the AI for concept questions, a range, and a self-check", () => {
    assert.ok(CARD_PROMPT.includes("범위:"));
    assert.ok(CARD_PROMPT.includes("개념을 이해했는지"));
    assert.ok(CARD_PROMPT.includes("다시 확인하고"));
  });
```

질문 문장(CARD_PROMPT)에 "범위:", "개념을 이해했는지", "다시 확인하고"(스스로 검토)가 들어 있는지.

```
  it("reads card reports from the sheet without making intervention rows", () => {
    const result = convertNativeBatch(defaultData(), {
      sessions: [],
      lockAttempts: [],
      interventions: [
        { id: "r1", createdAt: new Date(2026, 9, 13, 21).getTime(), target: "youtube", level: 2, reason: "선택하지 않음", interventionType: "card", alternativeAction: null, outcome: "card_report", usageSeconds: 0, stage: "card_report", cardId: "bad-card" },
      ],
    });
    assert.deepEqual(result.reports, ["bad-card"]);
    assert.equal(result.interventions.length, 0);
  });
});

describe("v0.8.1", () => {
  it("reads loose ChatGPT output with explanations", () => {
    let n = 0;
    const text = `[
  {“question”: “3-way handshake 첫 단계는?”, "options": ["SYN", "ACK", "FIN", "RST"], "answer": "1번", "explanation": "연결 요청은 SYN으로 시작해요.",},
  {"question": "포트 80은?", "options": ["SSH", "HTTP"], "answer": "HTTP"},
]`;
    const result = parseCardImport(text, new Date(2026, 8, 27), () => `c${++n}`);
    assert.equal(result.error, null);
    assert.equal(result.cards.length, 2);
    assert.equal(result.cards[0]?.answer, 0);
    assert.equal(result.cards[0]?.explanation, "연결 요청은 SYN으로 시작해요.");
    assert.equal(result.cards[1]?.answer, 1);
  });
```

"이 문제 이상해요" 신고(card_report)는 개입 기록으로 넣지 않고, 지울 카드 id 목록으로만 꺼낸다.

ChatGPT의 느슨한 출력 읽기: 둥근 따옴표(” ”), 끝 쉼표, 정답 "1번"이나 보기 글자 "HTTP"로 적어도 읽고 해설도 가져온다.

```
  it("counts rest from timed locks only, up to now", () => {
    const lock = intervention({
      interventionId: "l",
      timestamp: local(27, 10),
      outcome: "timed-block",
      blockDuration: 30,
    });
    const end = intervention({
```

쉰 시간은 시간 잠금만, 지금까지만 센다: 10:00 30분 잠금은 10:10엔 600초, 다음 날엔 1800초. 밤의 "오늘은 끝"은 쉰 시간에 넣지 않는다 (잠자는 시간이 쉰 시간으로 부풀지 않게).

```
      interventionId: "e",
      timestamp: local(27, 23),
      date: "2026-09-27",
      outcome: "day-end",
      blockDuration: 420,
    });
    const data = { ...defaultData(), interventions: [lock, end] };
    assert.equal(Math.round(restSecondsOn(data, "2026-09-27", new Date(2026, 8, 27, 10, 10).getTime())), 600);
    assert.equal(Math.round(restSecondsOn(data, "2026-09-27", new Date(2026, 8, 28).getTime())), 1800);
  });
```

```
  it("counts a lock streak back from the day", () => {
    const on = (day: number, outcome: InterventionLog["outcome"]) =>
      intervention({ interventionId: `s${day}`, timestamp: local(day, 12), date: `2026-09-${day}`, outcome });
    const data = {
      ...defaultData(),
      interventions: [on(24, "timed-block"), on(25, "day-end"), on(26, "timed-block"), on(22, "timed-block")],
    };
    assert.equal(lockStreak(data, "2026-09-26"), 3);
    assert.equal(lockStreak(data, "2026-09-27"), 3); // 오늘 아직 없으면 어제부터
    assert.equal(lockStreak(data, "2026-09-23"), 1);
  });
});
```

잠금 연속 일수: 24·25·26일 연속 잠금 → 26일 기준 3일. 27일에 아직 없으면 어제부터 세서 3. 23일 기준은 22일 하루만 → 1 (중간에 빈 날이 있으면 끊김).